Bundesverband der Deutschen Volksbanken und Raiffeisenbanken e. V.
Bundesverband deutscher Banken e. V.
Bundesverband Öffentlicher Banken Deutschlands e. V.
Deutscher Sparkassen- und Giroverband e. V.
Verband deutscher Pfandbriefbanken e. V.

Die Deutsche Kreditwirtschaft

Anlage 1

der Schnittstellenspezifikation für die Datenfernübertragung zwischen Kunde und Kreditinstitut gemäß DFÜ-Abkommen

„Spezifikation für die EBICS-Anbindung“

Version 3.0
Final Version, 29. März 2017

Diese Spezifikation ist ab dem 27. November 2018 gültig
und sie ersetzt die bisherige Version 2.5 zum 22. November 2021.

©Die Deutsche Kreditwirtschaft

EBICS- Spezifikation

deutsche Übersetzung

(Electronic Banking Internet Communication Standard)

Version 3.0

Final Version, 29. März 2017

Diese Spezifikation ist ab 27. November 2018 gültig.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Änderungsverfolgung

Die Tabelle enthält eine Übersicht der wesentlichen Änderungen von Version 2.5 auf Version 3.0.

Kapitel	Art*	Beschreibung
Gesamtes Dokument	Ä	Da die Verteilte Elektronische Unterschrift in Zukunft EBICS-weit genutzt werden wird, wird die Abkürzung VEU (basierend auf den deutschen Begriff) in EDS (Electronic Distributed Signature) umbenannt. In bestehenden HAC-Codes bleibt die Abkürzung bestehen. *Hinweis: In der deutschen Übersetzung wird weiterhin in erster Linie die deutsche Abkürzung VEU verwendet und EDS allenfalls in Klammer dazugesetzt.*
Neuer Anhang "Transport Layer Security"	Ä	Die verwendete TLS-Version (V 1.2) und die Liste der empfohlenen Cipher Suiten wurden in einen neuen EBICS Anhang migriert Daher kann Kapitel 11.3.1 der Spezifikation gelöscht werden.
3 und 16	Ä	Nur das Public-Key-Format X.509 darf verwendet werden (Löschung von proprietären Beschreibungen).
3.13 und 3.14	E	Beschreibungen zur Interpretation von BTF- und Flag-Kombinationen durch den Bankserver.
4.6.2	Ä	Im Falle von Bankschlüsseländerungen ist der EBICS-Teilnehmer nicht mehr zu einer „manuellen" Überprüfung verpflichtet.
verschiedene, insbesondere 4.7, 11, 14	Ä	Erweiterung der Mindestschlüssellänge für alle Signaturschlüssel
Verschiedene, insbesondere 4.8 (Löschung)	L	Löschung aller Referenzen und Beschreibungen hinsichtlich der FTAM-Migration
5.5.2, 5.6.2, 8-10 und weitere Passagen	Ä / E / L	Integration des BTF-Konzepts (statt der bankfachlichen Auftragsarten und der File Format Parameter). Dies ist die größte Änderung von den V 2.x-Versionen und der neuen Version V 3.0. FUL und FDL-Strukturen werden gelöscht, die verbleibenden Auftragsarten werden alle nur noch für administrative und technische Zwecke verwendet und daher im Konzept „administrative Auftragsarten" zur Unterscheidung genannt. Denn alle bankfachlichen Aufträge werden mittels Elementgruppe „Service" aus der BTF-Struktur ausgedrückt (BTF = Business Transaction Formats). Im Kontext des BTF-Konzeptes werden die Auftragsattribute überflüssig (und daher entfernt). Das Element <OrderAttribute> und alle Regeln seiner Belegung werden entfernt (Schema-relevant!). Die Bedeutung wird ersetzt durch das <SignatureFlag> (für DZHNN und OZHNN). Die Bedeutung von UZHNN (Übertragung einer Unterschrift) ist die implizite und exklusive Bedeutung der administrativen Auftragsarten HVE, HVS und SPR. Das bedeutet insbesondere, dass (fehlende) EUs nur für bereits vorhandene Aufträge via VEU (internationale Abkürzung EDS) eingereicht werden können!
5.5.1.1.2, 8.3.1, 10	E	Zusätzliche Auftragsinformationen (gesendet vom Kunden an die Bank)
5.5.1.2, Abschnitt I.e.c.	K	Klarstellung zur Prüfung von Auftragsattributen und Auftragsnummern

* F = Fehler; Ä = Änderung; K = Klarstellung; E = Erweiterung; L = Löschung

Kapitel	Art*	Beschreibung
9.3.2.1.3	K	Weitere Beispiele für das Element HostID (zur Illustration)
10.2.1, 10.3, 10.4	E	Klarstellung zu HAC: - Timestamp - ISO Codeliste - Ende-Kennzeichen in HAC
Verschiedene, insbesondere 11.2, 11.5.1.1, 14, 14.2 (Löschung)	L	Löschung aller Referenzierungen und Beschreibungen zur EU-Version A004
11.3.1.1	F / Ä	Aktualisierungen und Korrekturen hinsichtlich der TLS-Verschlüsselung (TLS-Handshake)
13	Ä	Änderungen in der Liste der administrativen Auftragsarten
Beispiele 56, 59, 60 und 70	F	Korrektur von Beispielen bezüglich des Defaults des Attributs LastSegment im Element SegmentNumber: (Richtig ist: LastSegment="false")
16	Ä	Aktualisierung des Glossars. **Zu beachten ist insbesondere der hinzugefügte Begriff "Zertifikat"**
Anhang 1 (downloadbar im Internet)	E / Ä / L	Verschiedene Änderungen in der Returncode-Liste (Details siehe in der Änderungsverfolgung des Anhangs 1)
EBICS_BTF-ExternalCodeList	E	Neuer Anhang für die EBICS-Spezifikation. Dieser ersetzt Anhang 2 ab EBICS V 3.0.
Gemeinsamer Implemenation Guide	A / E / Ext	Verschiedene Änderungen (Details siehe in der Änderungsverfolgung des IG)

Kapitel	Art*	Beschreibung
Schemadateien innerhalb H005	E / Ä	1. Target namespace für H005 URN-Adresse: urn:org:ebics:H005 2. Neue xsd-Namen für alle Schemadateien innerhalb H005 (z.B. ebics_request_H005) 3. Definition eine optionalen Elements “AdditionalOrderInfo” für zusätzliche Informationen zum Auftrag (unstrukturiert, bis zu 255 Zeichen) a. Neuer einfacher Datentyp String255 b. Erweiterung des EBICS-Requests (Erweiterung von DataTransferRequestType) und von HVU/HVZ 4. Verpflichtendes Element DataDigest zur Übertragung des Hashwertes mit jedem EBICS-Request (Erweiterung von DataTransferRequestType) 5. Integration von “IBAN only” in die administrative VEU (neuer Begriff: EDS) Auftragsarten HVZ und HVT: Der Bankcode ist nun in diesen Strukturen optional 6. Löschung aller Strukturen, die sich auf FTAM beziehen (administrative Auftragsart HSA) 7. Löschung der Elementgruppe “OrderSignature”, die nur bei A004 verwendbar war. In diesem Kontext erfolgt auch ein globaler Austausch des Attributs „SignatureVersion” (nun verpflichtend statt wie bisher optional mit Default “A004”). Neuer Namespace / neue Version für S001 → S002 (wegen der Löschung der Elementgruppe OrderSignature war dies erforderlich) 8. Wegen der EBICS-weiten Verwendung des Public Key Formats X.509 sind Änderungen in den betreffenden Elementgruppen erforderlich 9. Erweiterungen und Änderungen für das BTF-Konzept 10. Inhalt von “genericOrderParams” wird auf BTU/BTDOrderParams verlagert 11. Löschung der optionalen Struktur x509Data in der HPD Response

Inhalt

1 Übersicht und Zielsetzungen für EBICS 10
1.1 Zielsetzung der Kooperation 10
1.2 Grundsätzliche Zielsetzung von EBICS 10

2 Definitionen 12
2.1 Begriffe 12
2.2 Notationen 12
2.2.1 XML 12
2.2.2 Ablaufdiagramme 14
2.2.3 Sonstige Notationen 15
2.3 Datentypen 15

3 Designentscheidungen 17
3.1 OSI-Modell aus EBICS-Sicht 17
3.1.1 TCP/IP als paketorientierte Übertragungsschicht 17
3.1.2 TLS als Transportverschlüsselung 18
3.1.3 HTTP(S) als technisches Basisprotokoll 20
3.1.4 XML als Anwendungsprotokollsprache 20
3.2 Komprimierung, Verschlüsselung und Kodierung der Auftragsdaten 25
3.3 Segmentierung der Auftragsdaten 25
3.4 Wiederaufsetzen der Übertragung von Aufträgen (Recovery) [optional] 26
3.5 Elektronische Unterschrift (EU) der Auftragsdaten 26
3.5.1 EU der Teilnehmer 26
3.5.2 EU der Kreditinstitute [vorgesehen] 27
3.5.3 Darstellung der EUs in EBICS-Nachrichten 28
3.6 Vorabprüfung [optional] 29
3.7 Technische Teilnehmer 30
3.8 Authentifikationssignatur 31
3.9 X.509-Daten 32
3.10 Unterstützte administrative Auftragsarten 33
3.11 Auftragsparameter 33
3.12 Ablauf der EBICS-Transaktionen 36
3.13 Interpretation der BTF-Elementkombinationen durch den Bankserver 39
3.14 Interpretation von EU- und VEU/EDS-Flag-Kombinationen durch den Bankserver 39

4 Schlüsselmanagement 41

4.1 Übersicht der verwendeten Schlüssel 41
4.2 Darstellung der öffentlichen Schlüssel 42
4.3 Aktionen innerhalb des Schlüsselmanagements 43
4.4 Initialisierung 44
4.4.1 Teilnehmerinitialisierung 47
4.4.2 Abholung der öffentlichen Schlüssel des Kreditinstituts 64
4.5 Sperren eines Teilnehmers 71
4.5.1 Alternativen 71
4.5.2 Sperren eines Teilnehmers über SPR 71
4.6 Schlüsseländerungen 72
4.6.1 Änderung der Teilnehmerschlüssel 72
4.6.2 Änderung der Bankschlüssel 79
4.7 Übergang zu größeren Schlüssellängen 80
4.8 Zusammenfassung 81

5 EBICS-Transaktionen 82
5.1 Allgemeine Festlegungen 82
5.1.1 EBICS-Transaktionen 82
5.1.2 Transaktionsphasen und Transaktionsschritte 82
5.1.3 Durchführung der Aufträge 82
5.1.4 Transaktionsverwaltung 83
5.2 Zuordnung von EBICS-Request zu EBICS-Transaktion 84
5.3 Vorabprüfung von Aufträgen [optional] 85
5.4 Recovery – Wiederaufsetzen von Transaktionen [optional] 87
5.5 Upload-Transaktionen 88
5.5.1 Ablauf von Upload-Transaktionen 88
5.5.2 Wiederaufsetzen von Upload-Transaktionen 115
5.6 Download-Transaktionen 119
5.6.1 Ablauf von Download-Transaktionen 119
5.6.2 Wiederaufsetzen von Download-Transaktionen 139

6 Verschlüsselung 143
6.1 Verschlüsselung auf TLS-Ebene 143
6.2 Verschlüsselung auf Anwendungsebene 143

7 Segmentierung der Auftragsdaten 145
7.1 Verfahrensbeschreibung 145
7.2 Umsetzung in den EBICS-Nachrichten 146

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

8 Verteilte Elektronische Unterschrift (VEU bzw. EDS) 147
8.1 Beschreibung des Verfahrens 147
8.2 Technische Umsetzung der VEU (bzw. EDS) 149
8.3 Detailbeschreibung der administrativen VEU-Auftragsarten 151
8.3.1 HVU (VEU-Übersicht abholen) und HVZ (VEU-Übersicht mit Zusatzinformationen abholen) [verpflichtend] 151
8.3.2 HVD (VEU-Status abrufen) 176
8.3.3 HVT (VEU-Transaktionsdetails abrufen) 182
8.3.4 HVE (Elektronische Unterschrift hinzufügen) 204
8.3.5 HVS (VEU-Storno) 208
8.3.6 Verwendete Service-Strukturen (eingeschränkt/nicht eingeschränkt) ... 211

9 „Sonstige“ administrative EBICS-Auftragsarten 214
9.1 HAA (abrufbare BTF-Parameter abholen) 214
9.1.1 HAA-Request 214
9.1.2 HAA-Response 214
9.2 HPD (Bankparameter abholen) 215
9.2.1 HPD-Request 216
9.2.2 HPD-Response 217
9.3 HKD (Kunden- und Teilnehmerinformationen des Kunden abrufen) 224
9.3.1 HKD-Request 225
9.3.2 HKD-Response 225
9.4 HTD (Kunden- und Teilnehmerinformationen des Teilnehmers abrufen) 243
9.4.1 HTD-Request 243
9.4.2 HTD-Response 243
9.5 HEV (Unterstützte EBICS-Versionen abrufen) 247
9.5.1 HEV-Request 247
9.5.2 HEV-Response 247
9.5.3 Schema für HEV-Request / HEV-Response 247

10 EBICS-Kundenprotokoll (HAC) 250
10.1 Vorbemerkungen 250
10.2 Belegung von pain.002 für HAC 251
10.2.1 Belegung der Elementgruppe Group Header 251
10.2.2 Belegung der Elementgruppe Original Group Information and Status ... 252
10.2.3 Belegung der Elementgruppe Original Payment Information and Status 252
10.3 Anhang für HAC: External Reason Codes (Ergebnis der Aktion) 261
10.4 Anhang für HAC: Art und Ergebnis der Aktion (zulässige Paare) 263

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

11 Anhang: Kryptografische Verfahren 266
11.1 Authentifikationssignatur 266
11.1.1 Verfahren 266
11.1.2 Format 266
11.2 Elektronische Unterschriften 267
11.2.1 Verfahren 267
11.2.2 Format 267
11.2.3 EBICS-Autorisationsschemata für Unterschriftsklassen 267
11.3 Verschlüsselung 268
11.3.1 Verschlüsselung auf TLS-Ebene 268
11.3.2 Verschlüsselung auf Anwendungsebene 268
11.4 Replay-Vermeidung mittels Nonce und Timestamp 271
11.4.1 Verfahrensbeschreibung 271
11.4.2 Aktionen des Kundensystems 271
11.4.3 Aktionen des Banksystems 273
11.5 Initialisierungsbriefe 274
11.5.1 Initialisierungsbrief für INI (Beispiel) 274
11.5.2 Initialisierungsbrief für HIA (Beispiel) 275
11.6 Erzeugen der TransaktionsIDs 277

12 Übersicht ausgewählter EBICS-Details 278
12.1 Optionale EBICS-Features 278
12.1.1 Optionale administrative Auftragsarten 278
12.1.2 Optionale Funktionalitäten im Transaktionsablauf 278
12.2 EBICS-Bankparameter 278
12.3 Sicherheitsmedien bankfachlicher Schlüssel 279
12.4 Muster für TeilnehmerIDs, KundenIDs, AuftragsIDs, HostIDs 279

13 Anhang: Auftragsartenkennungen 281

14 Anhang: Signaturverfahren für die Elektronische Unterschrift 283
14.1 Die Elektronische Unterschrift der Version A005/A006 284
14.1.1 Vorbemerkung und Einleitung 284
14.1.2 RSA 285
14.1.3 Signaturalgorithmus 287
14.1.4 Signaturverfahren A005 und A006 der DK 288
14.1.5 Referenzen 295
14.1.6 XML-Struktur der Unterschriftsversionen A005/A006 296

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

15 Anhang: Standards und Verweise 298

16 Anhang: Glossar 300

17 Abbildungsverzeichnis 305

Die XML-Schemata (H005, H000 und S002) finden sich unter
http://www.ebics.org (siehe „technical-information“) oder mit deutschsprachigen Erläuterungen unter
http://www.ebics.de (siehe „Spezifikation“)

1 Übersicht und Zielsetzungen für EBICS

1.1 Zielsetzung der Kooperation

Die Deutsche Kreditwirtschaft (DK) und die französische Kreditwirtschaft, vertreten durch das Comité Français d'Organisation et de Normalisation Bancaires (CFONB), gründeten eine Gesellschaft (EBICS SCRL) zur gemeinsamen Nutzung von EBICS. Die Schweizer Kreditwirtschaft, vertreten durch SIX Interbank Clearing, trat der EBICS SCRL im Jahr 2015 bei.
EBICS wurde ursprünglich von der Deutschen Kreditwirtschaft entwickelt. Es ermöglicht Firmenkunden (Nichtverbrauchern) eine flexible, sichere und effektive Durchführung ihrer Bankgeschäfte und erlaubt ihnen, den für ihre individuellen Bedürfnisse geeigneten Dienstleister auszuwählen. Außerdem ist EBICS „multibankfähig". Dies bedeutet, dass Firmenkunden (Nichtverbrauchern) grundsätzlich mit Hilfe derselben Software mit jeder EBICS unterstützenden BankVerbindung aufnehmen können.
Prinzipiell ist die Spezifikation EBICS-weit gültig. In Ausnahmefällen kann jedoch eine für ein Land geltende Vorschrift eine individuelle Anpassung der Spezifikation erforderlich machen.
Als optional definierte Funktionalitäten können in dem einen Land unterstützt (und verpflichtend vorgeschrieben) und gleichzeitig in einem anderen Land nicht unterstützt werden.
Ein gemeinsamer Implementation Guide, der die Unterstützung von optionalen Funktionalitäten näher erläutert, wurde erstellt (siehe Kapitel 3 in diesem Guide).

1.2 Grundsätzliche Zielsetzung von EBICS

Die vorliegende EBICS-Spezifikation („Electronic Banking Internet Communication Standard") beschreibt die Funktionalität multibankfähiger, sicherer Kommunikation über das Internet
EBICS stellt keine speziellen Anforderungen an die konkrete Architektur der Kundensysteme; es können eigenständige Desktop-Applikationen genauso angebunden werden wie etwa Client/Server-Anwendungen oder Applet-Lösungen.

Das Verfahren „DFÜ mit Kunden" wird auf der Anwendungsebene ergänzt um das Konzept der Verteilten Elektronischen Unterschrift (VEU, englischer Begriff: Electronic Distributed Signature = EDS), das eine zeitlich und räumlich unabhängige Autorisierung von Aufträgen kundenübergreifend ermöglicht.

Die grundsätzlichen Merkmale des EBICS-Standards sind:

- Übertragung der fachlichen Daten (Geschäftsvorfälle) über Auftragsarten in den etablierten bankfachlichen Formaten
- Erweiterung des DFÜ-Abkommens um die Möglichkeit der „Verteilten Elektronischen Unterschrift (VEU / EDS)"
- Spezifikation der EBICS-spezifischen Protokollelemente in XML
- Versand der Nachrichten über HTTP („internetbasiert"); Nutzung von TLS als Basis-Transportsicherung zwischen dem Kunden- und dem Banksystem unter Verwendung der TLS-Server-Authentisierung
- kryptografische Absicherung jedes einzelnen Schritts einer Transaktion mittels Verschlüsselung und digitaler Signaturen auf Anwendungsebene.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Die EBICS-Spezifikation ist die Grundlage für die Entwicklung von Kunden- und Banksystemen, die über das EBICS-Protokoll kommunizieren. Als solche enthält sie herstellerunabhängige Verfahrensbeschreibungen und gewährleistet so die Interaktion zwischen Kunden- und Banksystemen unterschiedlicher Hersteller.

Diese EBICS-Spezifikation umfasst die EBICS-Protokollbeschreibung samt Details zu Schlüsselmanagement, VEU (EDS) und den XML-Schemata für die Auftragsdaten der administrativen (technischen) EBICS-Auftragsarten. Die kompletten XML-Schemata sind als eigenständige HTML-Dokumentation hinterlegt.

Die EBICS-Spezifikation schränkt die Implementierungsfreiheit der Kunden- und Banksysteme durch Vorgaben und Festlegungen nur da ein, wo Sicherheitsüberlegungen oder Abläufe jenseits der EBICS-Kommunikation dies erforderlich machen. In Abgrenzung zum EBICS Implementation Guide werden in der EBICS-Spezifikation keine Implementierungsalternativen aufgezeigt.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

2 Definitionen

2.1 Begriffe

Folgende Begriffe in Kapitälchenschrift haben in der Protokolldefinition eine spezielle Bedeutung:

- **MUSS/MÜSSEN:** beschreibt eine zwingende Anforderung; nur Implementierungen, die diese Anforderung erfüllen, sind EBICS-konform
- **SOLL/SOLLEN/SOLLTE/SOLLTEN:** benennt Anforderungen, die normalerweise zu befolgen sind; technisch oder fachlich begründete Ausnahmen sind im Einzelfall aber möglich
- **KANN/KÖNNEN:** bezeichnet unverbindliche Empfehlungen oder optionale Features.

Funktionalitäten oder Merkmale des EBICS-Protokolls werden als **optional** gekennzeichnet, wenn sie vom Kreditinstitut nicht unterstützt werden müssen. Kunden haben gegenüber den Kreditinstituten keinen rechtlichen Anspruch auf die entsprechende Funktionalität.

Funktionalitäten oder Merkmale des EBICS-Protokolls werden in einer bestimmten Version als **vorgesehen** gekennzeichnet, wenn sie für nachfolgende Versionen vorbereitet sind, jedoch in der gegebenen Version noch nicht verwendet werden dürfen.

Diese Spezifikation ist an Software-Hersteller gerichtet. Die oben erwähnten Bedeutungen beziehen sich auf die Anforderungen an ImplementierungenImplemtierungen, die die EBICS-Funktionalität unterstützen. Dieses Dokument ist keine Vorgabe an Banken zur konkreten Ausgestaltung von Kundenbedingungen.

2.2 Notationen

2.2.1 XML

2.2.1.1 XML-Schema

Folgende Symbolik zur grafischen Darstellung von XML-Schema wird verwendet:

- Elemente werden in Rechtecke gesetzt
- Attribute werden ebenfalls in Rechtecke gesetzt und von einem Kasten „attributes“ umgeben
- Elemente, Attribute und weitere Deklarationen, die zu einem komplexen Typ gehören, werden von einem gestrichelten, gelb hinterlegten Kasten umgeben
- Eine „Verzweigung” (entspricht `choice` bei XML-Schema) wird mit einem Achteck dargestellt, das ein Schaltsymbol für drei mögliche Schaltpositionen enthält. Rechts vom Symbol verzweigen die Verbindungslinien zu den möglichen Alternativen
- Eine „Sequenz“ (entspricht `sequence` bei XML-Schema) wird mit einem Achteck dargestellt, das ein Liniensymbol mit drei Punkten darauf enthält. Rechts vom Symbol verzweigen die Verbindungslinien zu den einzelnen Sequenzelementen

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

- Symbole mit durchgehender Umrandung deuten auf obligatorische Verwendung hin und entsprechen bei XML-Schema dem Attribut `minOccurs="1"` für Elemente bzw. `use="required"` für Attribute
- gestrichelte Symbole deuten auf optionale Verwendung hin und entsprechen bei XML-Schema dem Attribut `minOccurs="0"` für Elemente bzw. `use="optional"` für Attribute
- durchgestrichene Symbole deuten auf vorgesehene Verwendung hin und entsprechen bei XML-Schema der Attributkombination `minOccurs="0" maxOccurs="0"` für Elemente bzw. `use="prohibited"` für Attribute
- „m..n“ an der rechten unteren Ecke eines Elementsymbols begrenzt die Verwendung des Elements auf m- bis n-faches Vorkommen und entspricht `minOccurs="`*m*`" maxOccurs="`*n*`"` bei XML-Schema; bei „m..∞“ entsprechend `minOccurs="`*m*`" maxOccurs="unbounded"`
- Elementgruppen werden durch Achtecke dargestellt und entsprechen der `group`-Deklaration in XML-Schema
- Attributgruppen werden von Kästen mit dem jeweiligen Gruppennamen umgeben und entsprechen der `attributeGroup`-Deklaration in XML-Schema.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

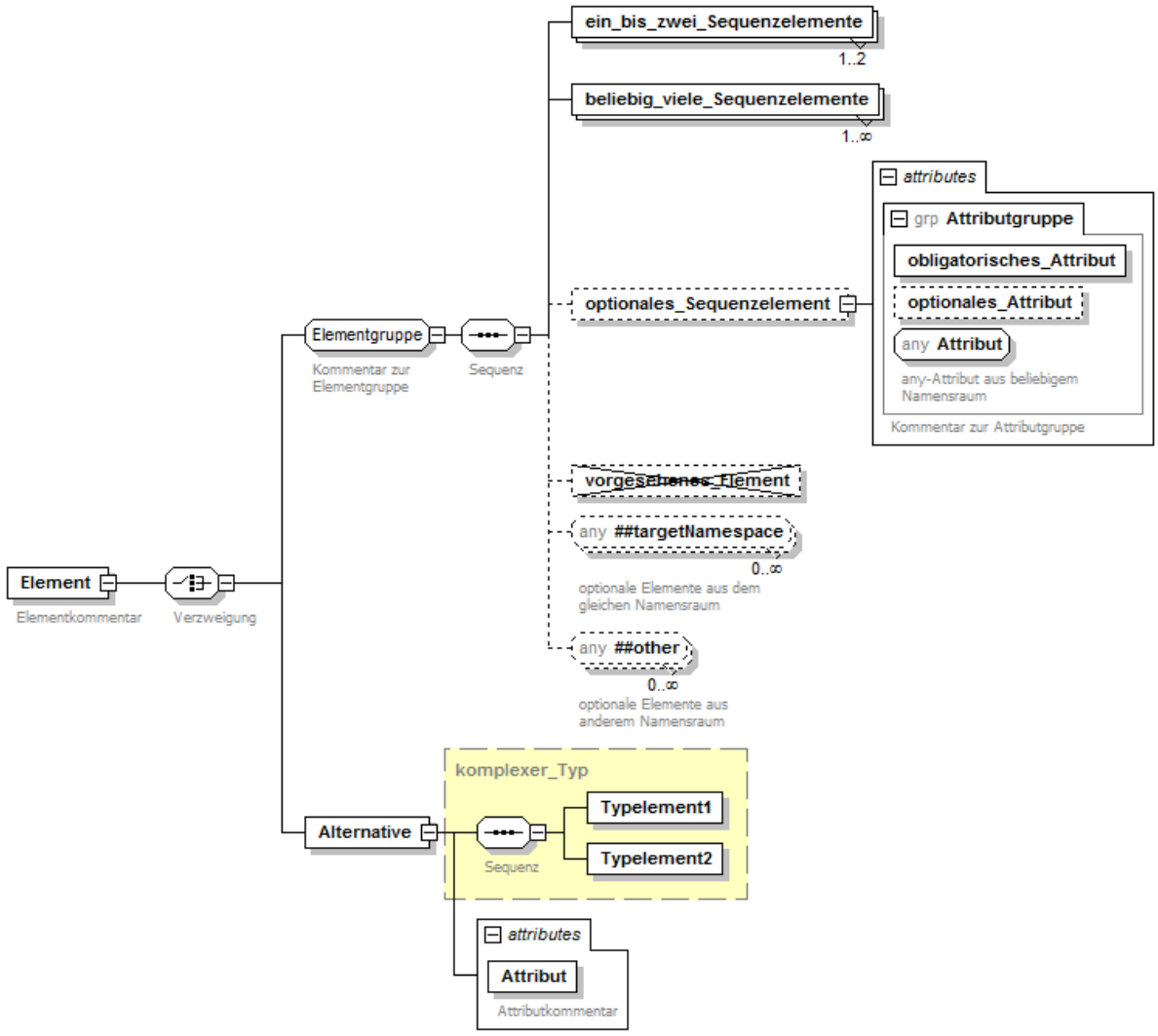


Abbildung 1: Symbolik für XML-Schema

2.2.1.2 XML-Dokumente

Einzelne Codeteile werden mit der Schriftart Courier versehen.
Falls ein Elementname oder –typ nicht komplett in eine Zeile passt, wird mit »
in die nächste Zeile weitergeleitet.

```
Komplette Codebeispiele werden in Courier 8pt und mit Rahmen gesetzt.
```

2.2.2 Ablaufdiagramme

Abläufe werden mit Hilfe von UML 2.0-Aktivitäten dargestellt. Im vorliegenden Dokument enthalten sie einen Start- und einen Endknoten. Ein Startknoten ist der Startpunkt eines Ablaufs, der Endknoten markiert das Ende eines gesamten Ablaufs.

Für die bessere Übersichtlichkeit werden Aktivitäten ineinander verschachtelt. Aktionen, die wiederum eine Aktivität enthalten, werden durch ein Gabelsymbol gekennzeichnet. Die Aktivität A in Abbildung 2 besteht aus drei Ablaufschritten (Aktionen). Schritt_2 ist selbst wiederum eine Aktivität aus 2 Ablaufschritten. Die Aktivität Schritt_2 wird somit innerhalb der Aktivität A aufgerufen, d.h. vom Startknoten von Schritt_2 bis zum Endknoten von Schritt_2 durchlaufen.

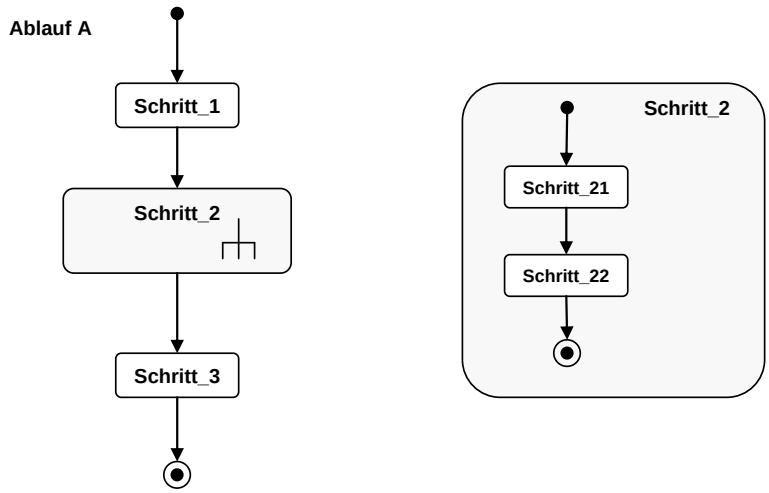


Abbildung 2 Verschachtelung von Aktivitäten

2.2.3 Sonstige Notationen

Bei Nennung neuer administrativer Auftragsarten wird mit dem Anhängsel „[verpflichtend]“ kenntlich gemacht, dass das Kreditinstitut diese administrative Auftragsart unterstützen MUSS. Mit dem Anhängsel „[optional]“ ist demgegenüber gemeint, dass das Kreditinstitut diese administrative Auftragsart unterstützen KANN.

Analog dazu wird vorgesehenen Merkmalen bzw. Funktionalitäten das Kennzeichen „[vorgesehen]“ angehängt.

2.3 Datentypen

XML-Schema definiert einen Satz von primitiven und abgeleiteten Datentypen, die zum Aufbau eigener Datentypen benutzt werden können.

Im Zusammenhang mit EBICS finden hauptsächlich die folgenden primitiven Datentypen Verwendung:

- **string**: Zeichenketten unbeschränkter Länge und beliebigen Aufbaus
- **boolean**: boolscher Wahrheitswert mit den Ausprägungen „true“ (=1) oder „false“ (=0)
- **decimal**: Dezimalzahlen beliebiger Genauigkeit
- **dateTime**: Zeitstempel mit Datum und Uhrzeit gemäß ISO 8601
  Der Aufbau ist wie folgt: YYYY-MM-DDTHH:MM:SS.sss**Z,**

wobei Z wiedergibt, dass ein Umrechnung auf UTC stattgefunden hat.
Sollte der Datumsstring nicht diesem Aufbau entsprechen ist die EBICS-Nachricht abzulehnen bzw. die EU-Prüfung als negativ zu bewerten.

- **date**: Datumsangabe gemäß ISO 8601
- **hexBinary**: Hexadezimalwert beliebiger Länge
- **base64Binary**: Datentyp zur Aufnahme von base64-kodierten Binärdaten
- **anyURI**: Uniform Resource Locator (z.B. URL, IP-Adresse).

Die folgenden vordefinierten Datentypen werden von primitiven Datentypen abgeleitet und finden im EBICS-Standard Verwendung:

- **normalizedString**: Zeichenkette, die am Anfang und Ende von Leerraum (blanks) bereinigt ist
- **token**: ein normalizedString, der keine Zeilenvorschübe und keine mehrfachen Leerräume hintereinander enthält
- **language**: Länderkennzeichen gemäß RFC 1766
- **nonNegativeInteger**: ganzzahlige, nicht negative Werte
- **positiveInteger**: ganzzahlige, positive Werte.

Mit Hilfe der genannten Datentypen werden in den EBICS-Schemata neue Datentypen definiert:

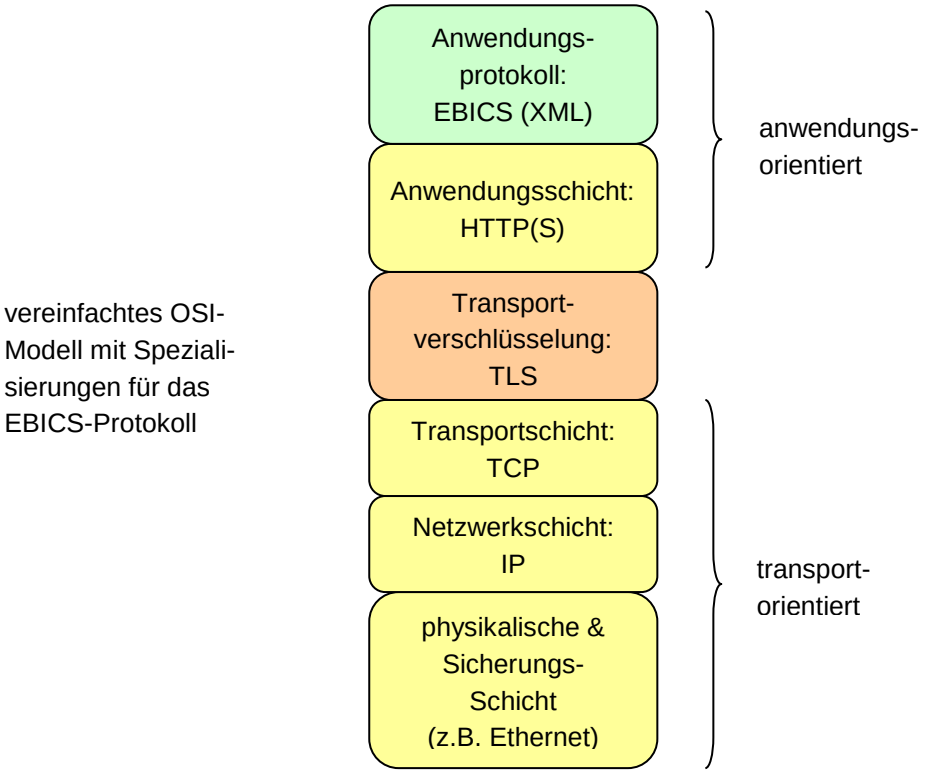
- **einfache (simple) Datentypen** definieren lediglich einschränkende oder erweiternde Merkmale bezüglich des Wertebereichs eines vorhandenen primitiven oder abgeleiteten Datentyps, d.h. sie leiten von einem bestehenden Datentyp ab
- **komplexe (complex) Datentypen** definieren neue Strukturen, die sich aus Feldern und Attributen unterschiedlicher (einfacher oder komplexer) Datentypen zusammensetzen.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

3 Designentscheidungen

Dieses Kapitel verdeutlicht Entscheidungen, die ausschlaggebenden Einfluss auf das Design des EBICS-Protokolls hatten. Darunter fallen sowohl netzwerkspezifische Details als auch Vorgaben fachlicher und technischer Natur.

3.1 OSI-Modell aus EBICS-Sicht



3.1.1 TCP/IP als paketorientierte Übertragungsschicht

Als Transportprotokoll wird TCP/IP verwendet. Mit IP (Internet Protocol) werden die auszutauschenden Daten als Pakete übertragen. Diese Paketübertragung wird durch TCP (Transmission Control Protocol) als Übertragungskontrollprotokoll überwacht.

Der Kommunikationsaufbau erfolgt unter Verwendung einer URL (Uniform Resource Locator). Alternativ kann auch eine IP-Adresse des jeweiligen Kreditinstituts benutzt werden. Die URL oder die IP-Adresse ist zusammen mit der EBICS Host-ID für den Verbindungsaufbau zum Bankrechner erforderlich und wird dem Kunden bei Vertragsabschluss von dem Kreditinstitut mitgeteilt.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

3.1.2 TLS als Transportverschlüsselung

TLS wurde von der Arbeitsgruppe Transport Layer Security WG im IETF (Security-Area) entwickelt. Es stellt einen ursprünglich von Netscape (zunächst unter dem Namen SSL) entwickelten, offenen Standard zur gesicherten Übertragung paketorientierter Daten dar. Das Ziel von TLS ist es, die Datensicherheit auf Schichten oberhalb von TCP/IP zu gewährleisten. Das Protokoll ermöglicht Datenverschlüsselung, Echtheitsbestätigung von Servern und Nachrichtenintegrität für TCP/IP-Kommunikation.

Es vereinigt folgende Grundeigenschaften:

1. Die TLS-Verbindung ist vertraulich: Beim TLS-Handshake wird unter Verwendung asymmetrischer Verschlüsselung (bei EBICS: RSA) ein gemeinsamer, geheimer Schlüssel vereinbart, der in der weiteren TLS-Sitzung als symmetrischer Schlüssel (bei EBICS: AES) dient.
2. Die Integrität der TLS-Verbindung ist gesichert: Der Nachrichtentransport beinhaltet eine Nachrichtenintegritätsüberprüfung mittels sogenannter „Message Authentication Codes“ (MACs). Sichere Hash-Funktionen (bei EBICS: SHA-1) kommen für die MAC-Berechnungen zum Einsatz.
3. Die Identität des Kreditinstituts wird durch die Verwendung von Serverzertifikaten und elektronischer Signaturen beglaubigt; mittels dieser TLS-Serverauthentisierung sind die Nachrichten des Kreditinstituts authentisiert.
4. TLS enthält Schutzmechanismen gegen Man-in-the-middle-Attacken auf der TLS-Verbindungsstrecke zwischen Kunden- und Banksystem. Dazu nutzt es interne Zähler und „verteilte Geheimnisse“ („shared secrets“) und sichert zudem den Handshake mit signierten Zusammenfassungen der bis dahin ausgetauschten Daten gegen einen solchen Angriff ab.

Für die Übertragung der EBICS-Nachrichten zwischen Kundensystem und Banksystem wird eine TLS-Verbindung zwischen beiden Systemen aufgebaut.

Eingesetzt wird TLS (Details sind dem aktuellen EBICS-Anhang „Transport Layer Security“ zu entnehmen) mit X.509v3-Serverzertifikaten, d.h. der Server MUSS sich mittels Zertifikat authentisieren. Die Art des Zertifikats MUSS für den Schlüsseltauschalgorithmus der gewählten Schlüssel geeignet sein.

Zugunsten einer besseren Marktakzeptanz verzichtet EBICS in der Version H005 auf die TLS-Client-Authentisierung. Eine spätere Erweiterung um die Fähigkeit zur TLS-Client-Authentisierung (und der damit verbundenen Ausgabe von X.509v3-Clientzertifikaten für TLS an Kundensysteme) wird nicht ausgeschlossen.

Die Begriffe Client und Server werden in den nächsten beiden Unterkapiteln synonym zu TLS-Client bzw. TLS-Server verwendet. Das Kundensystem übernimmt dabei die Rolle des TLS-Clients, das Banksystem die des TLS-Servers.

3.1.2.1 Vorabverteilung und Prüfung der Vertrauensanker

Die Herausgeber der TLS-Serverzertifikate können sowohl öffentliche als auch nichtöffentliche (z.B. bankinterne) CAs sein. Es liegt in der Verantwortung des Kreditinstituts, eine öffentliche CA als

Herausgeber ihres TLS-Serverzertifikats nur dann zuzulassen, wenn diese CA eine hinreichende Prüfung der Identität der Zertifikatsinhaber garantiert.

Es ist Aufgabe des Kreditinstituts, Teilnehmern, die mit dem Kreditinstitut über EBICS kommunizieren möchten, ein vertrauenswürdiges CA-Zertifikat vom Zertifizierungpfad des TLS-Serverzertifikats zukommen zu lassen. Dafür gibt es die beiden folgenden Möglichkeiten:

- Auslieferung des CA-Zertifikats an den Kunden/ Teilnehmer:

Das gilt sowohl für öffentliche wie für nichtöffentliche CAs.

- Auslieferung eines CA-Zertifikats (Bridge-CA), mit dessen Hilfe eine Liste von vertrauenswürdigen nicht-öffentlichen CA-Zertifikaten unterzeichnet wurde:

Das CA-Zertifikat des Kreditinstituts ist Bestandteil dieser Liste. Der Vorteil einer solchen signierten Liste kommt für den Teilnehmer dann zum Tragen, wenn die Liste die CA-Zertifikate mehrerer Kreditinstitute enthält, welchen der Teilnehmer bankfachliche Aufträge über EBICS erteilen möchte.

Die Auslieferung der CA-Zertifikate an den Kunden/ Teilnehmer SOLL auf elektronischem Wege passieren. Denkbar sind Auslieferungen über E-Mail oder Auslieferungen als Bestandteil der Kundensoftware bzw. als Updates der Kundensoftware.

Zusätzlich zum CA-Zertifikat KANN das Kreditinstitut auch das TLS-Serverzertifikat selbst an den Kunden/ Teilnehmer ausliefern. Während der Serverauthentisierung beim TLS-Verbindungsaufbau kann in diesem Fall das TLS-Serverzertifikat als Vertrauensanker verwendet werden.

Neben der Auslieferung von Zertifikaten MUSS das Kreditinstitut sicherstellen, dass der Teilnehmer die erhaltenen Zertifikate über einen zweiten, unabhängigen Kommunikationsweg überprüfen kann. Das kann beispielsweise über die Veröffentlichung dieser Zertifikate und ihrer Fingerprints im Internet erfolgen.

Im Gegenzug ist der Teilnehmer dafür verantwortlich, den Abgleich der über unterschiedliche Kommunikationswege eingegangenen Zertifikate vorzunehmen.

3.1.2.2 Serverauthentisierung

Im Rahmen des TLS-Handshakes überträgt der Server sein Zertifikat an den Client.
Voraussetzung für den Aufbau einer TLS-Verbindung zwischen Client und Server ist die erfolgreiche Prüfung des Serverzertifikats durch den Client.

Dabei handelt es sich um ein X.509v3-Zertifikat. Die Überprüfung von X.509v3-Zertifikaten im Allgemeinen ist in RFC 5280 definiert, die Besonderheiten der Überprüfung der TLS-Serverzertifikate im Rahmen des HTTPS-Handshakes sind in RFC 2818 beschrieben.

Der Teilnehmer ist dafür verantwortlich, Kundensoftware einzusetzen, die die Überprüfung des TLS-Serverzertifikats gemäß dieser Spezifikationen durchführt. Ebenso ist er dafür verantwortlich,

Kundensoftware einzusetzen, die das vom Kreditinstitut vorab erhaltene CA-Zertifikat bzw. TLS-Serverzertifikat bei dieser Prüfung als Vertrauensanker einsetzt.

3.1.3 HTTP(S) als technisches Basisprotokoll

Das Hypertext Transfer Protocol (HTTP) ist ein zustandsloses Datenaustausch-Protokoll zur Übertragung von Daten. HTTP wird im „World Wide Web“ (WWW) hauptsächlich zur Übertragung von Webseiten verwendet.

Die Kombination von HTTP und TLS als Transportverschlüsselung wird auch als „HTTPS“ (HTTP Secure) bezeichnet. Hierfür ist der Port 443 (SSL) reserviert, welcher bei den meisten Firewall-Konfigurationen uneingeschränkt nutzbar ist.

Die Zustandslosigkeit von HTTP erzwingt für das EBICS-Protokoll die Verwendung eigener Sitzungsparameter, die mehrere Kommunikationsschritte logisch zu einer Transaktion zusammenfassen.

Die Kommunikation zwischen Kunde und Kreditinstitut erfolgt klassisch via Client/Server-Rollen. Das Kreditinstitut übernimmt wie bisher auch die (passive) Serverrolle und der Kunde die (aktive) Clientrolle. Bei diesem Kommunikationsschema sendet der Client mittels HTTP-Request eine Anfrage an den Server; dieser antwortet mit einer HTTP-Response. Der Request kann generell als GET- (zusätzliche Daten werden in die URL kodiert) oder POST-Anfrage (zusätzliche Daten werden an den HTTP-Header angehängt) gestellt werden; im EBICS-Kontext findet ausschließlich POST Verwendung.

Bei EBICS MUSS sowohl vom Client als auch vom Server HTTP 1.1 eingesetzt werden.

3.1.4 XML als Anwendungsprotokollsprache

Das Anwendungsprotokoll EBICS setzt auf dem technischen Basisprotokoll HTTP(S) auf. Als Protokollsprache auf Anwendungsebene wurde XML (Extensible Markup Language) gewählt. Für diese Entscheidung sprechen folgende Gründe:

1. XML verwendet lesbare Marken („tags“). Deren Namen/Attribute können derart gewählt werden, dass sich die Bedeutung der „tags“ auch ohne Dokumentation erschließen lässt.
2. Kostenlose XML-Parser sind für die gängigen Betriebssysteme und Programmiersprachen erhältlich.
3. XML-Nachrichten können einfach um zusätzliche Elemente und Attribute erweitert werden. Hierbei ist es für die Erhaltung der syntaktischen Korrektheit („well-formedness“) der Gesamtnachricht nicht erforderlich, Anpassungen an den bestehenden Nachrichtenabschnitten vorzunehmen.
4. Für die Validierung von XML-Nachrichten steht XML-Schema als standardisierte Definitionssprache zur Verfügung.

Als Zeichenkodierung innerhalb der EBICS-XML-Nachricht MUSS UTF-8 verwendet werden. UTF-8 wird von jedem XML-Parser unterstützt und kodiert abwärtskompatibel zu ASCII.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Die Syntax der XML-Nachrichten wird mit Hilfe sogenannter XSD-Dateien (XML Schema Definition) fixiert. Für EBICS sind die folgenden XSD-Dateien definiert worden und unter http://www.ebics.de (siehe “Spezifikation”) abrufbar:

- „ebics_request_H005.xsd“ enthält das XML-Schema für die Anfragen des Kundensystems
- „ebics_response_H005.xsd“ definiert das XML-Schema für die Antworten des Banksystems
- „ebics_orders_H005.xsd“ enthält auftragsspezifische Datenstrukturen
- „ebics_types_H005.xsd“ führt einfache EBICS-Typdeklarationen auf.

Neben diesen Hauptschemata sind an gleicher Stelle noch die folgenden spezifischen Varianten für Transaktionen zu finden, welche speziell das Schlüsselmanagement betreffen:

- „ebics_keymgmt_request_H005.xsd“ definiert das XML-Schema für Anfragen des Kundensystems im Rahmen des Schlüsselmanagements
- „ebics_keymgmt_response_H005.xsd“ enthält das XML-Schema für die Antworten des Banksystems im Rahmen des Schlüsselmanagements.
  Der target namespace ist als “urn:org:ebics:H005” definiert.Für die Übermittlung der EU in strukturierter Form wurde ein Schema „ebics_signature.xsd“ definiert, es hat die Schema Target Location http://www.ebics.de.org/S002 und ist abrufbar unter http://www.ebics.de (siehe “Spezifikation”):
- Dieses Schema wurde als eigenständig definiert, um auch außerhalb EBICS verwendet werden zu können. Es ist auch der Import des o.g. Namespaces zur Nutzung der EU in EBICS erforderlich. Es hat das Prefix „esig“.
- Von „ebics_signature“ aus wird auf Strukturen des W3C-Standards "XML-Signature" referenziert mit den Prefix „ds“ (siehe dazu Kapitel 3.8). Dieses Schema ist an gleicher Stelle unter dem Namen „xmldsig-core-schema.xsd“ hinterlegt.

Jede der vier XSD-Dateien mit der Endung „_request_H005“ oder „_response_H005“ definiert eine oder mehrere Arten von EBICS XML-Nachrichten, die jeweils ein anderes XML-Wurzelelement („root element“) mit einem eindeutigen Namen haben.

Für Standard-EBICS-Nachrichten definiert „ebics_request_H005.xsd“ das Wurzelelement `ebicsRequest` für Anfragen des Kundensystems und „ebics_response_H005.xsd“ definiert das Wurzelelement `ebicsResponse` für Antworten des Banksystems. Für Transaktionen des Schlüsselmanagements enthält „ebics_keymgmt_request_H005.xsd“ drei zusätzliche XML-Nachrichten für Anfragen des Kundensystems mit den Wurzelelementen `ebicsUnsecuredRequest`, `ebicsUnsignedRequest` und `ebicsNoPubKeyDigestsRequest`.
„ebics_keymgmt_response_H005.xsd“ definiert für das Schlüsselmanagement das Wurzelelement `ebicsKeyManagementResponse` für Antworten des Banksystems.

„ebics_H005.xsd“ inkludiert diese vier XML-Schema Dateien und enthält daher die Gesamtmenge aller Definitionen der EBICS-Schemaversion H005. Dies kann ebenfalls auf http://www.ebics.de (siehe “Spezifikation”) heruntergeladen werden. Sein target namespace ist ebenfalls “urn:org:ebics:H005”. Mit

Hilfe dieser Datei kann verifiziert werden, dass alle globalen Definitionen im EBICS Namensraum (Elemente und Typen) eindeutige Namen haben. Diese Eigenschaft des EBICS XML-Protokolls vereinfacht die Verarbeitung von EBICS XML-Nachrichten durch Standard-XML-Werkzeuge, weil bereits der Name des XML-Wurzelelements zusammen mit der Angabe des EBICS Namensraums ausreicht, um das zulässige Format für die komplette XML-Nachricht zu bestimmen. Ein Standard-XML-Parser kann z.B. bereits anhand dieses XML-Fragments erkennen, gegen welche Definition in den EBICS XSD-Dateien das gesamte XML-Dokument validiert werden muss:

```
<ebicsRequest xmlns="urn:org:ebics:H005" Version="H005">
```

Die Schema Location besteht aus einem Referenz-Paar, getrennt durch ein Leerzeichen. Die erste Referenz ist der Name des Namespaces, die zweite Referenz ist ein Hinweis, wo das entsprechende Schema zu diesem Namespace zu finden ist, z.B. der lokale Dateiname ebics_request_H005.xsd:

```
<ebicsRequest xmlns=" urn:org:ebics:H005" Version="H000"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation=""urn:org:ebics:H005 ebics_request_H005.xsd"">
```

Die Referenzierung von EBICS-XML-Elementen und -Attributen wird an folgendem Beispiel aus der XML-Schema-Datei „ebics_request_H005.xsd“ für die EBICS-Wurzelstruktur für Standardanfragen des Kundensystems („root element“ `ebicsRequest` und dessen Unterelemente) erläutert:

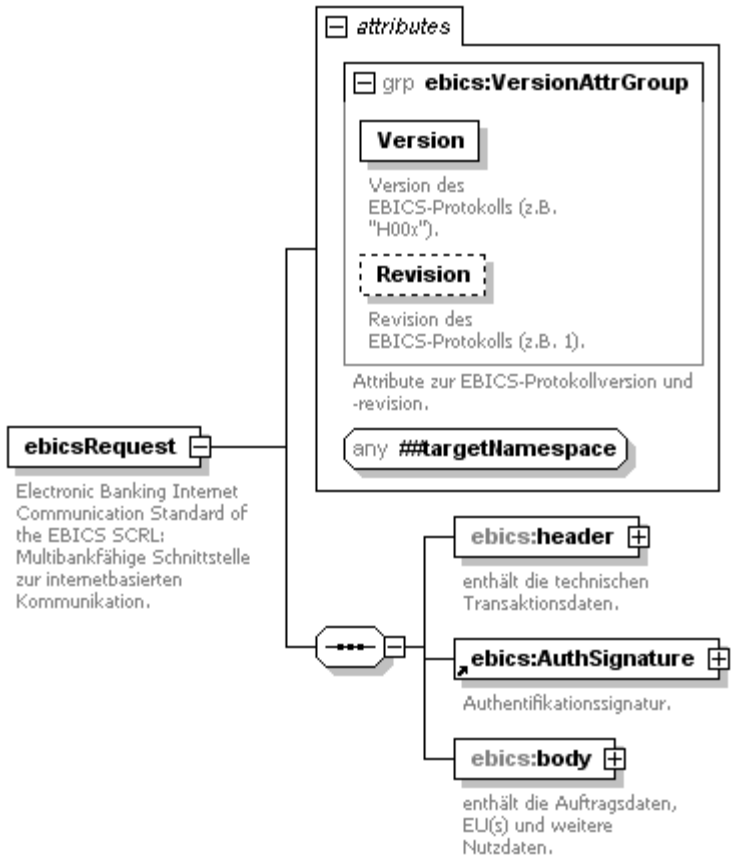


Abbildung 3: Wurzelstruktur des EBICS-Protokolls

Das XML-Wurzelelement für Standard-EBICS-Nachrichten mit Requests des Kundensystems heißt `ebicsRequest`. Es enthält einige Attribute mit fundamentalen Informationen, die zum Parsen der Nachricht als Ganzes benötigt werden (Attributgruppe `VersionAttrGroup`):

- `Version` für die EBICS-Protokollversion (z.B. „H005“)
- `Revision` für die EBICS-Protokollrevision: Dieses Attribut SOLLTE ebenfalls mitgeschickt werden, um mehrere (kompatible) Revisionen der gleichen Protokollversion fachlich unterscheiden zu können.

Die direkte Unterstruktur zu `ebicsRequest` bilden die folgenden Elemente:

- `header`: Das XML-Tag enthält technische Informationen (sog. „technische Steuerdaten“) in den Subtags:
  - `static` für diejenigen technischen Steuerdaten, die über die gesamte Transaktion hinweg konstant bleiben
    - `HostID` für die EBICS Host-ID zur Identifizierung des EBICS Bankrechners. Das Element `HostID` ist in allen EBICS Request-Nachrichten des Kundensystems enthalten (sowohl für Standard-Transaktionen als auch für systembedingte Transaktionen). Die EBICS Host-IDwird dem Kunden vom Kreditinstitut mitgeteilt.
  - `mutable` für die veränderlichen technischen Steuerdaten.

  Beide Subtags von `header` MÜSSEN in der obigen Reihenfolge erscheinen.
- `AuthSignature`: In diesem Element ist die Authentifikationssignatur gemäß dem Standard „XML-Signature“ untergebracht. Das XML-Tag MUSS in allen Nachrichten auftauchen mit Ausnahme der administrativen Auftragsarten INI, HIA und HPB (für diese administrativen Auftragsarten werden eigene XML-Schemata benutzt; siehe hierzu Kapitel 4.4).
- `body` enthält die eigentlichen Auftragsdaten, Unterschriften (EUs) und weitere Daten, die in direktem Bezug zum Auftrag stehen oder zu dessen Auswertung benötigt werden.

Die XML-Anfragen der Teilnehmer an die Kreditinstitute werden als EBICS-Requests bezeichnet, die XML-Antworten der Kreditinstitute als EBICS-Responses. Das HTTP-Binding eines EBICS-Requests und der zugehörigen EBICS-Response ist wie folgt: Der EBICS-Request wird in einen HTTP-POST-Request eingebettet, die EBICS-Response in die entsprechende HTTP-Response.

Ein typischer HTTP-Request sieht in EBICS wie folgt aus (Ausschnitt):

```
POST /ebics HTTP/1.1
Host: www.die-bank.de
Content-Type: text/xml; charset=UTF-8
Content-Length: 800

<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest xmlns=" urn:org:ebics:H005"
       xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
```

```
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="urn:org:ebics:H005 ebics_request_H005.xsd "
      Version="H005" Revision="1">
  <header authenticate="true">
    <static>
      <HostID>EBIXHOST</HostID>
      …
    </static>
    <mutable>
      …
    </mutable>
  </header>
  <AuthSignature>
    …
  </AuthSignature>
  <body>
    …
  </body>
</ebicsRequest>
```

Eine entsprechende mögliche HTTP-Response beschreibt der folgende Ausschnitt:

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=UTF-8
Content-Length: 1538

<?xml version="1.0" encoding="UTF-8"?>
<ebicsResponse xmlns="urn:org:ebics:H005"
       xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation=" urn:org:ebics:H005 ebics_response_H005.xsd"
       Version="H005" Revision="1">
  <header authenticate="true">
    <static>
      …
    </static>
    <mutable>
      …
    </mutable>
  </header>
  <AuthSignature>
    …
  </AuthSignature>
  <body>
    …
  </body>
</ebicsResponse>
```

Weitere Details zu der Struktur von EBICS-Protokollnachrichten und Transaktionsdetails sind in Kapitel 5 aufgeführt. Die Formate der XML-Nachrichten für Standardantworten des Banksystems und die systembedingten Nachrichten des Keymanagements verwenden andere XML-Wurzelelemente, die

weitgehend analog zum Standard-Request aufgebaut sind. Die kompletten XML-Schemata sind in der separaten HTML-Schemadokumentation zu finden.

Das Schema ebics_hev.xsd zur Abfrage der von der Bank unterstützten EBICS-Versionen steht unter http://www.ebics.org (siehe „technical-information“) zur Verfügung (siehe auch Kapitel 9.5)

3.2 Komprimierung, Verschlüsselung und Kodierung der Auftragsdaten

EBICS behandelt bankfachliche Nutzdaten transparent. Das bedeutet: Auftragsdaten werden unabhängig von der spezifischen Datenstruktur unterschiedlicher administrativer Auftragsarten als binärer Block behandelt und in die XML-Struktur eingebettet. Hierbei MÜSSEN diese Auftragsdaten vor dem Versand stets zunächst ZIP-komprimiert, dann hybrid verschlüsselt (gemäß Verfahren E002) und das Ergebnis schließlich base64-kodiert werden. Ausnahmen: Bei den administrativen Schlüsselmanagement-Auftragsarten INI, HIA und H3K erfolgt der Versand unverschlüsselt (siehe hierzu Kapitel 4.4.1.2.5.1 für INI, HIA und H3K). Die Standards, die den in EBICS gültigen ZIP-Algorithmus sowie das base64-Format definieren, sind im Anhang (Kapitel 15) spezifiziert.

Die derart erzeugte Datenrepräsentation ist dann ohne weitere Zeichenkonvertierung oder Strukturierung z.B. in das XML-Element `ebicsRequest/body/DataTransfer/OrderData` einzustellen.

Die ZIP-Komprimierung dient zur Verringerung des Datenvolumens, das übertragen werden muss.

Bei der hybriden Verschlüsselung werden die eigentlichen Daten symmetrisch verschlüsselt. Der hierzu verwendete Transaktionsschlüssel wird wiederum asymmetrisch verschlüsselt und z.B. in Form des XML-Elements `ebicsRequest/body/DataTransfer/DataEncryptionInfo/TransactionKey` beigefügt (siehe hierzu Kapitel 6.2).

Die Verschlüsselung der Auftragsdaten erfolgt zusätzlich zur TLS-Transportverschlüsselung. Dadurch ist gewährleistet, dass die Auftragsdaten sowohl auf dem Weg über öffentliche Netze (ergänzend zu TLS) als auch jenseits der TLS-geschützten Verbindungsstrecke vor unautorisiertem Lesezugriff geschützt sind.

base64 verwendet zur Kodierung des Binärstroms nur druckbare ASCII-Zeichen und stellt somit sicher, dass die Auftragsdaten unverfälscht ihr Ziel erreichen und dort authentisch ausgewertet werden können.

3.3 Segmentierung der Auftragsdaten

Unter Segmentierung wird die Aufteilung von großen Datenvolumina in kleinere, einzelne Übertragungssegmente verstanden.

Die Segmentierung der Auftragsdaten findet bei EBICS auf Anwendungsprotokollebene statt. Auftragsdaten dürfen nur dann in einer einzelnen EBICS-Nachricht übermittelt werden, wenn sie in komprimierter, verschlüsselter und base64-kodierter Form die vorgegebene feste Größe von 1 MB nicht

überschreiten. Das gilt für Sende- und Abholaufträge gleichermaßen. Wird die Grenze von 1 MB überschritten, MÜSSEN die komprimierten, verschlüsselten und base64-kodierten Auftragsdaten in Segmente aufgeteilt werden, deren Größe jeweils die festgelegte Segmentgröße von 1 MB nicht überschreitet. Die Segmente werden dann in fortlaufender Reihenfolge in einzelnen EBICS-Nachrichten übertragen.

Weitere Details zur Segmentierung der Auftragsdaten sind Kapitel 7 zu entnehmen.

3.4 Wiederaufsetzen der Übertragung von Aufträgen (Recovery) [optional]

Recovery ermöglicht es, die Übertragung eines Auftrags nach einem Transport- oder Verarbeitungsfehler fortzusetzen, ohne alle bereits erfolgreich versandten Auftragsdatensegmente erneut übertragen zu müssen.

Basierend auf dem Ablauf der Übertragung von Aufträgen in mehreren fest vorgegebenen Schritten definiert EBICS ein Recovery-Verfahren auf der XML-Anwendungsprotokollebene. Es ist ein optimistisches Recovery-Verfahren, das auf einen separaten Synchronisationsschritt verzichtet, da das Kundensystem in der Regel weiss, mit welchem Schritt die Übertragung des betroffenen Auftrags fortzusetzen ist.

Details zum Recovery sind Kapitel 5.4 zu entnehmen.

3.5 Elektronische Unterschrift (EU) der Auftragsdaten

Die „**E**lektronische **U**nterschrift“ (EU) der Auftragsdaten stellt die Authentizität der Auftragsdaten sicher, jenseits der TLS-Übertragungsstrecke und unabhängig von der Komprimierung, Verschlüsselung, Kodierung sowie Segmentierung der Auftragsdaten.

Bei Sendeaufträgen handelt es sich um die willentliche Signatur eines Teilnehmers, welche die inhaltliche Zustimmung (content commitment) des Teilnehmers dokumentiert, bei Abholaufträgen ist es die Signatur des Kreditinstituts.

EUs werden gemäß Anhang (Kapitel 14) erzeugt, in der EBICS-Version „H005“ muss mindestens die EU-Version „A005“ unterstützt werden (siehe Anhang (Kapitel 11.2)).

3.5.1 EU der Teilnehmer

Die Auftragsdaten der Sendeaufträge MÜSSEN vor dem Versand signiert, d.h. mit mindestens einer EU versehen werden. Ausnahmen bilden die administrativen Schlüsselmanagement-Auftragsarten INI und HIA, die nicht bankfachlich signiert werden.

Aus einer EU eines Teilnehmers kann das Banksystem mit Hilfe des öffentlichen Signaturschlüssels des Unterzeichners gemäß dem verwendeten und von der Bank bzgl. EBICS unterstützten Signaturverfahren den Hashwert der signierten Auftragsdaten extrahieren.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Für die EUs von Teilnehmern sind die folgenden **Unterschriftsklassen** definiert, die hier in der Reihenfolge abfallender Stärke aufgelistet werden („E“ ist die stärkste, „T“ die schwächste Unterschriftsklasse):

- **Einzelunterschrift (Typ „E”)**
- **Erstunterschrift (Typ „A“)**.
- **Zweitunterschrift (Typ „B“)**
- **Transportunterschrift (Typ „T“)**.

Durch die Zuordnung von Unterschriftsklassen zu Teilnehmern wird innerhalb des Kreditinstituts ein Berechtigungsmodell für EUs definiert. Beispielsweise haben Teilnehmer mit der Unterschriftsklasse A die Berechtigung zur Erstunterschrift von Aufträgen. Institutsindividuell KÖNNEN detailliertere Berechtigungsmodelle definiert werden, bei denen die Unterschriftsberechtigung eines Teilnehmers bezüglich der BTF-Parameterwerte und/ oder des Betragslimits und/ oder des benutzten Kontos parametrisierbar ist.

Die Unterschriftsklasse einer gegebenen EU eines Teilnehmers ist die stärkste Klasse, die dieser EU im Berechtigungsmodell des entsprechenden Kreditinstituts zugeordnet werden kann.

Unterschriftsberechtigungen des Typs „T“ werden Teilnehmern pauschal zugewiesen oder (in detaillierteren Berechtigungsmodellen) Teilnehmern in Kombination mit bestimmten BTF-Parameterwerten. Sie sind jedoch nicht abhängig von Konten oder Betragslimits. Transportunterschriften werden nicht zur bankfachlichen Autorisation von Aufträgen verwendet, sondern lediglich zu deren (autorisierten) Einreichung an das Banksystem.

Als **bankfachliche EUs** bezeichnet man EUs vom Typ „E“, „A“, oder „B“. Bankfachliche EUs dienen der Autorisierung von Aufträgen. Aufträge können mehrere bankfachliche EUs benötigen, die dann von unterschiedlichen Teilnehmern geleistet werden MÜSSEN. Die Unterzeichner eines Auftrags können zudem Teilnehmer unterschiedlicher Kunden sein.

Für jede unterstützte BTF-Parameterwertkombination wird zwischen Kreditinstitut und Kunde eine **Mindestanzahl erforderlicher bankfachlicher EUs** vereinbart.

Details zum Format der EU und ihrer Anwendung zur Auftragsautorisierung werden im Anhang (Kapitel 11.2) erläutert.

3.5.2 EU der Kreditinstitute [vorgesehen]

Die EU der Kreditinstitute ist eine *vorgesehene* Funktionalität von EBICS. Der Grund für den Verzicht ist die fehlende abschließende rechtliche Betrachtung dieser EU.

Sowohl im vorliegenden Feinkonzept als auch in den XML-Schemadateien von EBICS wurden Vorbereitungen getroffen, die die Umsetzung folgender Festlegungen in zukünftigen EBICS-Versionen erleichtern:

- verpflichtende EU des Hashwertes und der Displaydatei des zu unterzeichnenden Auftrags im Falle der administrativen Auftragsart HVD
  Siehe Kapitel 8.3.2.2.
- Vordefiniert im Schema, jedoch noch nicht verwendbar: EU der Abholdaten bei Abholaufträgen Schema, jedoch noch nicht verwendbar:Abholung des öffentlichen bankfachlichen Schlüssels des Kreditinstituts über die Auftragsart HPB
  Siehe Kapitel 4.4.2.2
- Überprüfung des Hashwerts des öffentlichen bankfachlichen Schlüssels des Kreditinstituts im Rahmen der Transaktionsinitialisierung.
  Bestandteil jeder Transaktionsinitialisierung ist die Überprüfung der öffentlichen Schlüssel des Kreditinstituts, über die der Teilnehmer verfügt. Ausgenommen sind die administrativen Auftragsarten des Schlüsselmanagement INI, HIA, HPB sowie die administrative Auftragsart HEV zur Abfrage der bankseitig unterstützen EBICS-Versionen.
  Siehe Kapitel 4.6.2 und Kapitel 5.5.1.2.1.
- Binärformat für die EU der Kreditinstitute analog zum Binärformat der Teilnehmer-EU
  Siehe Anhang (Kapitel 11.2.2).

3.5.3 Darstellung der EUs in EBICS-Nachrichten

Die EUs eines Auftrags werden mit Hilfe des XML-Elements`UserSignatureData` (für die Teilnehmer-EU, das in der Schemadatei„ebics_signature.xsd“ definiert ist) dargestellt. Die Struktur `BankSignatureData` (für die bankseitige EU) wird noch nicht verwendet (geplante Funktionalität). Sie substituieren jeweils das abstrakte Element `EBICSSignatureData` . Abbildung 4 enthält die grafische Darstellung von `EBICSSignatureData`: A005/A006 sindim Element `OrderSignatureData` enthalten. EUs werden gemäß Anhang (Kapitel 14) gebildet.
In der strukturierten Signaturinformation für Signaturverfahren ab A005/A006 ist die Kunden-ID bereits im Element `PartnerID` enthalten. Die Angabe einer abweichenden Kunden-ID für die kundenübergreifende EU ist ausschließlich mit der Auftragsart HVE über spezielle Auftragsparameter möglich.

Die bankfachliche EU des Kreditinstituts wird analog zur bekannten bankfachlichen Teilnehmer-EU gebildet (siehe Anhang (Kapitel 11.2.2) im Vergleich zu Anhang (Kapitel 14)), wobei in diesem Fall das Attribut `PartnerID` entfällt.

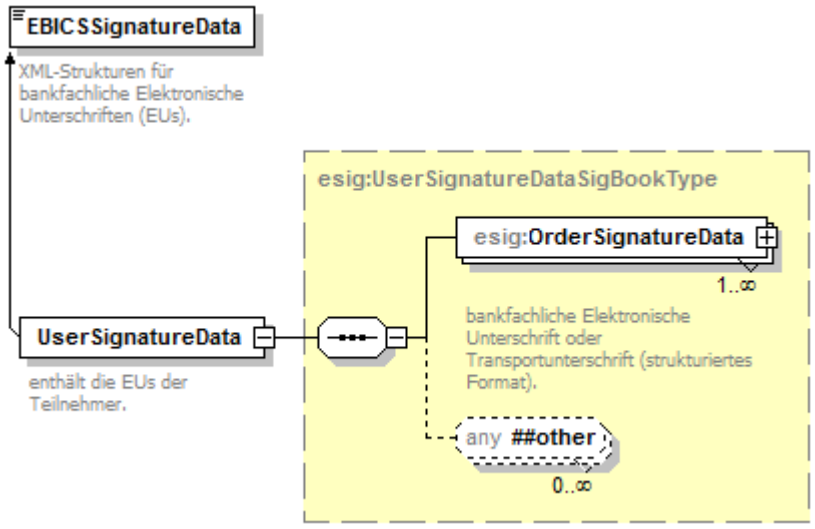


Abbildung 4: XML-Strukturen UserSignatureData für die EUs eines Auftrags (in strukturierter Form)

Um die EUs eines Auftrags in EBICS-Nachrichten einzubetten, sind die folgenden Schritte notwendig:

- Erstellung eines Instanzdokuments zu ebics_orders_H005.xsd bzw. ebics_signature.xsd, welches nur aus dem Element `BankSignatureData` (Bank-EU) bzw. `UserSignatureData` (Teilnehmer-EU) besteht.
- ZIP-Komprimierung, Verschlüsselung, base64-Kodierung des Instanzdokuments.
  Die Verschlüsselung erfolgt mit dem Transaktionsschlüssel `TransactionKey` aus dem XML-Zweig `ebicsRequest/body/DataTransfer/DataEncryptionInfo` (siehe Kapitel 6.2).
- Das Resultat wird in das Element `SignatureData` im Zweig `DataTransfer` des EBICS-Rumpfs eingestellt (siehe Kapitel 3.1.4).

3.6 Vorabprüfung [optional]

Der Teilnehmer KANN bei Upload-Aufträgen im ersten Transaktionsschritt Informationen versenden, die das Banksystem – sofern es diese Funktionalität unterstützt – zur Vorabprüfung des Auftrags nutzen KANN. Die Vorabprüfung kann eine oder mehrere der folgenden Prüfungen umfassen: Kontoberechtigungsprüfung, Limitprüfung, EU-Verifikation. Treten bei der Vorabprüfung (fachliche) Fehler auf, so ist die Fortsetzung der Übertragung des Auftrags sinnlos, zumal der Auftrag nicht durchgeführt werden kann.

Ob ein Kreditinstitut die Vorabprüfung generell unterstützt, können Teilnehmer über die Bankparameter-abfrage (administrative Auftragsart HPD, zurückgelieferte XML-Struktur `HPDResponseOrderData`, Attribut `ProtocolParams/PreValidation@supported`) erfahren. Mitgelieferte Parameter für Vorabprüfungen, die ein Kreditinstitut nicht unterstützt, werden von dem Kreditinstitut ignoriert.

Mehr zur administrativen Auftragsart HPD ist in Kapitel 9.2 zu finden. Für Details zur Vorabprüfung siehe Kapitel 5.3.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

3.7 Technische Teilnehmer

EBICS-Kundensysteme können selbst wieder als Client-Server-Systeme, sogenannte MultiUser-Systeme, realisiert werden. In diesem Fall übernimmt der Server die Rolle des EBICS-Clients in der Kommunikation mit dem Banksystem und ist als solcher verantwortlich für die Übermittlung von Aufträgen gemäß der EBICS Spezifikation.

Dieser kundenseitige Server stellt sich gegenüber dem Banksystem als „technischer Teilnehmer“ dar, der im Wesentlichen wie ein (menschlicher) Teilnehmer auf dem Banksystem administriert wird.

Die EBICS-Requests eines technischen Teilnehmers und eines menschlichen Teilnehmers unterscheiden sich nur dadurch, dass der technische Teilnehmer in allen EBICS-Requests das Feld SystemID mit seiner Teilnehmerkennung belegt und die Authentifikationssignatur für den EBICS-Request erstellt.

EBICS-Responses für den technischen Teilnehmer werden immer mit dem öffentlichen Verschlüsselungsschlüssel des technischen Teilnehmers verschlüsselt.

Für den technischen Teilnehmer gilt folgendes:

- Im EBICS-Request wird grundsätzlich (zusätzlich zu den Feldern PartnerID und UserID) das Feld SystemID mit der Teilnehmerkennung des technischen Teilnehmers versorgt. Am Vorhandensein des Feldes SystemID erkennt das Banksystem, dass der Request von einem technischen Teilnehmer versendet wurde.

- der technische Teilnehmer leistet die Authentifikationssignatur für den EBICS-Request (Ausnahme sind nur die Auftragsarten, die keine Authentifikationssignatur benötigen)

- der technische Teilnehmer kann alle EBICS-Requests für den im Feld UserID genannten Teilnehmer durchführen.

- der technische Teilnehmer kann keine bankfachliche Unterschrift leisten.

- der technische Teilnehmer kann Dateien mit einer eigenen Transportunterschrift einreichen (D-Datei oder Einreichung in die VEU (EDS)).

- der technische Teilnehmer kann Dateien mit den bankfachlichen Unterschriften menschlicher Teilnehmer einreichen. In diesem Fall muss der technische Teilnehmer keine Transportunterschrift leisten.

Für die Prüfungen auf dem Banksystem gilt folgendes:

- Anhand der Belegung des Feldes SystemID wird die Prüfung der Authentifikationssignatur des vom technischen Teilnehmer erstellten EBICS-Requests vorgenommen.

- Zur Prüfung der Auftragsberechtigung wird nur der Inhalt der Felder PartnerID und UserID herangezogen. Der Inhalt des Feldes SystemID wird hier nicht berücksichtigt.

  Nur wenn der technische Teilnehmer EBICS-Requests im eigenen Namen (im Feld UserID steht

die Teilnehmerkennung des technischen Teilnehmers) durchführt, benötigt er auch die entsprechende Auftragsberechtigung im Banksystem.

- Eine Kontoprüfung erfolgt für technische Teilnehmer nicht.

- Die Prüfung der elektronischen Unterschrift erfolgt, wie sonst auch, unabhängig von der Belegung der Felder SystemID und UserID.

3.8 Authentifikationssignatur

Die Authentifikation des Teilnehmers bzw. des Kundensystems und des Kreditinstituts in jedem Transaktionsschritt sind notwendig, um den bankseitigen Verbrauch von Ressourcen durch Unberechtigte und die unautorisierte Zustandsänderung von Aufträgen oder Daten zu unterbinden.

Die Authentifikationssignatur stellt als Haupt-XML-Zweig zwischen den EBICS-Kopf- und -Rumpfdaten einen integralen Bestandteil des EBICS-Protokolls dar. Sie wird gemäß dem XML-Signature-Standard erzeugt und hat mehrere Aufgaben zu erfüllen:

1. Identifikation und Authentisierung des (technischen) Teilnehmers: Das Banksystem MUSS sich mit Hilfe der Authentifikationsignatur von der Korrektheit der (technischen) Teilnehmerkennung ihm bekannter Teilnehmer bzw. Kundensysteme überzeugen.
2. Integrität der Steuerdaten/EU(s): Änderungen – auch jenseits der TLS-Übertragungsstrecke – an der/den EU(s) sowie den technischen und auftragsbezogenen Daten (mit Ausnahme der Auftragsdaten, welche nicht von der Authentifikationssignatur, sondern ausschließlich von der bankfachlichen Unterschrift erfasst werden) werden mit Hilfe der Authentifikationssignatur aufgedeckt, solange die XML-Struktur der signierten Daten unverändert bleibt.

Die Authentifikationssignatur wird (im Gegensatz zur EU, die die Auftragsdaten signiert) über die Steuerdaten und über die EU(s) gebildet und MUSS bei jedem Transaktionsschritt jeder administrativen Auftragsart (mit Ausnahme der systembedingten Auftragsarten INI, HIA, H3K und HPB, siehe hierzu Kapitel 4.4) sowohl vom Kunden- als auch vom Banksystem geleistet werden. Die Authentifikation der bankfachlichen EU(s) koppelt die mit dieser/diesen EU(s) signierten Auftragsdaten an die restlichen Protokollinformationen und verhindert so den unautorisierten Austausch von Aufträgen mitsamt ihren EU(s) innerhalb einer EBICS-Transaktion.

Details zu den verwendeten Algorithmen für die Authentifikationssignatur sind im Anhang (Kapitel 11.1) zu finden. Dort ist auch festgelegt, dass ein Kanonisierungsverfahren (C14N) die Daten vor der Signaturerzeugung und -verifikation in eine standardisierte Form überführt.
Zur Signaturbildung MÜSSEN – zusätzlich zu den XML-Signature-eigenen Strukturen – genau diejenigen Elemente (und deren Substrukturen) in die Authentifikationssignatur eingehen, die den Attributmarker `@authenticate="true"` besitzen. Die Vorkommen dieser Attributmarker sind in den XML-Schemata festgelegt.

Die Authentifikationssignatur jeder EBICS-Nachricht MUSS vom jeweiligen Empfänger der Nachricht überprüft werden.

Kann die Authentifikationssignatur eines EBICS-Requests nicht erfolgreich verifiziert werden, so kann das Banksystem nicht davon ausgehen, dass der EBICS-Request auch tatsächlich vom entsprechenden (technischen) Teilnehmer stammt.

Der Sender des EBICS-Request erhält in diesem Fall einen entsprechenden Fehlercode (EBICS_AUTHENTICATION_FAILED). Weitere Einzelheiten sind jeweils in den Kapiteln 5.5.1.2.1 und 5.5.1.2.2 enthalten, jeweils zu dem Unterpunkt „Prüfung der Authentizität der EBICS-Requests“.

Kann umgekehrt die Authentifikationssignatur des EBICS-Response nicht erfolgreich überprüft werden, so kann das Kundensystem nicht davon ausgehen, dass der EBICS-Response vom erwarteten Banksystem stammt. In diesem Fall MUSS die betroffene EBICS-Transaktion durch das Kundensystem abgebrochen werden.

Auf Kundenseite MÜSSEN dann die Einstellungen der Kundensoftware überprüft werden, die beim Verbindungsaufbau zum Banksystem unter Einhaltung der Anforderungen aus Kapitel 3.1.2.2 verwendet werden. Des Weiteren MUSS die Aktualität der gespeicherten öffentlichen Schlüssel des Kreditinstituts überprüft werden (siehe dazu auch Kapitel 5.5.1.2.1, Unterpunkt „Überprüfung der Hashwerte der Bankschlüssel“).

3.9 X.509-Daten

Das EBICS-Protokoll greift in der Version H005 für die kryptografischen Algorithmen (d.h. für Authentifikation, Verschlüsselung, Signatur) standardmäßig auf öffentliche Schlüssel im X.509-Format zurück, die im Rahmen der Teilnehmerinitialisierung zwischen Teilnehmer und Kreditinstitut ausgetauscht wurden (zum Schlüsselmanagement siehe Kapitel 4).
Es wird direkt auf die Strukturen des W3C-Standards referenziert.
Die Elementgruppe dazu befindet sich in den EBICS-XML-Schemata „ebics_request_H005.xsd“ und „ebics_keymgmt_request_H005.xsd“ z.B. im Pfad `ebicsRequest/body/X509Data`. Die Typdefinition benutzt die Vorgabe aus XML-Signature (siehe Abbildung 5).

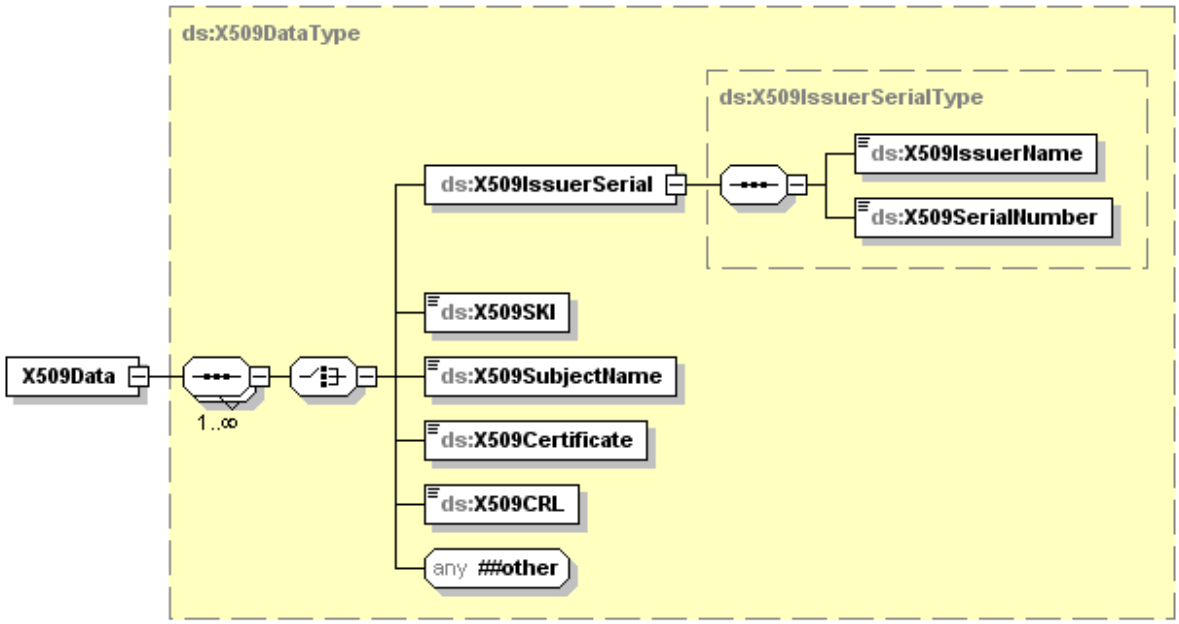


Abbildung 5: X509DataType

In der EBICS-Schema-Version H005 sind in den administrativen Auftragsarten des Schlüsselmanagements zusammen mit öffentlichen Schlüsseln auch X.509-Daten erforderlich.

3.10 Unterstützte administrative Auftragsarten

Es werden alle standardisierten, systembedingten und reservierten administrativen Auftragsarten gemäß vollständiger Liste (siehe Anhang Kapitel 13) durch transparente Einbettung der Auftragsdaten in die XML-Struktur unterstützt.

Administrative Auftragsarten werden in den Kapiteln 8 und 9 (HIA/H3K: Kapitel 4.4.1, HCA/HCS: Kapitel 4.6.1) ausführlich erläutert. Eine vollständige Liste befindet sich im Anhang (in Kapitel 13).Informationen zur bankseitigen Unterstützung (verpflichtend, optional oder konditional) siehe ebenfalls im Anhang (Kapitel 13).

3.11 Auftragsparameter

Für die Übertragung von Auftragsparametern, die nicht Bestandteil der Auftragsdaten sind, wurde in die festen Steuerdaten (unter `ebicsRequest/header/static/OrderDetails`) das Element `OrderParams` integriert. Je nach Auftragsart hat dieses abstrakte Element eine spezifische, konkrete Ausprägung (siehe auch

Abbildung 7).

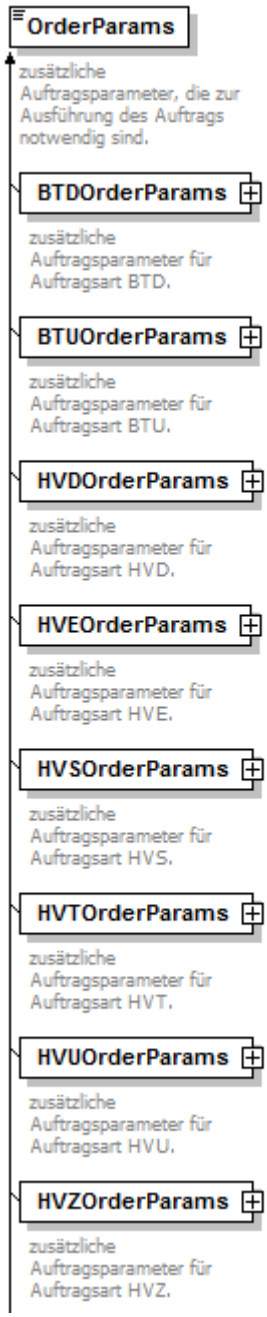


Abbildung 6: OrderParams

- `BTDOrderParams` für das Abholen von fachlichen Geschäftsvorfällen (BTF)
- `BTUOrderParams` für das Senden von fachlichen Geschäftsvorfällen (BTF)
- `HVDOrderParams` bei Auftragsparametern für die Auftragsart HVD
- `HVEOrderParams` bei Auftragsparametern für die Auftragsart HVE
- `HVSOrderParams` bei Auftragsparametern für die Auftragsart HVS
- `HVTOrderParams` bei Auftragsparametern für die Auftragsart HVT

- `HVUOrderParams` bei Auftragsparametern für die Auftragsart HVU

Die Strukturen für die Auftragsparameter zu BTU und BTD werden in Kapitel 5 erläutert.
Die Strukturen für die Auftragsparameter zu den administrativen Auftragsarten HVD, HVE, HVS, HVT und HVU werden im Kapitel 8.3 näher erläutert.



Abbildung 7: Mögliche Ausprägungen für die Auftragsparameter (OrderParams)

3.12 Ablauf der EBICS-Transaktionen

Dieses Kapitel enthält eine vereinfachte Beschreibung des Protokollablaufs für die Übermittlung eines DFÜ-Auftrags über EBICS, die den Festlegungen der vorhergehenden Kapitel Rechnung trägt.

Die Übermittlung erfolgt in einer EBICS-Transaktion, die aus mehreren Transaktionsschritten bestehen kann. Ein Transaktionsschritt ist ein Paar bestehend aus einem EBICS-Request und dem dazu gehörigen EBICS-Response.

Der erste Transaktionsschritt ist der Transaktionsinitialisierungsschritt. In diesem Schritt werden teilnehmerbezogene Berechtigungsprüfungen durchgeführt, wie etwa die Prüfung der Berechtigung für spezielle BTF-Parameterwerte. Die erfolgreiche Berechtigungsprüfung ist Voraussetzung für die Fortsetzung der Transaktion. Des Weiteren werden in diesem Transaktionsschritt die EUs des Auftrags übermittelt: bei Sendeaufträgen werden im EBICS-Request die EUs der Unterzeichner übertragen, bei Abholaufträgen eventuell die bankfachliche EU des Kreditinstituts im EBICS-Response.

Auf die Transaktionsinitialisierung folgen in der Regel mehrere Transaktionsschritte, in denen die Segmente der Auftragsdaten sequentiell und in fortlaufender Reihenfolge übertragen werden.

Sendeaufträge, die über EBICS an das Banksystem übermittelt werden, können auf zwei unterschiedliche Arten autorisiert werden:

Variante 1: Autorisierung mit Hilfe einer oder mehrerer bankfachlicher EUs

Die bankfachlichen EUs zu einer Auftragsdatei müssen von unterschiedlichen Teilnehmern geleistet werden, wobei im Falle der VEU (EDS) diese Teilnehmer im Sonderfall auch zu unterschiedlichen Kunden gehören können (kundenübergreifende Unterschrift). Die EUs können auf drei verschiedene Arten zum Kreditinstitut übertragen werden, dabei stammen alle mit einer einzelnen EBICS-Transaktion eingereichten EUs jeweils vom selben Kunden.

1. Einreichung der Nutzdaten des Auftrags zusammen einer oder mehreren EUs durch einen Sendeauftrag mit vorhandenem Signatur-Flag
   Alle auf diese Weise eingereichten EUs stammen vom Kunden des Einreichers des Auftrags.
   Sollten die übermittelten EUs für die bankfachliche Freigabe nicht ausreichen,
   wird der Auftrag
   a) an die VEU (EDS) übergeben, wenn das optionale Attribut des Signatur-Flags @requestEDS vorhanden ist, was bedeutet, dass der Kunde VEUs (EDSs) via HVE nachreichen kann,
   oder
   b) zurückgewiesen, wenn das optionale Attribut des Signatur-Flags @requestEDS nicht vorhanden ist.
2. Einreichung noch ausstehender bankfachlicher EUs mit Hilfe der administrativen Auftragsart **HVE**
   Werden EUs per HVE-Transaktion eingereicht, so müssen diese EUs jeweils vom Kunden des Einreichers dieser HVE-Transaktion stammen:
   HVE ermöglicht den Sonderfall der kundenübergreifenden Unterschrift, weil die über HVE

eingereichten EUs nicht notwendigerweise vom Kunden des Einreichers des Auftrags stammen müssen.

Variante 2: Autorisierung mittels eines handschriftlich unterschriebenen Begleitzettels

Das Signatur-Flag ist bei der Übertragung der Auftragsdatei nicht gesetzt.
Im Rahmen der EBICS-Transaktion wird eine EU der Unterschriftsklasse „T“ übertragen zusammen mit den Nutzdaten des Auftrags. Der Auftrag wird nicht an die VEU weitergereicht, sondern direkt an die bankspezifische Weiterverarbeitung.
Verfügt der einreichende Teilnehmer im Banksystem über die Berechtigung zur Durchführung einer bankfachlichen EU, so werden diese Unterschriften bei der Einreichung ohne Signatur-Flag grundsätzlich nur als Transportunterschrift gewertet. Es darf auch in diesem Fall keine Einleitung in die VEU (EDS) erfolgen.
Die Bedeutung und die Art der Verwendung des Signatur-Flags sind im Kapitel 3.14) beschrieben.

Die Übermittlung eines Sendeauftrags mit einem (komprimierten, verschlüsselten und base64-kodierten) Nutzdatenvolumen von 3MB wird beispielhaft mit Hilfe des Sequenzdiagramms aus Abbildung 8 dargestellt. Die EBICS-Transaktion zu einem Sendeauftrag wird beendet, sobald das letzte Nutzdatensegment erfolgreich an das Kreditinstitut übertragen wurde.

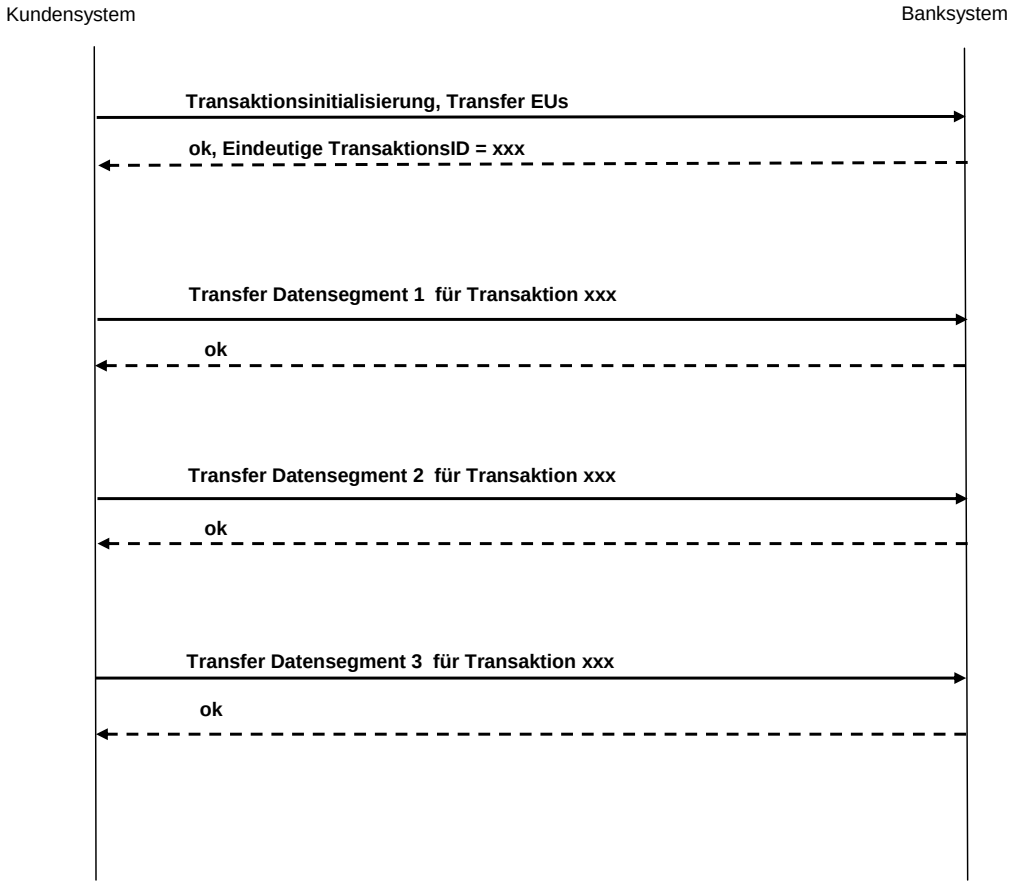


Abbildung 8: Beispielhafter Ablauf einer EBICS-Transaktion für einen Sendeauftrag

Die Übermittlung eines Abholauftrags mit einem (komprimierten, verschlüsselten und base64-kodierten) Nutzdatenvolumen von 3 MB wird beispielhaft anhand des Sequenzdiagramms in Abbildung 9 verdeutlicht.
Bei Abholaufträgen wird der Empfang der Abholdaten mit einem Quittierungsschritt bestätigt. Danach wird die EBICS-Transaktion zum Abholauftrag beendet.

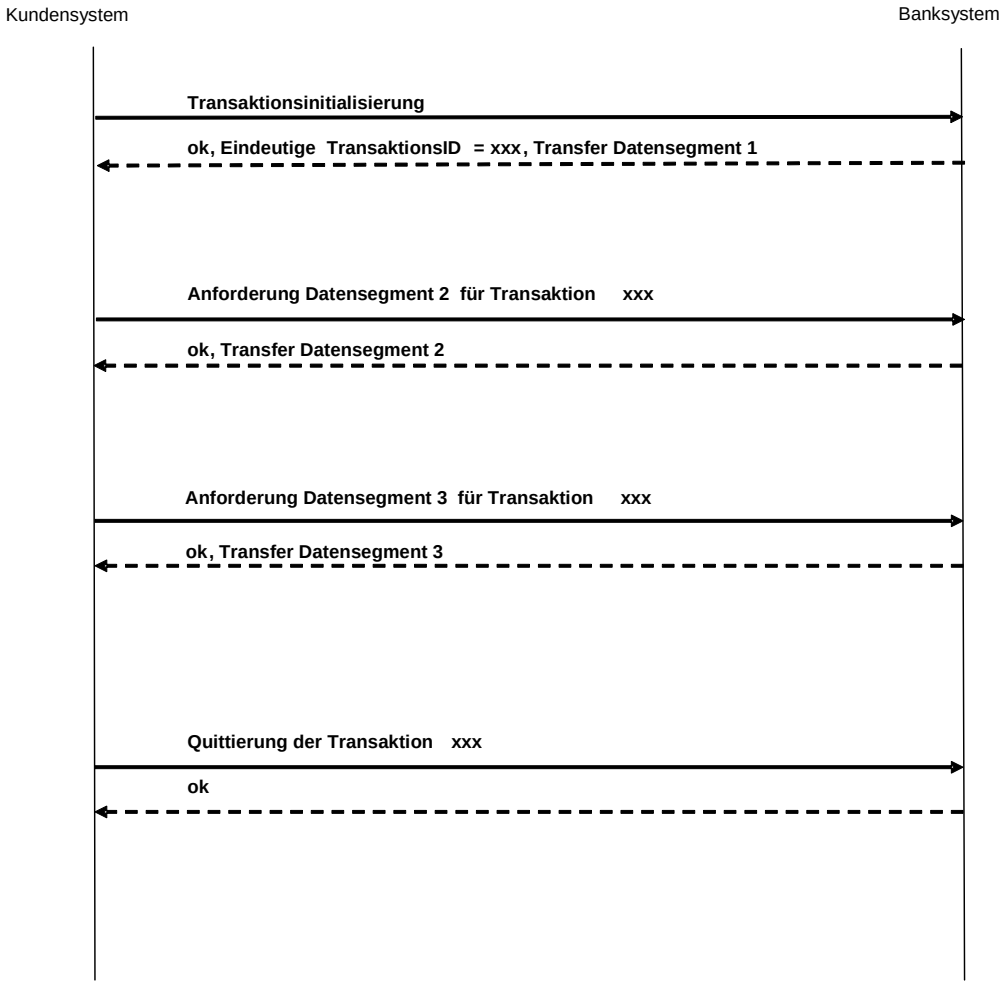


Abbildung 9: Beispielhafter Ablauf einer EBICS-Transaktion für einen Abholauftrag

Details zum Ablauf der EBICS-Transaktionen sind in Kapitel 5 enthalten.

3.13 Interpretation der BTF-Elementkombinationen durch den Bankserver

Dieses Kapitel beschreibt, wie die Belegung der BTF-Elementgruppen <Service> und <ServiceFilter> in einem EBICS-Request vom Bankserver interpretiert werden:

Administrative VEU-Download-Aufträge:
Nur vorhandene BTF-Elemente im Filter werden berücksichtigt, optionale BTF Elemente, die fehlen, werden als “wildcards” interpretiert.
Der Teilnehmer erhält alle zulässigen Matchings, eine Berechtigung des Teilnehmers vorausgesetzt.

Upload Requests:
Die BTF Elemente müssen korrekt und vollständig angegeben werden (passend zum Geschäftsvorfall, der ausgeführt werden soll). Dies bedeutet, dass auch im Request nicht angegebene (also fehlende optionale) Elemente eine Bedeutung haben!
Allerdings hängt es von der Bank (bzw. den vertraglichen Vereinbarungen zwischen Kunde und Bank) ab, wie Berechtigungen definiert und bankseitig hinterlegt werden:
Es ist unabhängig von der EBICS-Kommunikation, ob alle einzelnen zulässigen BTF-Kombinationen oder hierarchische Modelle, wo Gruppen von Geschäftsvorfällen vereinbart werden (z.B. Teilnehmer darf SEPA-Lastschriften einreichen unabhängig von Serviceoptionen und/oder Scopes), im Berechtigungskonzept hinterlegt werden.

Download von Dateien:
Die Angabe der BTF-Elemente muss vollständig und korrekt sein (hinsichtlich des Dateityps, den der Teilnehmer abholenabolen will).) Dies bedeutet, dass auch ein fehlendes (optionales) Element eine Bedeutung hat!

HKD / HTD:
Eine vollständige Deklaration jeder BTF-Elementwert-Kombination, für die der Teilnehmer/Kunde berechtigt ist, ist in der HKD/HTD-Response erforderlich. Dies bedeutet wieder, dass ein fehlendes Element eine Bedeutung hat.

3.14 Interpretation von EU- und VEU/EDS-Flag-Kombinationen durch den Bankserver

Diese Tabelle beschreibt die Bedeutung des Signatur (EU-)Flags und des VEU- (bzw. EDS-)VEU-) Flags:

EU Flag	VEU/ EDS-Flag	Bedingung	Reaktion	Entspricht dem ab EBICS V3.0 nicht mehr anwendbarem bisherigen Auftragsattribut (nur zur Information)
x	-	Anzahl EUs muss ausreichend	Abweisung, wenn Anzahl EUs nicht ausreicht (“Authorization failed” 090003)	OZHNN

		sein		
x	x	Kunde hat vertragliche Vereinbarung hinsichtlich der VEU/EDS (HVE-Berechtigungen sind eingeräumt)	Im Falle einer ausreichenden Anzahl von EUs wird das VEU-Flag ignoriert. Es erfolgt eine Abweisung, wenn es keine vertragliche VEU-Vereinbarung gibt und die Anzahl der EUs nicht ausreicht. (“EBICS Distributed Signature authorization failed” 091007)	OZHNN
-	-	Datei wird nicht innerhalb von EBICS signiert/ authorisiert	Hängt von der vertraglichen Vereinbarung ab: Wenn Unterschriften erforderlich sind und das Unterschreiben außerhalb von EBICS nicht vereinbart wurde, wird der Auftrag nicht autorisiert (“Authorization failed” 090003)	DZHNN

4 Schlüsselmanagement

4.1 Übersicht der verwendeten Schlüssel

Das EBICS-Protokoll sieht für jeden Teilnehmer drei RSA-Schlüsselpaare vor. Diese werden für die folgenden Verwendungszwecke eingesetzt:

- Bankfachliche/technische EU der Auftragsdaten, die der Teilnehmer/ das Clientsystem an das Banksystem sendet
- Authentifikation des Teilnehmers gegenüber dem Banksystem mittels Authentifikationssignatur
- Entschlüsselung des (symmetrischen) Transaktionsschlüssels, der zur Verschlüsselung der Auftragsdaten eingesetzt wird, die der Teilnehmer vom Banksystem abruft.

Ihrem Verwendungszweck folgend spricht man dann auch von dem

- öffentlichen / privaten bankfachlichen Schlüssel
- öffentlichen / privaten Authentifikationsschlüssel
- öffentlichen / privaten Verschlüsselungsschlüssel.

EBICS ermöglicht die Verwendung von drei unterschiedlichen Schlüsselpaaren je Teilnehmer. EBICS fordert dabei die Verwendung von mindestens zwei unterschiedlichen Schlüsselpaaren je Teilnehmer:

- Ein Schlüsselpaar wird exklusiv für die bankfachliche elektronische Signatur verwendet
- Für die Authentifikation des Teilnehmers gegenüber dem Banksystem UND die Entschlüsselung von Transaktionsschlüsseln ist die Verwendung eines einzigen Schlüsselpaares erlaubt.

Analog zu den Teilnehmerschlüsseln sieht EBICS drei unterschiedliche RSA-Schlüsselpaare des Banksystems vor. Diese werden für die folgenden Verwendungszwecke eingesetzt:

- bankfachliche EU der Auftragsdaten, die ein Teilnehmer vom Banksystem abruft
  In der EBICS-Version „H005“ ist die bankfachliche EU der Kreditinstitute nur vorgesehen (siehe Kapitel 3.5.2)
- Authentifikation des Kreditinstituts gegenüber dem Teilnehmer mittels Authentifikationssignatur
- Entschlüsselung des (symmetrischen) Transaktionsschlüssels zur Verschlüsselung bankfachlicher Nutzdaten, die ein Teilnehmer an das Kreditinstitut sendet.

Bezüglich des Einsatzes eines RSA-Schlüsselpaares für unterschiedliche Zwecke gelten dieselben Einschränkungen wie für Teilnehmerschlüssel.

Die Schlüssel eines Teilnehmers sind an Verfahren gekoppelt, die der Teilnehmer für die Erzeugung/Verifikation der EU, für die Erzeugung/Verifikation der Authentifikationssignatur sowie für die Verschlüsselung von Auftragsdaten einsetzen möchte. Diese Verfahren werden durch eindeutige

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Versionen identifiziert, so dass unterschiedliche Teilnehmer beispielsweise unterschiedliche Verfahren für die EU verwenden können. EBICS setzt voraus, dass die jeweiligen Verfahren je Teilnehmer im Banksystem verwaltet werden.

Die Version „H005“ des EBICS-Protokolls sieht die Verwendung der folgenden Verfahren vor:

- „X002“ für die Authentifikationssignatur
- „A005“ oder „A006“ für die EU
- „E002“ für die Verschlüsselung.

Details zu diesen Verfahren sind im Anhang (Kapitel 11.1, Kapitel 11.2, Kapitel 11.3) enthalten.

Teilnehmer desselben Kunden verwenden in der Regel dieselben Verfahren der Authentifikationssignatur, der Verschlüsselung sowie der EU.

Die Aufträge eines Teilnehmers können durch einen technischen Teilnehmer übermittelt werden, wenn beide Teilnehmer dieselben Verfahren der Authentifikationssignatur, der Verschlüsselung und der bankfachlichen Signatur einsetzen. Kundenseitig kann in diesem Fall die Verwaltung der öffentlichen Bankschlüssel für all die Teilnehmer zentralisiert werden, die mit denselben Verfahren arbeiten möchten.

4.2 Darstellung der öffentlichen Schlüssel

EBICS definiert die administrativen Auftragsarten HIA, HCA, HCS und HPB, deren bankfachliche Nutzdaten öffentliche Schlüssel des Teilnehmers bzw. des Kreditinstituts sind. Für diese Auftragsarten erfolgt die Einbettung der öffentlichen Schlüssel in EBICS-Nachrichten unter Verwendung neu definierter Typen auf Basis von XML-Schema (siehe Schema-Definitionsdatei ebics_types_H005.xsd bzw. ebics_signature.xsd). Diese Typen sind in der nachfolgenden Tabelle enthalten:

Schlüsseltyp	**XML-Typ**
Authentifikationsschlüssel	`AuthenticationPubKeyInfoType`
Bankfachlicher Schlüssel	`SignaturePubKeyInfoType`
Verschlüsselungsschlüssel	`EncryptionPubKeyInfoType`

Die grafische Darstellung der XML-Typen `AuthenticationPubKeyInfoType`, `SignaturePubKeyInfoType`, `EncryptionPubKeyInfoType` ist in Abbildung 10, Abbildung 11 und schließlich Abbildung 12 enthalten.

Die XML-Strukturen sind ähnlich aufgebaut: Sie enthalten Informationen zum verwendeten X.509-Zertifikat. Bestandteil des Authentifikationsschlüssels ist zudem die Version des Verfahrens zur Bildung/ Verifikation der Authentifikationssignatur (siehe Element `AuthenticationVersion`). Analog dazu ist die Version des Verschlüsselungsverfahrens Bestandteil des Verschlüsselungsschlüssels sowie die Version der bankfachlichen EU Bestandteil des bankfachlichen Schlüssels.

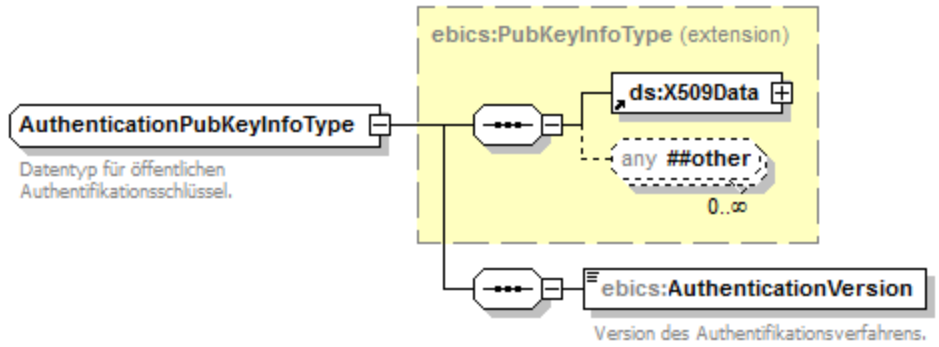


Abbildung 10: Definition des XML-Schema-Typs AuthenticationPubKeyInfoType

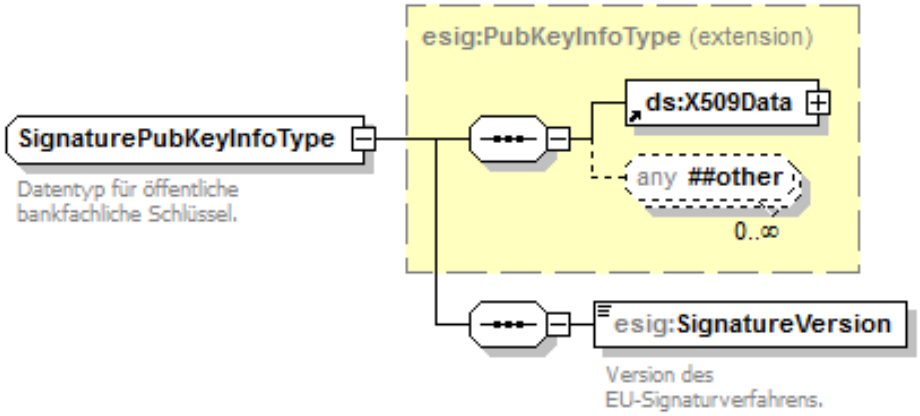


Abbildung 11: Definition des XML-Schema-Typs SignaturePubKeyInfoType

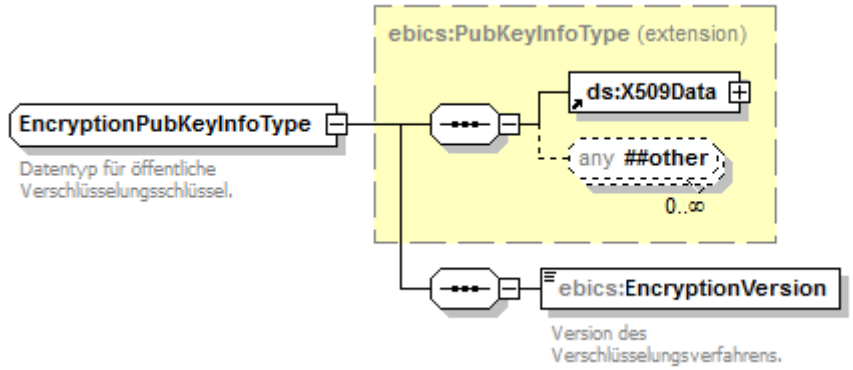


Abbildung 12: Definition des XML-Schema-Typs EncryptionPubKeyInfoType

4.3 Aktionen innerhalb des Schlüsselmanagements

Die eigentliche Durchführung der Sendeaufträge des Schlüsselmanagements muss synchron zu deren Übertragung über EBICS stattfinden. Die Durchführung muss somit abgeschlossen sein, bevor der letzte EBICS-Response dieser Übertragung an den Teilnehmer gesendet wird.

Diese Forderung gilt für INI, HIA und H3K (siehe Kapitel 4.4.1, um die Durchführung der Teilnehmer-initialisierung nicht unnötig zu verzögern. Sie gilt gleichermaßen für SPR (siehe Kapitel 4.5), um die Sperre des Teilnehmers sofort zu aktivieren. Nachträglich initiierte EBICS-Transaktionen für die Übermittlung eines bankfachlichen Auftrags werden auf Ebene des EBICS-Protokolls solange abgewiesen, bis der Teilnehmer wieder den Zustand „Bereit“ erreicht hat. (Vor dem Zeitpunkt der Sperrung erfolgreich verifizierte EUs des Teilnehmers bleiben auch nach der Sperrung gültig. Eine solche EU kann für die Autorisierung eines offenen Auftrags im Rahmen der VEU (EDS) weiterhin verwendet werden).

Schließlich gilt diese Forderung auch für PUB, HCS und HCA (siehe Kapitel 4.6.1), um unmittelbar darauf folgende EBICS-Transaktionen des betroffenen Teilnehmers, die die aktualisierten Schlüssel bereits einsetzen, auch erfolgreich durchführen zu können. (Vor dem Zeitpunkt der Durchführung von PUB bzw. HCS erfolgreich verifizierte EUs des Teilnehmers bleiben auch nach der Durchführung von PUB bzw. HCS und der damit verbundenen Änderung des bankfachlichen Teilnehmerschlüssels gültig. Eine solche EU kann für die Autorisierung eines offenen Auftrags im Rahmen der VEU (EDS) weiterhin verwendet werden).

4.4 Initialisierung

Um bankfachliche EBICS-Transaktionen mit einem bestimmten Kreditinstitut durchführen zu können, müssen für die Teilnehmer eines Kunden eine Reihe von Voraussetzungen erfüllt sein.

Grundvoraussetzung ist der Abschluss eines Vertrags zwischen Kunde und Kreditinstitut. In diesem Vertrag wird vereinbart, welche Geschäftsvorfälle (BTF-Parameterwerte) der Kunde mit dem Kreditinstitut abwickelt, welche Konten betroffen sind, welche Teilnehmer des Kunden mit dem System arbeiten und welche Berechtigungen diese Teilnehmer besitzen.

Sofern der Kunde noch nicht über ein entsprechendes Kundenprodukt verfügt, erhält er nach Abschluss des Vertrags die Client-Software sowie die Zugangsdaten des Kreditinstituts (Bankparameter). Das Kreditinstitut richtet die Kunden- und Teilnehmerstammdaten gemäß den vertraglichen Vereinbarungen im Banksystem ein. Der einzelne Teilnehmer erhält hierdurch den Status „**Neu**“.

Details der vertraglichen Vereinbarungen sind nicht Gegenstand dieses Standards, sie sind bankindividuell in der Beziehung Kunde-Kreditinstitut zu regeln.

Weitere Voraussetzungen sind die erfolgreiche Teilnehmerinitialisierung und Abholung der öffentlichen Schlüssel des Kreditinstituts durch den Teilnehmer. Die notwendigen Schritte auf Kreditinstitut-, Kunden- und Teilnehmerseite sowie die zeitlichen Abhängigkeiten dieser Schritte sind in Abbildung 13 enthalten. Abbildung 14 zeigt einen beipielhaften Ablauf anhand eines Sequenzdiagramms. Der Zustand der öffentlichen Bankschlüssel auf Teilnehmerseite ist auf der Lebenslinie des Teilnehmersystems dargestellt. Entsprechend sind der Zustand der öffentlichen Teilnehmerschlüssel auf Bankseite sowie der Zustand des Teilnehmers selbst auf der Lebenslinie des Banksystems dargestellt. Details dieser Abbildungen werden in den nachfolgenden Kapiteln erklärt.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

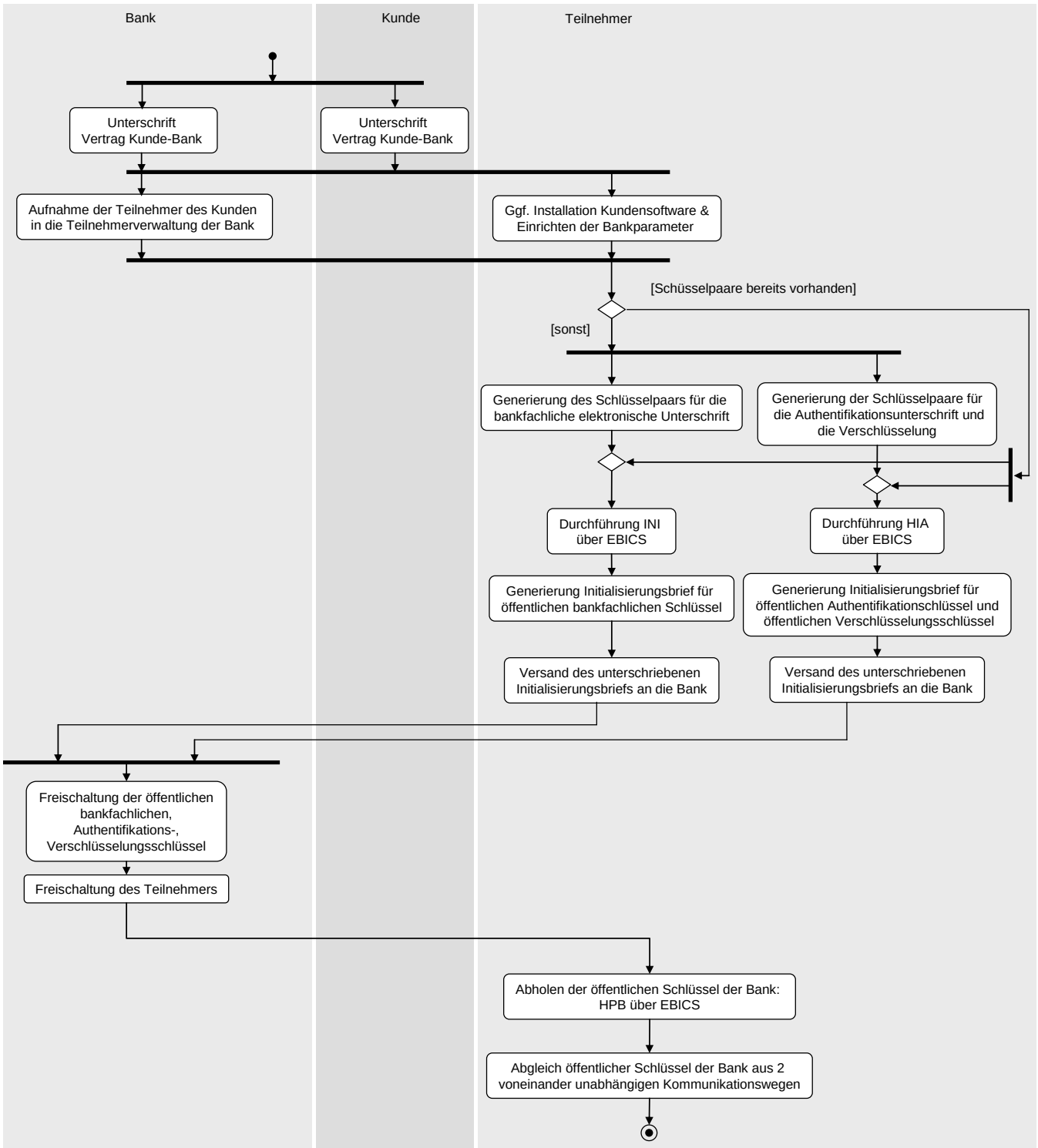


Abbildung 13: Notwendige Schritte im Vorfeld der eigentlichen Durchführung von Geschäftsvorfällen über EBICS (wenn mit INI/HIA initialisiert wird)

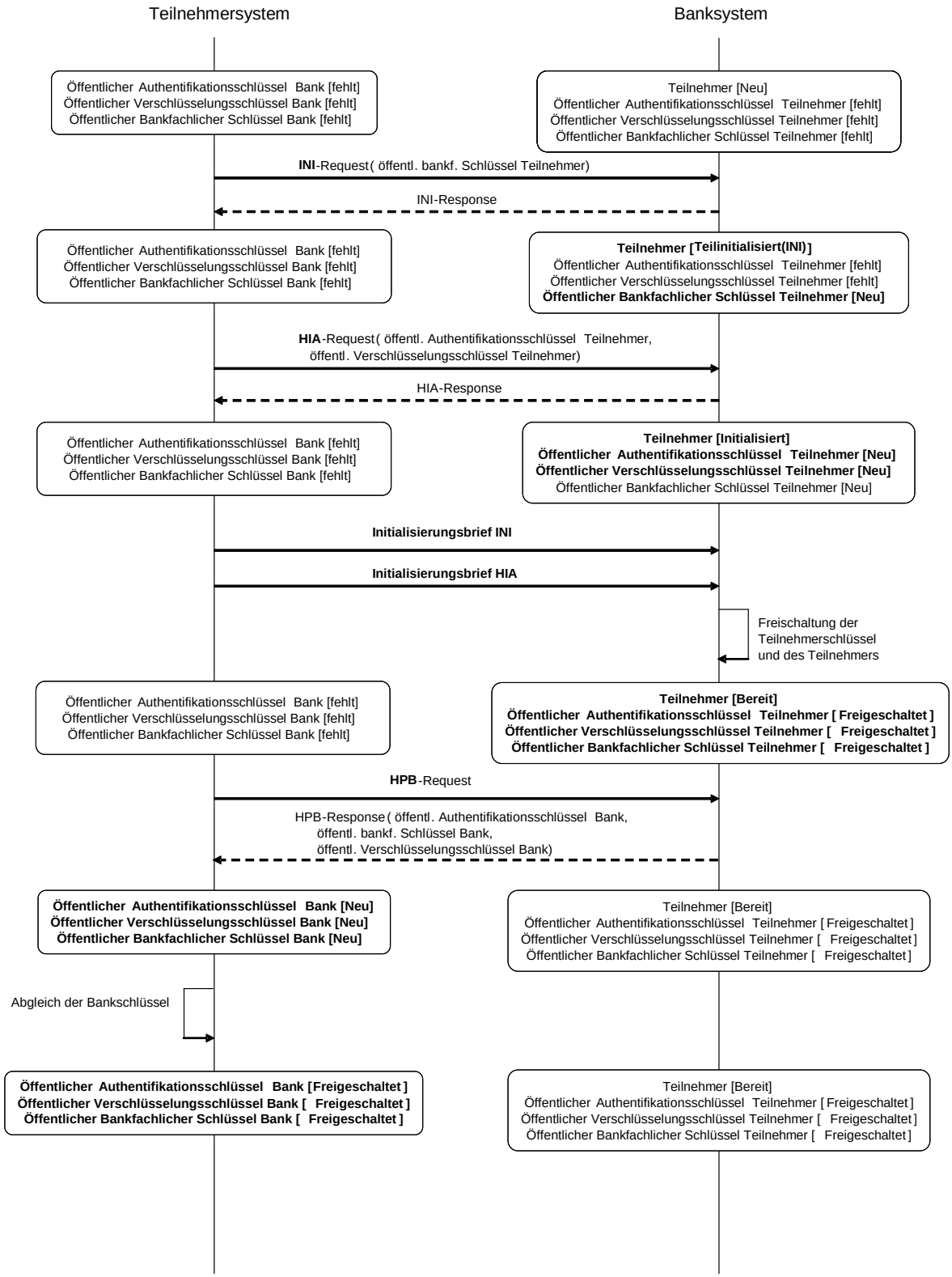


Abbildung 14: Beispielhafter Ablauf: Teilnehmerinitialisierung mit anschließender Abholung und Überprüfung der Bankschlüssel (wenn mit INI/HIA initialisiert wird)

4.4.1 Teilnehmerinitialisierung

4.4.1.1 Allgemeine Beschreibung

Für die Initialisierung des Teilnehmers beim Kreditinstitut ist die Übermittlung der öffentlichen Schlüssel des Teilnehmers an das Banksystem erforderlich. Bestandteil der Bankparameter sind die unterstützten Versionen für die EU, die Verschlüsselung und die Authentifikationssignatur. Der bankfachliche Schlüssel des Teilnehmers muss neu generiert werden, wenn der Teilnehmer nicht über einen passenden bankfachlichen Schlüssel verfügt oder einen bereits vorhandenen bankfachlichen Schlüssel nicht für die neue Bankverbindung nutzen möchte. Das gleiche gilt für den Verschlüsselungsschlüssel und den Authentifikationsschlüssel.

Der Teilnehmer übermittelt dem Kreditinstitut seine öffentlichen Schlüssel wie folgt

Alternative 1: (auf zwei voneinander unabhängigen Kommunikationswegen):

- über EBICS mittels folgender administrativer Auftragsarten:
- **INI**: Senden des öffentlichen bankfachlichen Schlüssels (Schlüssel für Autorisierung/EU)
- **HIA**: Senden des öffentlichen Authentifikationsschlüssels sowie des öffentlichen Verschlüsselungsschlüssels.

Die Übermittlung der öffentlichen Teilnehmerschlüssel an das Kreditinstitut über INI und HIA wird als **Teilnehmerinitialisierung** bezeichnet

- auf dem Postweg mit vom Teilnehmer unterschriebenen Initialisierungsbriefen.

Die Verwendung unterschriebener Initialisierungsbriefe ermöglicht dem Kreditinstitut:

- die Überprüfung der Authentizität der über EBICS übermittelten öffentlichen Teilnehmerschlüssel als Voraussetzung für die Freischaltung von Teilnehmern
- die Nachvollziehbarkeit der Schlüssel-Historie von Teilnehmern durch Aufbewahrung und Archivierung der Initialisierungsbriefe.

Die Reihenfolge der Durchführung von INI und HIA ist nicht festgelegt, im Rahmen einer Teilnehmerinitialisierung werden jedoch genau ein INI-Auftrag und genau ein HIA-Auftrag durchgeführt. Die Übermittlung der öffentlichen Teilnehmerschlüssel über zwei separate Aufträge in beliebiger Reihenfolge erfordert die Definition der Teilnehmerzustände **„Teilweise Initialisiert(INI)“** und **„Teilweise Initialisiert(HIA)“**. In Abhängigkeit davon, ob im Rahmen der Teilnehmerinitialisierung der erste erfolgreich durchgeführte Auftrag INI oder HIA ist, nimmt der Teilnehmer den entsprechenden Zustand ein.

Eine Initialisierung mit Alternative 1 ist immer zulässig, egal, ob es für die RSA Schlüssel eine CA-Zertifizierung gibt oder nicht. Allerdings müssen bei Alternative 1 immer Initialisierungsbriefe verwendet warden. Die öffentlichen Teilnehmerschlüssel müssen dann mit den administrativen Auftragsarten INI (öffentlicher bankfachlicher Schlüssel, also Schlüssel für Autorisierung/EU) und HIA (öffentliche Schlüssel für Authentifikation und Verschlüsselung) zu Bank geschickt werden. Um die Authentizität der Teilnehmerschlüssel zu garantieren, muss sichergestellt sein, dass die Bank diese Schlüssel über einen zweiten, davon unabhängigen, Kommunikationsweg erhält (Initialisierungsbriefe sowohl für INI als auch

für HIA). Hat die Bank die Schlüssel über diese beiden unabhängigen Kommunikationswege erhalten, vergleicht sie diese zuerst, bevor sie sie anerkennt.
Alternative 1 kann insbesondere genutzt werden, wenn Alternative 2 fehlschlägt.
Details und Workflow zur Alternative 1 sind in Kapitel 4.4.1.2 beschrieben.

Alternative 2 (Initialisierung in einem Schritt):
Diese Alternative ist nur möglich, wenn der bankfachliche (Autorisierungs-) Schlüssel auf einem Zertifikat basiert, das von einer Zertifizierungsinstanz (CA) ausgegeben wurde, die Authentizität dieses speziellen Schlüssels von der CA garantiert wird und Kunde und Bank vereinbart haben, dieses Zertifikat zu verwenden, so lange dieses auch gültig ist.
Dann kann auf einen Initialisierungsbrief verzichtet werden.

- via EBICS mittels der Auftragsart:
  - **H3K**: Übermittlung der öffentlichen Teilnehmerschlüssel für die bankfachliche EU (Autorisierung), für Authentifikation und Verschlüsselung.
  -

Für die Initialisierung via CA-Zertifikat vereinfacht die Auftragsart H3K den Ablauf:

1. Alle öffentlichen Teilnehmerschlüssel können in einem Schritt übermittelt werden (H3K-Request), wobei das mitgesendete von einer CA ausgegebene Zertifikat für den bankfachlichen EU (Autorisierungs-) Schlüssel dabei bereits für schon für eine EU genutzt werden kann.
2. INI- und HIA-Brief sind daher nicht notwendig.

Details und Workflow siehe Kapitel 4.4.1.3.

4.4.1.2 Initialisierung via INI und HIA

4.4.1.2.1 INI

Die Durchführung von INI ist zulässig, wenn der Zustand des betroffenen Teilnehmers „Neu“, „Gesperrt“ oder „Teilweise Initialisiert(HIA) ist. INI besteht aus einem einzigen EBICS-Request/-Response-Paar. Für den EBICS-Request von INI gilt:

- er erfordert keine Authentifikationssignatur, da der öffentiche Authentifikationsschlüssel des Teilnehmers vom Kreditinstitut noch nicht freigeschaltet wurde und damit nicht zur Prüfung verwendet werden kann
- er enthält keine bankfachliche Unterschrift, da der öffentliche bankfachliche Schlüssel des Teilnehmers in diesem Request erst übertragen wird. Dieser öffentliche bankfachliche Schlüssel des Teilnehmers kann vom Kreditinstitut nicht zur Prüfung der bankfachlichen Unterschrift verwendet werden, da seine Authentizität zu diesem Zeitpunkt noch nicht gesichert ist
- er enthält die Nutzdaten, d.h. den öffentlichen bankfachlichen Schlüsel des Teilnehmers in unverschlüsselter Form, da der öffentliche Verschlüsselungsschlüssel des Kreditinstituts dem Teilnehmer (zumindest bei der ersten Initialisierung) noch nicht vorliegt.

Das Ablaufdiagramm in Abbildung 15 stellt die bankseitige Verarbeitung dar, die beim Empfang eines INI-Requests stattfindet. Fehlersituationen, die aus einer ungültigen Kombination aus Kunden-

/TeilnehmerID oder einem unzulässigen Teilnehmerzustand resultieren, werden nicht direkt an den Sender des INI-Requests durchgereicht. Stattdessen erhält der Sender den technischen Fehlercode EBICS_INVALID_USER_OR_USER_STATE. INI liefert keine Fehler der Art „Unbekannter Teilnehmer“ oder „Unzulässiger Teilnehmerzustand“, um potenziellen Angreifern keine genaue Information über die Gültigkeit von TeilnehmerIDs oder den Zustand von Teilnehmern zu geben. Hingegen muss auf Seiten des Kreditinstituts eine interne Protokollierung erfolgen, die die genaue Fehlerursache dokumentiert.

Das Ablaufdiagramm sieht die Überprüfung des Teilnehmerzustands vor, um INI-Requests bereits auf EBICS-Ebene abzuweisen, wenn der Teilnehmerzustand für INI nicht zulässig ist. Zulässige Zustände für INI sind: „Neu“, „Gesperrt“ sowie „Teilweise initialisiert(HIA)“. Geprüft wird hier der Zustand des Teilnehmers aus den Headerdaten des Requests. Die Nutzdaten des INI-Requests (siehe Kapitel 4.4.1.2.5.1) enthalten letztlich den Teilnehmer, dessen bankfachlicher Schlüssel übermittelt werden soll. Der Teilnehmer aus den Headerdaten sollte deshalb mit dem Teilnehmer aus den Nutzdaten übereinstimmen. Das EBICS-Protokoll sieht keine Überprüfung dieser Übereinstimmung vor. Vor der eigentlichen Durchführung des Auftrags wird jedoch der Zustand des Teilnehmers (erneut) überprüft, welcher Teil der Nutzdaten von INI ist.

Die Durchführung eines INI-Auftrags kann die folgenden Fehlercodes liefern:

- EBICS_KEYMGMT_UNSUPPORTED_VERSION_SIGNATURE
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten eine unzulässige Version des Verfahrens zur bankfachlichen Signatur enthalten
- EBICS_KEYMGMT_KEYLENGTH_ERROR_SIGNATURE
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten einen bankfachlichen Schlüssel unzulässiger Länge enthalten
- EBICS_INVALID_ORDER_DATA_FORMAT
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten nicht dem dafür vorgesehenen Format entsprechen (siehe Kapitel 4.4.1.2.5.1)
- EBICS_INVALID_USER_OR_USER_STATE
  Dieser technische Fehler tritt dann auf, wenn die Nutzdaten einen Teilnehmer enthalten, der entweder unbekannt oder dessen Zustand für INI unzulässig ist. Zulässig sind die folgenden Teilnehmerzustände: „Neu“, „Gesperrt“, „Teilweise Initialisiert(HIA)“.
  - Für Return Codes, die die Verwendung von CA-Zertifikaten betreffen, wird auf Anhang 1 verwiesen.

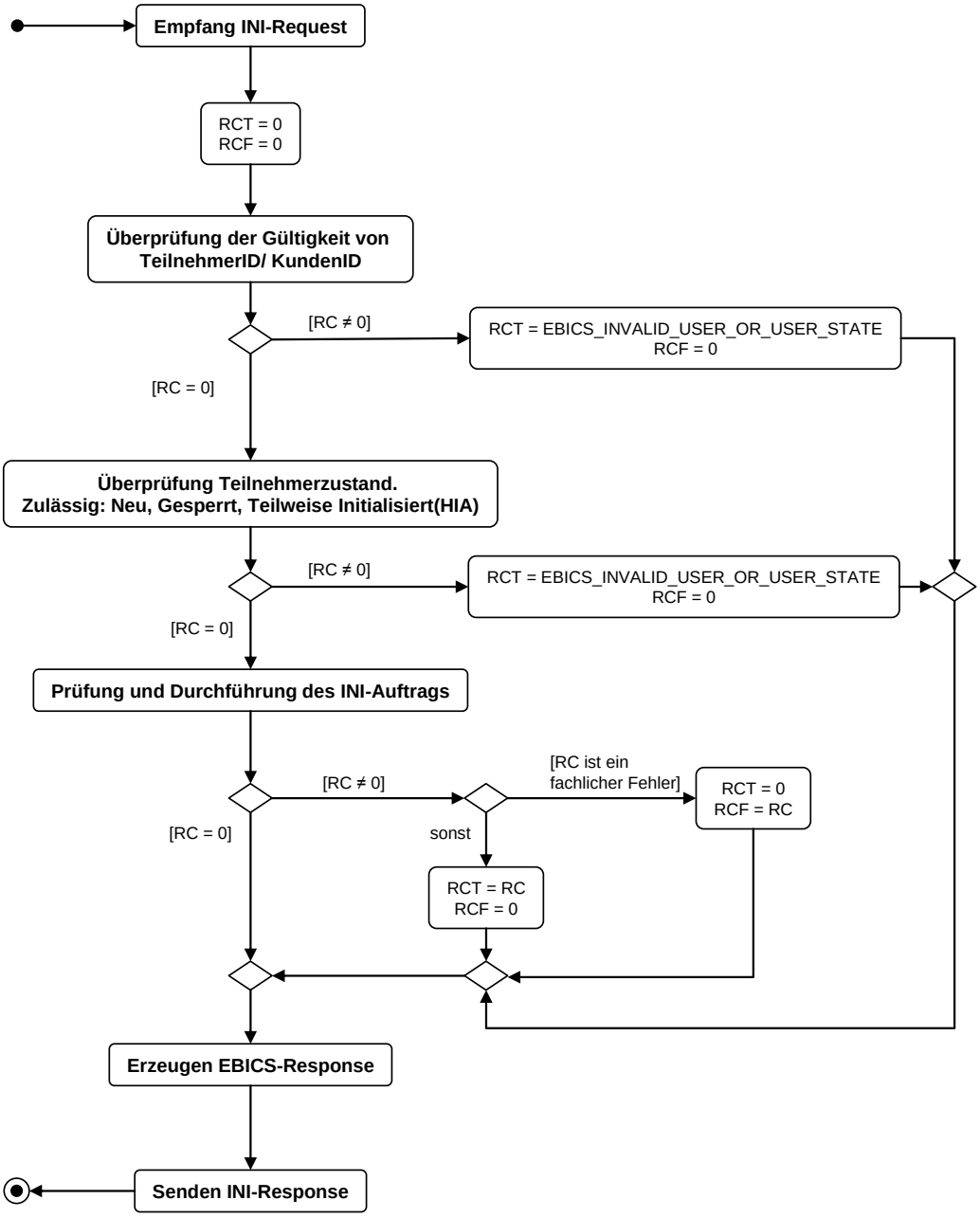


Abbildung 15: Bankseitige Verarbeitung eines INI-Requests

Die EBICS-Response für INI enthält keine Authentifikationssignatur des Kreditinstituts, da der öffentliche Authentifikationsschlüssel des Kreditinstituts dem Teilnehmer zur Prüfung noch nicht vorliegt.

In Abbildung 14 wird INI vor HIA durchgeführt, und dementsprechend gelangt der Teilnehmer vom Zustand „Neu“ in den Zustand „Teilweise initialisiert(INI)“. Der Zustand „Teilweise initialisiert(INI)“ bedeutet:

- das Banksystem verfügt über den öffentlichen bankfachlichen Schlüssel des Teilnehmers, den es allerdings noch nicht freigeschaltet hat

- das Banksystem verfügt (noch) nicht über den öffentlichen Authentifikationsschlüssel sowie den öffentlichen Verschlüsselungsschlüssel des Teilnehmers.

In diesem Zustand kann der Teilnehmer nur eine der beiden folgenden Aktionen durchführen:

- die administrative Auftragsart HIA durchführen und danach in den Zustand „Initialisiert“ übergehen:
Aufträge ungleich HIA, die der Teilnehmer in diesem Zustand einreicht, werden vom Banksystem zurückgewiesen. Bankfachliche Unterschriften des Teilnehmers zu bereits vorhandenen Aufträgen werden als ungültig gewertet, wenn sich der Teilnehmer zum Zeitpunkt der Überprüfung im Zustand „Teilweise initialisiert(INI)“ befindet.
- sich per Anruf beim Kreditinstitut sperren lassen:
Danach ist nur die erneute Initialisierung des Teilnehmers möglich.

Nach der erfolgreichen Durchführung von INI verschickt der Teilnehmer einen unterschriebenen Initialisierungsbrief für INI. Für Einzelheiten zum Inhalt des Initialisierungsbriefs siehe Kapitel 4.4.1.2.3.

4.4.1.2.2 HIA

Die Durchführung von HIA ist zulässig, wenn der Zustand des Teilnehmers „Neu“, „Gesperrt“ oder „Teilweise Initialisiert(INI)“ ist. HIA besteht aus einem einzigen EBICS-Request/ -Response-Paar. Für den EBICS-Request von HIA gilt:

- er enthält keine Authentifikationssignatur, da der öffentliche Authentifikationsschlüssel des Teilnehmers in diesem Request erst übertragen wird. Dieser öffentliche Authentifikationsschlüssel des Teilnehmers kann vom Kreditinstitut nicht zur Prüfung der Authentifikationsunterschrift verwendet werden, da seine Authentizität zu diesem Zeitpunkt noch nicht gesichert ist
- er enthält keine bankfachliche Unterschrift, da der öffentliche bankfachliche Schlüssel des Teilnehmers vom Kreditinstitut noch nicht freigeschaltet wurde und damit nicht zur Prüfung verwendet werden kann
- er enthält die Nutzdaten, d.h. den öffentlichen Verschlüsselungsschlüssel und den öffentlichen Authentifikationsschlüssel des Teilnehmers, in unverschlüsselter Form, da der öffentliche Verschlüsselungsschlüssel des Kreditinstituts dem Teilnehmer (zumindest bei der ersten Initialisierung) noch nicht vorliegt.

Das Ablaufdiagramm in Abbildung 16 stellt die bankseitige Verarbeitung dar, die beim Empfang eines HIA-Requests stattfindet. Analog zu INI werden auch hier Fehlersituationen, die aus einer ungültigen Kombination aus Kunden-/TeilnehmerID oder einem unzulässigen Teilnehmerzustand resultieren, nicht direkt an den Sender des HIA-Requests durchgereicht. Stattdessen erhält der Sender den technischen Fehlercode EBICS_INVALID_USER_OR_USER_STATE. HIA liefert keine Fehler der Art „Unbekannter Teilnehmer“ oder „Unzulässiger Teilnehmerzustand“, um potenziellen Angreifern keine genaue Information über die Gültigkeit von TeilnehmerIDs oder den Zustand von Teilnehmern zu geben. Ebenfalls analog zu INI muss auf Seiten des Kreditinstituts eine interne Protokollierung erfolgen, die die genaue Fehlerursache dokumentiert.

Das Ablaufdiagramm sieht die Überprüfung des Teilnehmerzustands vor, um HIA-Requests bereits auf EBICS-Ebene abzuweisen, wenn der Teilnehmerzustand für HIA nicht zulässig ist. Zulässige Zustände für HIA sind: „Neu“, „Gesperrt“ sowie „Teilweise initialisiert(INI)“. Geprüft wird hier der Zustand des Teilnehmers aus den Headerdaten des Requests. Die Nutzdaten des HIA-Requests (siehe Kapitel 4.4.1.2.5.1) enthalten letztlich den Teilnehmer, dessen Authentifikations- und Verschlüsselungsschlüssel übermittelt werden sollen. Der Teilnehmer aus den Headerdaten sollte deshalb mit dem Teilnehmer aus den Nutzdaten übereinstimmen. Das EBICS-Protokoll sieht keine Überprüfung dieser Übereinstimmung vor. Vor der eigentlichen Durchführung des Auftrags wird jedoch der Zustand des Teilnehmers (erneut) überprüft, welcher Teil der Nutzdaten von HIA ist.

Die Durchführung eines HIA-Auftrags kann die folgenden Fehlercodes liefern:

- EBICS_KEYMGMT_UNSUPPORTED_VERSION_ENCRYPTION
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten eine unzulässige Version des Verschlüsselungsverfahrens enthalten
- EBICS_KEYMGMT_UNSUPPORTED_VERSION_AUTHENTICATION
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten eine unzulässige Version des Verfahrens der Authentifikationssignatur enthalten
- EBICS_KEYMGMT_KEY_LENGTH_ERROR_ENCRYPTION
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten einen Verschlüsselungsschlüssel unzulässiger Länge enthalten
- EBICS_KEYMGMT_KEY_LENGTH_ERROR_AUTHENTICATION
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten einen Authentifikationsschlüssel unzulässiger Länge enthalten
- EBICS_INVALID_ORDER_DATA_FORMAT
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten nicht dem dafür vorgesehenen Format entsprechen (siehe Kapitel 4.4.1.2.5.1)
- EBICS_INVALID_USER_OR_USER_STATE
  Dieser technische Fehler tritt dann auf, wenn die Nutzdaten einen Teilnehmer enthalten, der entweder nicht gültig ist oder dessen Zustand für HIA unzulässig ist. Zulässig sind die folgenden Teilnehmerzustände: „Neu“, „Gesperrt“, „Teilweise Initialisiert(INI)“
- Für Return Codes, die Zertifikate betreffen, wird auf Anhang 1 verwiesen.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

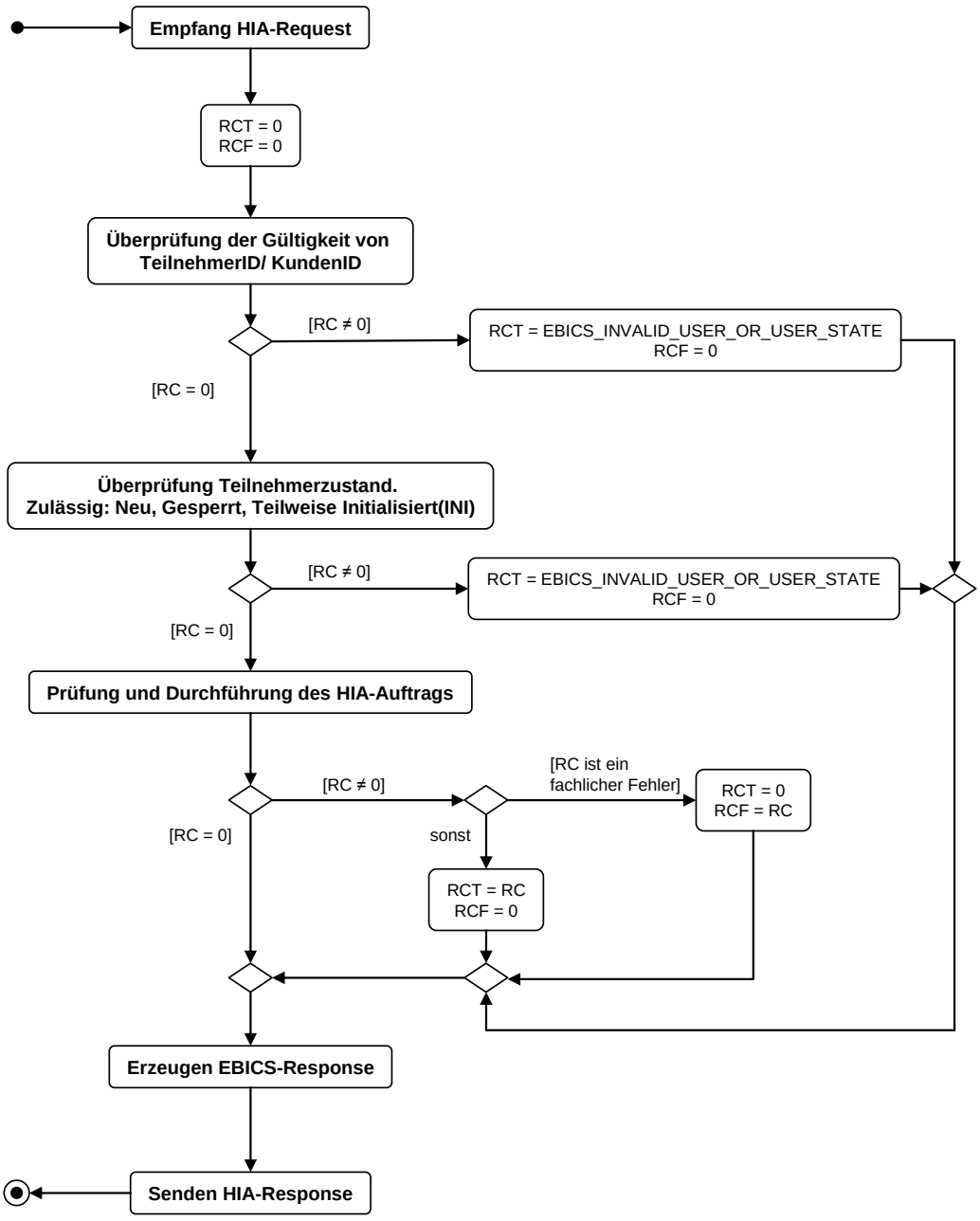


Abbildung 16: Bankseitige Verarbeitung eines HIA-Requests

Die EBICS-Response für HIA enthält keine Authentifikationssignatur des Kreditinstituts, da der öffentliche Authentifikationsschlüssel des Kreditinstituts dem Teilnehmer zur Prüfung noch nicht vorliegt.

Die Bedeutung des Zustands „Teilweise initialisiert (HIA)“, welcher in Abbildung 14 nicht berücksichtigt wurde, ist die folgende:

- Das Banksystem verfügt über den öffentlichen Authentifikationsschlüssel sowie den öffentlichen Verschlüsselungsschlüssel des Teilnehmers. Beide wurden noch nicht durch das Banksystem freigeschaltet

- Das Banksystem verfügt (noch) nicht über den öffentlichen bankfachlichen Schlüssel des Teilnehmers.

In diesem Zustand kann der Teilnehmer nur eine der beiden folgenden Aktionen durchführen:

- die administrative Auftragsart INI durchführen:
Aufträge ungleich INI, die der Teilnehmer in diesem Zustand einreicht, werden vom Banksystem zurückgewiesen. Bankfachliche Unterschriften des Teilnehmers zu bereits vorhandenen Aufträgen werden als ungültig gewertet, wenn sich der Teilnehmer zum Zeitpunkt der Überprüfung im Zustand „Teilweise initialisiert(HIA)“ befindet
- sich per Anruf beim Kreditinstitut sperren lassen:
Danach ist lediglich eine erneute Inititialisierung des Teilnehmers möglich.

Nach der erfolgreichen Durchführung von HIA verschickt der Teilnehmer einen unterschriebenen Initialisierungsbrief für HIA an das Kreditinstitut. Für Einzelheiten zum Inhalt des Initialisierungsbriefes siehe Kapitel 4.4.1.2.3.

4.4.1.2.3 Initialisierungsbriefe

Initialisierungsbriefe für INI enthalten den öffentlichen bankfachlichen Schlüssel des entsprechenden Teilnehmerzertifikats, Initialisierungsbriefe für HIA den öffentlichen Authentifikationsschlüssel sowie den öffentlichen Verschlüsselungsschlüssel der entsprechenden Teilnehmerzertifikats. Alle Zertifikate werden in PEM-Format dargestellt. Neben den öffentlichen Teilnehmerschlüssel enthalten die Initialisierungsbriefe die folgenden Daten:

- Benutzername (optional): kundensoftware-interner Name des Teilnehmers
- Datum: Datum der Durchführung des entsprechenden EBICS-Auftrags
- Uhrzeit: Uhrzeit der Durchführung des entsprechenden EBICS-Auftrags
- Empfänger-Bank
- Teilnehmer-ID
- Kunden-ID.

Zu jedem öffentlichen Teilnehmerschlüssel enthält der Initialisierungsbrief die folgenden Daten:

- Verwendungszweck des öffentlichen SchlüsselsS des Teilnehmerzertifikats:
  bankfachliche elektronische Unterschrift
  Authentifikationssignatur
  Verschlüsselung.

- Verfahren:
  Verfahren der bankfachlichen elektronischen Unterschrift: A005 oder A006
  Verfahren der Authentifikationssignatur: X002
  Verfahren der Verschlüsselung: E002.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

- Hashwert der öffentlichen Zertifikate in hexadezimaler Darstellung

  Der Initialisierungsbrief für INI enthält den SHA-256-Hashwert des Zertifikates für die EU. Die Bildung des Hashwerts für die verschiedenen Unterschriftsversionen wird im Anhang Kapitel 14 beschrieben. Der Initialisierungsbrief für HIA enthält den SHA-256-Hashwert des Zertifikates für die Authentifikationssignatur sowie den SHA-256-Hashwert des Zertifikates für die Verschlüsselung. Der SHA-256-Hashwert der Zertifikate für X002, E002 sowie A005 bzw. A006 wird wie folgt berechnet und angedruckt: Berechnung des SHA-256-Hashwertes des Zertifikats im DER Binärformat, die entstehende Byte-Anordnung (32 Bytes) wird in hexadezimaler Darstellung (64 Zeichen) und in Großbuchstaben dargestellt.

Initialisierungsbriefe für INI enthalten den öffentlichen Signaturschlüssel des entsprechenden Teilnehmerzertifikates, Initialisierungsbriefe für HIA den öffentlichen Authentifikationsschlüssel sowie den öffentlichen Verschlüsselungsschlüssel des jeweils entsprechenden Teilnehmerzertifikates. Beispiele für Initialisierungsbriefe sind im Anhang (Kapitel 11.5.1) enthalten.

4.4.1.2.4 Freischaltung des Teilnehmers durch das Kreditinstitut

Nach erfolgreicher Durchführung von INI und HIA befindet sich der Teilnehmer zunächst im Zustand **„Initialisiert“:** das Banksystem verfügt über alle notwendigen öffentlichen Schlüssel des Teilnehmers, die es allerdings noch nicht freigeschaltet hat. Teilnehmer im Zustand “Initialisiert” können keine Aufträge oder Unterschriften über EBICS einreichen: Jeder Versuch wird vom Banksystem zurückgewiesen.

Nach erfolgreicher Prüfung der Initialisierungsbriefe auf Seiten des Kreditinstituts werden die öffentlichen Teilnehmerschlüssel freigegeben und der Zustand des Teilnehmers im Banksystem auf **„Bereit“** gesetzt. Der Zustand „Bereit“ bedeutet, dass die Bankseite über alle notwendigen Informationen verfügt, um Einreichungen von Aufträgen oder Unterschriften durch den Teilnehmer erfolgreich durchführen zu können. Siehe dazu auch Abbildung 14. Insbesondere kann der Teilnehmer die sogenannten Bankparameter des Kreditinstituts über die administrative Auftragsart HPD abholen (siehe Kapitel 9.2).

Abbildung 17 verdeutlicht noch einmal die bisher beschriebenen Zustandsübergänge eines Teilnehmers. Das Löschen von Teilnehmern aus den Teilnehmerverwaltungen der Banksysteme ist nicht Gegenstand dieses Standards. Aus diesem Grund wird auf die Darstellung eines weiteren Zustands „Gelöscht“ sowie eines Endzustands verzichtet.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

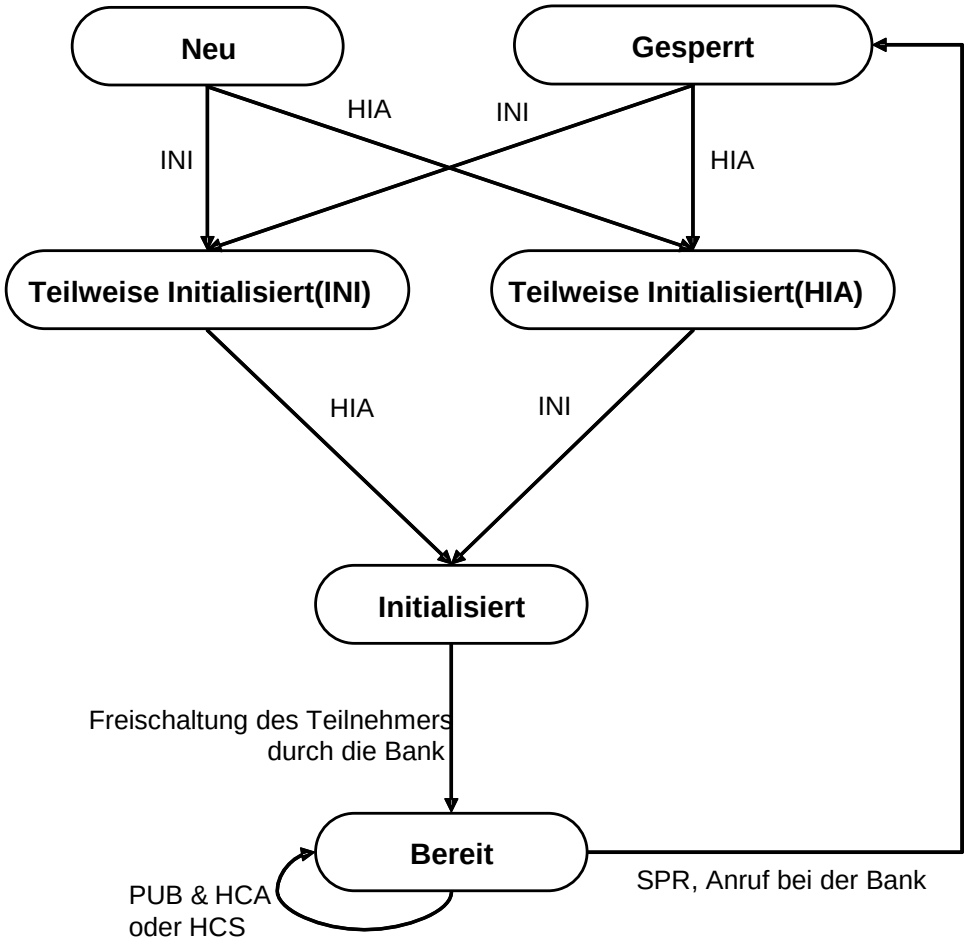


Abbildung 17: Zustandsübergangsdiagramm für Teilnehmer

Die (erneute) Durchführung von INI oder HIA ist im Teilnehmerzustand „Bereit“ nicht zulässig. Damit soll verhindert werden, dass Teilnehmer ungewollt vom Zustand „Bereit“ in den Zustand „Teilweise Initialisiert(INI)“ oder „Teilweise Initialisiert(HIA)“ versetzt werden. Das hätte nämlich zur Folge, dass betroffene Teilnehmer zunächst keine weiteren bankfachlichen Aufträge ausführen könnten.

Teilnehmer im Zustand „Bereit“ müssen zuerst ihren DFÜ-Zugang zum Banksystem sperren, bevor sie eine erneute Teilnehmerinitialisierung vornehmen können. Details zum Sperren von Teilnehmern sind in Kapitel 4.5 beschrieben.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

4.4.1.2.5 Beschreibung der EBICS-Nachrichten

4.4.1.2.5.1 Format der Auftragsdaten

Die Auftragsdaten für INI ist ein zu ebics_signature.xsd konformes Instanzdokument, das aus dem Toplevel-Element `SignaturePubKeyOrderData` besteht.
`SignaturePubKeyOrderData` ist mittels XML-Schema wie folgt definiert:

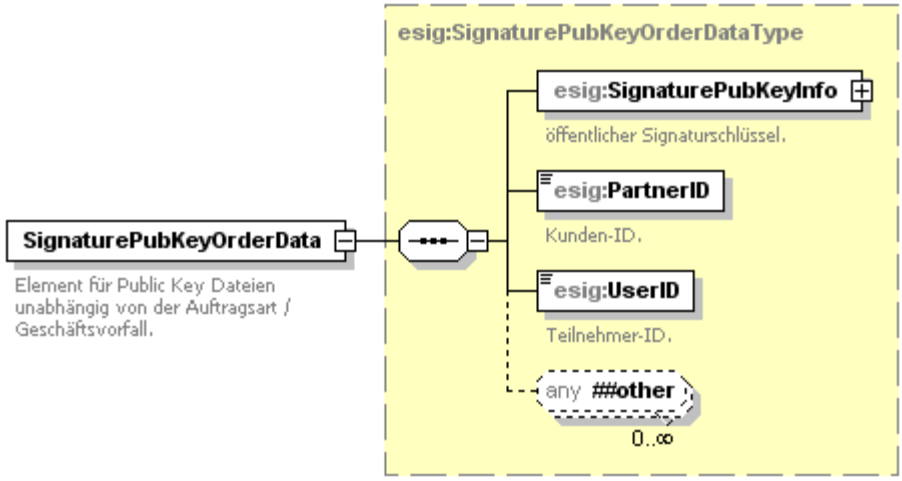


Abbildung 18: Definition des XML-Schema-Elements SignaturePubKeyOrderData der Auftragsdaten für INI (identisch mit PUB, siehe eigenes Kapitel)

Die Auftragsdaten für HIA sind ein zu ebics_orders_H005.xsd konformes Instanzdokument, das aus dem Toplevel-Element `HIARequestOrderData` besteht. `HIARequestOrderData` ist mittels XML-Schema wie folgt definiert:

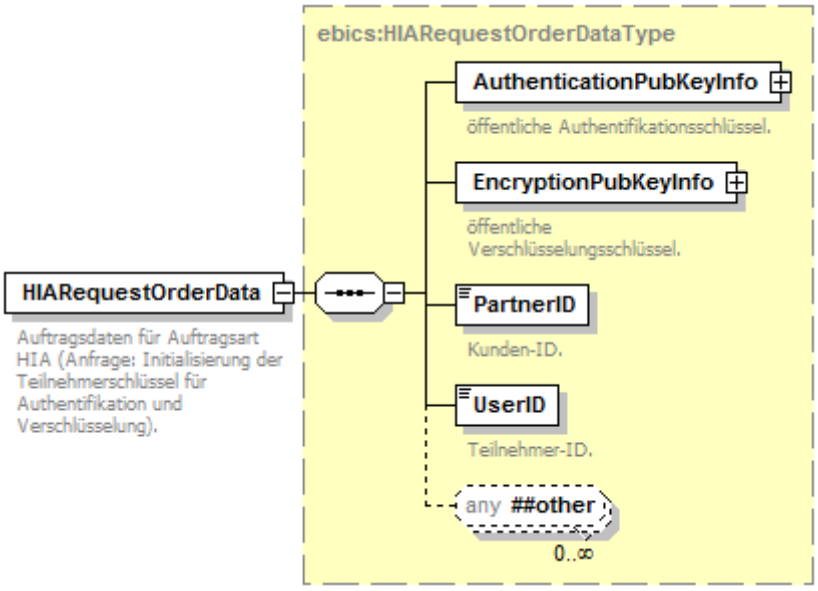


Abbildung 19: Definition des XML-Schema-Elements HIARequestOrderData der Auftragsdaten für HIA

Die Auftragsdaten von INI und HIA werden jeweils komprimiert und base64-kodiert in den entsprechenden EBICS-Request eingebettet.

4.4.1.2.5.2 Beschreibung und Beispiele für Nachrichten

Dieses Kapitel beschreibt die EBICS-Nachrichten für die administrativen Auftragsarten INI und HIA. INI- und HIA-Requests sind zu ebics_keymgmt_request_H005.xsd konforme Instanzdokumente mit dem Toplevel-Element `ebicsUnsecuredRequest`. INI- und HIA-Responses sind zu ebics_keymgmt_response_H005.xsd konforme Instanzdokumente mit dem Toplevel-Element `ebicsKeyManagementResponse`.

Hier werden die Daten aufgezählt, die Bestandteil dieser Nachrichten sind. In Klammern stehen jeweils die entsprechenden XML-Elemente in XPath-Notation. Es gelten die folgenden Konventionen:

- Daten, die grundsätzlich optional sind, werden mit „(optional)“ gekennzeichnet
- Daten, die nur unter bestimmten Voraussetzungen fehlen dürfen, werden stattdessen mit „(bedingt)“ gekennzeichnet
- Optionale XML-Elemente der EBICS-Nachrichten, die in der Beschreibung fehlen, dürfen in der EBICS-Nachricht nicht vorkommen
- Optionale XML-Elemente der EBICS-Nachrichten, die in der Beschreibung ohne die Kennzeichnung „(optional)“ oder „(bedingt)“ vorkommen, müssen immer entsprechend der Beschreibung gesetzt sein.

Ergänzt wird diese Beschreibung durch Beispiele.

Übertragung der folgenden Daten im INI-Request (siehe Beispiel in Abbildung *20*)

- Host-ID des EBICS Bankrechners (ebicsUnsecuredRequest/header/static/HostId)
- Teilnehmer (`ebicsUnsecuredRequest/header/static/PartnerID`, `ebicsUnsecuredRequest/header/static/UserID`), dessen öffentlicher bankfachlicher Schlüssel an das Kreditinstitut übermittelt werden soll
- (optional) technischer Teilnehmer (`ebicsUnsecuredRequest/header/static/PartnerID`, `ebicsUnsecuredRequest/header/static/SystemID`)
  `SystemID` kann in der Nachricht enthalten sein, wenn es sich bei dem Kundensystem um ein Multi-Usersystem handelt. Da INI-Requests keine Authentifikationssignatur enthalten und die Auftragsdaten unverschlüsselt sind, ist die Angabe von SystemID optional.
- (optional) Information zum Kundenprodukt (ebicsUnsecuredRequest/header/static/Product)
- Administrative Auftragsart (ebics`UnsecuredRequest`/header/static/OrderDetails/AdminOrderType) mit der Belegung “INI”

- Sicherheitsmedium für den bankfachlichen Schlüssel des Teilnehmers (`ebicsUnsecuredRequest/header/static/SecurityMedium`)
  Die zulässigen Belegungen sind im Anhang (Kapitel 12.3) enthalten
- Auftragsdaten (ebicsUnsecuredRequest/body/DataTransfer/OrderData).

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsUnsecuredRequest
 xmlns=" urn:org:ebics:H005" xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_keymgmt_request_H005.xsd"
 Version="H005" Revision="1">
 <header authenticate="true">
   <static>
     <HostID>EBIXHOST</HostID>
     <PartnerID>KUNDE001</PartnerID>
     <UserID>TLN100</UserID>
     <OrderDetails>
       <AdminOrderType>INI</AdminOrderType>
     </OrderDetails>
     <SecurityMedium>0200</SecurityMedium>
   </static>
   <mutable/>
 </header>
 <body>
   <DataTransfer>
     <!--INI-Datei, komprimiert und base64-kodiert-->
     <OrderData>
       …
     </OrderData>
   </DataTransfer>
 </body>
</ebicsUnsecuredRequest>
```

Abbildung 20: EBICS-Request für die administrative Auftragsart INI

Übertragung der folgenden Daten im INI-Response (siehe Beispiel in Abbildung 21):

- bankfachlicher Returncode (ebicsKeyManagementResponse/body/ReturnCode)
- Auftragsnummer (`ebicsKeyManagementResponse`/header/mutable/OrderID)
  Diese Nummer wird durch den Bankserver automatisch bestimmt.
- technischer Returncode (ebicsKeyManagementResponse/header/mutable/ReturnCode)
- technischer Reporttext (ebicsKeyManagementResponse/header/mutable/ReportText)
- (optional) Zeitstempel der letzten Aktualisierung der Bankparameter (`ebicsKeyManagementResponse/body/TimestampBankParameter`).

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsKeyManagementResponse
 xmlns="urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_keymgmt_response_H005.xsd"
 Version="H005" Revision="1">
 <header authenticate="true">
   <static/>
   <mutable>
     <OrderID>A101</OrderID>
     <ReturnCode>000000</ReturnCode>
```

```
      <ReportText>[EBICS_OK] OK</ReportText>
    </mutable>
  </header>
  <body>
    <ReturnCode authenticate="true">000000</ReturnCode>
  </body>
</ebicsKeyManagementResponse>
```

Abbildung 21: EBICS-Response für die Auftragsart INI

- **Übertragung der folgenden Daten im HIA-Request** (analog zu INI, siehe Beispiel in Abbildung 22):
  - Host-ID des EBICS Bankrechners (ebicsUnsecuredRequest/header/static/HostId)
  - Teilnehmer (`ebicsUnsecuredRequest/header/static/PartnerID`, `ebicsUnsecuredRequest/header/static/UserID`), dessen öffentlicher Authentifikationsschlüssel und öffentlicher Verschlüsselungsschlüssel an das Kreditinstitut übermittelt werden soll
  - (optional) technischer Teilnehmer (`ebicsUnsecuredRequest/header/static/PartnerID`, `ebicsUnsecuredRequest/header/static/SystemID`)
    `SystemID` kann in der Nachricht enthalten sein, wenn es sich bei dem Kundensystem um ein Multi-Usersystem handelt. Da HIA-Requests keine Authentifikationssignatur enthalten und die Auftragsdaten unverschlüsselt sind, ist die Angabe von `SystemID` optional
  - (optional) Information zum Kundenprodukt (`ebicsUnsecuredRequest/header/static/Product`)
  - Administrative Auftragsart (ebicsUnsecuredRequest/header/static/OrderDetails/AdminOrderType) mit der Belegung “HIA”
  - Sicherheitsmedium für den bankfachlichen Schlüssel des Teilnehmers (`ebicsUnsecuredRequest/header/static/SecurityMedium`) mit der Belegung „0000”.
    Das Sicherheitsmedium für den bankfachlichen Schlüssel des Teilnehmers ist mit „0000“ belegt, da HIA-Aufträge weder bankfachliche Schlüssel übermitteln noch EUs enthalten
  - Auftragsdaten `(ebicsUnsecuredRequest/body/DataTransfer/OrderData).`

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsUnsecuredRequest
 xmlns=" urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_keymgmt_request_H005.xsd"
 Version="H005" Revision="1">
 <header authenticate="true">
   <static>
     <HostID>EBIXHOST</HostID>
     <PartnerID>KUNDE001</PartnerID>
     <UserID>TLN100</UserID>
     <OrderDetails>
       <AdminOrderType>HIA</AdminOrderType>
```

```
      </OrderDetails>
      <SecurityMedium>0000</SecurityMedium>
    </static>
    <mutable/>
  </header>
  <body>
    <DataTransfer>
      <!--Zu ebics_orders_H005.xsd konformes (XML)-Instanzdokument mit Wurzelelement HIARequestOrderData,
komprimiert und base64-kodiert-->
      <OrderData>
        …
      </OrderData>
    </DataTransfer>
  </body>
</ebicsUnsecuredRequest>
```

Abbildung 22: EBICS-Request für die Auftragsart HIA

- **Übertragung der folgenden Daten im HIA-Response** (analog zu INI, siehe Beispiel in Abbildung 23):
  - bankfachlicher Returncode (ebicsKeyManagementResponse/body/ReturnCode)
  - Auftragsnummer (ebicsKeyManagementResponse/header/mutable/OrderID). Diese Nummer wird durch den Bankserver automatisch bestimmt.
  - technischer Returncode (ebicsKeyManagementResponse/header/mutable/ReturnCode)
  - technischer Reporttext (ebicsKeyManagementResponse/header/mutable/ReportText)
  - (optional) Zeitstempel der letzten Aktualisierung der Bankparameter (`ebicsKeyManagementResponse/body/TimestampBankParameter`).

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsKeyManagementResponse
 xmlns=" urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_keymgmt_response_H005.xsd"
 Version="H005" Revision="1">
 <header authenticate="true">
   <static/>
   <mutable>
     <OrderID>A101</OrderID>
     <ReturnCode>000000</ReturnCode>
     <ReportText>[EBICS_OK] OK</ReportText>
   </mutable>
 </header>
 <body>
   <ReturnCode authenticate="true">000000</ReturnCode>
 </body>
</ebicsKeyManagementResponse>
```

Abbildung 23: EBICS-Response für die Auftragsart HIA

4.4.1.3 Initialisierung via H3K

Wenn der bankfachliche (EU-) Schlüssel auf einem selbstsignierten Zertifikat basiert oder ein RSA-Schlüssel ohne Zertifikat ist, dann sind die öffentlichen Teilnehmerschlüssel weiterhin via INI / HIA an die Bank zu übermitteln. Um die Authentizität der öffentlichen Teilnehmerschlüssel zu garantieren, muss

hier sichergestellt sein, dass die Bank diese zusätzlich über einen zweiten unabhängigen Kommunikationsweg erhält (Initialisierungsbriefe sowohl für INI als auch für HIA). Hat die Bank die Schlüssel über diese beiden unabhängigen Kommunikationswege erhalten, vergleicht sie diese zuerst, bevor sie sie anerkennt.

Bei Nutzung von CA-Zertifikaten ist der Prozess für H3K wie folgt:

1) Das Zertifikat für die bankfachliche Unterschrift (EU) muss von einer Zertifizierungsinstanz (CA) ausgegeben sein. In diesem Fall ist kein Initialisierungsbrief für den Schlüssel notwendig. Allerdings können die Zertifikate für Verschlüsselung und die Authentifikationssignatur sowohl selbst-signiert (self-signed) als auch von einer CA ausgegeben sein.
2) Für die Übermittlung der öffentlichen Teilnehmerschlüssel für Verschlüsselung und die Authentifikationssignatur gilt, dass für diese, wenn sie mit einer (gültigen) EU unterschrieben sind, ebenfalls kein Initialisierungsbrief notwendig ist.
3) Die notwendigen bankseitigen Prüfungen (vor der erstmaligen Verwendung der Schlüssel) sind:
    a. Gibt es eine Vereinbarung zur Nutzung des CA-Zertifikats?
    b. Sind alle administrativen Schritte für den Kunden/Teilnehmer auf dem EBICS-Bankrechner erledigt und ist der Teilnehmer dem EBICS-Bankrechner bekannt?
    c. Ist das Zertifikat gültig?
4) Unter Berücksichtigung von 1) bis 3) kann die neue Auftragsart H3K definiert werden:
    a. Diese kombiniert INI und HIA (Übermittlung von drei öffentlichen Teilnehmerschlüsseln, alle Schlüssel basieren auf Zertifikaten); für den EU-Schlüssel muss das Zertifikat von einer CA ausgegeben sein.
    b. H3K enthält bereits eine Elektronische Unterschrift (durch die von einer CA zertifizierten EU-Schlüssel)

Dieses Konzept setzt bei den Zertifikaten, die im H3K-Prozess verwendet werden (und dort als EU-Schlüssel verwendet werden) eine hohe Authentizität voraus, gleiches gilt für die Zertifizierungsinstanz (CA), die diese Zertifikate ausgibt:

- Ausgabe des Zertifikats:
    - Strenge Vorgaben hinsichtlich Identifikation (die Person und die Organisation betreffend, die das Zertifikat anfordert)
    - Alle Daten des Zertifikats wurden sorgfältig durch die Registrierungsinstanz validiert.
- Namensregeln:
    - Für den Namen in Zertifikat (SubjectName) muss ein festes Schema vorhanden sein, das eine eindeutige (automatische) Zuordnung einer natürlichen Person zu einem Unternehmen zulässt.
- Kryptografische Anforderungen:
    - Z.B. Schlüssellängen
- Validierungsanforderungen:
    - Aktualität und Verfügbarkeit von Listen mit ungültig erklärten Zertifikaten (Zertifikatssperrlisten; revocation lists)

Für die elektronische Initialisierung (ohne INI-Brief) neuer EBICS-Teilnehmer mit der administrativen Auftragsart H3K ist die Verwendung von CA-Zertifikaten verpflichtend.
Wenn ein Zertifikat nicht von der Bank selbst ausgegeben wurde, ist eine notwendige Voraussetzung für die elektronische Initialisierung, dass der neue EBICS-Teilnehmer seiner Bank sein Zertifikat für die (bankfachliche) EU, das seinen eindeutigen Eigentümernamen (subject name) und den Namen der CA enthält, und das relevante CA Rootzertifikat (Stammzertifikat) bekannt gegeben hat. Der neue EBICS-Teilnehmer unterschreibt mit diesem Zertifikat die Zertifikate für Authentifikation und Verschlüsselung.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Die Anforderungen, die in der Zertifikatspolicy erfüllt sein müssen, werden bilateral zwischen Kunde und Bank abgestimmt. Die Interoperabilität von verschiedenen Trust Domains (Vertrauensdomänen) kann nur erreicht werden, wenn entsprechende technische, organisatorische und gesetzliche Anforderungen definiert sind. Diese Anforderungen sind nicht Gegenstand der EBICS-Spezifikation, da sie für den Kommunikationsstandard selbst nicht relevant sind.

In der Schemadatei ebics_orders_H005.xsd enthält die Elementgruppe `H3KRequestOrderData` drei Zertifikate. In der Schemadatei ebics_keymgmt_request_H005 enthält die Struktur ebics:UnsignedRequest `H3KRequestOrderData` (komprimiert und base-64 kodiert) und eine Signatur. In der Schemadatei ebics_ types_H005.xsd wird der Datentyp `SignatureCertificateInfoType`, der für die Übermittlung eines X.509-Zertifikats benötigt wird, definiert. `AuthenticationCertificateInfoType` und `EncryptionInfoType` sind Erweiterungen dieses Typs und jeder dieser Typen wird in `H3KOrderData` verwendet.

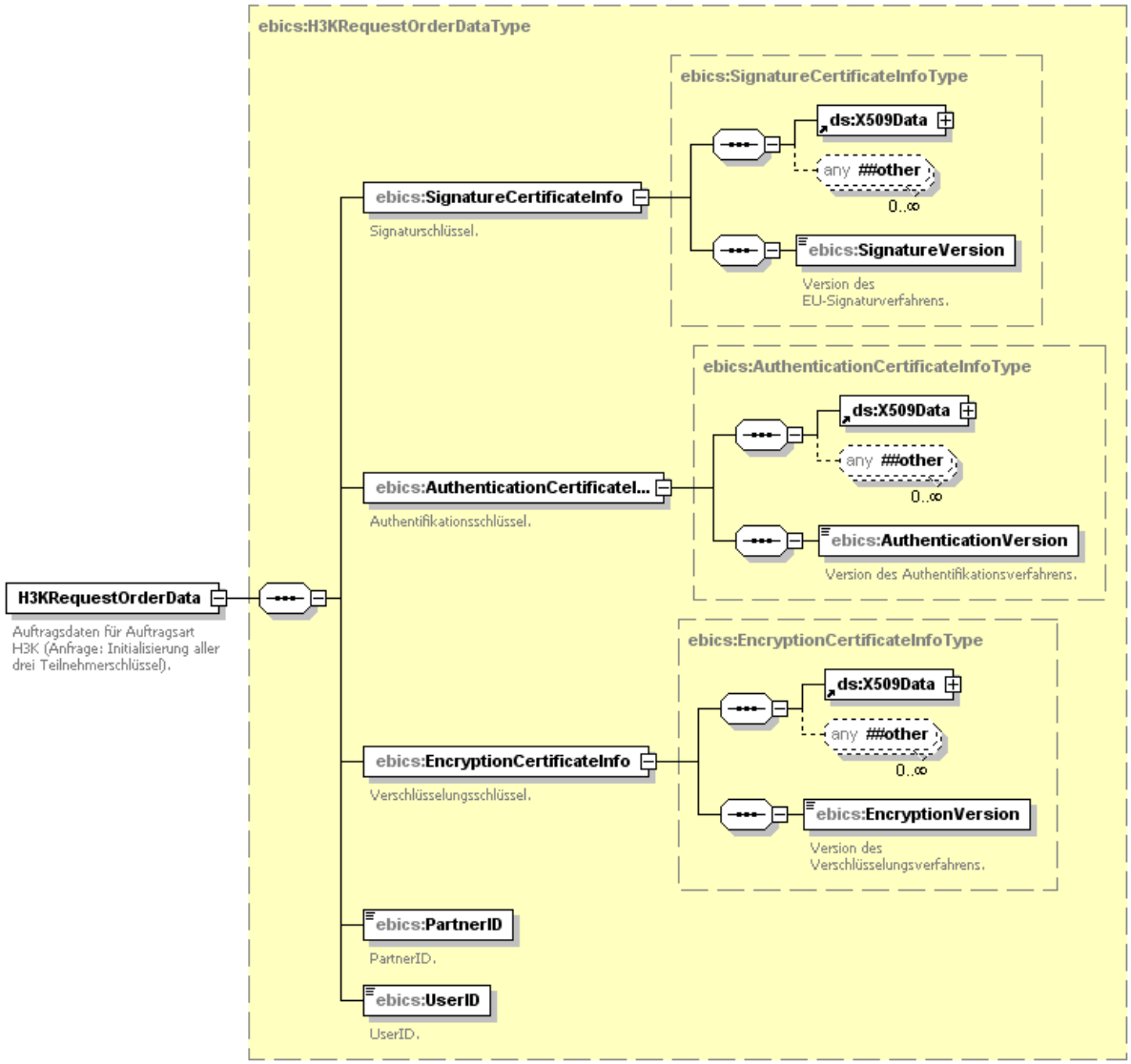


Abbildung 24: Definition des XML-Schema-Elements H3KRequestOrderData für H3KOrder Data

Bei der Teilnehmerinitialisierung sieht der bankseitige Prüfprozess wie folgt aus:

1- Überprüfung der Struktur `H3KRequestOrderData` und Extrahieren des für die EU (Autorisierung) vorgesehenen Zertifikats.

2- Überprüfung, ob dieses Zertifikat von einer zulässigen Zertifizierungsinstanz (CA) stammt.

3- Überprüfung, ob die Unterschrift in SignatureData zum öffentlichen Teilnehmerschlüssel für die EU in `H3KRequestOrderData` passt.

4- Überprüfung, ob die Kundeninformationen im Zertifikat mit den früheren Angaben übereinstimmen (`ebicsUnsignedRequest` → `body` → `DataTransfer`)
Note: Die Bank wählt für die Prüfung der Übereinstimmung notwendige Mittel und Informationen. Beispielsweise kann diese Information in einem Vertrag angegeben oder durch das Hochladen von Zertifikaten auf die Website der Bank veröffentlicht worden sein.

   a. Bei Übereinstimmung gibt der Bankrechner den Return Code EBICS_OK (Code-Nummer 000000) zurück. Der Status des Teilnehmers wird automatisch auf „**Bereit**“ gesetzt. Der Teilnehmer braucht keine Initialisierungsbriefe zu senden (auch kein anderer Validierungsprozess ist notwendig)

   b. Wenn der Bankrechner das Zertifikat (zum öffentlichen Teilnehmerschlüssel für die EU) nicht erfolgreich mit den früheren Angaben abgleichen kann, sendet er eine Ablehnungs-Response und der H3KRequest wird abgebrochen. Der Bankrechner gibt den Return Code EBICS_CERTIFICATES_VALIDATION_ERROR (Code-Nummer 091219) zurück. Der Status des Teilnehmers bleibt gleich („**Neu**“). Der Teilnehmer hat zwei Möglichkeiten, weiter vorzugehen:

      i. Erneute Durchführung eines H3K-Requests mit einem korrekten Zertifikat (für den öffentlichen Teilnehmerschlüssel für die EU), das von einer CA ausgegeben wurde

      ii. Oder Durchführung der Initialisierung mit INI und HIA (INI/HIA ist also insbesondere als Ausweichprozess nutzbar)

Das zur EU zu verwendende Zertifikat muss gültig sein (vor allem darf das Gültigkeitsdatum nicht abgelaufen sein). Ansonsten muss der Kunde sein (von einer CA ausgegebenes) Zertifikat aktualisieren bzw. ein neues angeben.

Zudem können alle Fehlermeldungen, die sich auf die Prozesse des Schlüsselmanagements beziehen, auch im H3K Prozess vorkommen.

4.4.2 Abholung der öffentlichen Schlüssel des Kreditinstituts

4.4.2.1 Allgemeine Beschreibung

Der Teilnehmer holt sämtliche öffentlichen Schlüssel des Kreditinstituts mittels einer eigens dafür vorgesehenen administrativen Auftragsart (HPB) ab. Die Abholung der öffentlichen Bankschlüssel erfordert den Teilnehmerzustand „Bereit“, da erst dann die Verfahren feststehen, die der Teilnehmer für die Authentifikationssignatur, bankfachliche Unterschrift und Verschlüsselung einsetzen möchte.

Die Durchführung von HPB erfordert lediglich ein einziges EBICS-Request-/ EBICS-Response-Paar. Der EBICS-Request von HPB enthält die Authentifikationsunterschrift des Teilnehmers selbst oder die eines technischen Teilnehmers desselben Kunden über die Steuerdaten.

Das Ablaufdiagramm in Abbildung 25 stellt die bankseitige Verarbeitung dar, die beim Empfang eines HPB-Requests stattfindet. Der Replay-Test erfolgt wie bei den bankfachlichen Auftragsarten (siehe dazu Anhang Kapitel 11.4). Demnach liefert HPB den technischen Fehler EBICS_TX_MESSAGE_REPLAY, wenn der HPB-Request eine wieder hergestellte Nachricht ist. Ebenso erfolgt die Überprüfung der Kunden-/TeilnehmerID, des Teilnehmerzustands sowie der Authentifikationssignatur wie im Falle bankfachlicher Auftragsarten innerhalb des Ablaufschritts “Überprüfung der Authentizität des EBICS-Requests” (siehe Kapitel 5.5.1.2.1, Abbildung 42).

Diese kann die folgenden Fehlercodes liefern:

- EBICS_AUTHENTICATION_FAILED
  Dieser technische Fehler tritt dann auf, wenn die Authentifikationssignatur des Teilnehmers (oder des technischen Teilnehmers) nicht erfolgreich geprüft werden konnte
- EBICS_USER_UNKNOWN
  Dieser technische Fehler tritt dann auf, wenn die Authentifikationssignatur des technischen Users erfolgreich verifiziert werden konnte, der (nichttechnische) Teilnehmer jedoch dem Kreditinstitut unbekannt ist
- EBICS_INVALID_USER_STATE
  Dieser technische Fehler tritt dann auf, wenn die Authentifikationssignatur des technischen Users erfolgreich verifiziert werden konnte und der (nichttechnische) Teilnehmer dem Kreditinstitut bekannt ist, jedoch nicht im Zustand „Bereit“ ist.

Nach der erfolgreichen Durchführung der „Überprüfung der Authentizität der Nachricht“ liefert die eigentliche Durchführung des HPB-Auftrags keine weiteren spezifischen technischen oder bankfachlichen Fehler.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

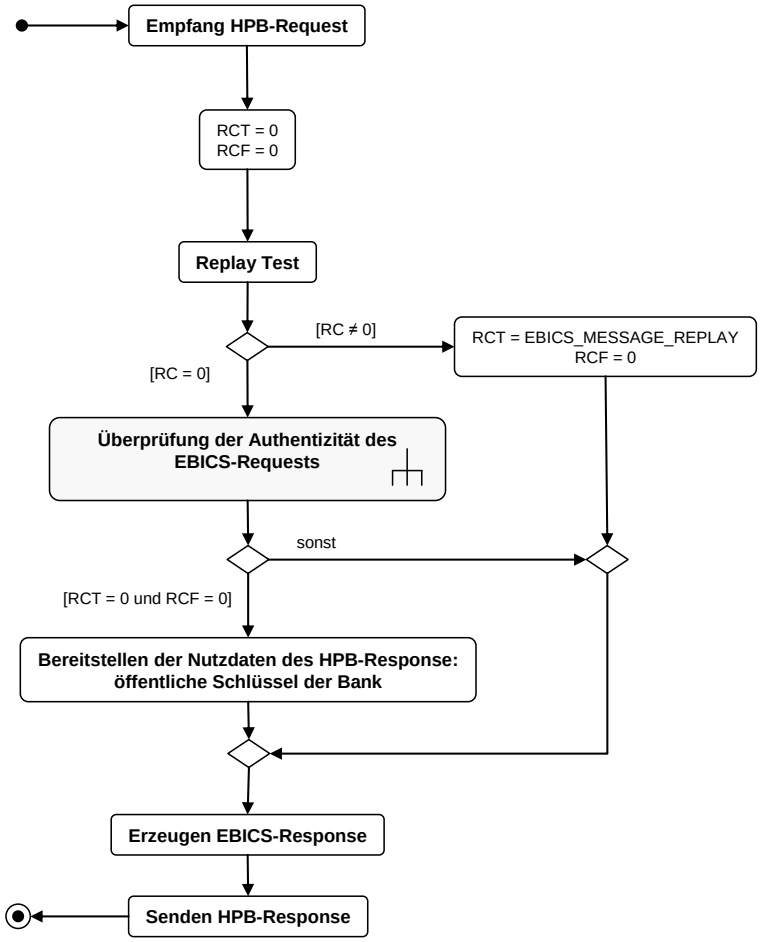


Abbildung 25: Bankseitige Verarbeitung eines HPB-Requests

Für die EBICS-Response gilt:

- sie enthält keine Authentifikationsunterschrift, da der öffentliche Authentifikationsschlüssel des Kreditinstituts in dieser Response erst übertragen wird. Der öffentliche Authentifikationsschlüssel des Kreditinstituts kann vom Teilnehmer noch nicht zur Prüfung der Authentifikationssignatur verwendet werden, da seine Authentizität zu diesem Zeitpunkt noch nicht gesichert ist
- sie enthält keine bankfachliche elektronische Unterschrift der Nutzdaten, d.h. den öffentlichen Bankschlüssel, da der bankfachliche Schlüssel des Kreditinstituts in dieser Response erst übertragen wird. Der öffentliche bankfachliche Schlüssel des Kreditinstituts kann vom Teilnehmer noch nicht zur Prüfung der bankfachlichen EU verwendet werden, da seine Authentizität zu diesem Zeitpunkt noch nicht gesichert ist
- sie enthält die Nutzdaten, d.h. die öffentlichen Bankschlüssel, in verschlüsselter Form, da der öffentliche Verschlüsselungsschlüssel des Teilnehmers durch das Kreditinstitut bereits freigeschaltet wurde.

Nach der erfolgreichen Durchführung von HPB verfügt der Teilnehmer über alle notwendigen öffentlichen Bankschlüssel, die er vor dem ersten Einsatz jedoch noch überprüfen muss: Wie in Abbildung 14 dargestellt, ist der Zustand dieser Schlüssel auf Teilnehmerseite „Neu“. Im Zustand „Neu“

dürfen Bankschlüssel nicht für die Kommunikation über EBICS eingesetzt werden, da ihre Authentizität nicht sichergestellt ist.

Um die Authentizität der Bankschlüssel zu gewährleisten, ist durch das Kreditinstitut sicherzustellen, dass der Teilnehmer die öffentlichen Bankschlüssel und/oder die Hashwerte über einen zweiten, unabhängigen Kommunikationsweg (z.B. über die Internetseite des Kreditinstituts) erhält. Es liegt in der Verantwortung des Teilnehmers, die Prüfung der Bankschlüssel vorzunehmen. Das Verfahren der Prüfung der Bankschlüssel ist nicht Bestandteil des vorliegenden Standards. Wenn die Bank von einer CA ausgegebene Zertifikate bereitstellt, ist der Kunde verpflichtet, deren Gültigkeit zu überprüfen (Anmerkungen siehe im Common Implementation Guide). Es ist abhängig von der Implementierung der EBICS-Clientsysteme, welche sicherstellen, dass Teilnehmer die öffentlichen Schlüssel erst nach einer erfolgreichen Prüfung einsetzen.

Nach der erfolgreichen Prüfung verändert sich der Zustand der öffentlichen Bankschlüssel auf Teilnehmerseite von “Neu” auf “Freigeschaltet”. Diese Zustandsänderung ist ebenfalls in Abbildung 14 dargestellt. Nur Bankschlüssel im Zustand „Freigeschaltet“ dürfen für die Kommunikation über EBICS eingesetzt werden.

In der EBICS-Version „H005“ ist die EU der Kreditinstitute zwar vorgesehen, wird aber noch nicht verwendet (siehe Kapitel 3.5.2). Abbildung 14 berücksichtigt den Zustand des öffentlichen bankfachlichen Schlüssels der Bank auf Teilnehmerseite in Vorbereitung zukünftiger EBICS-Versionen.

4.4.2.2 Beschreibung der EBICS-Nachrichten

4.4.2.2.1 Format der Auftragsdaten

Die Auftragsdaten für HPB sind ein zu ebics_orders_H005.xsd konformes Instanzdokument, das aus dem Toplevel-Element `HPBResponseOrderData` besteht. `HPBResponseOrderData` ist mittels XML-Schema wie folgt definiert:

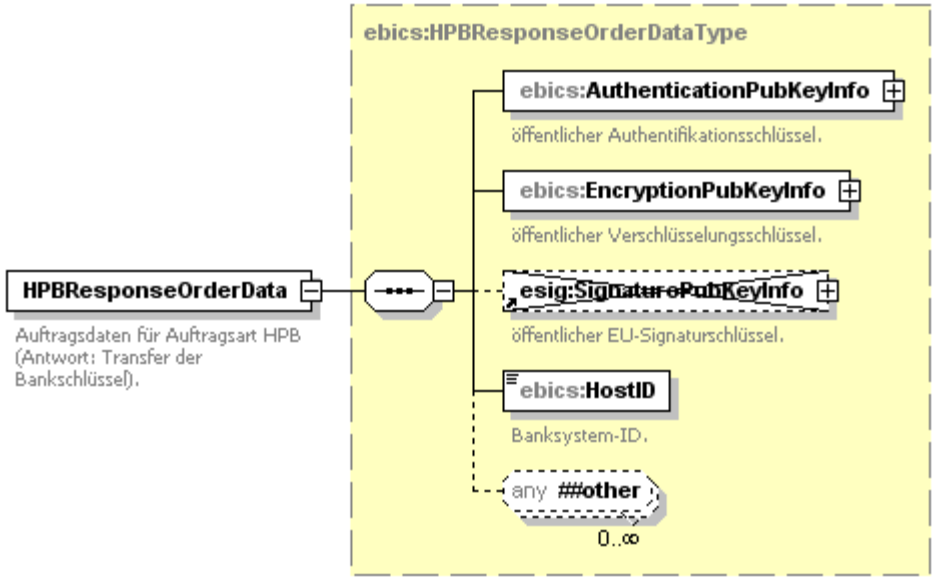


Abbildung 26: Definition des XML-Schema-Elements HPBResponseOrderData der Auftragsdaten für HPB

In der Version „H005“ des EBICS-Protokolls ist die EU der Kreditinstitute zwar vorgesehen, wird aber noch nicht verwendet (siehe dazu Kapitel 3.5.2). Das Element `SignaturePubKeyInfo` ist in Vorbereitung zukünftiger EBICS-Versionen mit der maximalen Häufigkeit (maxOccurs) gleich 0 definiert.

Die Auftragsdaten werden komprimiert, verschlüsselt und base64-kodiert in den entsprechenden HPB-Response eingebettet.

4.4.2.2.2 Beschreibung und Beispielnachrichten

Dieses Kapitel beschreibt die EBICS-Nachrichten für die administrative Auftragsart HPB. HPB-Requests sind zu ebics_keymgmt_request_H005.xsd konforme Instanzdokumente mit dem Toplevel-Element `ebicsNoPubKeyDigestsRequest`. Hingegen sind HPB-Responses zu ebics_keymgmt_response_H005.xsd konforme Instanzdokumente mit dem Toplevel-Element `ebicsKeyManagementResponse`.

Hier werden die Daten aufgezählt, die Bestandteil dieser Nachrichten sind. In Klammern stehen jeweils die entsprechenden XML-Elemente in XPath-Notation. Es gelten die folgenden Konventionen:

.
- Daten die grundsätzlich optional sind, werden mit „(optional)“ gekennzeichnet.

- Daten, die nur unter bestimmten Voraussetzungen fehlen dürfen, werden stattdessen mit „(bedingt)“ gekennzeichnet

- Optionale XML-Elemente der EBICS-Nachrichten, die in der Beschreibung fehlen, dürfen in der EBICS-Nachricht nicht vorkommen
- Optionale XML-Elemente der EBICS-Nachrichten, die in der Beschreibung ohne die Kennzeichnung „(optional)“ oder „(bedingt)“ vorkommen, müssen immer entsprechend der Beschreibung gesetzt sein.

Ergänzt wird diese Beschreibung durch ein Beispiel für ein EBICS-Request-/-Response-Paar für die Auftragsart HPB.

- **Übertragung der folgenden Daten im HPB-Request** (siehe Beispiel inAbbildung 28):
    - Host-ID des EBICS Bankrechners (ebicsNoPubKeyDigestsRequest/header/static/HostId)
    - Kombination aus Nonce und Timestamp, erforderlich für die Vermeidung von Replay alter EBICS-Nachrichten (`ebicsNoPubKeyDigestsRequest/header/static/Nonce`, `ebicsNoPubKeyDigestsRequest/header/static/Timestamp`)
    - Teilnehmer (`ebicsNoPubKeyDigestsRequest/header/static/PartnerID`, `ebicsNoPubKeyDigestsRequest/header/static/UserID`), der den HPB-Request initiiert
    - (bedingt) technischer Teilnehmer (`ebicsNoPubKeyDigestsRequest/header/static/PartnerID`,

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

ebicsNoPubKeyDigestsRequest/header/static/SystemID)
SystemID muss genau dann vorhanden sein, wenn es sich bei dem Kundensystem um ein Multi-Usersystem handelt. Der technische Teilnehmer ist für das Erstellen der EBICS-Requests (inklusive der Authentifikationssignaturen) zuständig, die zu Aufträgen gehören, die vom Teilnehmer eingereicht oder bankfachlich unterzeichnet werden.

- (optional) Information zum Kundenprodukt (ebicsNoPubKeyDigestsRequest/header/static/Product)
- Administrative Auftragsart (ebicsNoPubKeyDigestsRequest/header/static/OrderDetails/AdminOrderType) mit der Belegung “HPB”
- Sicherheitsmedium für den bankfachlichen Schlüssel des Teilnehmers (ebicsNoPubKeyDigestsRequest/header/static/SecurityMedium) mit der Belegung “0000”.
  Das Sicherheitsmedium für den bankfachlichen Schlüssel des Teilnehmers ist mit „0000“ belegt, da HPB-Aufträge keine EUs erfordern und auch keine bankfachlichen Teilnehmerschlüssel übermitteln
- Authentifikationssignatur des technischen Teilnehmers, falls ein solcher vorhanden ist, ansonsten die Authentifikationssignatur des Teilnehmers selbst (ebicsNoPubKeyDigestsRequest/AuthSignature)
  Die Authentifikationsignatur umfasst alle XML-Elemente des EBICS-Requests, deren Attributwert für @authenticate gleich „true“ ist. Die Definition des XML-Schemas „ebics_keymgmt_request_H005.xsd“ gewährleistet, dass der Wert des Attributs @authenticate für genau die Elemente gleich „true“ ist, die auch unterschrieben werden müssen

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsNoPubKeyDigestsRequest
 xmlns=" urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_keymgmt_request_H005.xsd"
 Version="H005" Revision="1">
 <header authenticate="true">
   <static>
     <HostID>EBIXHOST</HostID>
     <Nonce>234AB2340FD2C23035764578FF3091FA</Nonce>
     <Timestamp>2005-01-30T15:40:45.123Z</Timestamp>
     <PartnerID>KUNDE001</PartnerID>
     <UserID>TLN100</UserID>
     <OrderDetails>
       <AdminOrderType>HPB</AdminOrderType>
     </OrderDetails>
     <SecurityMedium>0000</SecurityMedium>
   </static>
   <mutable/>
 </header>
 <AuthSignature>
   <ds:SignedInfo>
     <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
     <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256"/>
     <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
```

```
        <ds:Transforms>
          <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
        </ds:Transforms>
        <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
        <ds:DigestValue> …hier Hashwert Authentifikation..</ds:DigestValue>
      </ds:Reference>
    </ds:SignedInfo>
    <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
  </AuthSignature>
  <body/>
</ebicsNoPubKeyDigestsRequest>
```

Abbildung 27: EBICS-Request für die administrative Auftragsart HPB

- **Übertragung der folgenden Daten im EBICS-Response für HPB** (siehe Beispiel in Abbildung 28):
  - bankfachlicher Returncode (ebicsKeyManagementResponse/body/ReturnCode)
  - technischer Returncode (ebicsKeyManagementResponse/header/mutable/ReturnCode)
  - technischer Reporttext (ebicsKeyManagementResponse/header/mutable/ReportText)
  - (bedingt) Information zur Verschlüsselung der Auftragsdaten und eventuell der EU der Auftragsdaten (`ebicsKeyManagementResponse/body/DataTransfer/DataEncryptionInfo`), falls keine Fehler technischer oder bankfachlicher Natur aufgetreten sind. `DataEncryptionInfo` enthält insbesondere auch den asymmetrisch verschlüsselten Transaktionsschlüssel (`ebicsKeyManagementResponse/body/DataTransfer/DataEncryptionInfo/TransactionKey`)
  - (bedingt) die Nutzdaten (`ebicsKeyManagementResponse/body/DataTransfer/OrderData`), falls keine Fehler technischer oder bankfachlicher Natur aufgetreten sind
  - (optional) Zeitstempel der letzten Aktualisierung der Bankparameter (`ebicsKeyManagementResponse/body/TimestampBankParameter`).

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsKeyManagementResponse
 xmlns="urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_keymgmt_response_H005.xsd "
 Version="H005" Revision="1">
 <header authenticate="true">
   <static/>
   <mutable>
     <ReturnCode>000000</ReturnCode>
     <ReportText>[EBICS_OK] OK</ReportText>
   </mutable>
 </header>
 <body>
   <DataTransfer>
     <DataEncryptionInfo authenticate="true">
       <EncryptionPubKeyDigest Version="E002" Algorithm="http://www.w3.org/2001/04/xmlenc#sha256">..hier
Hashwert des öffentlichen Schlüssels für die Verschlüsselung ..</EncryptionPubKeyDigest>
       <!--asymmetrisch verschlüsselter Transaktionsschlüssel-->
       <TransactionKey>…</TransactionKey>
```

```
      </DataEncryptionInfo>
      <!--Zu ebics_orders_H005.xsd konformes (XML-)Instanzdokument mit Wurzelelement HPBResponseOrderData,
komprimiert, verschlüsselt, base64-kodiert-->
      <OrderData>
        …
      </OrderData>
    </DataTransfer>
    <ReturnCode authenticate="true">000000</ReturnCode>
  </body>
</ebicsKeyManagementResponse>
```

Abbildung 28: EBICS-Response für die administrative Auftragsart HPB

4.5 Sperren eines Teilnehmers

4.5.1 Alternativen

Besteht der Verdacht der Kompromittierung der Teilnehmerschlüssel, MUSS der Teilnehmer seine Zugangsberechtigung zu allen Banksystemen sperren, die den/die kompromittierten Schlüssel verwenden.

Teilnehmer, die ihren DFÜ-Zugang zu einem Banksystem sperren möchten, können dies auf zwei Arten tun:
Im Zustand „Bereit“ kann der Teilnehmer die Sperrung über EBICS mit Hilfe der systembedingten Auftragsart SPR vornehmen.

- Die SPR Transaktion ist eine Standard-Upload Transaktion, die ausschließlich eine EU-Datei mit der Unterschrift des zu sperrenden Teilnehmers über eine Dummy-Datei übermittelt. Die Dummy-Datei enthält genau ein Leerzeichen und wird nicht übertragen. Der entsprechende EBICS-Request muss neben dieser Unterschrift des zu sperrenden Teilnehmers auch eine Authentifikationssignatur enthalten. Die Authentifikationssignatur darf auch von einem technischen Teilnehmer geleistet werden.
  Der SPR-Auftrag umfasst keine weiteren Nutzdaten und somit auch keine Auftragsdatei.
- Der Teilnehmer hat zudem die Möglichkeit, die Sperre über einen zweiten Kommunikationsweg vorzunehmen, z.B. telefonisch über einen bestimmten Ansprechpartner des Kreditinstituts. Bei Verlust oder Beschädigung des Teilnehmerschlüssels ist nur diese Alternative möglich.

Nach der erfolgreichen Durchführung der Sperrung hat der Teilnehmer den Zustand „Gesperrt“ und eine erneute Initialisierung des Teilnehmers ist erforderlich.

4.5.2 Sperren eines Teilnehmers über SPR

SPR ist eine reguläre Upload-Transaktion, deren Ablauf (einschließlich des Verhaltens in Fehlersituationen) in Kapitel 5.5 detailliert beschrieben ist. Nachfolgend sind daher nur Abweichungen und Ergänzungen dargestellt.

Für SPR wird nur eine EU-Datei übertragen. Die Verarbeitung findet bereits in der Initialisierungsphase statt, d.h. das Banksytem liefert in der Response keine Transaktions-ID.

Das Banksystem muss sicherstellen, dass der SPR-Request die Authentifikationssignatur des zu sperrenden Teilnehmers bzw. des technischen Teilnehmer enthält. Die Überprüfung der Kunden-/TeilnehmerID, des Teilnehmerzustands sowie der Authentifikationssignatur wird innerhalb des Ablaufschritts „Überprüfung der Authentizität des EBICS-Requests“ durchgeführt (für Einzelheiten siehe Kapitel 5.5.1.2.1, Abbildung 42).

Die EU-Datei muss eine gültige Elektronische Unterschrift des zu sperrenden Teilnehmers über eine Datei mit einem Leerzeichen enthalten.

Die anschließende eigentliche synchrone Durchführung der Sperrung des Teilnehmers liefert keine weiteren spezifischen technischen oder bankfachlichen Fehler.

4.6 Schlüsseländerungen

4.6.1 Änderung der Teilnehmerschlüssel

Bis zur EBICS-Version 2.3 mussten die Schlüsseländerungen über die beiden administrativen Auftragsarten PUB (Änderung des bankfachliches Schlüssels) und HCA (Änderung des Authentifikationsschlüssels und des Verschlüsselungsschlüssels) durchgeführt werden, die unabhängig voneinander ausgeführt werden konnten. Um das Key-Management sowohl für die Kunden- als auch für die Bankseite zu vereinfachen, wurde mit EBICS-Version 2.4 die Auftragsart HCS eingeführt, mit der alle 3 Schlüssel in einer Transaktion geändert werden können. HCS umfasst fachlich daher PUB und HCA. HCS bzw. PUB und HCA erfordern jeweils die bankfachliche EU des betroffenen Teilnehmers in der jeweils unterstützen EU-Version (z.B. A005, A006), nicht jedoch den zusätzlichen Versand von Initialisierungsbriefen. Aus Kompatibilitätsgründen können die administrativen Auftragsarten PUB und HCA auch weiterhin alternativ zu HCS genutzt werden.
Abhängig vom seinem Zustand hat ein Teilnehmer zwei Möglichkeiten, seine öffentlichen Teilnehmerschlüssel auf dem Banksystem zu aktualisieren:

- Im Zustand “Gesperrt” MUSS grundsätzlich eine Teilnehmerinitialisierung vorgenommen werden, um bankfachliche Aufträge über EBICS übermitteln zu können. Die Sperrung der Zugangsberechtigung mit anschließender Teilnehmerinitialisierung ist somit eine Alternative, Teilnehmerschlüssel zu aktualisieren. Teilnehmerinitialisierung erfolgt unter Verwendung der administrativen Auftragsarten INI und HIA und erfordert den zusätzlichen Versand von Initialisierungsbriefen. Der Ablauf der Teilnehmerinitialisierung ist in Kapitel 4.4.1 beschrieben. Informationen zum Thema Sperrung der Zugangsberechtigung eines Teilnehmers sind in Kapitel 4.5 enthalten
- Im Zustand “Bereit” ist es den Teilnehmern möglich, ihre öffentlichen Teilnehmerschlüssel unter Verwendung der administrativen Auftragsarten PUB, HCS und HCA zu aktualisieren, ohne den Umweg über die Teilnehmerinitialisierung gehen zu müssen. PUB, HCS und HCA erfordern jeweils die EU des betroffenen Teilnehmers, nicht jedoch den zusätzlichen Versand von Initialisierungsbriefen. Das vereinfacht einerseits das Verfahren der Schlüsseländerung, andererseits entfällt dadurch die Möglichkeit, Initialisierungsbriefe für die Dokumentation der Keyhistorie von Teilnehmern einzusetzen. Es liegt in der Verantwortung des Kreditinstituts, die Schlüsseländerungen mittels PUB, HCS und HCA

so zu dokumentieren, dass sie nachvollziehbar bleiben.

Gegenstand dieses Kapitels ist die detaillierte Beschreibung der Schlüsseländerung mittels PUB, HCS und HCA.

4.6.1.1 Allgemeine Beschreibung

Teilnehmer im Zustand „Bereit“ können ihre öffentlichen Teilnehmerschlüssel unter Verwendung der folgenden administrativen Auftragsarten aktualisieren:

- **PUB:** Aktualisierung des öffentlichen bankfachlichen Schlüssels
- **HCA:** Aktualisierung des öffentlichen Authentifikationsschlüssels und des öffentlichen Verschlüsselungsschlüssels.
- **HCS:** Aktualisierung des öffentlichen bankfachlichen Schlüssels, des öffentlichen Authentifikationsschlüssels und des öffentlichen Verschlüsselungsschlüssels.

PUB, HCS und HCA sind reguläre Upload-Transaktionen, deren Ablauf (einschließlich des Verhaltens in Fehlersituationen) in Kapitel 5.5 detailliert beschrieben ist. Darin enthalten ist die Abbildung 47, die den Ablauf der bankseitigen Verarbeitung der EBICS-Requests während der Datentransferphase einer Upload-Transaktion beschreibt. Teil dieses Ablaufs ist der Ablaufschritt „Überprüfung und Durchführung des Auftrags“. Für die administrative Auftragsart PUB liefert dieser Schritt die folgenden Fehlercodes:

- EBICS_KEYMGMT_UNSUPPORTED_VERSION_SIGNATURE
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten eine unzulässige Version des Verfahrens zur bankfachlichen Signatur enthalten
- EBICS_KEYMGMT_KEYLENGTH_ERROR_SIGNATURE
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten einen bankfachlichen Schlüssel unzulässiger Länge enthalten
- EBICS_INVALID_ORDER_DATA_FORMAT
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten nicht dem dafür vorgesehenen Format entsprechen (siehe Kapitel 4.4.1.2.5.1)
- EBICS_USER_UNKNOWN
  Dieser technische Fehler tritt dann auf, wenn der Teilnehmer, der Bestandteil der Nutzdaten von PUB ist, kein registrierter Teilnehmer ist
- EBICS_INVALID_USER_STATE

  Dieser technische Fehler tritt dann auf, wenn der Teilnehmer, welcher Bestandteil der Nutzdaten von PUB ist, sich nicht im Zustand „Bereit“ befindet
- EBICS_SIGNATURE_VERIFICATION_FAILED
  Dieser fachliche Fehler tritt auf, wenn die EU des betroffenen Teilnehmers nicht erfolgreich verifiziert werden konnte.

  Für Return Codes, die CA-Zertifikate betreffen, wird auf Anhang 1 verwiesen.

  Für HCA liefert der Ablaufschritt „Prüfung und Durchführung des Auftrags“ die folgenden Fehlercodes:

- EBICS_KEYMGMT_UNSUPPORTED_VERSION_ENCRYPTION
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten eine unzulässige Version des Verschlüsselungsverfahrens enthalten
- EBICS_KEYMGMT_UNSUPPORTED_VERSION_AUTHENTICATION
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten eine unzulässige Version des Verfahrens der Authentifikationssignatur enthalten
- EBICS_KEYMGMT_KEY_LENGTH_ERROR_ENCRYPTION
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten einen Verschlüsselungsschlüssel unzulässiger Länge enthalten
- EBICS_KEYMGMT_KEY_LENGTH_ERROR_AUTHENTICATION
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten einen Authentifikationsschlüssel unzulässiger Länge enthalten
- EBICS_INVALID_ORDER_DATA_FORMAT
  Dieser fachliche Fehler tritt dann auf, wenn die Nutzdaten nicht dem dafür vorgesehenen Format entsprechen (siehe Kapitel 4.4.1.2.5.1)
- EBICS_USER_UNKNOWN
  Dieser technische Fehler tritt dann auf, wenn der Teilnehmer, der Bestandteil der Nutzdaten von HCA ist, kein registrierter Teilnehmer ist
- EBICS_INVALID_USER_STATE
  Dieser technische Fehler tritt dann auf, wenn der Teilnehmer, welcher Bestandteil der Nutzdaten von HCA ist, sich nicht im Zustand „Bereit“ befindet
- EBICS_SIGNATURE_VERIFICATION_FAILED
  Dieser fachliche Fehler tritt auf, wenn die EU des betroffenen Teilnehmers nicht erfolgreich verifiziert werden konnte.
- Für Return Codes, die CA-Zertifikate betreffen, wird auf Anhang 1 verwiesen.

Da HCS eine Kombination aus PUB und HCA ist, sind im Ablaufschritt „Überprüfung und Durchführung des Auftrags“ alle unter PUB und HCA genannten Fehlercodes möglich.

Entweder PUB und HCA oder HCS müssen von dem Teilnehmer eingereicht werden, dessen Schlüssel aktualisiert werden sollen. PUB, HCS und HCA erfordern jeweils genau eine EU, die von dem Teilnehmer geleistet werden muss, dessen Schlüssel aktualisiert werden sollen. Die Unterschriftsklasse dieser EU ist unerheblich.

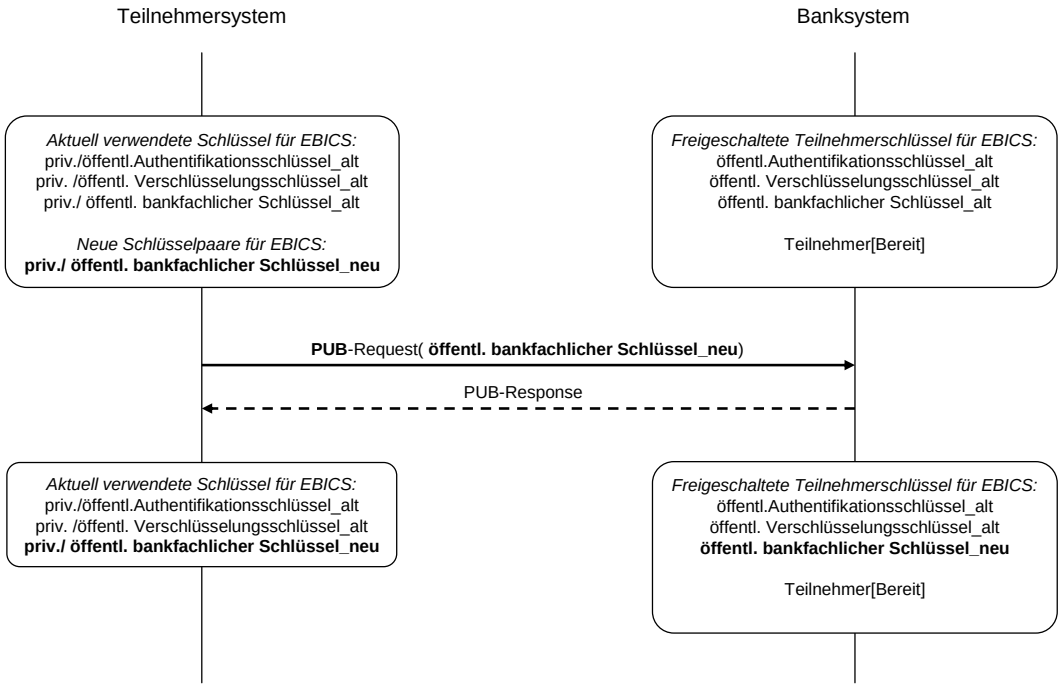


Abbildung 29: Änderung des bankfachlichen Teilnehmerschlüssels mittels PUB

Abbildung 29 stellt den Zustand der öffentlichen Teilnehmerschlüssel und des Teilnehmers vor und nach der Durchführung von PUB dar. Für die Durchführung von PUB gilt:

- Die Nutzdaten, d.h. der neue öffentliche bankfachliche Schlüssel des Teilnehmers, wird komprimiert, verschlüsselt und schließlich base64-kodiert in die EBICS-Nachrichten eingebettet
- Die Nutzdaten werden von dem Teilnehmer mittels EU signiert, dessen öffentlicher bankfachlicher Schlüssel aktualisiert werden soll. Für diese EU wird der alte, zu dem Zeitpunkt jedoch freigeschaltete bankfachliche Schlüssel dieses Teilnehmers verwendet.

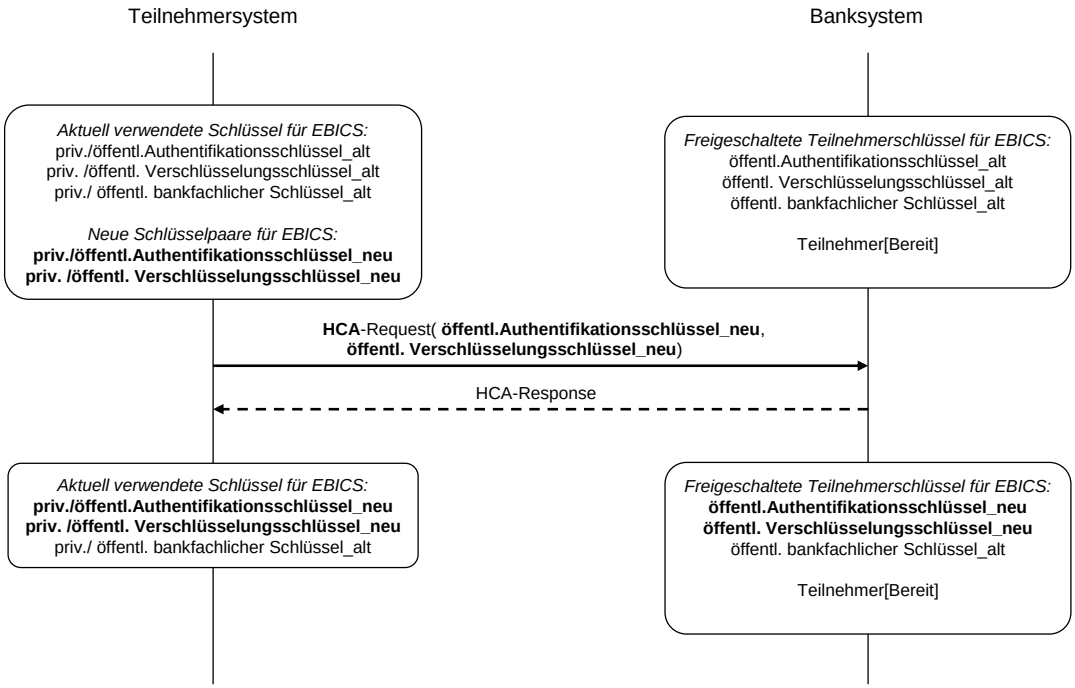


Abbildung 30: Änderung des Authentifikationsschlüssels sowie des Verschlüsselungsschlüssels mittels HCA

In Abbildung 30 ist der Zustand der Teilnehmerschlüssel und des Teilnehmers vor und nach der Durchführung von HCA dargestellt. Zudem gilt für die Durchführung von HCA:

- Die Nutzdaten, d.h. der neue öffentliche Authentifikationsschlüssel des Teilnehmers sowie dessen neuer öffentlicher Verschlüsselungsschlüssel, werden komprimiert, verschlüsselt und schließlich base64-kodiert in die EBICS-Nachrichten eingebettet
- HCA-Requests enthalten die Authentifikationssignatur des betroffenen Teilnehmers oder eines technischen Teilnehmers. Die Authentifikationsunterschrift des betroffenen Teilnehmers wird mit dem alten, zu dem Zeitpunkt jedoch freigeschalteten Authentifikationsschlüssel erzeugt. Die EBICS-Responses des Kreditinstituts enthalten die Authentifikationssignatur des Kreditinstituts.

Mittels HCS werden alle Schlüssel geändert. HCS ist alternativ zu PUB und HCA zu verstehen, bei denen die Schlüssel für die bankfachliche Elektronische Unterschrift (PUB) und für die Authentifikationssignatur und Verschlüsselung (HCA) getrennt geändert werden. Demnach sieht der Ablauf wie folgt aus:

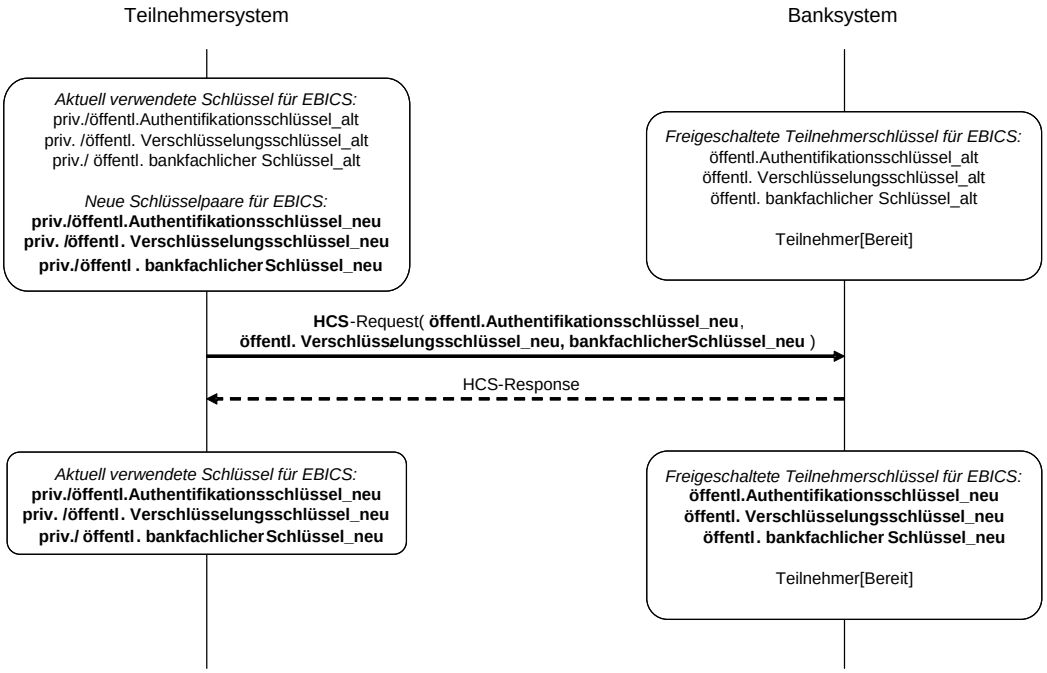


Abbildung 31: Änderung des bankfachlichen Teilnehmerschlüssels sowie Authentifikationsschlüssel und Verschlüsselungsschlüssel mittels HCS

4.6.1.2 Format der Auftragsdaten

Bei Verwendung der EU in strukturierter Form (ab dem Signaturverfahren A005/A006) sind die Auftragsdaten für PUB ein zu ebics_signature.xsd konformes Instanzdokument, das aus dem Toplevel-Element `SignaturePubKeyOrderData` besteht (analog INI, XML-Darstellung siehe daher auch Kapitel 4.4.1.2.5)

`SignaturePubKeyOrderData` ist mittels XML-Schema wie folgt definiert:

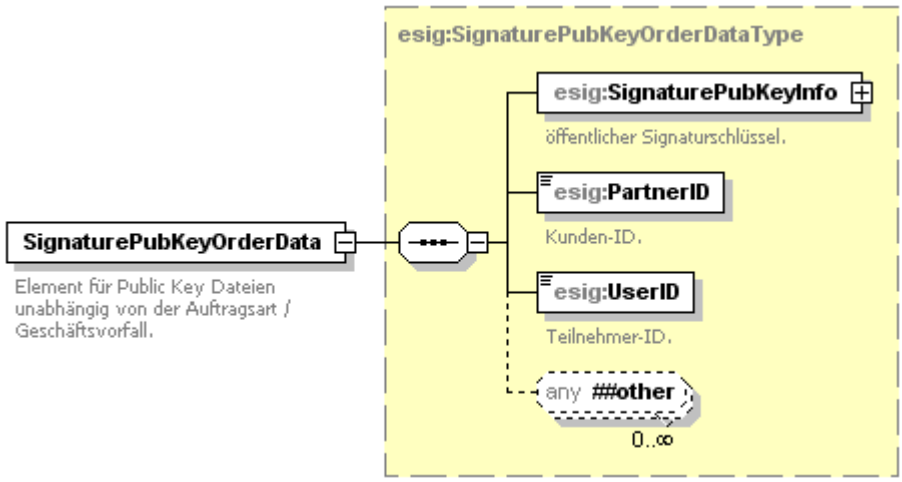


Abbildung 32: Definition des XML-Schema-Elements SignaturePubKeyOrderData der Auftragsdaten für PUB (identisch mit INI, siehe eigenes Kapitel)

Die Auftragsdaten für HCA sind ein zu ebics_orders_H005.xsd konformes Instanzdokument, das aus dem Toplevel-Element `HCARequestOrderData` besteht. `HCARequestOrderData` ist mittels XML-Schema wie folgt definiert:

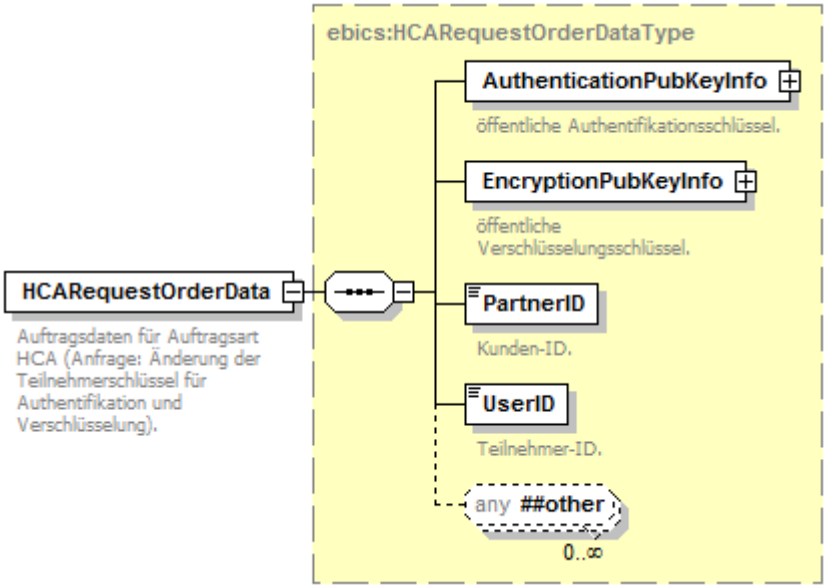


Abbildung 33: Definition des XML-Schema-Elements HCARequestOrderData der Auftragsdaten für HCA

Die Auftragsdaten für HCS sind ein zu ebics_orders_H005.xsd konformes Instanzdokument, das aus dem Toplevel-Element `HCSRequestOrderData` besteht. `HCSRequestOrderData` ist mittels XML-Schema wie folgt definiert:

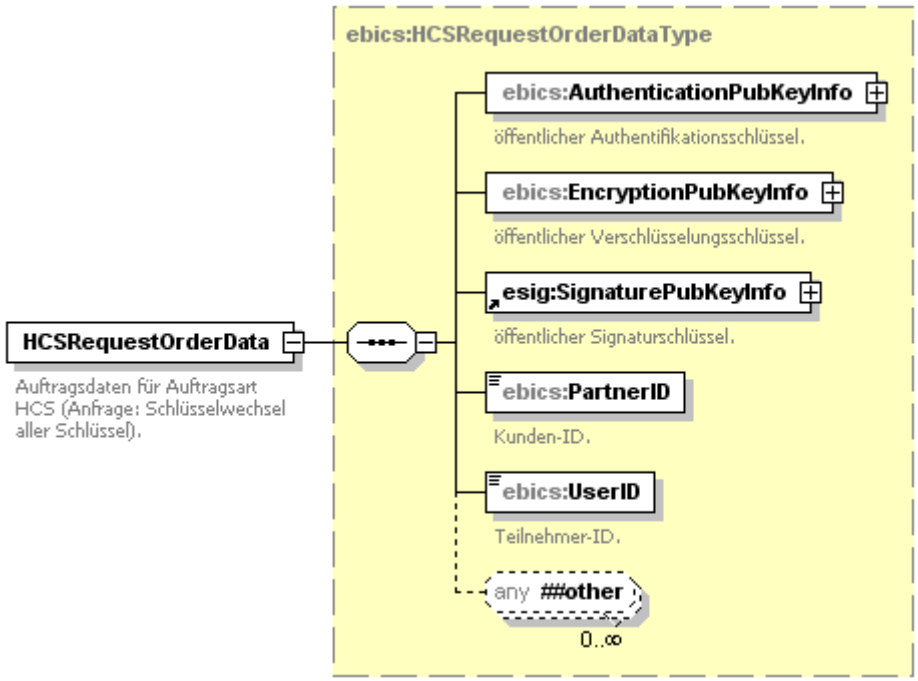


Abbildung 34: Definition des XML-Schema-Elements HCSRequestOrderData der Auftragsdaten für HCS

Die Auftragsdaten von PUB, HCS und HCA werden komprimiert, verschlüsselt und base64-kodiert in den entsprechenden EBICS-Request eingebettet.

4.6.2 Änderung der Bankschlüssel

Der Ablauf für die Aktualisierung der Bankschlüssel ist nicht Bestandteil des vorliegenden Standards. Die Gültigkeitsdauer der Bankschlüssel ist nicht Teil der EBICS-Schnittstelle. Aus Sicht des EBICS-Protokolls existiert zu jedem Zeitpunkt und zu jeder zulässigen Kombination von Verfahren der Authentifikationssignatur, der Verschlüsselung und der EU ein Satz aktuell gültiger Bankschlüssel. In der Version H005 besteht dieser aus genau den folgenden Schlüsseln:

- privater/ öffentlicher Verschlüsselungsschlüssel für das Verfahren E002
- privater / öffentlicher Authentifikationsschlüssel für das Verfahren X002
- privater/öffentlicher bankfachlicher Schüssel für das Verfahren A005 oder A006.

In EBICS gibt es keine Übergangsfristen, in denen mehrere Schlüssel für dasselbe Verfahren gültig sind. Auf Bankseite geänderte Schlüssel werden in EBICS sofort gültig.

Der Teilnehmer ist für die Abholung der jeweils aktuellen Bankschlüssel über HPB verantwortlich. Nach der Durchführung von HPB ist der Zustand der Bankschlüssel auf Teilnehmerseite gleich „Neu“. In diesem Zustand dürfen die Bankschlüssel (noch) nicht in der Kommunikation über EBICS eingesetzt werden. Das Kreditinstitut MUSS die neuen Schlüssel und/oder deren Hashwerte auf einem zweiten, unabhängigen Kommunikationsweg zugängig machen. Der Teilnehmer MUSS wie bei der erstmaligen

Abholung der Bankschlüssel unmittelbar nach der Initialisierung den Abgleich der über unterschiedliche Kommunikationswege erhaltenen Schlüssel vornehmen. Nach der erfolgreichen Überprüfung der Bankschlüssel ist deren Zustand auf Teilnehmerseite „Freigeschaltet“. Im Zustand „Freigeschaltet“ können die Bankschlüssel in der Kommunikation über EBICS verwendet werden.

Um sicherzustellen, dass der Teilnehmer über die aktuellen Bankschlüssel verfügt, sieht der Ablauf einer EBICS-Transaktion (mit Ausnahme von INI, HIA, HPB, HSA) im ersten EBICS-Request die Übermittlung der Hashwerte der öffentlichen Schlüssel des Kreditinstituts (XML-Struktur `ebicsRequest/header/static/BankPubKeyInfo`) vor, über die der Teilnehmer verfügt. Das Banksystem überprüft die Aktualität dieser Schlüssel und liefert das Ergebnis dieser Überprüfung an den Teilnehmer zurück. Sollte einer dieser Schlüssel nicht mehr aktuell sein, wird die Transaktion mit dem technischen Returncode EBICS_BANK_PUBKEY_UPDATE_REQUIRED abgebrochen. Der Teilnehmer muss dann die Bankschlüssel mit HPB abholen.

In der Version „H005“ des EBICS-Protokolls ist die EU der Kreditinstitute nur vorgesehen (siehe Kapitel 3.5.2). Die XML-Struktur `BankPubKeyInfo` enthält in Vorbereitung zukünftiger EBICS-Versionen den Hashwert des öffentlichen bankfachlichen Schlüssels mit der maximalen Häufigkeit gleich 0. Weitere Einzelheiten zur Überprüfung der Hashwerte sind in Kapitel 5.5.1.2 beschrieben.

4.7 Übergang zu größeren Schlüssellängen

Um die Sicherheit des RSA-Verfahrens zu gewährleisten, müssen die Schlüssellängen kontinuierlich erhöht werden. Siehe dazu etwa die regelmäßigen Veröffentlichungen der „Übersicht über geeignete Algorithmen“ der Regulierungsbehörde für Telekommunikation und Post.

Gegenstand dieses Kapitels ist der Übergang zu Schlüsseln größerer Länge in EBICS.

In Version H005 müssen bankfachlichen Schlüssel mit einer Mindestlänge von 2048 Bit verwendet werden.

EBICS legt in der Version „H005“ für Authentifikationsschlüssel und Verschlüsselungsschlüssel eine Mindestlänge von 2048 Bit (=2KBit) und eine Maximallänge von 16 KBit fest. Die Mindestlänge muss angepasst werden, sobald Schlüssel dieser Länge aus Sicherheitsgründen nicht mehr verwendet werden sollen. Die Maximallänge muss angepasst werden, sobald Schlüssel der Länge größer als diese Maximallänge erlaubt unterstützt werden sollen.

Die Nutzdatenformate der administrativen Auftragsarten HIA, HPB, HCA und HCS ermöglichen beliebig große Schlüssellängen. Das bedeutet, dass diese Nutzdatenformate nach einer Erhöhung der Schlüssellängen keiner Anpassung bedürfen.

Neue öffentliche Authentifikationsschlüssel oder Verschlüsselungsschlüssel größerer Länge werden dem Banksystem über HIA, HCS oder HCA genauso übermittelt wie neue Authentifikationsschlüssel gleich bleibender Länge.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Ebenso werden die neuen öffentlichen Schlüssel des Kreditinstituts über HPB abgeholt, unabhängig davon, ob die Länge des Authentifikationsschlüssels oder des Verschlüsselungsschlüssels des Kreditinstituts sich verändert hat.

4.8 Zusammenfassung

In der nachfolgenden Tabelle sind die wichtigsten Merkmale der(administrativen) Auftragsarten des Schlüsselmanagements zusammengefasst:

Administrative Auftrags-art	**Auftragsdatenformat**	**Authentifikationssignatur Teilnehmer / Kreditinstitut**	**EU der Auftragsdaten**
INI	INI-Datei (gemäß Anhang Kapitel 14)	nein/nein	nein
HIA	`ebics: HIARequestOrder» Data`	nein/nein	nein
HSA	`ebics: HSARequestOrder» Data`	nein/nein	ja
HPB	`ebics: HPBRequestOrder» Data`	ja/nein	nein
PUB	siehe INI	ja/ja	ja
HCA	`ebics: HCARequestOrder» Data`	ja/ja	ja
HCS	`ebics: HCSRequestOrder» Data`	ja/ja	ja
H3K	`ebics: H3KRequestOrder» Data`	nein/nein	ja (Zertifikat)
SPR	Schlüsselmanagement-Auftrag, der nur eine EU enthält	ja/ja	ja

5 EBICS-Transaktionen

Die Beschreibungen und Festlegungen dieses Kapitels gelten für alle Geschäftsvorfälle (identifiziert über BTF) und alle administrativen Aufträge mit Ausnahme der nachfolgenden administrativen Schlüsselmanagement-Auftragsarten: INI, HIA, HPB, PUB, SPR, HCA, H3K und HCS.

5.1 Allgemeine Festlegungen

5.1.1 EBICS-Transaktionen

EBICS-Transaktionen dienen der Übermittlung von Aufträgen an das bankfachliche Zielsystem. Entsprechend der Einteilung von Aufträgen in Sende- und Abholaufträge unterscheidet EBICS zwischen Upload- und Download-Transaktionen: Upload-Transaktionen übertragen bankfachliche Nutzdaten und/oder bankfachliche Unterschriften an das bankfachliche Zielsystem, umgekehrt werden im Rahmen einer Download-Transaktion bankfachliche Nutzdaten und/oder bankfachliche Unterschriften vom bankfachlichen Zielsystem abgeholt.

5.1.2 Transaktionsphasen und Transaktionsschritte

Jede EBICS-Transaktion durchläuft unterschiedliche Transaktionsphasen. Die Phasen einer Upload-Transaktion sind Initialisierung und Datentransfer, die Phasen einer Download-Transaktion Initialisierung, Datentransfer und schließlich Quittierung. Eine Transaktionsphase kann aus einem Transaktionsschritt oder mehreren zusammenhängenden Transaktionsschritten bestehen, wobei unter einem Transaktionsschritt ein Paar aus EBICS-Request und zugehöriger EBICS-Response zu verstehen ist. So umfasst die Initialisierung den ersten Transaktionsschritt, der Datentransfer kann sich hingegen über mehrere weitere Transaktionsschritte erstrecken, in denen jeweils ein Nutzdatensegment übertragen wird.

EBICS-Transaktionen können aus einem einzigen Transaktionsschritt bestehen, beispielsweise wenn sie zu einem Auftrag nur die bankfachlichen elektronischen Unterschriften übermitteln.

5.1.3 Durchführung der Aufträge

5.1.3.1 Zeitliche Abhängigkeiten zwischen Übermittlung und Durchführung von Sendeaufträgen

EBICS unterstützt die zeitliche Entkopplung der Einreichung von bankfachlichen Sendeaufträgen über EBICS von deren eigentlichen Ausführung auf den Backendsystemen des Kreditinstituts. Die innerhalb einer EBICS-Transaktion eingereichten EUs und Nutzdatensegmente werden zunächst vorverarbeitet. Diese **Vorverarbeitung** ist nicht Bestandteil von EBICS, sie ist abhängig von der Implementierung des Banksystems, die Zwischenspeicherung der Nutzdatensegmente ist beispielsweise ein Teil davon. Nach der Übertragung des letzten Nutzdatensegments werden die gesamten Nutzdaten, Auftragsparameter sowie EUs zunächst an eine Komponente des Banksystems weitergereicht, deren Verantwortlichkeit in

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

der **Verwaltung offener Aufträge** liegt. Die Realisierung ist abhängig von der Implementierung des Banksystems, sie ist nicht Bestandteil von EBICS.

Im Unterschied zu den bankfachlichen Sendeaufträgen wird gefordert, dass die **Durchführung** der Sendeaufträge administrativer Auftragsarten vor dem Senden der letzten EBICS-Response der Upload-Transaktion abgeschlossen sein MUSS. Diese Forderung gilt neben den (administrativen) Auftragsarten des Schlüsselmanagements auch für die Abholaufträge der Auftragsarten der VEU (EDS), um das Verfahren der Verteilten EU möglichst effizient abwickeln zu können und beteiligten Teilnehmern einen möglichst aktuellen Status der verteilten EUs zu einem Auftrag liefern zu können.

5.1.3.2 Zeitliche Abhängigkeiten zwischen Übermittlung und Durchführung von Abholaufträgen

Die Abholdaten sind Bestandteil der EBICS-Responses des Kreditinstituts. Mit jedem EBICS-Transaktionsschritt stellt das Banksystem ein weiteres Nutzdatensegment bereit. Um den Abholvorgang zu beschleunigen, können die Abholdaten vom Banksystem im Voraus erzeugt (wie beispielsweise im Falle von Kontoauszügen) oder erst auf Anfrage hin generiert werden.

5.1.4 Transaktionsverwaltung

Die Steuerung des Ablaufs einer EBICS-Transaktion obliegt im Normalfall dem Kundensystem, die einzelnen Transaktionsschritte einer EBICS-Transaktion werden jeweils vom Kundensystem initiiert. Das Banksystem kann in Sonderfällen den Ablauf einer Transaktion ebenfalls steuern, z.B. indem es dem Kundensystem im Recovery-Fall einen möglichen Wiederaufsetzpunkt mitteilt.

Im Banksystem müssen die EBICS-Transaktionen verwaltet werden, um Folgendes zu ermöglichen:

Zuordnung der einzelnen Transaktionsschritte zu einer bestimmten EBICS-Transaktion

Aufzeichnung des Ablaufs der EBICS-Transaktion zur Verwaltung der Transaktionszustände mit dem Ziel, das Fortschreiten der EBICS-Transaktion sicherzustellen.

- Wiederaufsetzen einer EBICS-Transaktion.

Daraus ergeben sich die folgenden Verantwortlichkeiten für die EBICS-Transaktionsverwaltung des Banksystems:

Erzeugung von EBICS-Transaktionen während der Transaktionsinitialisierung. Siehe dazu Kapitel 5.2.

Abbruch von EBICS-Transaktionen, wenn die Weiterführung aufgrund von Fehlersituationen nicht sinnvoll oder nicht möglich ist

Beenden von EBICS-Transaktionen, wenn alle Transaktionsschritte erfolgreich durchgeführt werden konnten

Kontrolle des Ablaufs von EBICS-Transaktionen, um deren Ablauf gemäß Kapitel 5.5.1 sicherzustellen

Unterstützung des Verfahrens zum Wiederaufsetzen von EBICS-Transaktionen gemäß Kapitel 5.5.2 und 5.6.2, falls das Banksystem Recovery unterstützt.

5.2 Zuordnung von EBICS-Request zu EBICS-Transaktion

Die erste Phase jeder EBICS-Transaktion ist die Initialisierung. Sie wird durch den ersten EBICS-Request der Transaktion ausgelöst und umfasst:

Prüfungen, deren erfolgreiche Durchführung die notwendige Voraussetzung für die Annahme des Auftrags durch das Kreditinstitut ist

Weitere Verarbeitungsschritte, die notwendig sind, um EBICS-Transaktionen, die aus mehr als einem Transaktionsschritt bestehen, in die Transaktionsverwaltung aufzunehmen.

Beispiele solcher Prüfungen sind die Prüfung des Zustands und der Berechtigung des Teilnehmers für bestimmte BTF-Parameter, der den Auftrag einreicht. Der genaue Umfang dieser Prüfungen/ Verarbeitungsschritte ist für Upload-Transaktionen in Kapitel 5.5.1.2.1 und für Download-Transaktionen in Kapitel 5.6.1.2.1 beschrieben.

Wurden alle notwendigen Prüfungen erfolgreich durchgeführt und besteht die Transaktion aus mehreren Transaktionsschritten, so erzeugt die Transaktionsverwaltung des Banksystems eine EBICS-Transaktion mit einer banksystemweit eindeutigen TransaktionsID (Details zur Generierung der TransaktionsID sind im Anhang (Kapitel 11.6) enthalten). Diese wird dem Teilnehmer über die Antwortnachricht des Kreditinstituts mitgeteilt. Die Transaktionsverwaltung des Banksystems ordnet dieser Transaktion die folgenden Daten fest zu, die Bestandteil der Headerdaten des EBICS-Requests sind:

KundenID/ TeilnehmerID/ technische TeilnehmerID

Administrative Auftragsart

- Auftragsparameter (hängt ab von der Art der Transaktion, die administrativ oder fachlich sein kann, siehe 3.11)
- Auftragsnummer (nur erlaubt bei den administrativen Auftragsarten HVE und HVS)
  Die Auftragsnummer ist nur vorhanden wenn sich die Datei, die zur Bank übertragen wird, auf einen Auftrag bezieht, zu dem es schon eine Auftragsnummer gibt (nur der Fall bei der Übertragung einer EU-Datei mittels HVE und HVS), um diese Dateien abzugleichen. Grundsätzlich ist die Auftragsnummer ein Teil der Header-Daten der EBICS-Response (von Sendevorgängen). Sie wird vom Bankrechner automatisch generiert.

Diese Daten sind der Transaktion fest zugeordnet und können im Verlauf der Transaktion nicht verändert werden.

Außerhalb der Initialisierung enthalten EBICS-Requests diese TransaktionsID für die Zuordnung zur passenden EBICS-Transaktion. Insgesamt enthalten sie die folgenden den Transaktionsschritt identifizierenden Elemente:

- die banksystemweit eindeutige TransaktionsID

- die Transaktionsphase (Initialisierung, Datentransfer, Quittierung) innerhalb der Transaktion
- die laufende Nummer des Datensegments bankfachlicher Daten, falls es sich um die Transaktionsphase Datentransfer handelt.

Die detaillierte Beschreibung des Aufbaus der EBICS-Requests von Upload-Transaktionen und Download-Transaktionen ist in den Kapiteln 5.5.1.1 bzw. 5.6.1.1 enthalten.

5.3 Vorabprüfung von Aufträgen [optional]

Um zu vermeiden, dass ein Teilnehmer große Datenbestände an das Banksystem überträgt und auf Bankseite erst nach der Übertragung festgestellt wird, dass die Unterzeichner des Sendeauftrags nicht über die erforderlichen Berechtigungen verfügen, KANN das Banksystem optional die Funktionalität der Vorabprüfung unterstützen. Die Angabe, ob ein Banksystem Vorabprüfungen durchführt, ist Bestandteil seiner abrufbaren Bankparameter (siehe dazu Kapitel 12.2). Wird die Vorabprüfung von Sendeaufträgen unterstützt, so ist der Umfang der Vorabprüfung die Entscheidung der einzelnen Kreditinstitute. Möglich ist die Unterstützung einer oder mehrerer der folgenden Prüfungen:

Kontoberechtigungsprüfung
Mit Hilfe der Kontoberechtigungsprüfung wird überprüft, ob für jeden Unterzeichner die folgende Bedingung erfüllt ist:

- der Unterzeichner hat das Recht, eine EU der Mindestklasse „B“ für Aufträge der gegebenen BTF-Parameterwerte für jedes der Auftraggeberkonten des gegebenen Auftrags zu leisten.

Limitprüfung
Durch die Limitprüfung wird sichergestellt, dass für jeden Unterzeichner die folgende Bedingung erfüllt ist:

- der Unterzeichner hat das Recht, eine EU der Mindestklasse „B“ für Aufträge der gegebenen BTF-Parameterwerte und in der jeweils vorliegenden Höhe für jedes der Auftraggeberkonten des gegebenen Auftrags zu leisten.

Prüfung der EUs
Die EU-Prüfung verifiziert die EUs der Unterzeichner des Auftrags und überprüft, ob die EUs jeweils von unterschiedlichen Teilnehmern stammen.
Für eine erfolgreiche Vorabprüfung benötigt der Teilnehmer eine der Unterschriftsklassen ‚E’, ‚A’ oder ‚B’. Wenn Aufträge nur eingereicht werden (also Unterschriftsklasse ‚T’), wird die Vorabprüfung nicht durchlaufen. Der Returncode EBICS_SIGNATURE_VERIFICATION_FAILED wird zurückgeliefert, wenn die Signatur nicht gültig ist.

Wenn der Unterzeichner den vorliegenden Auftrag bereits unterschrieben hat, wird der Returncode EBICS_DUPLICATE_SIGNATURE zurückgeliefert.

Die Vorabprüfung eines Sendeauftrags ist Teil des ersten Transaktionsschritts im Rahmen der entsprechenden Upload-Transaktion. Das Ergebnis der Vorabprüfung steht im bankfachlichen Returncode der entsprechenden EBICS-Response des ersten Transaktionsschritts. Die Vorabprüfung erfolgt vor der Übertragung der Nutzdaten des Auftrags auf Basis der Angaben des Kundensystems

über die noch ausstehenden Auftragsdaten. Sie ersetzt nicht die entsprechenden Prüfungen auf Grundlage der eigentlichen Nutzdaten nach deren Übertragung an das Banksystem.

Das Kundensystem KANN den Umfang der Vorabprüfungen weiter einschränken. Die Vorabprüfungen Kontoberechtigungsprüfung, Limitprüfung oder bankfachliche EU-Prüfung werden bankseitig nur dann ausgeführt, wenn die dazu notwendigen Daten vom Kundensystem zur Verfügung gestellt werden. Die Daten der Vorabprüfung werden im ersten EBICS-Request einer Upload-Transaktion über das (optionale) Element `ebicsRequest/body/PreValidation` (siehe ebics_request_H005.xsd) übermittelt. Derer Typ heißt `PreValidationRequestType` (siehe ebics_types_H005.xsd) und ist eine Liste aus den folgenden optionalen Elementen:

- `DataDigest`

  Dieses Element enthält den Hashwert der Auftragsdaten, die von den Unterzeichnern des Auftrags mittels Transportunterschrift oder bankfachlicher EU signiert wurden. Bei der Vorabprüfung eines Auftrags erfolgt die Prüfung der EUs allein auf Basis dieses Hashwerts, dessen Korrektheit zum Zeitpunkt der Vorabprüfung nicht überprüft werden kann. Für das von den Unterzeichnern des Auftrags verwendete (und bzgl. EBICS von der Bank unterstützte) Signaturverfahren kann ein Hashwert eingestellt werden, welcher im Hashverfahren des jeweiligen Signaturverfahrens zu bilden ist. Das zugehörige Signaturverfahren wird anhand des Attributs `SignatureVersion` identifiziert. `DataDigest` kann mehrfach vorkommen, wenn die Unterzeichner verschiedene Unterschriftsversionen verwenden (das ist der Fall, wenn nicht alle Teilnehmer eines Kunden mit dem gleichen Signaturverfahren signieren. Daher ist die korrekte Befüllung des Attributes `SignatureVersion` je Wert von `DataDigest` wichtig.

- `AccountAuthorisation`

  Dieses Element enthält ein Auftraggeberkonto des gegebenen Auftrags.
  Es müssen ein oder zwei Kontonummern angegeben werden unter:
  - AccountAuthorisation/AccountNumber:
  Hier ist die Angabe des deutschen und/oder internationalen Formats (IBAN) möglich
  - AccountAuthorisation/NationalAccountNumber:
  Hier ist die Angabe in freiem Format für Kontonummern möglich, die weder den deutschen noch den nationalen Vorgaben entsprechen.

  Es müssen eine oder zwei Bankleitzahlen angegeben werden unter:
  - AccountAuthorisation/BankCode
  Hier ist die Angabe des deutschen und/oder internationalen Formats (BIC) möglich
  - AccountAuthorisation/NationalBankCode
  Hier ist die Angabe in freiem Format für Bankleitzahlen möglich, die weder den deutschen noch den nationalen Vorgaben entsprechen.

  Optional kann auch der Kontoinhaber (`AccountAuthorisation/AccountHolder`) mitgeliefert werden. Diese Kontoinformationen werden von der Kontoberechtigungsprüfung und der

Limitprüfung benötigt. Für die Limitprüfung ist zudem die Angabe der Summe der Einzelaufträge zu dem gegebenen Auftraggeberkonto erforderlich. Dieser Betrag ist in dem (optionalen) Element `AccountAuthorisation/Amount` enthalten. Die Währung des Betrags ist der Wert des optionalen Attributs `AccountAuthorisation/Amount@Currency`, falls dieses vorhanden ist. Sonst ist die Währung der Wert des Attributs `AccountAuthorisation@Currency`, welches die Währung des Kontos beinhaltet.

Die einzelnen Vorabprüfungen benötigen die TeilnehmerID/ KundenID jedes einzelnen Unterzeichners. Diese sind Bestandteil des XML-Typs OrderSignature, welcher zur Darstellung einzelner EUs verwendet wird. Für weitere Einzelheiten zur Einbettung von EUs in EBICS-Nachrichten siehe Kapitel 3.5.

5.4 Recovery – Wiederaufsetzen von Transaktionen [optional]

In diesem Kapitel werden die Grundzüge der Recovery-Vorgehensweise beschrieben, die sowohl für Upload- als auch für Download-Transaktionen gelten.

Mit dem Recovery-Mechanismus von EBICS wird erreicht, dass bereits vom Kunden- oder Banksystem empfangene Auftragsdaten einer Transaktion nicht erneut übertragen werden müssen, wenn eine der folgenden Fehlersituationen eintritt:

- Transportfehler
- Verarbeitungsfehler der EBICS-Nachricht, die die Auftragsdaten enthält:
  Bei Upload-Transaktionen handelt es sich um Verarbeitungsfehler der EBICS-Requests, die auf Bankseite auftreten können, bei Download-Transaktionen um Verarbeitungsfehler der EBICS-Responses, die auf Seite des Kunden/ Teilnehmers auftreten. Beispielsweise können Fehler bei der (Zwischen-) Speicherung der Nutzdaten auftreten.

  Recovery ist ein wichtiger Aspekt des Protokolls, da die Größenordnung der Auftragsdaten durchaus im Bereich von mehreren hundert MegaByte liegen kann.

  Der Mechanismus setzt die Kenntnis der TransaktionsID der betroffenen EBICS-Transaktion voraus und basiert auf der Definition von **Wiederaufsetzpunkten** von Transaktionen:

- Bei Upload-Transaktionen ist der Wiederaufsetzpunkt der letzte Transaktionsschritt im Rahmen der Transaktion, dessen EBICS-Request aus Sicht des Banksystems erfolgreich empfangen und dessen zugehörige EBICS-Response erfolgreich versendet wurde. Der Wiederaufsetzpunkt wird bestimmt durch den Zustand der Transaktion auf dem Banksystem.

  Bei Download-Transaktionen kann es mehrere Wiederaufsetzpunkte geben, dabei handelt es sich um all die bisherigen Transaktionsschritte der betroffenen Transaktion, deren EBICS-Request aus Sicht des Banksystems bereits erfolgreich empfangen und deren zugehörige EBICS-Response erfolgreich versendet wurde.

Nach Transport- oder Verarbeitungsfehlern kann ein Wiederaufsetzpunkt verwendet werden, um Transaktionen mit dem Transaktionsschritt fortzusetzen, der in der sequenziellen Abfolge der Transaktionsschritte auf diesen Wiederaufsetzpunkt folgt.

Jeder EBICS-Request zu einer offenen Transaktion, der nicht zum Zustand dieser Transaktion passt, wird von der EBICS-Transaktionsverwaltung auf Bankseite als Recovery-Versuch gewertet.

Um ein Fortschreiten der EBICS-Transaktionen zu gewährleisten, MUSS die Anzahl der möglichen Recovery-Versuche je Transaktion durch einen Maximalwert begrenzt werden. Die Transaktionsverwaltung des Banksystems ist verantwortlich für die Verwaltung des entsprechenden Zählers je Transaktion. Transaktionen, deren Zähler die erlaubte Grenze überschreitet, werden durch die Transaktionsverwaltung des Banksystems abgebrochen. Zusätzlich KANN das Banksystem je Teilnehmer die Anzahl der offenen Transaktionen mit einem positiven Recovery-Zähler durch einen Maximalwert begrenzen. Der Zähler für die bereits erfolgten Recovery-Versuche je Transaktion und/oder der Zähler der offenen Transaktionen im Recovery-Modus je Teilnehmer sowie die erlaubten Maximal-zahlen sind nicht Bestandteil der EBICS-Nachrichten. Sie sind vielmehr Teil der Implementierung der EBICS-Transaktionsverwaltung auf der Bankseite.

Analog dazu begrenzt die Transaktionssteuerung des Kundensystems die Anzahl der Versuche, einen bestimmten Transaktionsschritt einer EBICS-Transaktion erfolgreich durchzuführen. Zähler und erlaubte Maximalzahl sind auch in diesem Fall nicht Bestandteil des EBICS-Nachrichten, sondern lediglich der Implementierung der Transaktionssteuerung auf Kundenseite.

Details zum Wiederaufsetzen von Upload- sowie Download-Transaktionen sind in den Kapiteln 5.5.2 bzw. 5.6.2 enthalten.

5.5 Upload-Transaktionen

5.5.1 Ablauf von Upload-Transaktionen

Der Ablauf einer Upload-Transaktion ist in Abbildung 35 anhand eines Sequenzdiagramms dargestellt. Die Übertragung der Nutzdatensegmente erfolgt innerhalb einer Schleife, die abgebrochen wird, sobald das letzte Nutzdatensegment übertragen wurde (beachte Teilausdruck „[letztes Datensegment wurde übertragen]“ der Abbruchbedingung). Der Ablauf verdeutlicht, dass innerhalb einer Upload-Transaktion nicht notwendigerweise auch Nutzdaten übertragen werden müssen (beachte Teilausdruck „[AdminOrderType = HVE or HVS]“ der Abbruchbedingung). Das ist der Fall, wenn bankfachlichebankfachliche EUs via HVE (Hinzufügen EU) oder HVS (Stornierung eines Auftrags) zu einem existierenden Auftrag übertragen werden.

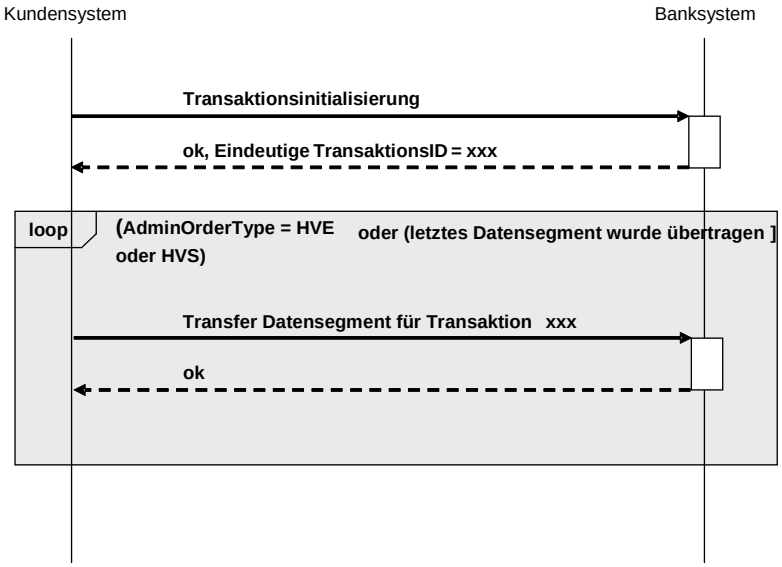


Abbildung 35: Fehlerfreier Ablauf einer Upload-Transaktion

5.5.1.1 Beschreibung der EBICS-Nachrichten

Die nachfolgende Beschreibung der Transaktionsschritte des Uploads einer BTF-Transaktion verwendet zur Verdeutlichung Beispielnachrichten für die Durchführung des Services „SCT“ (SEPA_Überweisung). Sie nimmt Bezug auf Elemente dieser Beispielnachrichten unter Verwendung der XPath-Notation.

Die folgenden Kapitel beschreiben die Nachrichten der einzelnen Transaktionsphasen. Es werden die Daten aufgezählt, die Bestandteil dieser Nachrichten sind. Daten, die grundsätzlich optional sind, werden mit „(optional)“ gekennzeichnet. Daten, die nur unter bestimmten Voraussetzungen fehlen dürfen, werden stattdessen mit „(bedingt)“ gekennzeichnet. Optionale XML-Elemente, die in der Beschreibung einer EBICS-Nachricht zu einer bestimmten Transaktionsphase fehlen, dürfen in dieser EBICS-Nachricht nicht vorkommen. Optionale XML-Elemente, die in der Beschreibung einer EBICS-Nachricht zu einer bestimmten Transaktionsphase vorkommen, MÜSSEN in dieser EBICS-Nachricht immer entsprechend gesetzt sein.

EBICS-Requests von Upload-Transaktionen sind zu ebics_request_H005.xsd konforme (XML-) Instanzdokumente, bestehend aus dem Toplevel-Element `ebicsRequest`, welches in ebics_request_H005.xsd deklariert ist. EBICS-Responses von Upload-Transaktionen sind zu

ebics_response_H005.xsd konforme Instanzdokumente, die aus dem Toplevel-Element `ebicsResponse` bestehen, das wiederum in ebics_response_H005.xsd deklariert ist.

5.5.1.1.1 EBICS-Nachrichten der Transaktionsinitialisierung

- **Übertragung der folgenden Daten im EBICS-Request (siehe** Abbildung 36**):**
  - Host-ID des EBICS Bankrechners (`ebicsRequest/header/static/HostID`)
  - Transaktionsphase (`ebicsRequest/header/mutable/TransactionPhase`) mit der Belegung „Initialisation“
  - Kombination aus Nonce und Timestamp, erforderlich für die Vermeidung von Replay alter EBICS-Nachrichten (`ebicsRequest/header/static/Nonce`, `ebicsRequest/header/static/Timestamp`)
  - Anzahl zu übertragender Datensegmente (`ebicsRequest/header/static/NumSegments`)
  - Teilnehmer (`ebicsRequest/header/static/PartnerID`, `ebicsRequest/header/static/UserID`), der einen neuen Auftrag einreicht bzw. zu einem bereits vorhandenen Auftrag bankfachliche EUs liefert.
  - (bedingt) technischer Teilnehmer (`ebicsRequest/header/static/PartnerID`, `ebicsRequest/header/static/SystemID`)
    `SystemID` muss genau dann vorhanden sein, wenn es sich bei dem Kundensystem um ein Multi-Usersystem handelt. Der technische Teilnehmer ist für das Erstellen der EBICS-Requests (inklusive der Authentifkationssignaturen) zuständig, die zu Aufträgen gehören, die vom Teilnehmer eingereicht oder bankfachlich unterzeichnet werden.
  - (optional) Information zum Kundenprodukt (`ebicsRequest/header/static/Product`)
  - Administrative Auftragsart (ebicsRequest/header/static/OrderDetails/AdminOrderType)
  - (bedingt) Auftragsnummer (ebicsRequest/header/static/OrderDetails/OrderID)
    OrderID (Auftragsnummer) ist nur vorhanden, wenn eine Datei zur Bank übertragen wird, die sich auf einen Auftrag mit bereits existierender Auftragsnummer bezieht (nur zulässig für AdminOrderType = HVE oder HVS).
  - Auftragsparameter (`ebicsRequest/header/static/OrderDetails/OrderParams`);
    die Ausprägung der Auftragsparameter ist abhängig von der administrativen Auftragsart (siehe auch Kapitel 3.11). Für das Senden von (bank-)fachlichen Geschäftsvorfällen (BTF) sind die Regeln im Detail in Kapitel 5.5.1.1.3 beschrieben.
  - Hashwerte der öffentlichen Schlüssel des Kreditinstituts, über die der Teilnehmer verfügt (ebicsRequest/header/static/BankPubKeyDigests/Authentication, ebicsRequest/header/static/BankPubKeyDigests/Encryption, ebicsRequest/header/static/BankPubKeyDigests/Signature).
    Zu jedem dieser Hashwerte wird sowohl der verwendete Hashalgorithmus als auch die Version des entsprechenden Authentifikations- bzw. Verschlüsselungs- bzw.

Signaturverfahrens angegeben.
Die Darstellung des SHA-256-Hashwertes der Bankzertifikate für X002 und E002 wird berechnet aus dem SHA-256-Hashwertes des Zertifikats im DER Binärformat, die entstehende Byte-Anordnung (32 Bytes) wird in hexadezimaler Darstellung (64 Zeichen) und in Großbuchstaben dargestellt.
In der Version „H005“ des EBICS-Protokolls ist die EU der Kreditinstitute nur vorgesehen (siehe dazu Kapitel 3.5.2). Das Element `BankPubKeyDigests/Signature` ist in Vorbereitung zukünftiger EBICS-Versionen bereits in dieser Beschreibung enthalten, obwohl seine maximale Häufigkeit (maxOccurs) in der Version „H005“ gleich 0 ist

- Sicherheitsmedium für den bankfachlichen Schlüssel des Teilnehmers (`ebicsRequest/header/static/SecurityMedium`)
- Authentifikationssignatur des technischen Teilnehmers, falls ein solcher vorhanden ist, ansonsten die Authentifikationssignatur des einreichenden Teilnehmers selbst (`ebicsRequest/AuthSignature`)
  Die Authentifikationsignatur umfasst alle XML-Elemente des EBICS-Requests, deren Attributwert für `@authenticate` gleich „true“ ist. Die Definition des XML-Schemas „ebics_request_H005.xsd“ gewährleistet, dass der Wert des Attributs `@authenticate` für genau die Elemente gleich „true“ ist, die auch unterschrieben werden müssen
- (optional) Daten für die Vorabprüfung des Auftrags (`ebicsRequest/body/PreValidation`)
- Information zur Verschlüsselung der EUs und Auftragsdaten (`ebicsRequest/body/DataTransfer/DataEncryptionInfo`), die insbesondere auch den asymmetrisch verschlüsselten Transaktionsschlüssel (`ebicsRequest/body/DataTransfer/DataEncryptionInfo/TransactionKey`) enthält
- EUs der Nutzdaten des Auftrags (`ebicsRequest/body/DataTransfer/SignatureData`)
  SignatureData enthält ein Instanzdokument, das zu „ebics_orders_H005.xsd“ konform ist und `UserSignatureData` als Toplevel-Element enthält. Vor der Einbettung in den EBICS-Request wurde dieses Instanzdokument mit ZIP komprimiert, für das Kreditinstitut verschlüsselt und schließlich base64-kodiert (siehe Anhang (Kapitel 11.2.2)). Ein Beispiel eines solchen Instanzdokuments, welches eine einzige EU enthält, ist in Abbildung 37 enthalten. Die Belegung für das Attribut `PartnerID` im `UserSignatureData` Dokument muss identisch sein zur Kunden-ID des Einreichers im Element `ebicsRequest/header/static/PartnerID`.
- Hashwert der Auftragsdaten (`ebicsRequest/body/DataTransfer/DataDigest`)
  Das Attribut `ebicsRequest/body/DataTransfer/DataDigest@SignatureVersion` spezifiziert das Verfahren zur Hashwertberechnung.

- Zusätzliche Auftragsinformationen (optional)
  (ebicsRequest/body/DataTransfer/AdditionalOrderInfo)

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest
xmlns="urn:org:ebics:H005"
xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="urn:org:ebics:H005 ebics_request_H005.xsd"
Version="H005" Revision="1">
  <header authenticate="true">
    <static>
      <HostID>EBIXHOST</HostID>
      <Nonce>BDA2312973890654FAC9879A89794E65</Nonce>
      <Timestamp>2005-01-30T15:30:45.123Z</Timestamp>
      <PartnerID>PARTNER1</PartnerID>
      <UserID>USER0001</UserID>
      <Product Language="de" InstituteID="Institutskennung">Kundenproduktkennung</Product>
      <OrderDetails>
        <AdminOrderType>BTU</AdminOrderType>
BTUOrderParams>
           <Service>
             <ServiceName>SCT</ServiceName>
              <MsgName>pain.001</MsgName>
           </Service>
 </BTUOrderParams>      </OrderDetails>
      <BankPubKeyDigests>
        <Authentication Version="X002"
Algorithm="http://www.w3.org/2001/04/xmlenc#sha256">1H/rQr2Axe9hYTV2n/tCp+3UIQQ=</Authentication>
        <Encryption Version="E002"
Algorithm="http://www.w3.org/2001/04/xmlenc#sha256">2lwiueWOIER823jSoiOkjl+woeI=</Encryption>
      </BankPubKeyDigests>
      <SecurityMedium>0000</SecurityMedium>
      <NumSegments>2</NumSegments>
    </static>
    <mutable>
      <TransactionPhase>Initialisation</TransactionPhase>
    </mutable>
  </header>
  <AuthSignature>
    <ds:SignedInfo>
      <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
      <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256">
      </ds:SignatureMethod>
      <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
        <ds:Transforms>
          <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
        </ds:Transforms>
        <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
        <ds:DigestValue> …hier Hashwert Authentifikation..</ds:DigestValue>
      </ds:Reference>
    </ds:SignedInfo>
    <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
  </AuthSignature>
  <body>
    <PreValidation authenticate="true">
      <DataDigest SignatureVersion="A0066"> MTIzNDU2Nzg5MDEyMzQ1Njc4OTAxMjM0NTY3ODkwMTI=</DataDigest>
    </PreValidation>
    <DataTransfer>
      <DataEncryptionInfo authenticate="true">
```

```
        <EncryptionPubKeyDigest Version="E002" Algorithm="http://www.w3.org/2001/04/xmlenc#sha256">..hier
Hashwert des öffentlichen Bankschlüssels für die Verschlüsselung ..</EncryptionPubKeyDigest>
        <TransactionKey>EIGI4En6KEB6ArEzw+iq4N1wm6EptcyxXxStA…</TransactionKey>
       </DataEncryptionInfo>
      <SignatureData authenticate="true">n6KEB6ArEzw+iq4N1wm6EptcyxXxStAO…</SignatureData>
<DataDigest SignatureVersion="A006">MTIzNDU2Nzg5MDEyMzQ1Njc4OTAxMjM0NTY3ODkwMTI=</DataDigest>  </body>
</ebicsRequest>
```

Abbildung 36: EBICS-Request für die Transaktionsinitialisierung für einen BTF-Upload-Auftrag

```
<?xml version="1.0" encoding="UTF-8"?>
<UserSignatureData
xmlns="http://www.ebics.org/S002"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www.ebics.org/S002
http://www.ebics.org/S002/ebics_signature.xsd">
<UserSignatureData>
    <OrderSignatureData>
        <SignatureVersion>A005</SignatureVersion>
        <SignatureValue>EUXkQa……</SignatureValue>
        <PartnerID>PARTNER1</PartnerID>
        <UserID>User1</UserID>
    </OrderSignatureData>
</UserSignatureData>
```

Abbildung 37: XML-Dokument, das die EUs der Unterzeichner des Sendeauftragsenthält

- Übertragung der folgenden Daten in der EBICS-Response (siehe auch Beispiel in Abbildung 38)

bankfachlicher Returncode (`ebicsResponse/body/ReturnCode`)

Auftragsnummer (`ebicsResponse/header/mutable/OrderId`)

technischer Returncode (`ebicsResponse/header/mutable/ReturnCode`)

technischer Reporttext (`ebicsResponse/header/mutable/ReportText`)

(bedingt) banksystemweit eindeutige TransaktionsID (`ebicsResponse/header/static/TransactionID`), falls die folgenden Bedingungen erfüllt sind:

- Während der Transaktionsinitialisierung sind keine Fehler technischer oder bankfachlicher Art aufgetreten
- Innerhalb der aktuellen Transaktion werden in weiteren darauf folgenden Transaktionsschritten Nutzdatensegmente übertragen, es sei denn AdminOrderType ist gleich HVE oder HVS.

Transaktionsphase (`ebicsResponse/header/mutable/TransactionPhase`) mit der Belegung „Initialisation“

Authentifikationssignatur des Kreditinstituts (`ebicsResponse/AuthSignature`)
Die Authentifikationsignatur umfasst alle XML-Elemente der EBICS-Response, deren Attributwert für `@authenticate` gleich „true“ ist. Die Definition des XML-Schemas „ebics_response_H005.xsd“ gewährleistet, dass der Wert des Attributs `@authenticate` für genau die Elemente gleich „true“ ist, die auch unterschrieben werden müssen

(optional) Zeitstempel der letzten Aktualisierung der Bankparameter (`ebicsResponse/body/TimestampBankParameter`).

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<ebicsResponse
xmlns="urn:org:ebics:H005"
H005 ebics_response_H005.xsd"
Version="H005" Revision="1">
  <header authenticate="true">
    <static>
      <TransactionID>ABCDEF41394644363445313243ABCDEF</TransactionID>
    </static>
    <mutable>
      <TransactionPhase>Initialisation</TransactionPhase>
      <OrderId>OR01</OrderId>
      <ReturnCode>000000</ReturnCode>
      <ReportText>[EBICS_OK] OK</ReportText>
    </mutable>
  </header>
  <AuthSignature>
    <ds:SignedInfo>
      <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
      <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256"/>
      <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
        <ds:Transforms>
          <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
        </ds:Transforms>
        <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
        <ds:DigestValue> …hier Hashwert Authentifikation..</ds:DigestValue>
      </ds:Reference>
    </ds:SignedInfo>
    <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
  </AuthSignature>
  <body>
    <ReturnCode authenticate="true">000000</ReturnCode>
  </body>
</ebicsResponse>
```

Abbildung 38: EBICS-Response der Transaktionsinitialisierung für einen BTF-Upload

5.5.1.1.2 EBICS-Nachrichten der Phase Datentransfer eines Nutzdatensegments

- Übertragung der folgenden Daten im EBICS-Request (siehe Beispiel in Abbildung 39):
  - Host-ID des EBICS Bankrechners (`ebicsRequest/header/static/HostID`)

Daten zur Identifikation des aktuellen Transaktionsschritts:

- TransaktionsID (ebicsRequest/header/static/TransactionID)
- Transaktionsphase (`ebicsRequest/header/mutable/TransactionPhase`) mit der Belegung „Transfer“
- laufende Nummer des Nutzdatensegments (`ebicsRequest/header/mutable/SegmentNumber`)
  Das Attribut ebicsRequest/header/mutable/SegmentNumber`@`lastSegment gibt an, ob es sich dabei um das letzte Datensegment handelt.

- Authentifikationssignatur des technischen Teilnehmers, falls ein solcher für die aktuelle Transaktion definiert wurde, ansonsten die Authentifikationssignatur des einreichenden Teilnehmers selbst (`ebicsRequest/AuthSignature`)

Die Authentifikationsignatur umfasst alle XML-Elemente des EBICS-Requests, deren Attributwert für `@authenticate` gleich „true“ ist. Die Definition des XML-Schemas „ebics_request_H005.xsd“ gewährleistet, dass der Wert des Attributs `@authenticate` für genau die Elemente gleich „true“ ist, die auch unterschrieben werden müssen

- das eigentliche Nutzdatensegment (`ebicsRequest/body/DataTransfer/OrderData`) (siehe Kapitel 3.3 sowie Kapitel 7 für Einzelheiten der Segmentierung von Nutzdaten).

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest
 xmlns="urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_request_H005.xsd"
 Version="H005" Revision="1">
 <header authenticate="true">
   <static>
     <HostID>EBIXHOST</HostID>
     <TransactionID>ABCDEF413946443634453132 43ABCDEF</TransactionID>
   </static>
   <mutable>
     <TransactionPhase>Transfer</TransactionPhase>
     <SegmentNumber lastSegment="true">4</SegmentNumber>
   </mutable>
 </header>
 <AuthSignature>
   <ds:SignedInfo>
     <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
     <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256">
     </ds:SignatureMethod>
     <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
       <ds:Transforms>
         <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
       </ds:Transforms>
       <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
       <ds:DigestValue> …hier Hashwert Authentifikation..</ds:DigestValue>
     </ds:Reference>
   </ds:SignedInfo>
   <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
 </AuthSignature>
 <body>
   <DataTransfer>
     <OrderData>RUJJQ1MtUmVxdWVzdCBm/HIgZGllINxiZXJ0…</OrderData>
   </DataTransfer>
 </body>
</ebicsRequest>
```

Abbildung 39: EBICS-Request für die Übertragung des letzten Nutzdatensegments für einen BTF-Upload

- Übertragung der folgenden Daten in der EBICS-Response (siehe auch Beispiel in Abbildung 40)
  - bankfachlicher Returncode (ebicsResponse/body/ReturnCode)
    Auftragsnummer (ebicsResponse/header/mutable/OrderId)
  - technischer Returncode (ebicsResponse/header/mutable/ReturnCode)
  - technischer Reporttext (ebicsResponse/header/mutable/ReportText)

Daten zur Identifikation eines Transaktionsschritts:
Falls der technische Returncode den Wert EBICS_TX_RECOVERY_SYNC hat, identifiziert dieser

Transaktionsschritt den Wiederaufsetzpunkt der Upload-Transaktion. Sind jedoch wie im vorliegenden Beispiel weder technische noch fachliche Fehler aufgetreten, so spiegelt dieser Transaktionsschritt den gerade aktuellen Transaktionsschritt wider

- TransaktionsID (`ebicsResponse/header/static/TransactionID`)
- Transaktionsphase (`ebicsResponse/header/mutable/TransactionPhase`)
- (bedingt) laufende Nummer des Nutzdatensegments (`ebicsResponse/header/mutable/SegmentNumber`), falls der Wert für `TransactionPhase` ungleich „Initialisation“ ist
  Das Attribut `ebicsResponse/header/mutable/SegmentNumber@lastSegment` gibt an, ob es sich dabei um das letzte Nutzdatensegment handelt.

Authentifikationssignatur des Kreditinstituts (`ebicsResponse/AuthSignature`)
Die Authentifikationsignatur umfasst alle XML-Elemente der EBICS-Response, deren Attributwert für `@authenticate` gleich „true“ ist. Die Definition des XML-Schemas „ebics_response_H005.xsd“ gewährleistet, dass der Wert des Attributs `@authenticate` für genau die Elemente gleich „true“ ist, die auch unterschrieben werden müssen.

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsResponse
 xmlns=" rn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_response_H005.xsd"
 Version="H005" Revision="1">
 <header authenticate="true">
   <static>
     <TransactionID>ABCDEF413946443634453132 43ABCDEF</TransactionID>
   </static>
   <mutable>
     <TransactionPhase>Transfer</TransactionPhase>
     <SegmentNumber lastSegment="true">4</SegmentNumber>
       <OrderId>OR01</OrderId>
     <ReturnCode>000000</ReturnCode>
     <ReportText>[EBICS_OK] OK</ReportText>
   </mutable>
 </header>
 <AuthSignature>
   <ds:SignedInfo>
     <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
     <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256"/>
     <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
       <ds:Transforms>
         <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
       </ds:Transforms>
       <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
       <ds:DigestValue> …hier Hashwert Authentifikation..</ds:DigestValue>
     </ds:Reference>
   </ds:SignedInfo>
   <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
 </AuthSignature>
 <body>
   <ReturnCode authenticate="true">000000</ReturnCode>
 </body>
</ebicsResponse>
```

Abbildung 40: EBICS-Response der Übertragung des letzten Nutzdatensegments für einen BTF-Upload

5.5.1.1.3 Upload-Request-Struktur für bankfachliche Geschäftsvorfälle und Formate (BTF)

Die folgenden Regeln spezifizieren die Abhängigkeit der BTF-Elemente untereinander:

1) Alle Codes für das Element <ServiceName> werden durch die EBICS SCRL (per externer Codeliste) definiert. Demzufolge wird ihre Bedeutung global gleich verstanden. Ein Beispiel für global verwendete Services sind die SEPA-Services.
2) Bei Services ohne global abgestimmtes Regelwerk ist ein <Scope> anzugeben, in dem der Service genutzt wird.
3) Ein Service mit zusätzlichen (marktspezifischen oder bilateral abgestimmten) Modifikationen benötigt einen entsprechenden Scope zur Kennzeichnung dieser Tatsache. Modifikationen können z.B. ein abweichender Namespace oder spezielle Einschränkungen sein..
4) Verwendbare Scopes sind ebenfalls in einer externen Codeliste spezifiziert.
5) Für das Element <ServiceOption> ist die folgende Struktur verbindlich:

a) 3-stellige Zeichenkette: Globale codes (aus der externen Codeliste)
b) 4-stellige Zeichenkette: Codes, die durch einen Markt (Scope) definiert werden (externe Codelisten).
   Der Markt kann durch das angegebene <Scope>-Element identifiziert werden.
c) 5- bis 10-stellige Zeichenkette: Bilateral vereinbarte Codes (zwischen einer einzelnen Bank und deren Kunden). Eine Veröffentlichung ist nicht verpflichtend.
d) Alle Codes für Service-Optionen (global, auf einen Markt bezogen, bilateral) müssen auf die Services verweisen, die mit ihnen verwendbar sind.
e) Die Verwendung von 4-10-stelligen Service-Optionen erforderterfordert ein Scope-Element. Dies bedeutet, dass <ServiceName> nur ohne <ServiceOption> oder zusammen mit einer 3-stelligen Service-Option (d.h. global festgelegt/abgestimmt) ein Geschäftsvorfall mit globaler Bedeutung sein kann.

Zusammenfassend kann man sagen, dass sich das Element <Scope> auf die Pflichtelemente <ServiceName> und <MsgName> bezieht sowie – falls vorhanden – auch auf <ServiceOption>. Dies ist ebenfalls gültig für Abholaufträge, Details siehe Kapitel 5.6.1.1.4.

Die Werte der Auftragsparameter stehen in `ebicsRequest/header/static/OrderDetails` und sind vom Typ `BTUOrderParams` verwendbar für Senden und der administrativen Auftragsart “BTU”:

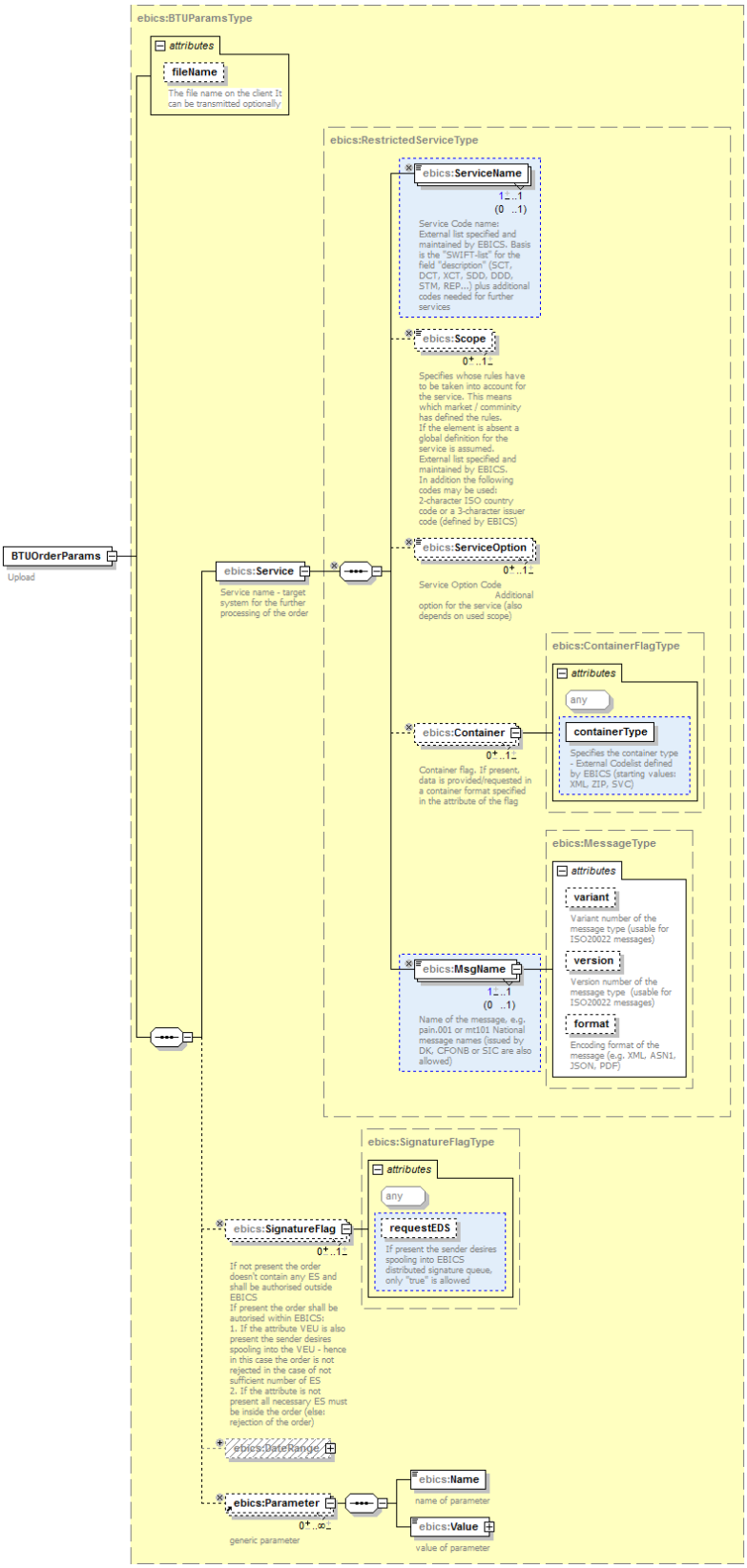


Abbildung 41: BTF-Struktur für Senden (verwendet eingeschränkte Service-Struktur)

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispi
BTUOrderParams	ebics:BTUParamsType (komplex)	1	Notwendig zur Kennzeichnung einer Geschäftsvorfalltransaktion und des verwendeten / zugehörigen Formats	- (complex)
BTUOrderParams@FileNm	ebics:FileNameStringType (simple) - restriction base: minLength value="1" maxLength value="256"	0..1	Original-Dateiname auf Kundensystemseite	
Service	RestrictedServiceType (komplex)	1	Kennzeichnet, wie die Datei auf Seiten des Zielsystems zu verarbeiten ist	- (komplex)
ServiceName	ebics: ServiceNameStringType (simple) - restriction base: minLength value="3" maxLength value="3" pattern = [A-Z0-9]	1	Name des ServicesServices	ServiceNar Thema eine externen C (gepflegt vo EBICS) Beispiel: "SCT" = SE Überweisur
Scope	ebics: ScopeStringType (simple) restriction base: minLength value="2" maxLength value="3"	0..1	Kennzeichnet, wer die Regeln vorgegeben hat, die für den Service hier zu beachten sind, d.h. welcher Markt/welche Community die zu beachtenden Regeln definiert hat. Eine externe Scope-Liste wird von EBICS spezifiziert und gepflegt. Ein fehlender Scope bedeutet, dass es sich um global akzeptierte (gleich verstandene) Regelwerke handelt	Scope-Cod werden in e externen C (gepflegt vo EBICS) spe 2-stellige IS Ländercode 3-stellige C andere Sco "BIL" bedeu "bilateral" abgestimm Ein fehlend Scope bed "global". St fehlenden S kann hier a Scope-Cod gesetzt wer
ServiceOption	ebics:CodeStringType (simple) restriction base: minLength value="3" maxLength value="10" pattern = [A-Z0-9]	0..1	Optionale Eigenschaft(en) eines Services	Options-Co werden in e externen C (gepflegt vo EBICS) spe Beispiel:

				"URG" = u
Container	ebics:ContainerFlagType	0..1	Kennzeichnung, ob ein Container verwendet wird	Es ist nur d true erlau Fehlen bedeutet fa
Container@containerType	ebics:ContainerstringType (simple) restriction base: minLength value="3" maxLength value="3" pattern = [a-z]	1..1	Kennzeichnung, welche Art von Container verwendet wird	Im Falle de Nutzung de Container-F die Belegur Attributs verpflichter (interne Co "XML" "ZIP" "SVC"
MsgName	ebics: MessageNameStringType (simple) restriction base: minLength value="1" maxLength value="10" pattern = [a-z\.0-9]	1	Nachrichtenname, entweder beginnend mit dem BA-Code (ISO) oder MT (FIN) oder einer Zeichenkette	"pain.001", "mt103" (Proprietäre Nachrichter (spezifiziert den angege Scope) sind zulässig
MsgName@version	ebics:NumString (simple) restriction base: minLength value="2" maxLength value="2" pattern = [0-9]	0..1	Verwendete ISO-Version der Nachricht (wenn es sich um kein ISO-Format handelt, wird dieses Attribut ignoriert)	"03"
MsgName@variant	ebics:NumString (simple) restriction base: minLength value="3" maxLength value="3" pattern = [0-9]	0..1	Wird zusammen mit <MsgName> betrachtet, (wenn es sich um kein ISO-Format handelt, wird dieses Attribut ignoriert)	"001"
MsgName@format	ebics:CodeString (simple) restriction base: minLength value="1" maxLength value="4" pattern = [A-Z0-9]	0..1	Wird zusammen mit <MsgName> betrachtet, ist für jede Art von Format zulässig, sollte aber nur verwendet werden, wenn diese Angabe eine Abweichung zum Standard darstellt (insbesondere, wenn kein XML für ISO 20022 verwendet wird).	„XML", „ASN „JSON", „PD
SignatureFlag	SignatureFlagType	0..1	Kennzeichnet das Vorhandensein von EUs (siehe Regeln in Kombination mit dem Flag requestEDS)	Es ist nur d true zuläs Fehlen .bed false

SignatureFlag@ requestEDS	boolean	0..1	Wenn vorhanden, dann soll der Auftrag innerhalb von EBICS freigegeben werden (Details zu den Reaktionen des Bankrechners siehe Kapitel 3.14)	Nur der We zulässig. Nichtvorhar des Flags b false
Parameter		0..1	Frei definierbare Parameter. Eine beliebige Anzahl von Wertepaaren kann spezifiziert werden.	
Name	boolean	1..1	Name des Parameters	
Value	anySimpleType	1..1	Wert des Parameters	
Value@type	NCName	1..1	Datentyp des Wertes	Defaultempfe ist string

5.5.1.2 Verarbeitung der EBICS-Nachrichten

Kapitel 5.5.1.1 beschreibt den Inhalt der EBICS-Nachrichten, die im Rahmen einer Upload-Transaktion ausgetauscht werden. Gegenstand dieses Kapitels ist die bankseitige Verarbeitung dieser EBICS-Nachrichten. Im Sequenzdiagramm in Abbildung 35 sind Aktionssequenzen hervorgehoben, deren Ablauf hier näher beschrieben wird.

Um die Beschreibung der Abläufe zu vereinfachen, wird angenommen, dass jeder Verarbeitungsschritt einen Returncode (RC) liefert, dessen Wert gleich 0 („000000“, EBICS_OK) ist, wenn dieser Schritt erfolgreich durchgeführt werden konnte. In Abhängigkeit vom RC werden der technische Returncode (RCT) und der bankfachliche Returncode (RCF) gesetzt, deren Werte dann in die EBICS-Nachrichten einfließen.

Die Überprüfung der Validität der EBICS-Requests erfolgt auf Grundlage der XML-Schema Definitionsdatei „ebics_request_H005.xsd“ sowie unter Berücksichtigung der Einschränkungen, die in Kapitel 5.5.1.1 zu den einzelnen Requests festgelegt wurden. Normalerweise findet die Validitätsprüfung parallel und/oder verzahnt mit den anderen Ablaufschritten der Verarbeitung der EBICS-Requests statt. Die nachfolgenden Abläufe verzichten auf die Darstellung eines Ablaufschritts vom Typ „Validitätsprüfung des EBICS-Requests“ zugunsten einer möglichst einfachen Darstellung. Demnach können diese Abläufe durch folgende zusätzliche technische Fehler abgebrochen werden:

- EBICS_INVALID_XML
  Die empfangene EBICS XML-Nachricht entspricht syntaktisch nicht den Vergaben des EBICS XML-Schemas: das XML ist nicht wohlgeformt bzw. nicht valide gemäß Schema,
  z.B. Upload-Request ohne das Element HostId (laut Schema erforderlich).
- EBICS_INVALID_REQUEST
  Die empfangene EBICS XML-Nachricht entspricht syntaktisch nicht den EBICS Vorgaben,
  z.B. Upload-Request ohne Element NumSegments (laut Schema optional aber laut Kapitel 5.5.1.1.1 erforderlich).

- EBICS_INVALID_REQUEST_CONTENT
  Die empfangene EBICS XML-Nachricht entspricht semantisch nicht den EBICS Vorgaben, obwohl es gemäß Schema syntaktisch korrekt ist,

5.5.1.2.1 Verarbeitung in der Initialisierungsphase

In Abbildung 45 ist die bankseitige Verarbeitung des EBICS-Requests dargestellt, welcher in der Initialisierungsphase einer Upload-Transaktion vom Kundensystem an das Banksystem versendet wird. Die einzelnen Verarbeitungsschritte werden im Folgenden näher erläutert:

I. Erzeugung einer EBICS-Transaktion (siehe Abbildung 44)

Dieser Verarbeitungsschritt ist sowohl für Upload- als auch für Download-Transaktionen relevant. Die nachfolgende Beschreibung berücksichtigt beide Transaktionstypen, um dann in den nachfolgenden Kapiteln zum Thema Download-Transaktionen auf diese Beschreibung verweisen zu können.

I.a. Überprüfung des Auftrags (sowohl bankfachlich als auch administrativ)

Die Überprüfung der Autragsart liefert den technischen Returncode EBICS_INVALID_ORDER_IDENTIFIER im Falle einer ungültigen administrativen Auftragsart oder einer unzulässigen BTF-Parameterwertkombination bzw. den technischen Returncode EBICS_UNSUPPORTED_ORDER_TYPE im Fall eines gültigen, jedoch optionalen Auftrags, der vom Banksystem nicht unterstützt wird.

I.b. Replay Test

Der Replay-Test liefert den technischen Returncode EBICS_TX_MESSAGE_REPLAY, falls es sich bei dem EBICS-Request um einen wieder eingespielten Request handelt. Details zur Replay-Vermeidung sind im Anhang (Kapitel 11.4) beschrieben.

I.c. Prüfung der Authentizität des EBICS-Requests (siehe Abbildung 42):

Die Authentifikationssignatur wird von einem technischen Teilnehmer geleistet, wenn ein solcher Bestandteil der Steuerdaten ist. Andernfalls wird die Authentifikationssignatur vom (nichttechnischen) Teilnehmer der EBICS-Transaktion erzeugt, welcher den Auftrag einreicht bzw. bankfachliche EUs zu einem bereits bestehenden Auftrag nachliefert. Um die Authentifikationssignatur eines beliebigen (technischen oder nichttechnischen) Teilnehmers prüfen zu können, muss die entsprechende Kombination aus Kunden- und TeilnehmerID im Banksystem registriert sein und der Zustand des Teilnehmers auf „Bereit“ stehen. In Fehlersituationen, die aus einer ungültigen Kombination aus Kunden-/TeilnehmerID oder einem unzulässigen Teilnehmerzustand resultieren, erhält der Sender den technischen Returncode EBICS_AUTHENTICATION_FAILED.

Die Verifikation der Authentifikationssignatur beinhaltet:

- die Prüfung, ob mit der Authentifikationssignatur alle erforderlichen Elemente der EBICS-Nachricht unterschrieben wurden: Das sind alle XML-Elemente des EBICS-Requests, deren Attributwert für `@authenticate` gleich „true“ ist
- die Verifikation der Authentifikationsunterschrift selbst.

Dieser Verarbeitungsschritt bricht mit dem technischen Fehler EBICS_AUTHENTICATION_FAILED ab, wenn die Authentifikationssignatur nicht erfolgreich verifiziert werden kann.

Stammt die erfolgreich verifizierte Authentifikationssignatur von einem technischen Teilnehmer, so wird zusätzlich die Gültigkeit und der Zustand des (nichttechnischen) Teilnehmers überprüft. Fehler, die aus einer ungültigen Kombination aus Kunden-/TeilnehmerID oder einem unzulässigen Teilnehmerzustand resultieren, werden dem Sender des EBICS-Requests mit Hilfe der technischen Fehlercodes EBICS_USER_UNKNOWN bzw. EBICS_INVALID_USER_STATE mitgeteilt.

Begründung: Wenn die Authentifikationssignatur nicht erfolgreich verifiziert werden kann, stammt der EBICS-Request potenziell von einem Angreifer. Fehler wie beispielsweise „Unbekannter Teilnehmer“ oder „Unzulässiger Teilnehmerzustand“ werden in diesem Fall nicht direkt an den Sender des EBICS-Requests weitergereicht, um potenziellen Angreifern keine genaue Information über die Gültigkeit von TeilnehmerIDs oder den Zustand von Teilnehmern zu geben. Nachdem die Authentifikationssignatur des technischen Teilnehmers jedoch erfolgreich verifiziert werden konnte, werden die Fehler EBICS_USER_UNKNOWN bzw. EBICS_INVALID_USER_STATE, die sich auf den nichttechnischen Teilnehmer der EBICS-Transaktion beziehen, an den authentisierten Sender weitergegeben.

I.d. Überprüfung der Hashwerte der Bankschlüssel

Mit dieser Überprüfung soll verhindert werden, dass ein Teilnehmer Aufträge einreicht, obwohl er nicht über die aktuellen öffentlichen Schlüssel des Kreditinstituts verfügt. In der Version „H005“ von EBICS ist die EU der Kreditinstitute nur vorgesehen (siehe Kapitel 3.5.2). Deshalb werden in der Version „H005“ nur die Hashwerte des öffentlichen Authentifikationsschlüssels und des öffentlichen Verschlüsselungsschlüssels überprüft. Nachfolgende EBICS-Versionen, die die EU der Kreditinstitute unterstützen, müssen in diesem Verarbeitungsschritt auch den Hashwert des öffentlichen bankfachlichen Schlüssels des Kreditinstituts überprüfen. Zu diesem Zweck übermittelt der Teilnehmer die Hashwerte der öffentlichen Schlüssel des Kreditinstituts, über die er verfügt. Das Banksystem überprüft diese Hashwerte. Sollten sie nicht mit den Hashwerten der aktuellen öffentlichen Schlüssel übereinstimmen, wird die Transaktionsinitialisierung mit dem technischen Returncode EBICS_BANK_PUBKEY_UPDATE_REQUIRED abgebrochen.

Verfügt der Teilnehmer nicht über den aktuellen Authentifikationsschlüssel des Kreditinstituts, so kann er die Authentifikationssignatur der EBICS-Response des Kreditinstituts nicht erfolgreich verifizieren. Dennoch sollte beim Eintreffen des Fehlers EBICS_BANK_PUBKEY_UPDATE_REQUIRED die Aktualität der Bankschlüssel überprüft werden und notfalls die neuesten Schlüssel mit Hilfe der administrativen Auftragsart HPB abgeholt werden.

I.e. Teilnehmerbezogene Prüfungen des Auftrags (siehe Abbildung 43)

I.e.a. Überprüfung der Berechtigung für den Geschäftsvorfall (bzw. den administrativen Auftrag)

Hier wird überprüft, ob der Teilnehmer das Recht hat, Aufträge (administrative Aufträge oder BTF) einzureichen. Schlägt diese Überprüfung fehl, dann wird die Transaktionsinitialisierung mit dem fachlichen Fehler EBICS_AUTHORISATION_ORDER_FAILED abgebrochen.

Bei Sendeaufträgen ist die Berechtigungsprüfung erfolgreich, wenn der Teilnehmer für den gegebenen Auftrag zumindest eine EU-Berechtigung der Klasse „T“ besitzt.

Hinweis: Hier wird nicht die EU-Berechtigung der eigentlichen Unterzeichner des Auftrags überprüft. Diese Prüfung ist Teil der (optionalen) Vorabprüfung eines Auftrags.

Bei Downloadaufträgen ist die Auftragsberechtigung nicht an eine EU-Berechtigung gekoppelt. Hier wird überprüft, ob der Teilnehmer eine Berechtigung für die gegebene Auftragsart besitzt.

I.e.b.Bankfachliche Vorabprüfung

Diese Prüfung betrifft nur Sendeaufträge, Details der Vorabprüfung sind in Kapitel 5.3 beschrieben.

Wenn das Banksystem die optionale Vorabprüfung nicht unterstützt, das Kundensystem jedoch Daten zur Vorabprüfung liefert, wird der Hinweis EBICS_NO_ONLINE_CHECKS an das Kundensystem zurückgegeben. Diese fachliche Information hat keinen Einfluss auf die laufende Transaktion, der Auftrag wird fortgesetzt.

Die bankfachliche Vorabprüfung eines Sendeauftrags liefert im Fehlerfall die folgenden fachlichen Returncodes:

- EBICS_SIGNATURE_VERIFICATION_FAILED
  Dieser fachliche Fehler tritt dann auf, wenn die EU eines Unterzeichners des Auftrags nicht erfolgreich verifiziert werden konnte

- EBICS_INVALID_SIGNATURE_FILE_FORMAT
  Die übergebenen EU-Dateien entsprechen nicht dem festgelegten Format

- EBICS_ PARTNER_ID_MISMATCH
  Die PartnerID (=KundenID) der EU-Datei stimmt nicht mit der PartnerID (=KundenID) des Einreichers überein

- EBICS_ACCOUNT_AUTHORISATION_FAILED
  Dieser fachliche Fehlercode wird zurückgegeben, wenn für einen der Unterzeichner die Kontoberechtigungsprüfung fehlschlägt

- EBICS_AMOUNT_CHECK_FAILED
  Dieser fachliche Fehler tritt auf, wenn für einen der Unterzeichner die Limitprüfung fehlschlägt

- EBICS_SIGNER_UNKNOWN
  Dieser fachliche Fehler tritt auf, wenn einer der Unterzeichner kein gültiger Teilnehmer ist

- EBICS_INVALID_SIGNER_STATE
  Dieser fachliche Fehler tritt auf, wenn der Zustand eines Unterzeichners ungleich „Bereit“ ist.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

- Für Return Codes, die CA-Zertifikate betreffen, wird auf Anhang 1 verwiesen.

I.e.c.Überprüfung der Auftragsnummer

Die folgenden Überprüfungen betreffen nur die bankfachlichen Sendeaufträge:

1) Bankfachliche Aufträge dürfen nur ohne Auftragsnummer gesendet werden.

2) Dateien, die nur eine EU, jedoch keine weiteren Daten enthalten, sind nicht zulässig. Dies gilt nur für die administrativen Auftragsarten HVE und HVS (sowie auch SPR)

3) Für HVE und HVS ist zu beachten, dass sich das Verbot des Übertragens von Auftragsnummern bei HVE und HVS natürlich nicht auf die in der VEU befindlichen Auftrag/Auftragsnummer bezieht. Es ist klar, dass hier immer eine Auftragsnummer als Referenzierung (via Orderparams) mitgegeben werden muss.
Folglich sind mögliche Fehlermeldungen für Upload-Requests:

- Bei einem SPR Upload-Request (und für HVE und HVS gleichermaßen) , der mit einer Auftragsnummer übertragen wird, wird die Fehlermeldung EBICS_INVALID_REQUEST_CONTENT zurückgegeben

- Wenn die administrative Aufträge HVE und HVS mit einer dem Bankrechner unbekannten Auftragsnummer übertragen werden, wird die Fehlermeldung EBICS_ORDER_ID_UNKNOWN zurückgegeben

- Bei einem HVE oder HVS-Upload-Request , der mit einer dem Bankrechner bekannten Auftragsnummer übertragen wird, der Auftrag jedoch einen unzulässigen Verarbeitungsstatus hat (weil der Auftrag voll autorisiert oder zurückgewiesen wurde) wird die Fehlermeldung EBICS_ORDERID_ALREADY_FINAL zurückgegeben.

Regel für die Upload-Response: Jede Upload-Response enthält eine Transactions-ID und die durch den Bankrechner vergebene Auftragsnummer (=OrderId, dies gilt auch für Fehlerfälle).

I.f. Erzeugen einer neuen EBICS-Transaktion mit eindeutiger TransaktionsID

Wenn alle vorhergehenden Prüfungen erfolgreich durchgeführt wurden und noch weitere Transaktionsschritte folgen, erzeugt die Transaktionsverwaltung für EBICS-Transaktionen auf Bankseite eine neue EBICS-Transaktion mit einer banksystemweit eindeutigen TransaktionsID. Details zur Generierung der TransaktionsID sind im Anhang (Kapitel 11.6) enthalten.

II. Vorverarbeitung

Die Vorverarbeitung betrifft hier die übermittelten EUs sowie die Auftragsparameter von allen Aufträgen, die in diesem Kapitel betrachtet werden. Eine Ausnahme bilden die administrativen Auftragsarten HVE und HVS. Die Vorverarbeitung ist nicht Bestandteil der EBICS Spezifikation und somit abhängig von der Implementierung des Banksystems. Beispielsweise ist die Zwischenspeicherung der EUs und Auftragsparameter ein Teil dieser Vorverarbeitung.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

III. Weiterleitung an die Verwaltung offener Aufträge

Bei den administrativen Auftragsarten HVE und HVS werdennur EUs zu einem gegebenen Auftrag übertragen, und so besteht die EBICS-Transaktion aus einem einzigen Request-/Response-Paar. In diesem Fall werden die übermittelten Auftragsparameter und EUs direkt an die Verwaltung offener Aufträge weitergeleitet, und die Transaktion wird beendet. Die Komponente Verwaltung offener Aufträge ist nicht Bestandteil des EBICS-Standards.

IV. Erzeugen der EBICS-Response

Dieser Verarbeitungsschritt erzeugt die EBICS-Response, die anschließend an das Kundensystem geschickt wird. Im Fehlerfall enthält diese EBICS-Nachricht den entsprechenden technischen oder fachlichen Fehlercode vorausgegangener Verarbeitungsschritte. Der Inhalt dieser EBICS-Nachricht ist in Kapitel 5.5.1.1.1 genauer beschrieben.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

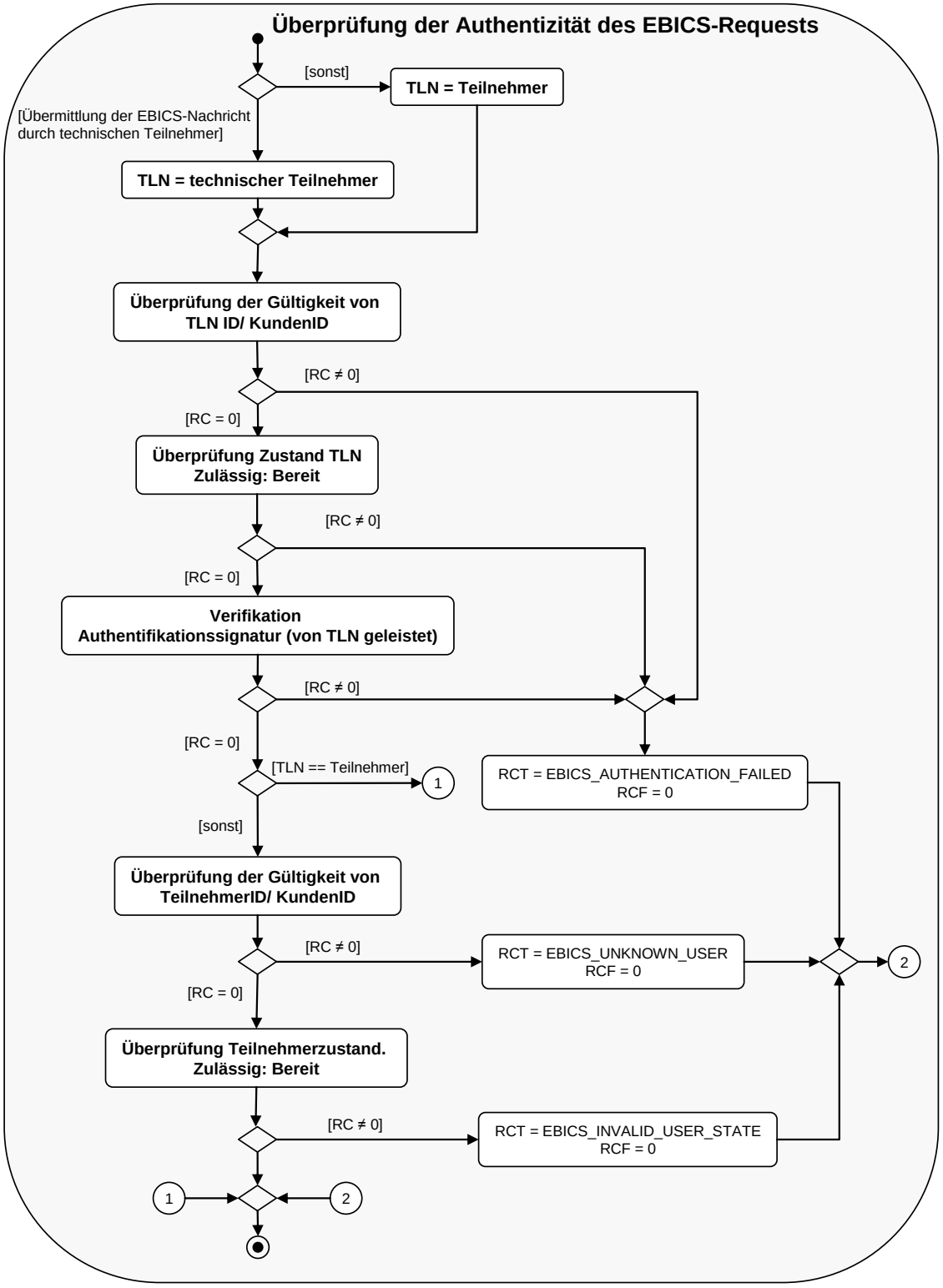


Abbildung 42: Detaillierte Beschreibung des Ablaufschritts „Überprüfung der Authentizität des EBICS-Requests“

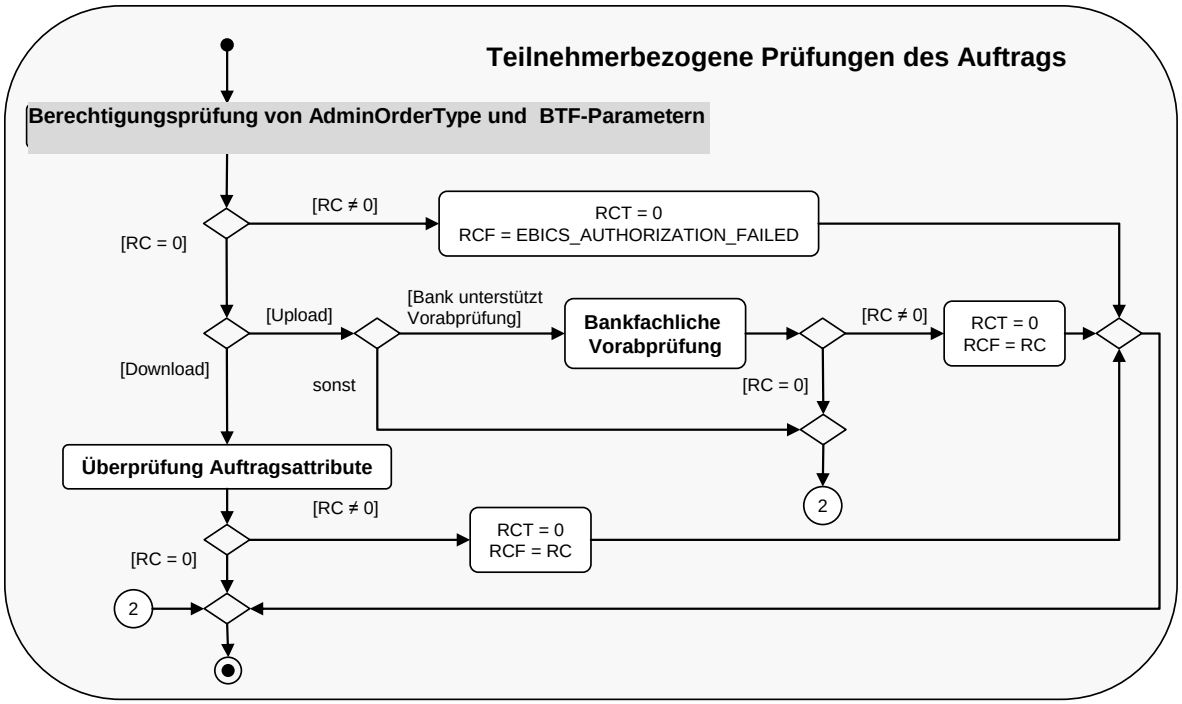


Abbildung 43: Detaillierte Beschreibung des Ablaufschritts „Teilnehmerbezogene Prüfungen des Auftrags“

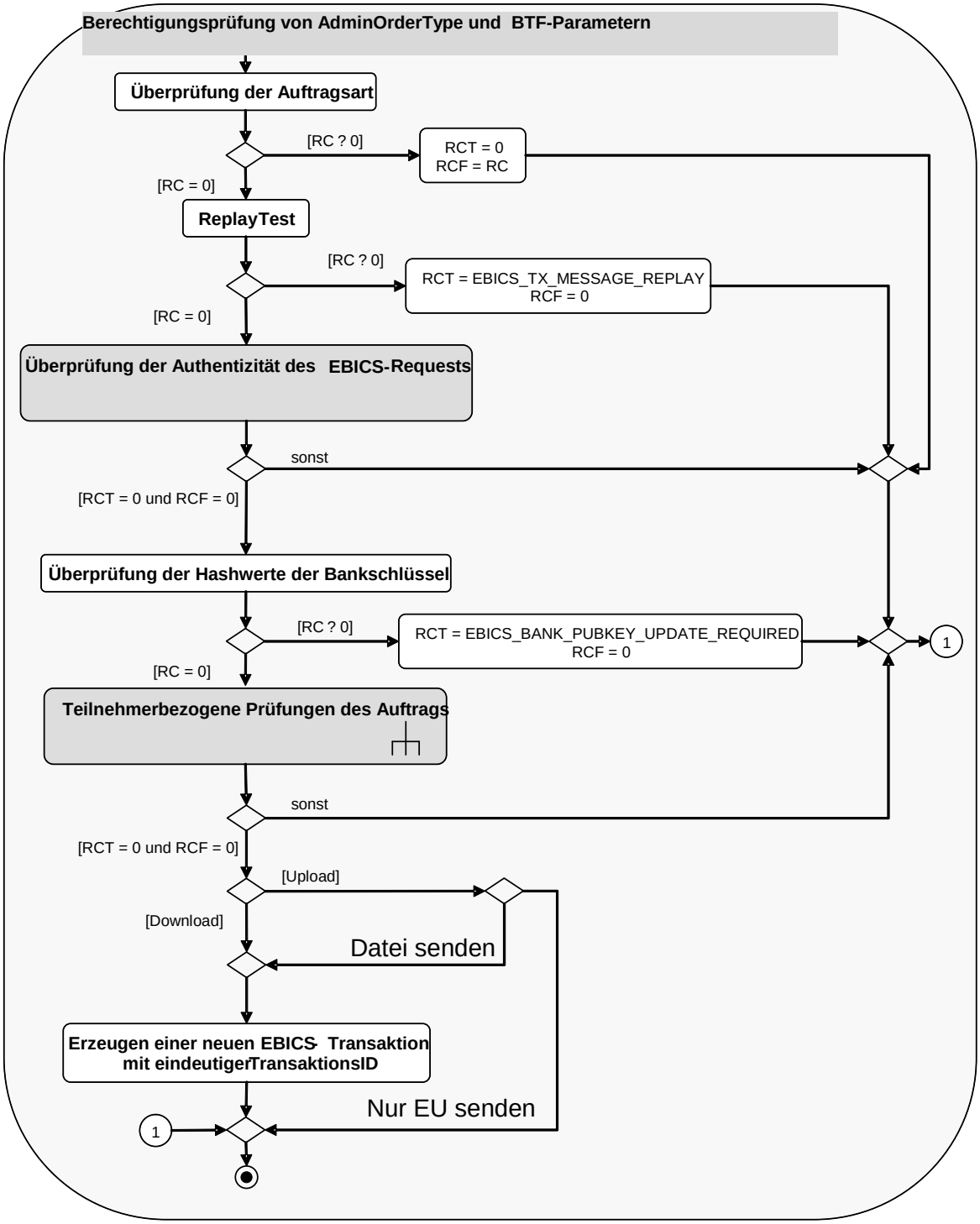


Abbildung 44: Detaillierte Beschreibung des Ablaufschritts „Erzeugung einer EBICS-Transaktion“

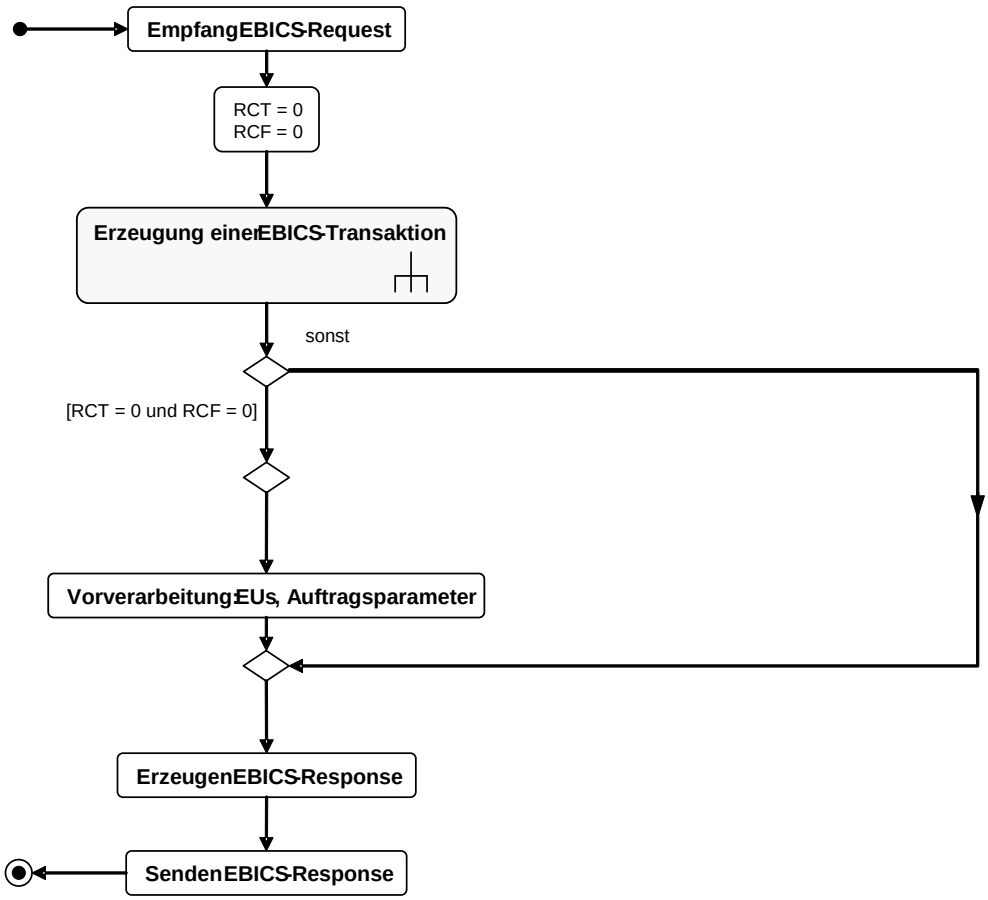


Abbildung 45: Verarbeitung des EBICS-Requests aus der Transaktionsinitialisierung

5.5.1.2.2 Verarbeitung in der Phase des Datentransfers

Die bankseitige Verarbeitung des EBICS-Requests, welcher in der Phase des Datentransfers einer EBICS-Transaktion vom Kundensystem an das Banksystem versendet wird, ist in Abbildung 47 sowie Abbildung 48 dargestellt. Die einzelnen Verarbeitungsschritte werden im Folgenden näher erläutert:

I. Überprüfung der EBICS-Transaktion (siehe

II. Abbildung 46

Dieser Verarbeitungsschritt ist sowohl für Upload- wie auch Download-Transaktionen relevant. Die nachfolgende Beschreibung berücksichtigt beide Transaktionstypen, um dann in den nachfolgenden Kapiteln zum Thema Download Transaktionen auf diese Beschreibung verweisen zu können.

II.a. Überprüfung der TransaktionsID

Hier wird überprüft, ob die EBICS-Transaktion mit der entsprechenden ID in der EBICS-Transaktionsverwaltung des Banksystems als offene, noch nicht abgeschlossene Transaktion existiert. Ist das nicht der Fall, so wird der technische Fehlercode EBICS_TX_UNKNOWN_TXID an den Sender des EBICS-Request geliefert.

II.b. Überprüfung der Authentizität des EBICS-Request (siehe Abbildung 42)

Diese Prüfung der Authentizität des EBICS-Requests erfolgt genau wie in der Initialisierungsphase der Transaktion (siehe Kapitel 5.5.1.2.1, Punkt I.c) – mit dem Unterschied, dass die benötigten Daten (z.B. Kunden-/TeilnehmerID) nicht Teil der Headerdaten des EBICS-Requests sind, sondern in der Transaktionsverwaltung des Kreditinstituts zu der gegebenen TransaktionsID gespeichert werden. Kann die Prüfung nicht erfolgreich durchgeführt werden, so enthält die EBICS-Response einen entsprechenden Fehlercode gemäß dem Ablauf aus Abbildung 42. Es handelt sich dabei um einen der Fehler EBICS_AUTHENTICATION_FAILED, EBICS _USER_UNKNOWN oder EBICS_INVALID_USER_STATE. Nicht authentisierte Requests haben keine Auswirkung auf den Zustand der Transaktion in der Transaktionsverwaltung des Banksystems. Daten, die den Zustand einer Transaktion ausmachen, wie etwa der nächste erwartete Transaktionsschritt oder der aktuelle Recovery-Zähler, werden nicht verändert. Dadurch wird verhindert, dass Angreifer mit Hilfe nicht authentisierter EBICS-Requests Einfluss auf eine Transaktion nehmen können. Die Transaktion kann durch den Teilnehmer fortgesetzt werden, als ob der EBICS-Request mit der ungültigen Authentifikationssignatur nicht empfangen worden wäre.

II.c. Überprüfung TxPhase/ TxSchritt aus dem EBICS-Request

An dieser Stelle wird überprüft, ob der Transaktionsschritt aus dem EBICS-Request zum aktuellen Zustand der EBICS-Transaktion auf Banksystemseite passt, wenn man eine bestimmte sequenzielle Abfolge der Transaktionsschritte voraussetzt.

Im Falle einer Upload-Transaktion wird von der sequenziellen Abfolge gemäß Abbildung 35 ausgegangen. Die Überprüfung der Transaktionsphase/ des Transaktionsschritts ist genau dann erfolgreich, wenn:

- der zuletzt vom Teilnehmer initiierte Transaktionsschritt erfolgreich abgeschlossen wurde, d.h. die Initialisierung bzw. die Übermittlung des n-ten Datensegments erfolgreich war

- der Transaktionsschritt aus dem EBICS-Request der nächste Transaktionsschritt in der sequenziellen Abfolge der Transaktionsschritte ist, d.h. es sich um die Übermittlung des 1-ten bzw. (n+1)-ten Datensegments handelt.

Die normale sequenzielle Abfolge von Transaktionsschritten einer Download-Transaktion ist in Abbildung 52 dargestellt. Die Überprüfung der Transaktionsphase/ des Transaktionsschritts ist genau dann erfolgreich, wenn die beiden folgenden Bedingungen erfüllt sind:

- Der zuletzt vom Teilnehmer initiierte Transaktionsschritt wurde erfolgreich durchgeführt, d.h. die Initialisierung (und somit Übermittlung des ersten Datensegments) bzw. die Anforderung des n-ten Datensegments im Rahmen des Datentransfers waren erfolgreich
- Der Transaktionsschritt aus dem EBICS-Request ist der nächste Transaktionsschritt in der sequenziellen Abfolge der Transaktionsschritte, d.h. es handelt sich um die Anforderung des (n+1)-ten Datensegments oder um die Quittierung der abgeholten Daten, wenn n das letzte Datensegment darstellt.

III. Auswertung des Ergebnisses der Überprüfung der EBICS-Transaktion

Ist die Überprüfung des Transaktionsschritts nicht erfolgreich verlaufen, dann:

- wird überprüft, ob die Upload-Transaktion wieder aufgesetzt werden kann, falls das Banksystem Recovery von Transaktionen unterstützt. Diese Prüfung erfolgt gemäß der Beschreibung aus Kapitel 5.5.2. Kann die Transaktion wieder aufgesetzt werden, so wird der technische Fehlercode EBICS_TX_RECOVERY_SYNC zurückgeliefert, andernfalls wird die Transaktion mit dem technischen Fehlercode EBICS_TX_ABORT abgebrochen
- wird die Upload-Transaktion mit dem fachlichen Fehlercode EBICS_RECOVERY_NOT_SUPPORTED abgebrochen, falls das Banksystem Recovery von Transaktionen nicht unterstützt. Setzt man MAX gleich 0, so wird im Ablaufdiagramm auch der Fall berücksichtigt, in dem Recovery nicht unterstützt wird.

IV. Überprüfung der Segmentnummer und Segmentgröße

Die laufende Nummer des übermittelten Nutzdatensegments (`ebicsRequest/header/mutable/SegmentNumber`) muss kleiner oder gleich der Gesamtanzahl zu übermittelnder Datensegmente sein. Falls die Nummer des übermittelten Nutzdatensegments mit der Gesamtanzahl übereinstimmt, muss zudem der Wert des Attributs `ebicsRequest/header/mutable/SegmentNumber@lastSegment` gleich „true“ sein. Ist eine der beiden Bedingungen nicht erfüllt, so wird die Transaktion mit dem technischen Fehlercode EBICS_TX_SEGMENT_NUMBER_EXCEEDED abgebrochen.

Ist die laufende Nummer des übermittelten Nutzdatensegments kleiner als die Gesamtanzahl zu übermittelnder Nutzdatensegmente und der Wert des Attributs `ebicsRequest/header/mutable/SegmentNumber@lastSegment` dennoch „true“, dann wird der technische Returncode der Fehlerklasse „Hinweis“ EBICS_TX_SEGMENT_NUMBER_UNDERRUN zurückgeliefert.

Die Größe des übermittelten Nutzdatensegments darf die in EBICS „H005“ fest vorgegebene Segmentgröße von 1 MB nicht überschreiten. Sonst wird die Transaktion mit dem technischen Fehlercode EBICS_SEGMENT_SIZE_EXCEEDED abgebrochen.

V. Vorverarbeitung

Die Vorverarbeitung betrifft hier das übermittelte Nutzdatensegment. Die Vorverarbeitung von Nutzdatensegmenten ist nicht Bestandeil des EBICS Spezifikation. Sie ist abhängig von der Implementierung des Banksystems, die Zwischenspeicherung des Nutzdatensegments kann ein Teil der Vorverarbeitung sein.

VI. Weiterleitung an die Verwaltung offener Aufträge

Wenn das übermittelte Nutzdatensegment das letzte war und es sich um einen bankfachlichen Sendeauftrag handelt, dann werden sämtliche im Rahmen der EBICS-Transaktion übermittelten Auftragsparameter, EUs sowie Auftragsdaten an die Verwaltung offener Aufträge weitergeleitet. Die EBICS-Transaktion kann danach beendet werden.
Die Komponente Verwaltung offener Aufträge ist nicht Bestandteil des EBICS-Standards.

(Optionale) Doppeleinreichungskontrolle: Da der Hashwert der Auftragsdaten während der Transferphase übertragen wird, hat der Bankserver die Möglichkeit, über den Hashwert zu überprüfen, ob der Auftrag bereits schon einmal gesendet wurde (falls notwendig, sind weitere Auftragsinformationen für die Prüfung hinzuzuziehen). Wenn der Auftrag bereits auf dem Bankrechner existiert (der gleiche Hashwert wurde bereits schon einmal übertragen), wird der Request mit dem Returncode EBICS_ORDER_ALREADY_EXISTS abgewiesen.

Man beachte, dass diese Prüfungen optional sind. Doppeleinreichungen können auch über andere Mechanismen im Rahmen der Implementierung überwacht werden.

VII. Überprüfung und Durchführung des Auftrags

Wenn das übermittelte Nutzdatensegment das letzte war, und es sich um einen systembedingten Sendeauftrag handelt, dann wird auf der Basis der übermittelten Nutzdaten der Auftrag synchron geprüft und durchgeführt. Die gelieferten technischen oder fachlichen Fehlercodes sind abhängig von der Auftragsart und sind in den Kapiteln definiert, die diese Auftragsarten beschreiben.

VIII. Erzeugen der EBICS-Response

Dieser Verarbeitungsschritt erzeugt die EBICS-Response, die anschließend an das Kundensystem geschickt wird. Im Fehlerfall enthält diese EBICS-Nachricht den technischen/ fachlichen Fehlercode vorausgegangener Verarbeitungsschritte. Der Inhalt dieser EBICS-Nachricht ist in Kapitel 5.5.1.1.2 genauer beschrieben.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

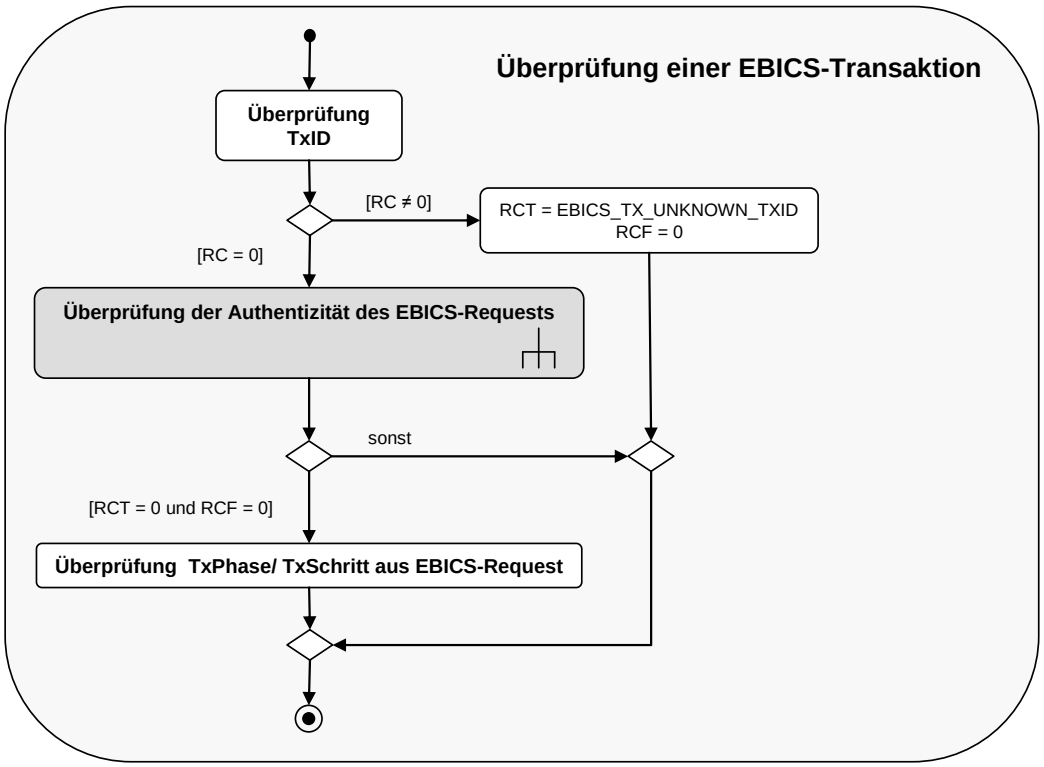


Abbildung 46: Detaillierte Beschreibung des Ablaufschritts „Überprüfung einer EBICS-Transaktion“

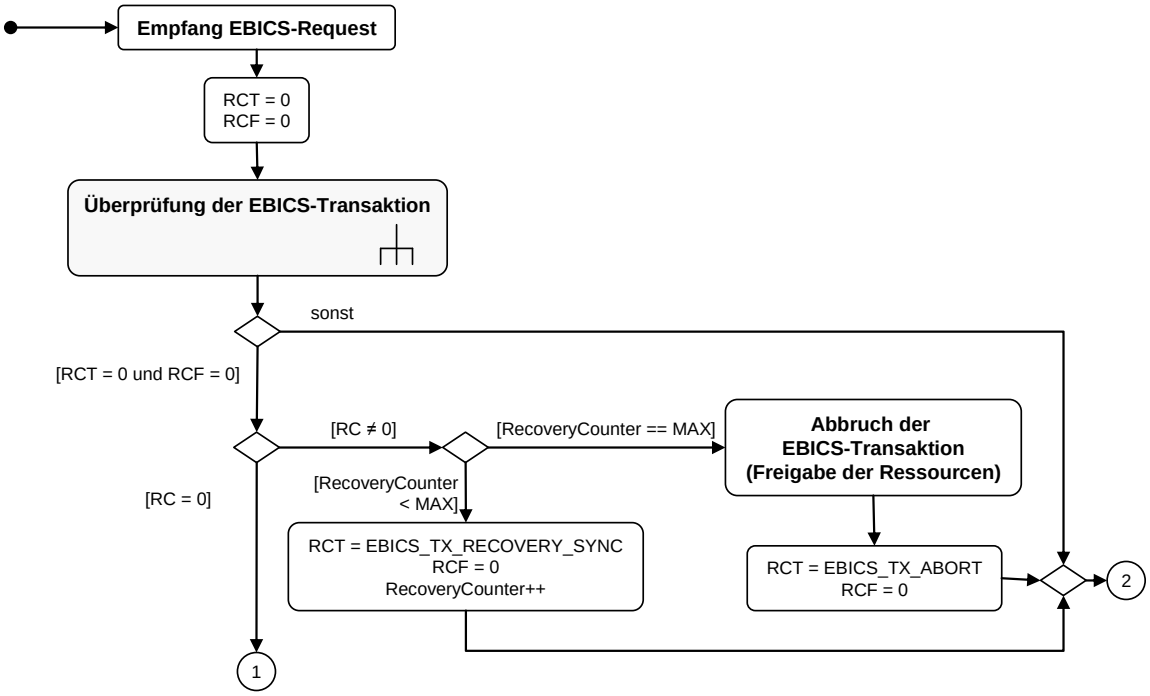


Abbildung 47: Verarbeitung eines EBICS-Requests zur Übermittlung eines Nutzdatensegments (1.Teil)

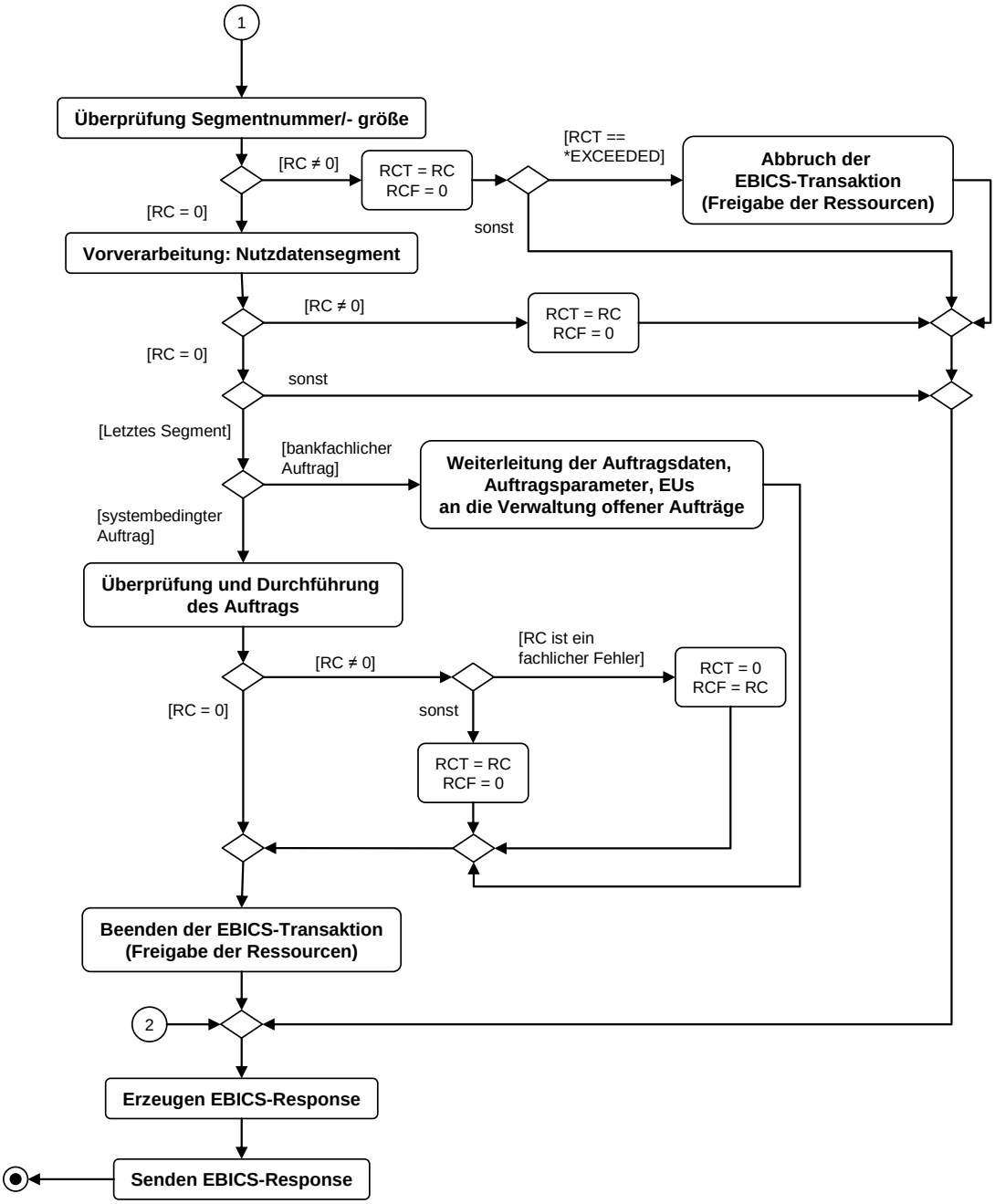


Abbildung 48: Verarbeitung eines EBICS-Requests zur Übermittlung eines Nutzdatensegments (2.Teil)

5.5.2 Wiederaufsetzen von Upload-Transaktionen

Das Kundensystem kann den Recovery-Mechanismus einsetzen, wenn eine der folgenden Fehlersituationen eintritt:

Transportfehler bei der Übertragung eines EBICS-Requests in der Datentransfer-Phase der Transaktion

Timeout bzw. Transportfehler beim Empfang einer EBICS-Response in der Datentransfer-Phase der Transaktion

Verlust des Zustands der Transaktion auf Teilnehmerseite.

Auf Bankseite kann die fehlerhafte Verarbeitung eines EBICS-Requests im Rahmen der Datentransferphase, beispielsweise verursacht durch Fehler in der Vorverarbeitung eines übermittelten Nutzdatensegments, die erneute Übertragung dieses Requests erfordern. Es handelt sich hier um einen Sonderfall von Recovery, da das Kundensystem die Notwendigkeit, die Übertragung zu wiederholen, nicht ohne weiteres erkennt. Mit dem Recovery-Mechanismus von EBICS kann man diesen Sonderfall abhandeln.

EBICS verwendet einen optimistischen Ansatz beim Wiederaufsetzen einer Upload-Transaktion und verzichtet auf einen separaten Synchronisationsschritt mit dem Banksystem. Tritt eine der obigen Fehlersituationen ein, geht das Kundensystem zunächst davon aus, den Wiederaufsetzpunkt der Transaktion aufgrund der im Kundensystem (eventuell persistent) gespeicherten Transaktionsdaten zu kennen.

Wenn das Kundensystem annimmt, der Wiederaufsetzpunkt sei die Übermittlung des n-ten Nutzdatensegments, dann ist der nächste initiierte Transaktionsschritt die Übermittlung des (n+1)-ten Nutzdatensegments. EBICS-Requests im Rahmen des Recovery von Upload-Transaktionen unterscheiden sich nicht von den EBICS-Requests eines normalen, fehlerfreien Ablaufs einer Upload-Transaktion.

In Abbildung 49 ist beispielhaft der Ablauf einer Transaktion dargestellt, die mehrfach ein Wiederaufsetzen erfordert. Das Wiederaufsetzen geschieht jeweils ohne explizite Synchronisation zwischen Kunden- und Banksystem. Das 2. Nutzdatensegment wird dreimal übertragen, da das Kundensystem die entsprechende EBICS-Response wegen eines Timeout- oder eines Transportfehlers nicht empfangen konnte. Bei der zweiten und dritten Übertragung des 2. Nutzdatensegments geht das Kundensystem davon aus, dass der Wiederaufsetzpunkt die Übertragung des 1. Nutzdatensegments ist. Der Wert des RecoveryCounters ist nach der dritten und erfolgreichen Übertragung des 2. Nutzdatensegments gleich 2, da die letzten beiden Übertragungen des 2. Nutzdatensegments vom Banksystem als Recovery-Versuch gewertet werden. Die Transaktion scheitert schließlich an der zu hohen Zahl von Wiederaufsetzversuchen.

Ist die Annahme bzgl. des Wiederaufsetzpunktes falsch, so enthält die EBICS-Response der Übermittlung des (n+1)-ten Datensegments neben dem technischen Returncode EBICS_TX_RECOVERY_SYNC den eigentlichen Wiederaufsetzpunkt der Transaktion. Ist dieser Wiederaufsetzpunkt beispielsweise die Übermittlung des Nutzdatensegments k, so kann die Transaktion nach dieser Synchronisation einfach mit der Übermittlung der Segmente k+1, k+2, usw. fortgesetzt werden.

In Abbildung 50 ist der erfolgreiche Ablauf einer Transaktion dargestellt, der ein Wiederaufsetzen der Transaktion nach einer expliziten Synchronisation zwischen Kunden- und Banksystem enthält. Hier überträgt das Kundensystem das Nutzdatensegment 1 in einem Zustand, in dem das Banksystem eigentlich das Segment 3 erwartet. Die EBICS-Response des Kreditinstituts (siehe Abbildung 51) enthält somit den Wiederaufsetzpunkt der Transaktion und der ist in diesem Fall die Übertragung des 2.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Nutzdatensegments. Danach fährt das Kundensystem mit der Übertragung des Nutzdatensegments 3 fort und beendet die Transaktion mit der Übertragung des letzten Segments 4.

Unabhängig davon, ob ein Kundensystem Fehler im Ablauf einer Transaktion feststellt, kann das Banksystem die erneute Übertragung eines EBICS-Request erzwingen. Das wird dadurch erreicht, dass die zugehörige EBICS-Response analog zu den obigen Recovery-Situationen neben dem Wiederaufsetzpunkt der Transaktion den technischen Returncode EBICS_TX_RECOVERY_SYNC enthält.

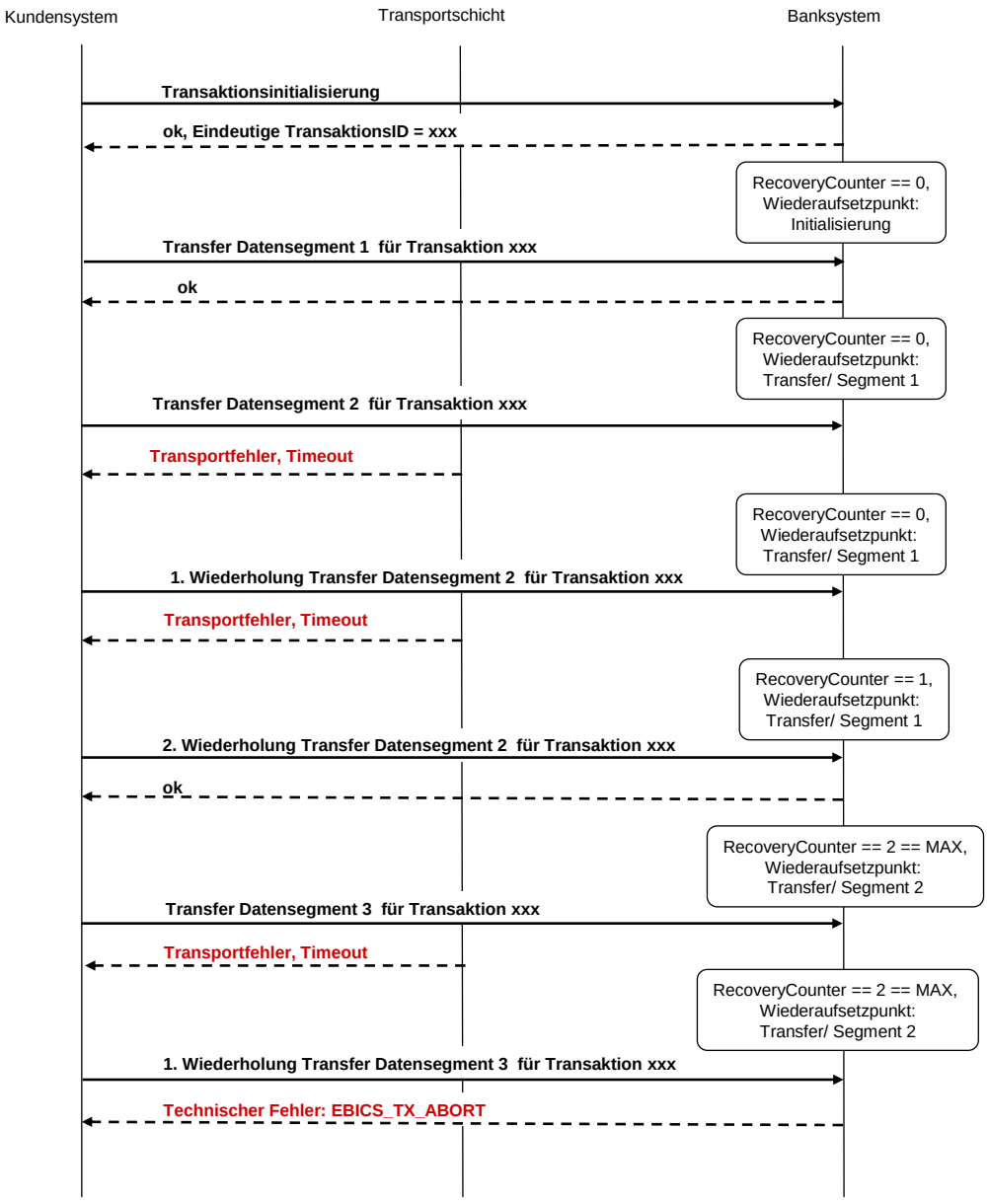


Abbildung 49: Abbruch des Recovery einer Upload-Transaktion aufgrund der Überschreitung der maximal erlaubten Zahl von Recovery-Versuchen

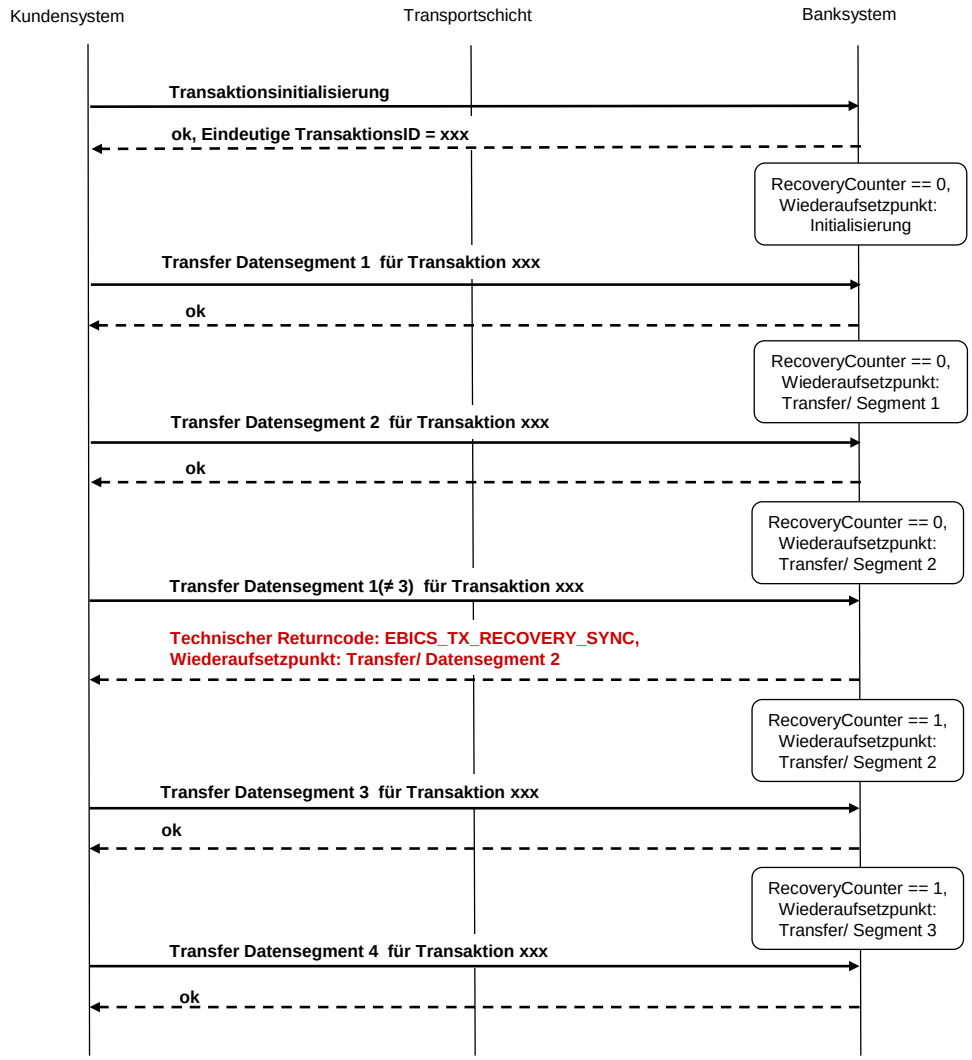


Abbildung 50: Recovery einer Upload-Transaktion mit expliziter Synchronisation zwischen Kunden- und Banksystem

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsResponse
 xmlns="urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation=" urn:org:ebics:H005 ebics_response_H005"
 Version="H005" Revision="1">
 <header authenticate="true">
   <static>     <TransactionID>ABCDEF413946443634453132 43ABCDEF</TransactionID>
   </static>
   <mutable>
     <TransactionPhase>Transfer</TransactionPhase>
     <SegmentNumber lastSegment="false">2</SegmentNumber>
     <OrderId>OR01</OrderId>
     <ReturnCode>061101</ReturnCode>
     <ReportText>[EBICS_TX_RECOVERY_SYNC] Synchronisation erforderlich</ReportText>
   </mutable>
 </header>
 <AuthSignature>
   <ds:SignedInfo>
     <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
```

```
      <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256"/>
      <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
        <ds:Transforms>
          <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
        </ds:Transforms>
        <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
        <ds:DigestValue> …hier Hashwert Authentifikation..</ds:DigestValue>
      </ds:Reference>
    </ds:SignedInfo>
    <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
  </AuthSignature>
  <body>
    <ReturnCode authenticate="true">000000</ReturnCode>
  </body>
</ebicsResponse>
```

Abbildung 51: EBICS-Response mit technischem Fehler EBICS_TX_RECOVERY_SYNC

5.6 Download-Transaktionen

5.6.1 Ablauf von Download-Transaktionen

Der Ablauf einer Download-Transaktion ist in Abbildung 52 anhand eines Sequenzdiagramms dargestellt. Dieses Sequenzdiagramm zeigt den Austausch von EBICS-Nachrichten in den einzelnen Phasen einer Download-Transaktion. Das erste Nutzdatensegment ist bereits in der EBICS-Response der Transaktionsinitialisierung enthalten. Alle weiteren Nutzdatensegmente werden in einer Schleife übertragen, die abbricht, sobald das letzte Datensegment vom Kundensystem empfangen wurde. (Siehe Abbruchbedingung der Schleife „[letztes Datensegment wurde empfangen]“). Schließlich wird der erfolgreiche Empfang aller Nutzdatensegmente vom Kundensystem quittiert.

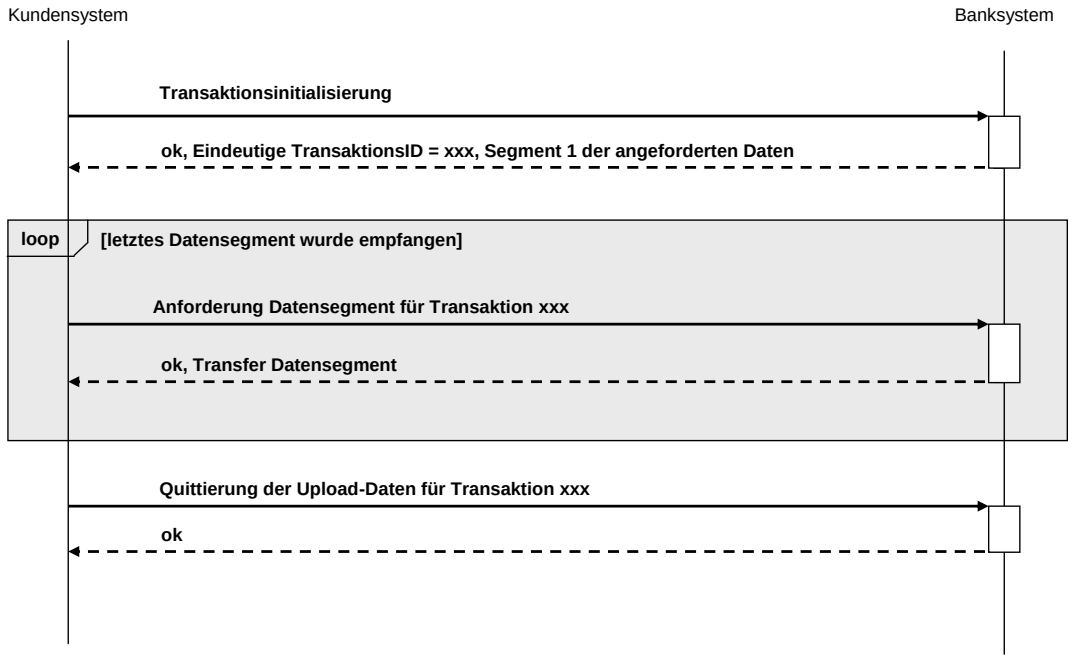


Abbildung 52: Fehlerfreier Ablauf einer Download-Transaktion

5.6.1.1 Beschreibung der EBICS-Nachrichten

Die nachfolgende Beschreibung der Transaktionsschritte einer Download-Transaktion verwendet zur Verdeutlichung Beispielnachrichten für den Download eines MT940-Kontoauszugs, der nach Deutschem (DK-) Regelwerk bereitgestellt wird. Sie nimmt Bezug auf Elemente dieser Beispielnachrichten unter Verwendung der XPath-Notation.

Die folgenden Kapitel beschreiben die EBICS-Nachrichten der einzelnen Phasen einer Download-Transaktion. Es werden die Daten aufgezählt, die Bestandteil dieser Nachrichten sind. Daten, die grundsätzlich optional sind, werden mit „(optional)“ gekennzeichnet. Daten, die nur unter bestimmten Voraussetzungen fehlen dürfen, werden stattdessen mit „(bedingt)“ gekennzeichnet. Optionale XML-Elemente, die in der Beschreibung einer EBICS-Nachricht zu einer bestimmten Transaktionsphase fehlen, dürfen in dieser EBICS-Nachricht nicht vorkommen. Optionale XML-Elemente, die in der Beschreibung einer EBICS-Nachricht zu einer bestimmten Transaktionsphase vorkommen, MÜSSEN für diese EBICS-Nachricht immer entsprechend gesetzt sein.

EBICS-Requests von Download-Transaktionen sind zu ebics_request_H005.xsd konforme (XML-)Instanzdokumente bestehend aus dem Toplevel-Element `ebics`, welches in ebics_request_H005.xsd deklariert ist. EBICS-Responses von Download-Transaktionen sind zu ebics_response_H005.xsd konforme Instanzdokumente, die aus dem Toplevel-Element `ebics` bestehen, das wiederum in ebics_response_H005.xsd deklariert ist.

5.6.1.1.1 EBICS-Nachrichten der Transaktionsinitialisierung

Übertragung der folgenden Daten im EBICS-Request (siehe auch Beispiel in Abbildung 53):

- Host-ID des EBICS Bankrechners (`ebicsRequest/header/static/HostID`)
- Transaktionsphase (`ebicsRequest/header/mutable/TransactionPhase`) mit der Belegung „Initialisation“
- Kombination aus Nonce und Timestamp für die Vermeidung von Replay alter EBICS-Nachrichten (`ebicsRequest/header/static/Nonce`, `ebicsRequest/header/static/Timestamp`)
- Teilnehmer (`ebicsRequest/header/static/PartnerID`, `ebicsRequest/header/static/UserID`), welcher den Auftrag einreicht oder bankfachliche EUs zu einem bestehenden Auftrag nachliefert
- (bedingt) technischer Teilnehmer (`ebicsRequest/header/static/PartnerID`, `ebicsRequest/header/static/SystemID`)
  `SystemID` muss genau dann vorhanden sein, wenn es sich bei dem Kundensystem um ein Multi-User-System handelt. Der technische Teilnehmer ist für das Erstellen der EBICS-Requests (inklusive der Authentifikationssignaturen) zuständig, die zu Aufträgen gehören, die vom Teilnehmer eingereicht oder bankfachlich unterzeichnet werden
- (optional) Information zum Kundenprodukt (`ebicsRequest/header/static/Product`)

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

- Administrative Auftragsart (ebicsRequest/header/static/OrderDetails/AdminOrderType)
- Auftragsparameter (`ebicsRequest/header/static/OrderDetails/OrderParams`).
  Die Ausprägung der Auftragsparameter ist abhängig von der administrativen Auftragsart. Für das Abholen von bankfachlichen Aufträgen (BTF) sind die Auftragsparameter und das Regelwerk ihrer Verwendung im Detail in Kapitel 5.6.1.1.4 beschrieben.
- Hashwerte der öffentlichen Schlüssel des Kreditinstituts, über die der Teilnehmer verfügt (ebicsRequest/header/static/BankPubKeyDigests/Authentication, ebicsRequest/header/static/BankPubKeyDigests/Encryption, ebicsRequest/header/static/BankPubKeyDigests/Signature).
  Zu jedem dieser Hashwerte wird sowohl der verwendete Hashalgorithmus als auch die Version des entsprechenden Authentifikations- bzw. Verschlüsselungs- bzw. Signaturverfahrens angegeben.
  In der Version „H005“ des EBICS-Protokolls ist die EU der Kreditinstitute nur vorgesehen (siehe dazu Kapitel 3.5.2). Das Element BankPubKeyDigests/Signature ist in Vorbereitung zukünftiger EBICS-Versionen bereits in dieser Beschreibung enthalten, obwohl seine maximale Häufigkeit (maxOccurs) in der Version „H005“ gleich 0 ist
- Sicherheitsmedium für den bankfachlichen Schlüssel des Teilnehmers (`ebicsRequest/header/static/SecurityMedium`)
- Authentifikationssignatur des technischen Teilnehmers, falls ein solcher vorhanden ist, ansonsten die Authentifikationssignatur des Teilnehmers selbst (`ebicsRequest/AuthSignature`)
  Die Authentifikationssignatur umfasst alle XML-Elemente des EBICS-Requests, deren Attributwert für `@authenticate` gleich „true“ ist. Die Definition des XML-Schemas „ebics_request_H005.xsd“ gewährleistet, dass der Wert des Attributs `@authenticate` für genau die Elemente gleich „true“ ist, die auch unterschrieben werden müssen

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest
 xmlns="urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_request_H005.xsd"
 Version="H005" Revision="1">
 <header authenticate="true">
   <static>
     <HostID>EBIXHOST</HostID>
     <Nonce>98498A65465C645E645F64565462C645</Nonce>
     <Timestamp>2005-01-30T15:40:45.123Z</Timestamp>
     <PartnerID>PARTNER1</PartnerID>
     <UserID>USER0001</UserID>
     <Product Language="de" InstituteID="Institutskennung">Kundenproduktkennung</Product>
     <OrderDetails>
       <AdminOrderType>BTD</AdminOrderType>
BTDOrderParamsBTD>
         <Service>
           <ServiceName>EOP</ServiceName>
           <Scope>DE</Scope>
           <MsgName>mt940</MsgName>
  </Service>           <Start>2016-09169-01</Start>
```

```
            <End>2016-09169-30</End>
          </DateRange>
        </BTDOrderParamsBTD>
      </OrderDetails>
      <BankPubKeyDigests>
        <Authentication Version="X002"
Algorithm="http://www.w3.org/2001/04/xmlenc#sha256">1H/rQr2Axe9hYTV2n/tCp+3UIQQ=</Authentication>
        <Encryption Version="E002"
Algorithm="http://www.w3.org/2001/04/xmlenc#sha256">2joEROI3092OIFP394+WOIer2WI=</Encryption>
      </BankPubKeyDigests>
      <SecurityMedium>0000</SecurityMedium>
    </static>
    <mutable>
      <TransactionPhase>Initialisation</TransactionPhase>
    </mutable>
  </header>
  <AuthSignature>
    <ds:SignedInfo>
      <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
      <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256"/>
      <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
        <ds:Transforms>
          <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
        </ds:Transforms>
        <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
        <ds:DigestValue> …hier Hashwert Authentifikation..</ds:DigestValue>
      </ds:Reference>
    </ds:SignedInfo>
    <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
  </AuthSignature>
  <body/>
</ebicsRequest>
```

Abbildung 53: EBICS-Request für die Transaktionsinitialisierung für das Abholen eines Kontoauszugs im Format MT940

Übertragung der folgenden Daten in der EBICS-Response (siehe Beispiel in Abbildung 54):

- bankfachlicher Returncode (`ebicsResponse/body/ReturnCode`)
- technischer Returncode (ebicsResponse/header/mutable/ReturnCode)
- technischer Reporttext (ebicsResponse/header/mutable/ReportText)
- (bedingt) banksystemweit eindeutige TransaktionsID (`ebicsResponse/header/static/TransactionID`), falls während der Transaktions-initialisierung keine Fehler technischer oder bankfachlicher Art aufgetreten sind
- Transaktionsphase (`ebicsResponse/header/mutable/TransactionPhase`) mit der Belegung „Initialisation“
- (bedingt) Gesamtzahl zu übertragender Nutzdatensegmente (`ebicsResponse/header/static/NumSegments`), falls keine Fehler technischer oder bankfachlicher Art aufgetreten sind
- (bedingt) laufende Nummer des in dieser Response übermittelten Nutzdatensegments (`ebicsResponse/header/mutable/SegmentNumber`), falls keine Fehler technischer oder bankfachlicher Art aufgetreten sind.

`SegmentNumber` ist in der Initialisierungsphase stets mit 1 belegt. Das Attribut `ebicsResponse/header/mutable/SegmentNumber@lastSegment` gibt an, ob es sich dabei um das letzte Datensegment handelt

- Authentifikationssignatur des Kreditinstituts (`ebicsResponse/AuthSignature`)
  Die Authentifikationssignatur umfasst alle XML-Elemente der EBICS-Response, deren Attributwert für `@authenticate` gleich „true“ ist. Die Definition des XML-Schemas „ebics_response_H005.xsd“ gewährleistet, dass der Wert des Attributs `@authenticate` für genau die Elemente gleich „true“ ist, die auch unterschrieben werden müssen
- (bedingt) Information zur Verschlüsselung der Auftragsdaten und eventuell der EU der Auftragsdaten (`ebicsResponse/body/DataTransfer/DataEncryptionInfo`), falls keine Fehler technischer oder bankfachlicher Natur aufgetreten sind.
  `DataEncryptionInfo` enthält insbesondere auch den asymmetrisch verschlüsselten Transaktionsschlüssel (`ebicsResponse/body/DataTransfer/DataEncryptionInfo/TransactionKey`)
- (bedingt) das erste Nutzdatensegment (`ebicsResponse/body/DataTransfer/OrderData`), falls keine Fehler technischer oder bankfachlicher Natur aufgetreten sind
- (bedingt) die bankfachliche EU der Nutzdaten durch das Kreditinstitut (`ebicsResponse/body/DataTransfer/SignatureData`), falls keine Fehler technischer oder bankfachlicher Natur aufgetreten sind.
  In der Version „H005“ des EBICS-Protokolls ist die EU der Kreditinstitute nur vorgesehen (siehe dazu Kapitel 3.5.2).
- (optional) Zeitstempel der letzten Aktualisierung der Bankparameter (`ebicsResponse/body/TimestampBankParameter`).

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsResponse
xmlns="urn:org:ebics:H005"
xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="urn:org:ebics:H005 ebics_response_H005.xsd "
Version="H005" Revision="1">
<header authenticate="true">
  <static>
    <TransactionID>FEDCBA4139464436445313243FEDCBA</TransactionID>
    <NumSegments>2</NumSegments>
  </static>
  <mutable>
    <TransactionPhase>Initialisation</TransactionPhase>
    <SegmentNumber lastSegment="false">1</SegmentNumber>
    <ReturnCode>000000</ReturnCode>
    <ReportText>[EBICS_OK] OK</ReportText>
  </mutable>
</header>
<AuthSignature>
  <ds:SignedInfo>
    <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
    <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256">
    </ds:SignatureMethod>
```

```
    <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
      <ds:Transforms>
        <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
      </ds:Transforms>
      <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
      <ds:DigestValue> …hier Hashwert Authentifikation..</ds:DigestValue>
    </ds:Reference>
  </ds:SignedInfo>
    <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
</AuthSignature>
<body>
  <DataTransfer>
    <DataEncryptionInfo authenticate="true">
      <EncryptionPubKeyDigest Version="E002" Algorithm="http://www.w3.org/2001/04/xmlenc#sha256">..hier
Hashwert des öffentlichen Bankschlüssels für die Verschlüsselung ..</EncryptionPubKeyDigest>
      <TransactionKey>En6KEB6ArEzw+iq4N1wm6Eptcyx…</TransactionKey>
      </DataEncryptionInfo>
    <OrderData>…</OrderData>
  </DataTransfer>
  <ReturnCode authenticate="true">000000</ReturnCode>
</body>
</ebicsResponse>
```

Abbildung 54: EBICS-Response der Transaktionsinitialisierung für das Abholen eines Kontoauszugs im Format MT940

5.6.1.1.2 EBICS-Nachrichten der Phase des Datentransfers

Übertragung der folgenden Daten im EBICS-Request (siehe auch Beispiel in Abbildung 55):

- Host-ID des EBICS Bankrechners (`ebicsRequest/header/static/HostID`)
- Daten zur Identifikation des aktuellen Transaktionsschritts:
  - TransaktionsID (ebicsRequest/header/static/TransactionID)
  - Transaktionsphase (`ebicsRequest/header/mutable/TransactionPhase`) mit der Belegung „Transfer“
  - laufende Nummer des Nutzdatensegments, das in diesem Transaktionsschritt abgeholt werden soll (`ebicsRequest/header/mutable/SegmentNumber`) Das Attribut `ebicsRequest/header/mutable/SegmentNumber@lastSegment` hat für diesen EBICS-Request keine Bedeutung
- Authentifikationssignatur des technischen Teilnehmers, falls ein solcher vorhanden ist, ansonsten die Authentifikationssignatur des Teilnehmers selbst (`ebicsRequest/AuthSignature`)
  Die Authentifikationssignatur umfasst alle XML-Elemente des EBICS-Requests, deren Attributwert für `@authenticate` gleich „true“ ist. Die Definition des XML-Schemas „ebics_request_H005.xsd“ gewährleistet, dass der Wert des Attributs `@authenticate` für genau die Elemente gleich „true“ ist, die auch unterschrieben werden müssen.

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest
<ebicsRequest
 xmlns="urn:org:ebics:H005"
```

```
xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="urn:org:ebics:H005 ebics_request_H005.xsd"
Version="H005" Revision="1">
<header authenticate="true">
  <static>
    <HostID>EBIXHOST</HostID>
    <TransactionID>FEDCBA4139464436344531324 3FEDCBA</TransactionID>
  </static>
  <mutable>
    <TransactionPhase>Transfer</TransactionPhase>
    <SegmentNumber lastSegment="false">2</SegmentNumber>
  </mutable>
</header>
<AuthSignature>
  <ds:SignedInfo>
    <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
    <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256">
    </ds:SignatureMethod>
    <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
      <ds:Transforms>
        <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
      </ds:Transforms>
      <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
      <ds:DigestValue> …hier Hashwert Authentifikation../ds:DigestValue>
    </ds:Reference>
  </ds:SignedInfo> <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
</AuthSignature>
<body/>
</ebicsRequest>
```

Abbildung 55: EBICS-Request für die Übertragung des nächsten Nutzdatensegments für den Kontoauszug im Format MT940

Übertragung der folgenden Daten in der EBICS-Response (siehe Beispiel in Abbildung 56)

- bankfachlicher Returncode (`ebicsResponse/body/ReturnCode`)
- technischer Returncode (ebicsResponse/header/mutable/ReturnCode)
- technischer Reporttext (ebicsResponse/header/mutable/ReportText)
- Daten zur Identifikation eines Transaktionsschritts
  Falls der technische Returncode den Wert EBICS_TX_RECOVERY_SYNC hat, identifiziert dieser Transaktionsschritt den letzten Wiederaufsetzpunkt der Download-Transaktion. Sind jedoch weder technische noch fachliche Fehler aufgetreten, so spiegelt dieser Transaktionsschritt den gerade aktuellen Transaktionsschritt wider:
  - TransaktionsID (ebicsResponse/header/static/TransactionID)
  - Transaktionsphase (ebicsResponse/header/mutable/TransactionPhase) mit der Belegung “Transfer”
  - laufende Nummer des Nutzdatensegments (`ebicsResponse/header/mutable/SegmentNumber`).
    Es handelt sich hierbei um die Nummer des Nutzdatensegments, die vom EBICS-Request angefordert wurde oder im Falle des Fehlers EBICS_TX_RECOVERY_SYNC um die Nummer des letzten Nutzdatensegments,

welches aus Sicht des Banksystems erfolgreich an das Kundensystem übermittelt wurde. Im Fall des Fehlers EBICS_TX_RECOVERY_SYNC ist der Wert für `SegmentNumber` immer gleich 1, falls der Wert für `TransactionPhase` „Initialisation“ ist.
Das Attribut `ebicsResponse/header/mutable/SegmentNumber@lastSegment` gibt jeweils an, ob es sich um das letzte Nutzdatensegment handelt.

- Authentifikationssignatur des Kreditinstituts (`ebicsResponse/AuthSignature`)
  Die Authentifikationssignatur umfasst alle XML-Elemente der EBICS-Response, deren Attributwert für `@authenticate` gleich „true“ ist. Die Definition des XML-Schemas „ebics_response_H005.xsd“ gewährleistet, dass der Wert des Attributs `@authenticate` für genau die Elemente gleich „true“ ist, die auch unterschrieben werden müssen
- (bedingt) das angeforderte Nutzdatensegment (`ebicsResponse/body/DataTransfer/OrderData`), falls keine Fehler technischer oder bankfachlicher Natur aufgetreten sind.

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsResponse
 xmlns="urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_response_H005.xsd"
 Version="H005" Revision="1">
 <header authenticate="true">
   <static>
     <TransactionID>FEDCBA4139464436344531324 3FEDCBA</TransactionID>
   </static>
   <mutable>
     <TransactionPhase>Transfer</TransactionPhase>
     <SegmentNumber lastSegment="true">2</SegmentNumber>
     <ReturnCode>000000</ReturnCode>
     <ReportText>[EBICS_OK] OK</ReportText>
   </mutable>
 </header>
 <AuthSignature>
   <ds:SignedInfo>
     <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
     <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256">
     </ds:SignatureMethod>
     <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
       <ds:Transforms>
         <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
       </ds:Transforms>
       <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
       <ds:DigestValue> …hier Hashwert Authentifikation..</ds:DigestValue>
     </ds:Reference>
   </ds:SignedInfo>
   <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
 </AuthSignature>
 <body>
   <DataTransfer>
     <OrderData>…</OrderData>
   </DataTransfer>
   <ReturnCode authenticate="true">000000</ReturnCode>
```

```
 </body>
</ebicsResponse>
```

Abbildung 56: EBICS-Response der Übertragung des letzten Nutzdatensegments für den Kontoauszug im Format MT940

5.6.1.1.3 EBICS-Nachrichten der Quittierungsphase

Übertragung der folgenden Daten im EBICS-Request (siehe auch Beispiel in Abbildung 57):

- Host-ID des EBICS Bankrechners (`ebicsRequest/header/static/HostID`)
- Daten zur Identifikation des aktuellen Transaktionsschritts:
  - TransaktionsID (ebicsRequest/header/static/TransactionID)
  - Transaktionsphase (`ebicsRequest/header/mutable/TransactionPhase`) mit der Belegung „Receipt“
- Authentifikationssignatur des technischen Teilnehmers, falls ein solcher vorhanden ist, ansonsten die Authentifikationssignatur des Teilnehmers selbst (`ebicsRequest/AuthSignature`)
  Die Authentifikationssignatur umfasst alle XML-Elemente des EBICS-Requests, deren Attributwert für `@authenticate` gleich „true“ ist. Die Definition des XML-Schemas „ebics_request_H005.xsd“ gewährleistet, dass der Wert des Attributs `@authenticate` für genau die Elemente gleich „true“ ist, die auch unterschrieben werden müssen
- Quittung (`ebicsRequest/body/TransferReceipt/ReceiptCode`):
  Der Wert der Quittung ist 0 („positive Quittung“), wenn die Abholung und Verarbeitung der Nutzdaten erfolgreich war. Andernfalls ist der Wert der Quittung 1 („negative Quittung“).

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest
 xmlns="urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_request_H005.xsd"
 Version="H005" Revision="1">
 <header authenticate="true">
   <static>
     <HostID>EBIXHOST</HostID>
     <TransactionID>FEDCBA4139464436344531324 3FEDCBA</TransactionID>
   </static>
   <mutable>
     <TransactionPhase>Receipt</TransactionPhase>
   </mutable>
 </header>
 <AuthSignature>
   <ds:SignedInfo>
     <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
     <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256">
     </ds:SignatureMethod>
     <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
       <ds:Transforms>
         <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
       </ds:Transforms>
       <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
       <ds:DigestValue> …hier Hashwert Authentifikation..</ds:DigestValue>
     </ds:Reference>
```

```
    </ds:SignedInfo>
    <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
  </AuthSignature>
  <body>
    <TransferReceipt authenticate="true">
      <ReceiptCode>0</ReceiptCode>
    </TransferReceipt>
  </body>
</ebicsRequest>
```

Abbildung 57: EBICS-Request für die Quittierung der Download-Daten

Übertragung der folgenden Daten in der EBICS-Response (siehe Beispiel in Abbildung 58)

- bankfachlicher Returncode (`ebicsResponse/body/ReturnCode`)
- technischer Returncode (ebicsResponse/header/mutable/ReturnCode)
- technischer Reporttext (ebicsResponse/header/mutable/ReportText)
- Daten zur Identifikation eines Transaktionsschritts:
  Falls der technische Returncode den Wert EBICS_TX_RECOVERY_SYNC hat, identifiziert dieser Transaktionsschritt den letzten Wiederaufsetzpunkt der Download-Transaktion. Sind jedoch weder technische noch fachliche Fehler aufgetreten, so spiegelt dieser Transaktionsschritt den gerade aktuellen Transaktionsschritt wider, also die Quittierung der Abholdaten.
- TransaktionsID (ebicsResponse/header/static/TransactionID)
- Transaktionsphase (ebicsResponse/header/mutable/TransactionPhase) mit der Belegung “Receipt”
- (bedingt) laufende Nummer des Nutzdatensegments (`ebicsResponse/header/mutable/SegmentNumber`), falls der Fehler EBICS_TX_RECOVERY_SYNC aufgetreten ist und demzufolge der Wert für `TransactionPhase` „Initialisation“ oder „Transfer“ ist.
  Es handelt sich hierbei um die Nummer des Nutzdatensegments, welches aus Sicht des Banksystems zuletzt erfolgreich an das Kundensystem übermittelt wurde. Der Wert für `SegmentNumber` ist immer gleich 1, falls der Wert für `TransactionPhase` „Initialisation“ ist.
  Das Attribut `ebicsResponse/header/mutable/SegmentNumber@lastSegment` gibt jeweils an, ob es sich um das letzte Nutzdatensegment handelt.
- Authentifikationssignatur des Kreditinstituts (`ebicsResponse/AuthSignature`)
  Die Authentifikationssignatur umfasst alle XML-Elemente der EBICS-Response, deren Attributwert für `@authenticate` gleich „true“ ist. Die Definition des XML-Schemas „ebics_response_H005.xsd“ gewährleistet, dass der Wert des Attributs `@authenticate` für genau die Elemente gleich „true“ ist, die auch unterschrieben werden müssen.

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsResponse
 xmlns="urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
```

```
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_response_H005.xsd"
 Version="H005" Revision="1">
 <header authenticate="true">
   <static>
     <TransactionID>FEDCBA4139464436344531324 3FEDCBA</TransactionID>
   </static>
   <mutable>
     <TransactionPhase>Receipt</TransactionPhase>
     <ReturnCode>011000</ReturnCode>
     <ReportText>[EBICS_POSTPROCESS_DONE] positive Quittung erhalten </ReportText>
   </mutable>
 </header>
 <AuthSignature>
   <ds:SignedInfo>
     <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
     <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256">
     </ds:SignatureMethod>
     <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
       <ds:Transforms>
         <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
       </ds:Transforms>
       <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
       <ds:DigestValue> …hier Hashwert Authentifikation..</ds:DigestValue>
     </ds:Reference>
   </ds:SignedInfo>
   <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
 </AuthSignature>
 <body>
   <ReturnCode authenticate="true">000000</ReturnCode>
 </body>
</ebicsResponse>
```

Abbildung 58: EBICS-Response der Quittierung der Download-Daten

5.6.1.1.4 Download-Request-Struktur für bankfachliche Geschäftsvorfälle und Formate (BTF)

Der Standardprozess ist in Kapitel 5.6.1 beschrieben. Die Werte der Auftragsparameter finden sich unter `ebicsRequest/header/static/OrderDetails`
und sind vom Typ `BTDOrderParams` – Diese werden für das Abholen und mit der administrativen Auftragsart “BTD” verwendet:

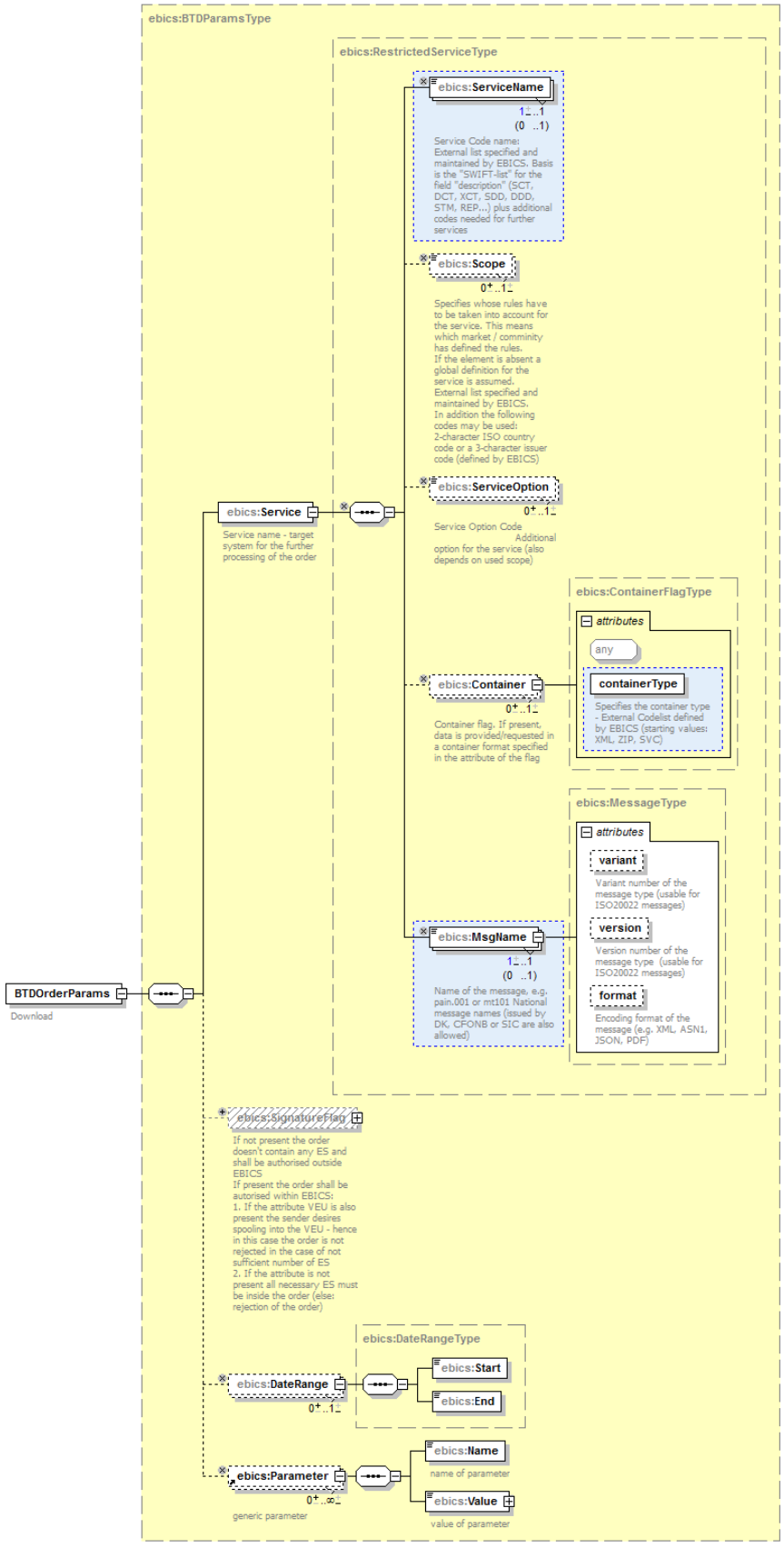


Abbildung 59: BTF-Struktur für Abholung (verwendet eingeschränkte Service-Struktur)

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
BTDOrderParams	ebics:BTDParamsTypeBTUParamsType (komplex)	1	Notwendig zur Kennzeichnung eines Abhol-Geschäftsvorfalls und des verwendeten / zugehörigen Formats	- (complex)
Service	RestrictedServiceType (komplex)	1	Kennzeichnet, wie die Datei auf Seiten des Zielsystems zu verarbeiten ist	- (komplex)
ServiceName	ebics: ServiceNameStringType (simple) - restriction base: minLength value="3" maxLength value="3" pattern = [A-Z0-9]	1	Name der Services	ServiceName ist Thema einer externen Codeliste (gepflegt von EBICS) Beispiel:: “REP” = Report
Scope	ebics: ScopeStringType (simple) restriction base: minLength value="2" maxLength value="3"	0..1	Kennzeichnet, wer die Regeln vorgegeben hat, die für den Service hier zu beachten sind, d.h. welcher Markt/welche Community die zu beachtenden Regeln definiert hat. Eine externe Scope-Liste wird von EBICS spezifiziert und gepflegt. Ein fehlender Scope bedeutet, dass es sich um global akzeptierte (gleich verstandene) Regelwerke handelt	Scope-Codes werden in einer externen Codeliste (gepflegt von EBICS) spezifiziert 2-stellige ISO-Ländercodes 3-stellige Codes für andere Scopes “BIL” bedeutet “bilateral” abgestimmt Ein fehlender Scope bedeutet “global”. Statt eines fehlenden scopes kann hier auch der scope-Code “GLB” gesetzt werden.
ServiceOption	ebics:CodeStringType (simple)	0..1	Optionale Eigenschaft(en)	Options-Codes werden in einer externen

	restriction base: minLength value="3" maxLength value="10" pattern = [A-Z0-9]		eines Service	Codeliste (gepflegt von EBICS) spezifiziert Beispiel: “B2B” bedeutet im Kontext ServiceName = “REP”, dass der Report nur Informationen über Firmenlastschriften enthält.
`Container`	`ebics:ContainerFlagType`	0..1	Kennzeichnung, ob ein Container verwendet wird	Es ist nur der Wert `true` erlaubt. Fehlen bedeutet `false`
`Container@containerType`	`ebics:ContainerstringType` (simple) restriction base: minLength value="3" maxLength value="3" pattern = [a-z]	1..1	Kennzeichnung, welche Art von Container verwendet wird	Im Falle der Nutzung des Container-Flags ist die Belegung des Attributs verpflichtend (interne Codeliste) “XML” “ZIP” “SVC”
`MsgName`	`ebics: MessageNameStringType` (simple) restriction base: minLength value="1" maxLength value="10" pattern = [a-z\.0-9]	1..1	Nachrichtenname, entweder beginnend mit dem BA-Code (ISO) oder MT (FIN) oder einer Zeichenkette	“pain.002”, “mt940” (Proprietäre) Nachrichtennamen, (spezifiziert durch den angegebenen Scope) sind zulässig
`MsgName@version`	`ebics:NumString` (simple) restriction base: minLength value="2" maxLength value="2" pattern = [0-9]	0..1	Verwendete ISO-Version der Nachricht (wenn es sich um kein ISO-Format handelt, wird dieses Attribut ignoriert)	“03”
`MsgName@variant`	`ebics:NumString` (simple) restriction base: minLength value="3" maxLength value="3" pattern = [0-9]	0..1	Wird zusammen mit <MsgName> betrachtet, (wenn es sich um kein ISO-Format handelt, wird dieses Attribut ignoriert)	“001”
`MsgName@format`	`ebics:CodeString` (simple) restriction base: minLength value="1" maxLength value="4" pattern = [A-Z0-9]	0..1	Wird zusammen mit <MsgName> betrachtet, ist für jede Art von Format zulässig, sollte aber nur	„XML“, „ASN1“, „JSON“, „PDF“

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

			verwendet werden, wenn diese Angabe eine Abweichung zum Standard darstellt (insbesondere, wenn kein XML für ISO 20022 verwendet wird).	
dateRange	DateRangeType (complex)	0..1	Spezifiziert einen Datumsbereichfür die angeforderte Nachricht	
start	DateType	1..1	Startdatum (inkl.)	2018-10-11
end	DateType	1..1	Endedatum (inkl.)	2018-10-11
Parameter		0..1	Frei definierbare Parameter. Eine beliebige Anzahl von Wertepaaren kann spezifiziert werden.	
Name	boolean	1..1	Name des Parameters	
Value	anySimpleType	1..1	Wert des Parameters	
Value@type	NCName	1..1	Datentyp des Wertes	Defaultempfehlung ist string

5.6.1.2 Verarbeitung der EBICS-Nachrichten

Kapitel 5.6.1.1 beschreibt den Inhalt der EBICS-Nachrichten, die im Rahmen einer Download-Transaktion ausgetauscht werden. Gegenstand dieses Kapitels ist die Verarbeitung der EBICS-Nachrichten. Im Sequenzdiagramm in Abbildung 52 sind Aktionssequenzen hervorgehoben, deren Ablauf hier näher beschrieben wird.

Um die Beschreibung der Abläufe zu vereinfachen, wird angenommen, dass jeder Verarbeitungsschritt einen Returncode (RC) liefert, dessen Wert gleich EBICS_OK (000000) ist, wenn dieser Schritt erfolgreich durchgeführt werden konnte. In Abhängigkeit von RC werden der technische Returncode (RCT) und der bankfachliche Returncode (RCF) gesetzt, deren Werte dann in EBICS-Nachrichten einfließen.

- Die Überprüfung der Validität der EBICS-Requests erfolgt auf Grundlage der XML-Schema-Definitionsdatei „ebics_request_H005.xsd“ sowie unter Berücksichtigung der Einschränkungen, die in Kapitel 5.6.1.1 zu den einzelnen Requests festgelegt wurden. Normalerweise findet die Validitätsprüfung parallel und/oder verzahnt mit den anderen Schritten der Verarbeitung der EBICS-Requests statt. Die nachfolgenden Abläufe verzichten auf die Darstellung eines Ablaufschritts vom

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Typ „Validitätsprüfung des EBICS-Requests“ zugunsten einer möglichst einfachen Darstellung. Demnach können diese Abläufe durch folgende zusätzlichen technischen Fehler abgebrochen werden: EBICS_INVALID_XML, EBICS_INVALID_REQUEST oder EBICS_INVALID_REQUEST_CONTENT. Für Return Codes, die CA-Zertifikate betreffen, wird auf Anhang 1 verwiesen.

5.6.1.2.1 Verarbeitung in der Initialisierungsphase

In Abbildung 60 ist die bankseitige Verarbeitung des EBICS-Requests dargestellt, welcher in der Initialisierungsphase einer Download-Transaktion vom Kundensystem an das Banksystem versendet wird. Die einzelnen Verarbeitungsschritte werden im Folgenden näher erläutert:

I. Erzeugung einer EBICS-Transaktion (siehe 5.5.1.2.1 Punkt 1 sowie Abbildung 44)

II. Beenden der EBICS-Transaktion

Sollten die angefordeten Abholdaten nicht verfügbar sein, so wird die EBICS-Transaktion mit dem fachlichen Returncode EBICS_NO_DOWNLOAD_DATA_AVAILABLE beendet.

III. Bereitstellen von Daten

In diesem Verarbeitungsschritt wird das erste Nutzdatensegment zwecks Einbettung in die EBICS-Response bereitgestellt. Falls das Kreditinstitut die bankfachliche EU für die aktuelle Auftragsart und den aktuellen Teilnehmer (Einreicher) einsetzt, wird zusätzlich die bankfachliche EU des Kreditinstituts über die Nutzdaten bereitgestellt.

In der Version „H005“ des EBICS-Protokolls ist die EU der Kreditinstitute nur vorgesehen (siehe dazu Kapitel 3.5.2). Sie wird hier nur in Vorbereitung zukünftiger EBICS-Versionen berücksichtigt.

Die Bereitstellung der Abholdaten ist nicht Bestandteil von EBICS, sie ist vielmehr abhängig von der Implementierung des Banksystems.

IV. Erzeugen der EBICS-Response

Dieser Verarbeitungsschritt erzeugt die EBICS-Response, die anschließend an das Kundensystem gesendet wird. Wenn alle bisherigen Verarbeitungsschritte erfolgreich waren, enthält diese EBICS-Nachricht das erste Segment der Nutzdaten und eventuell auch die bankfachliche Unterschrift der (gesamten) Nutzdaten. Im Fehlerfall transportiert diese EBICS-Nachricht den entsprechenden technischen oder fachlichen Fehlercode. Einzelheiten zum Inhalt dieser EBICS-Nachricht sind in Kapitel 5.6.1.1.1 beschrieben.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

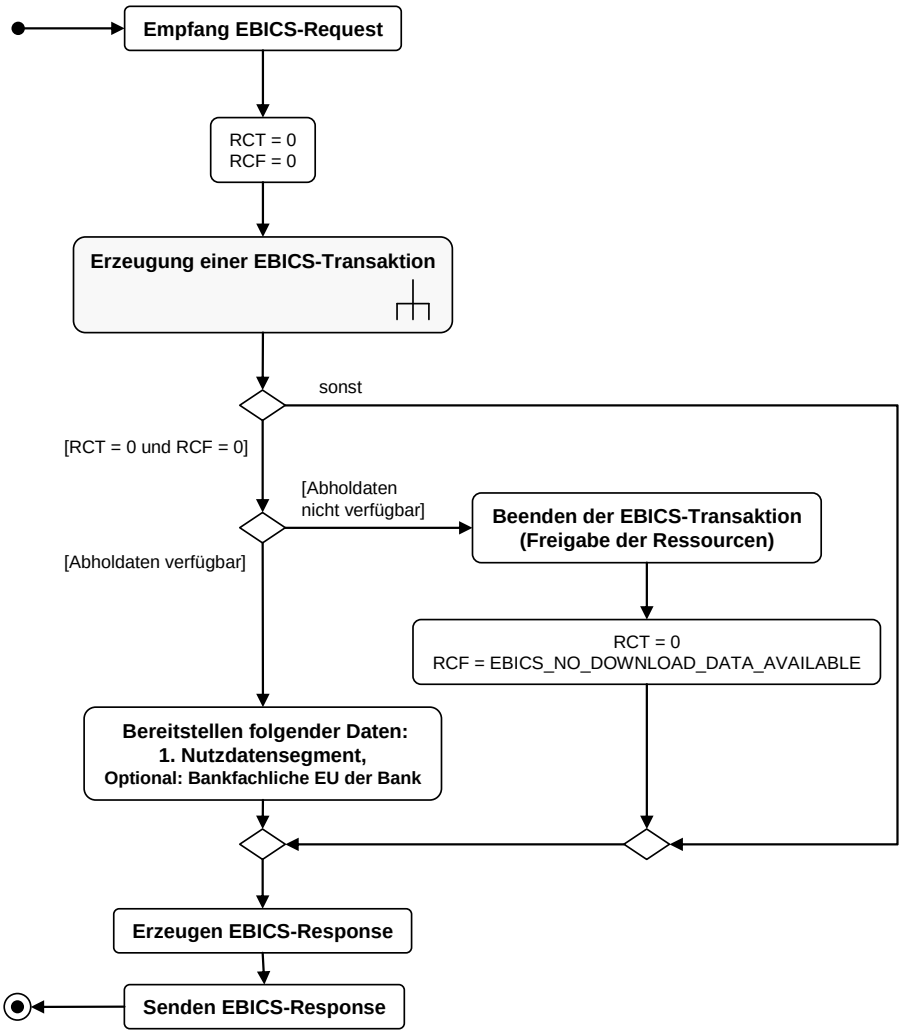


Abbildung 60: Verarbeitung des EBICS-Requests der Initialisierungsphase einer Download-Transaktion

5.6.1.2.2 Verarbeitung in der Phase des Datentransfers

In Abbildung 62 ist die bankseitige Verarbeitung des EBICS-Request dargestellt, welcher in der Datentransferphase einer EBICS-Transaktion vom Kundensystem an das Banksystem versendet wird. Die einzelnen Verarbeitungsschritte werden im Folgenden näher erläutert:

I. Überprüfung der Download-Transaktion (siehe Abbildung 61)

I.a. Überprüfung der EBICS-Transaktion (siehe 5.5.1.2.2, Punkt 1 sowie Abbildung 46)

I.b. Auswertung des Ergebnisses der Überprüfung der EBICS-Transaktion

Ist die Überprüfung des Transaktionsschritts nicht erfolgreich, dann:

wird überprüft, ob die Download-Transaktion wiederaufgesetzt werden kann, falls das Banksystem Recovery von Transaktionen unterstützt. Die Prüfung erfolgt gemäß der Beschreibung aus Kapitel 5.6.2. Ist diese Prüfung erfolgreich, so wird der technische

Returncode EBICS_TX_RECOVERY_SYNC zurückgeliefert, andernfalls wird die Transaktion mit dem technischen Returncode EBICS_TX_ABORT abgebrochen.

Wird die Download-Transaktion mit dem fachlichen Fehlercode EBICS_RECOVERY_NOT_SUPPORTED abgebrochen, falls das Banksystem Recovery von Transaktionen nicht unterstützt. Wenn man im Ablaufdiagramm MAX gleich 0 setzt, wird auch der Fall berücksichtigt, in dem Recovery nicht unterstützt wird.

II. Bereitstellen von Daten

In diesem Verarbeitungsschritt wird das angeforderte Nutzdatensegment zwecks Einbettung in die EBICS-Response bereitgestellt. Die Bereitstellung der Abholdaten ist nicht Bestandteil von EBICS, sie ist vielmehr abhängig von der Implementierung des Banksystems.

III. Erzeugen der EBICS-Response

Dieser Verarbeitungsschritt erzeugt die EBICS-Response, die anschließend an das Kundensystem gesendet wird. Wenn alle bisherigen Verarbeitungsschritte erfolgreich waren, enthält diese EBICS-Nachricht das im entsprechenden EBICS-Request angeforderte Segment der Nutzdaten. Im Fehlerfall transportiert diese EBICS-Nachricht den entsprechenden technischen oder fachlichen Fehlercode. Der Inhalt dieser EBICS-Nachricht ist in Kapitel 5.6.1.1.2 detailliert beschrieben.

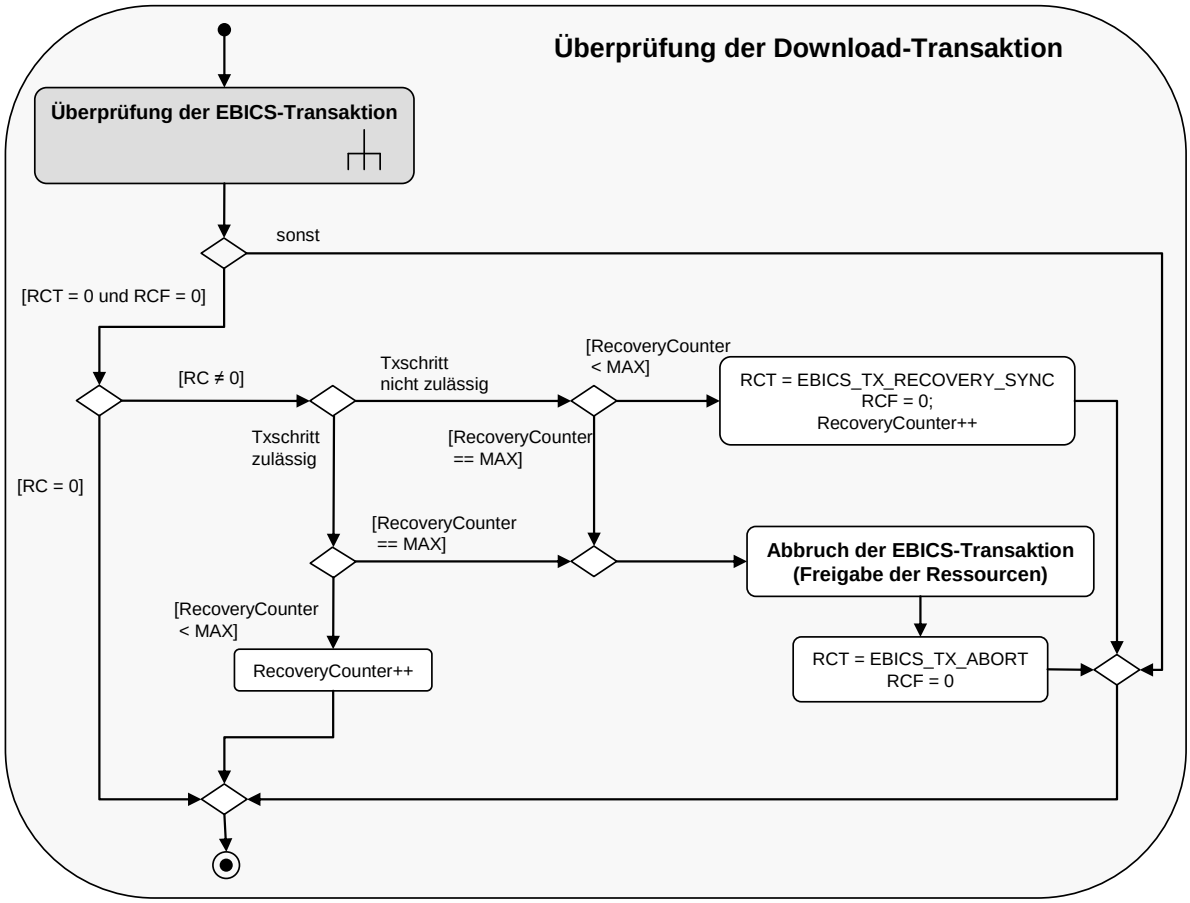


Abbildung 61: Detaillierte Beschreibung des Ablaufschritts „Überprüfung der Download-Transaktion“

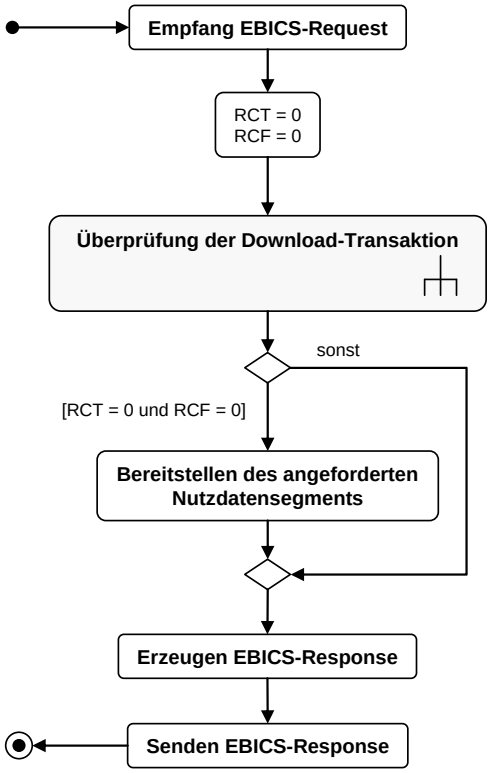


Abbildung 62: Verarbeitung eines EBICS-Requests zur Anforderung eines Nutzdatensegments

5.6.1.2.3 Verarbeitung in der Quittierungsphase

In Abbildung 63 ist die bankseitige Verarbeitung des EBICS-Request dargestellt, welcher in der Quittierungsphase einer EBICS-Transaktion vom Kundensystem an das Banksystem versendet wird.

Die einzelnen Verarbeitungsschritte werden im Folgenden näher erläutert:

I. Überprüfung der Download-Transaktion (siehe Beschreibung aus Kapitel 5.6.1.2.2, Punkt 1)

II. Download Nachbearbeitung

Die positive Quittung bedeutet, dass die Auftragsdaten vom Kundensystem erfolgreich abgeholt und verarbeitet werden konnten. Im Unterschied zur negativen Quittung hat sie zur Folge, dass auf dem Banksystem nun abschließende Arbeiten durchgeführt werden können, wie beispielsweise die Markierung der Auftragsdaten als „abgeholt. Unabhängig von der Art der Quittung wird die EBICS-Transaktion vom Banksystem beendet.

III. Beenden der EBICS-Transaktion

IV. Erzeugen der EBICS-Response

Dieser Verarbeitungsschritt erzeugt die EBICS-Response, die anschließend an das Kundensystem gesendet wird. Im Falle der positiven Quittung wird der technische Returncode EBICS_DOWNLOAD_POSTPROCESS_DONE, im Falle der negativen Quittung der technische

Returncode EBICS_DOWNLOAD_POSTPROCESS_SKIPPED zurückgeliefert. Im Fehlerfall transportiert diese EBICS-Nachricht den entsprechenden technischen oder fachlichen Fehlercode. Der Inhalt dieser EBICS-Nachricht ist in Kapitel 5.6.1.1.3 genauer beschrieben.

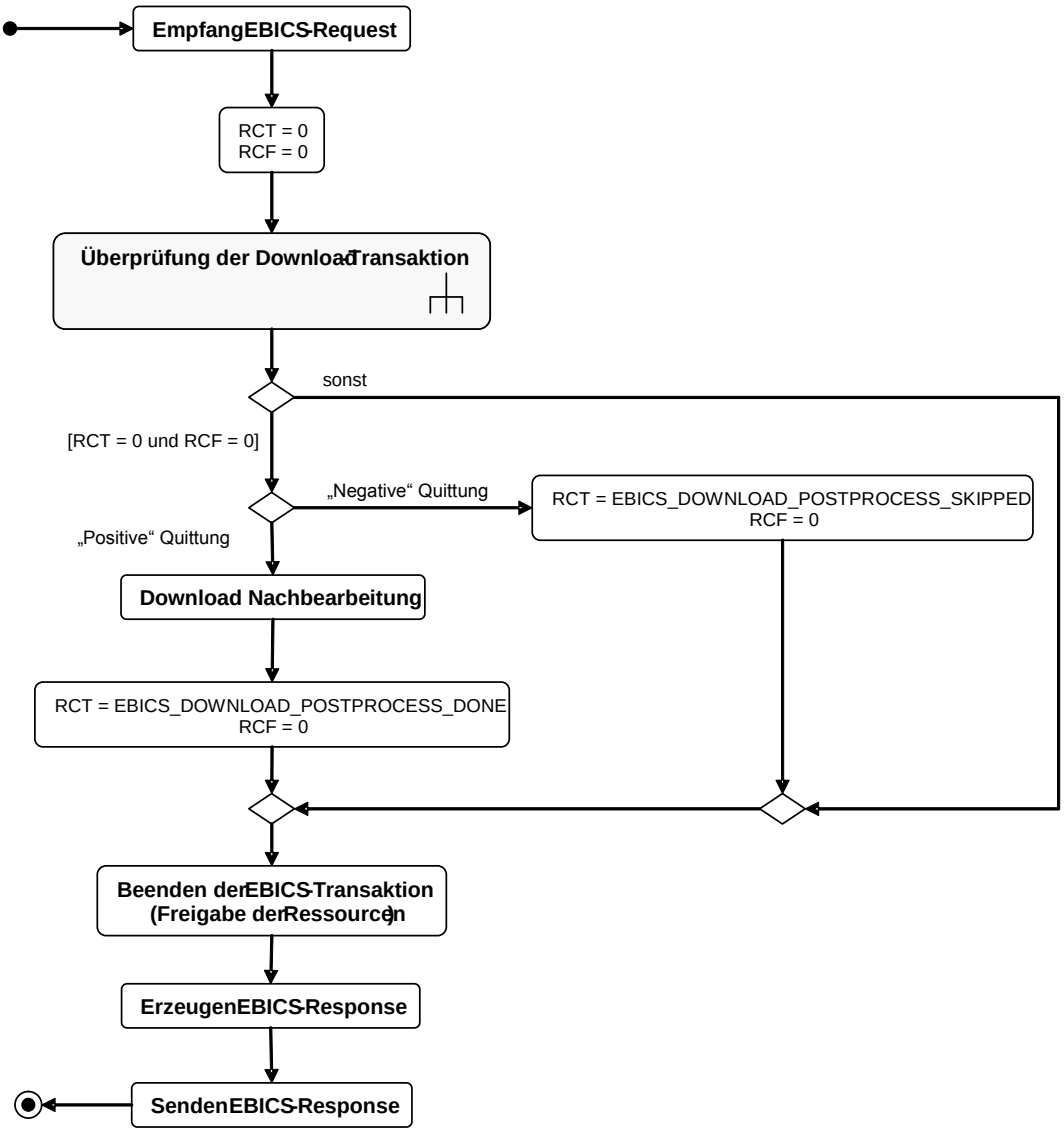


Abbildung 63: Verarbeitung eines EBICS-Requests zur Quittierung im Rahmen einer Download-Transaktion

5.6.2 Wiederaufsetzen von Download-Transaktionen

Recovery von Download-Transaktionen wird immer vom Kundensystem angestoßen. Die Ursachen für ein Recovery sind analog zu denen für Upload-Transaktionen:

- Transportfehler bei der Übertragung eines EBICS-Request in der Datentransfer- oder Quittierungsphase der Transaktion
- Timeout bzw. Transportfehler beim Empfang einer EBICS-Response in der Datentransfer- oder Quittierungsphase der Transaktion
- Verlust von bereits empfangenen Nutzdatensegmenten auf Teilnehmerseite
- Temporärer Fehler in der Verarbeitung einer empfangenen EBICS-Response, der eine erneute Übertragung erfordert.

Tritt eine der obigen Fehlersituationen ein, dann wählt das Kundensystem in Abhängigkeit von der Anzahl der vorhandenen Nutzdatensegmente auf Teilnehmerseite den geeigneten Wiederaufsetzpunkt aus. Ist der gewählte Wiederaufsetzpunkt die Anforderung des n-ten Nutzdatensegments, dann ist der nächste vom Teilnehmer initiierte Transaktionsschritt die Anforderung des (n+1)-ten Nutzdatensegments oder die Quittierung der Abholung sämtlicher Nutzdatensegmente, falls n das letzte Nutzdatensegment ist. EBICS-Requests im Rahmen des Recovery von Download-Transaktionen unterscheiden sich nicht von den EBICS-Requests eines normalen fehlerfreien Ablaufs einer Download-Transaktion.

In Abbildung 64 wird der beispielhafte Ablauf einer Transaktion dargestellt, die ein mehrfaches Wiederaufsetzen erfordert. Das Wiederaufsetzen geschieht jeweils ohne explizite Synchronisation zwischen Kunden- und Banksystem. Das 3. Nutzdatensegment wird dreimal angefordert, da das Kundensystem die entsprechende EBICS-Response wegen eines Timeout oder eines Transportfehlers nicht empfangen konnte. Bei der zweiten und dritten Anforderung des 3. Nutzdatensegments geht das Kundensystem davon aus, dass der Wiederaufsetzpunkt die Anforderung des 2. Nutzdatensegments ist. Der Wert des RecoveryCounters ist nach der dritten (und erfolgreichen) Anforderung des 3. Nutzdatensegments gleich 2, da die letzten beiden Anforderungen des 3. Nutzdatensegments vom Banksystem als Recovery-Versuch gewertet werden. Die Transaktion scheitert schließlich an der zu hohen Zahl von Wiederaufsetzversuchen.

Ist der gewählte Wiederaufsetzpunkt aus Sicht des Banksystems nicht zulässig, so enthält die EBICS-Response neben dem technischen Returncode EBICS_TX_RECOVERY_SYNC den letzten möglichen Wiederaufsetzpunkt der Download-Transaktion. Die zulässigen Wiederaufsetzpunkte einer Download-Transaktion sind in Kapitel 5.4 definiert. Ist der gewählte Wiederaufsetzpunkt beispielsweise die Anforderung des Nutzdatensegments mit der laufenden Nummer k, so kann die Transaktion mit der Anforderung der Nutzdatensegmente mit den laufenden Nummern i+1, i+2, usw. fortgesetzt werden, wobei i <= k gelten muss. Gilt allerdings i < k, wird also das i-te Nutzdatensegment wiederholt angefordert, dann wird der Zähler für die Anzahl der durchgeführten Recovery-Versuche um eins erhöht.

In Abbildung 65 ist der erfolgreiche Ablauf einer Transaktion dargestellt, die das Wiederaufsetzen der Transaktion nach einer expliziten Synchronisation zwischen Kunden- und Banksystem enthält. Hier

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

fordert das Kundensystem das 5. Nutzdatensegment in einem Zustand an, ohne vorher das 4. Nutzdatensegment angefordert zu haben. Die EBICS-Response des Kreditinstituts (siehe Abbildung 66) enthält somit den Wiederaufsetzpunkt der Transaktion, der in diesem Fall die Anforderung des 3. Nutzdatensegments darstellt. Danach fährt das Kundensystem mit der Anforderung des 4. Nutzdatensegments fort und beendet die Transaktion nach dem Erhalt des letzten Segments 5.

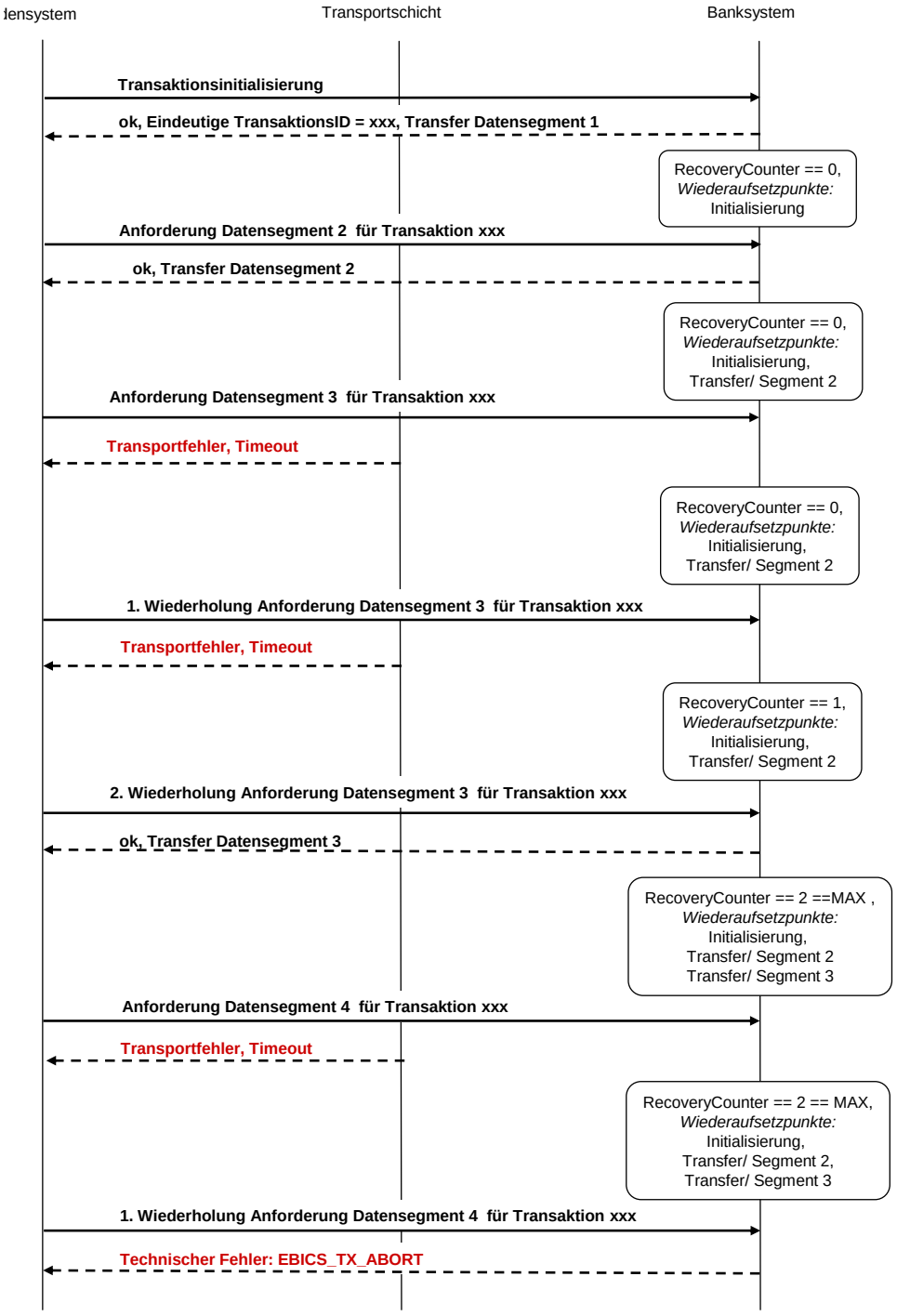


Abbildung 64: Abbruch des Recovery einer Download-Transaktion aufgrund der Überschreitung der maximal erlaubten Zahl von Recovery-Versuchen

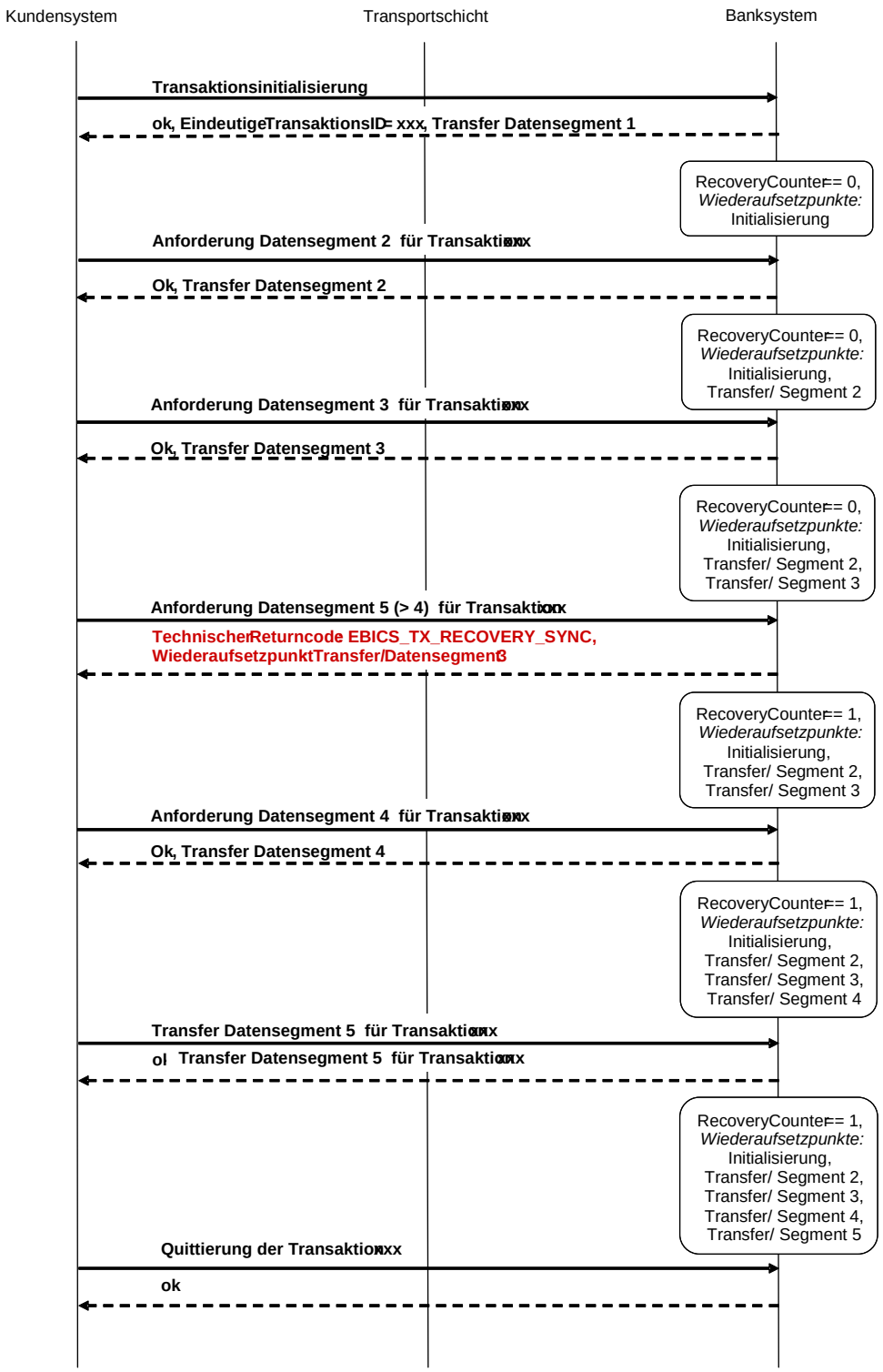


Abbildung 65: Recovery einer Download-Transaktion mit expliziter Synchronisation zwischen Kunden- und Banksystem

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsResponse
 xmlns=" urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation=" urn:org:ebics:H005 ebics_response_H005.xsd "
 Version="H005" Revision="1">
 <header authenticate="true">
   <static>     <TransactionID>FEDCBA4139464436344531324 3FEDCBA</TransactionID>
   </static>
   <mutable>
     <TransactionPhase>Transfer</TransactionPhase>
     <SegmentNumber lastSegment="false">3</SegmentNumber>
     <ReturnCode>061101</ReturnCode>
     <ReportText>[EBICS_TX_RECOVERY_SYNC] Synchronisation erforderlich</ReportText>
   </mutable>
 </header>
 <AuthSignature>
   <ds:SignedInfo>
     <ds:CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
     <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256">
     </ds:SignatureMethod>
     <ds:Reference URI="#xpointer(//*[@authenticate='true'])">
       <ds:Transforms>
         <ds:Transform Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315"/>
       </ds:Transforms>
       <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
       <ds:DigestValue> …hier Hashwert Authentifikation..</ds:DigestValue>
     </ds:Reference>
   </ds:SignedInfo>
   <ds:SignatureValue> …hier Signaturwert Authentifikation..</ds:SignatureValue>
 </AuthSignature>
 <body>
   <ReturnCode authenticate="true">000000</ReturnCode>
 </body>
</ebicsResponse>
```

Abbildung 66: EBICS-Response mit dem technischen Fehler EBICS_TX_RECOVERY_SYNC

6 Verschlüsselung

EBICS sieht Verschlüsselung auf zwei unterschiedlichen Protokoll-Ebenen vor: auf der Ebene des XML-basierten Anwendungsprotokolls von EBICS sowie auf einer Ebene zwischen Anwendungs- und Transportebene, nämlich auf der TLS-Ebene.

6.1 Verschlüsselung auf TLS-Ebene

Der Einsatz von TLS auf den externen Übertragungsstrecken zwischen Kunden- und Banksystem sichert die Geheimhaltung und die Integrität der EBICS-Nachrichten auf diesen Strecken. Die kryptografischen Verfahren, die zum Aufbau einer TLS-Session zwischen Kunden- und Banksystem verwendet werden, sind im Anhang (Kapitel 11.3.1) beschrieben.

6.2 Verschlüsselung auf Anwendungsebene

Die Auftragsdaten bankfachlicher Aufträge werden grundsätzlich als sensibel eingestuft und deshalb in verschlüsselter Form in die EBICS-Nachrichten eingebettet. Das ermöglicht ihre Geheimhaltung auf banksysteminternen sowie kundensysteminternen Übertragungsstrecken, auf denen die Kommunikation nicht notwendigerweise auf TLS basiert.

Die Auftragsdaten systembedingter Aufträge des Schlüsselmanagements werden verschlüsselt, sobald der Sender der Auftragsdaten über den (hinreichend überprüften) Verschlüsselungsschlüssel des Empfängers verfügt. Die Auftragsdaten von INI- oder HIA- Aufträgen werden somit unverschlüsselt in die EBICS-Nachricht eingebettet, die Auftragsdaten von HPB-, PUB- , HCS oder HCA-Aufträgen hingegen verschlüsselt.

Die Auftragsdaten systembedingter Aufträge der Verteilten Elektronischen Unterschrift werden ebenfalls in verschlüsselter Form in die EBICS-Nachricht eingebettet.

Die elektronischen Unterschriften eines Auftrags, d.h. die Transportunterschrift bzw. die bankfachlichen EUs werden analog zu den Auftragsdaten bankfachlicher Aufträge immer verschlüsselt.

Außer den Auftragsdaten und den EUs werden auf Anwendungsebene keine weiteren Daten verschlüsselt.

Zu verschlüsselnde Auftragsdaten sowie EUs eines Auftrags werden zunächst mittels ZIP komprimiert, anschließend verschlüsselt und schließlich base64-kodiert in die EBICS-Nachricht eingebettet. Dabei erfolgt die Komprimierung und anschließende Verschlüsselung der Auftragsdaten vor deren Segmentierung. Das eingesetzte Verschlüsselungsverfahren ist ein Hybridverfahren: die Daten werden symmetrisch verschlüsselt, der eingesetzte symmetrische Schlüssel wird dem Empfänger der Daten asymmetrisch verschlüsselt mitgeteilt. Details des Verschlüsselungsverfahrens werden im Anhang (Kapitel 11.3.2) beschrieben.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Im Falle einer Upload-Transaktion wird im Kundensystem ein zufälliger symmetrischer Schlüssel generiert, welcher ausschließlich im Rahmen dieser Transaktion sowohl für die Verschlüsselung der EUs als auch für die Verschlüsselung der Nutzdaten eingesetzt wird. Dieser Schlüssel wird asymmetrisch mit dem öffentlichen Verschlüsselungsschlüssel des Kreditinstituts verschlüsselt und während der Initialisierungsphase der Transaktion vom Kundensystem an das Banksystem übertragen.

Analog wird im Falle einer Download-Transaktion im Banksystem ein zufälliger symmetrischer Schlüssel generiert, der für die Verschlüsselung der abzuholenden Nutzdaten und für die Verschlüsselung der vorgesehenen bankfachlichen Unterschrift des Kreditinstituts verwendet wird. Dieser Schlüssel wird asymmetrisch verschlüsselt und während der Initialisierungsphase der Transaktion vom Banksystem an das Kundensystem übertragen. Die asymmetrische Verschlüsselung erfolgt mit dem öffentlichen Verschlüsselungsschlüssel des technischen Teilnehmers, falls die EBICS-Nachrichten der Transaktion durch einen technischen Teilnehmer versendet werden. Andernfalls erfolgt die asymmetrische Verschlüsselung mit dem öffentlichen Verschlüsselungsschlüssel des nichttechnischen Teilnehmers, d.h. des Einreichers des Auftrags.

Ab EBICS 2.4 muss das Kundensystem in einem Request den Hashwert des öffentlichen Bankschlüssels E002 verwenden. Dieser Hashwert wird vom Kundensystem gemäß E002-Verfahren mittels SHA-256 gebildet.
Die Transaktion wird mit dem Returncode EBICS_INVALID_REQUEST_CONTENT abgebrochen, wenn noch E001 in einem Request verwendet wird.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

7 Segmentierung der Auftragsdaten

7.1 Verfahrensbeschreibung

In der VersionH005 des EBICS-Standards MÜSSEN Auftragsdaten, die in komprimierter, verschlüsselter und base64-kodierter Form mehr als 1 MB Speicherplatz benötigen, unabhängig von der Transferrichtung (Upload/Download) vor dem Versand segmentiert werden.

Bei der Segmentierung ist folgendes Vorgehen einzuhalten:

1. Die Auftragsdaten werden ZIP-komprimiert
2. Die komprimierten Auftragsdaten werden gemäß Kapitel 6.2 verschlüsselt
3. Die komprimierten, verschlüsselten Auftragsdaten werden base64-kodiert.
   Hierbei sind im resultierenden kodierten Datenblock bei EBICS nur die 65 druckbaren Zeichen des Base64-Alphabets aus RFC 2045 erlaubt. Insbesondere sind sogenannte „white space characters“ wie Leerzeichen, Tabs, Wagenrückläufe und Zeilenvorschübe („CR/LF“) unzulässig
4. Das Resultat ist hinsichtlich des Datenvolumens zu prüfen:

4i. Bleibt das resultierende Datenvolumen unterhalb der Grenze von 1 MB = 1.048.576 Bytes, so können die Auftragsdaten komplett innerhalb eines Versandschritts als ein Datensegment übermittelt werden

4ii. Übersteigt das resultierende Datenvolumen 1.048.576 Bytes, so sind die Daten sequenziell in Segmente von jeweils maximal 1.048.576 Bytes base64-konform aufzuteilen.

Mit Schritt 4i. ist sichergestellt, dass auch diejenigen Auftragsdaten einheitlich im Rahmen der Segmentierung behandelt werden, welche in komprimierter, verschlüsselter und kodierter Form die zulässige maximale Segmentgröße von 1 MB nicht überschreiten.

Der Empfänger vollzieht die algorithmischen Berechnungen in umgekehrter Reihenfolge, um die ursprünglichen Auftragsdaten wieder herzustellen:

1. Das gerade empfangene Datensegment wird an die bereits empfangenen Datensegmente angehängt (konkateniert)
2. Der komplette Datenblock wird base64-dekodiert
3. Das Ergebnis der base64-Dekodierung wird gemäß Kapitel 6.2 entschlüsselt
4. Das Ergebnis der Entschlüsselung wird ZIP-dekomprimiert, um die ursprünglichen Auftragsdaten zu erhalten.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

7.2 Umsetzung in den EBICS-Nachrichten

Der Sender der Auftragsdaten nummeriert die gemäß Kapitel 7.1 erzeugten Datensegmente sequenziell aufsteigend, beginnend mit 1.

Der Server bricht die Transaktion mit dem technischen Fehlercode EBICS_TX_SEGMENT_NUMBER_EXCEEDED ab, sofern der Client bei einer **Upload-Transaktion** die in der Initialisierungsphase angegebene Gesamtanzahl zu übertragender Segmente im Feld `ebicsRequest/header/static/NumSegments` zu niedrig angegeben hat, d.h. falls für den aktuellen Transaktionsschritt gilt:

- `ebicsRequest/header/mutable/SegmentNumber = ebicsRequest/header/static/NumSegments` (aus der Initialisierungsphase) und `ebicsRequest/header/mutable/SegmentNumber@lastSegment≠"true"`, oder
- `ebicsRequest/header/mutable/SegmentNumber > ebicsRequest/header/static/NumSegments` (aus der Initialisierungsphase).

Der Server beendet die Transaktion regulär mit dem technischen Hinweis-Returncode EBICS_TX_SEGMENT_NUMBER_UNDERRUN, sofern der Client bei einer **Upload-Transaktion** die in der Initialisierungsphase angegebene Gesamtanzahl zu übertragender Segmente im Feld `ebicsRequest/header/static/NumSegments` zu hoch angesetzt hat, d.h. falls für den aktuellen Transaktionsschritt gilt:

- `ebicsRequest/header/mutable/SegmentNumber < ebicsRequest/header/static/NumSegments` (aus der Initialisierungsphase) und `ebicsRequest/header/mutable/SegmentNumber@lastSegment="true"`.

Der Server bricht die Transaktion mit dem technischen Fehlercode EBICS_SEGMENT_SIZE_EXCEEDED ab, sofern der Client bei einer **Upload-Transaktion** die zulässige Segmentgröße von 1 MB im aktuellen Transaktionsschritt überschritten hat.

Bei **Download-Transaktionen** obliegt es dem Kundensystem, auf Unstimmigkeiten bzgl. der Segmentanzahl oder -größe zu reagieren:

- Ist die tatsächliche Anzahl übertragener Segmente bis zur Attributbelegung `ebicsResponse/header/mutable/SegmentNumber@lastSegment="true"` seitens des Servers niedriger als die Angabe aus der Initialisierungsphase, so SOLLTE der Client die laufende Transaktion dennoch ordnungsgemäß mit der Quittierungsphase fortsetzen.
- Überschreitet der Server die in der Initialisierungsphase postulierte Gesamtanzahl Segmente, so KANN der Client die Transaktion trotzdem durch Anfragen für weitere Segmente fortführen. Alternativ bzw. bei unverhältnismäßig großer Abweichung der aktuellen Segmentanzahl zur Vorgabe KANN der Client die Transaktion unterbrechen, indem er keine weiteren Anfragen sendet.
- Überschreitet der Server die zulässige Segmentgröße von 1 MB, so SOLLTE der Client die Transaktion abbrechen.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

8 Verteilte Elektronische Unterschrift (VEU bzw. EDS)

Die bankseitige Unterstützung für die Verteilte Elektronische Unterschrift ist verbindlich für EBICS-konforme Implementationen, d.h. jedes Kreditinstitut MUSS die für die VEU notwendigen administrativen Auftragsarten (internationaler Begriff EDS = Electronic Distributed Signature) anbieten. Die Information, ob das Kreditinstitut auch die optionale administrative VEU-Auftragsart „HVT“(VEU-Transaktionsdetails abrufen) unterstützt, ist den Bankparameterdaten zu entnehmen (siehe Kapitel 9.2.2, Parameter “DistribSigTransactionDetails”).

8.1 Beschreibung des Verfahrens

Die Verteilte Elektronische Unterschrift (VEU bzw. EDS) erlaubt es, Aufträge orts- und zeitunabhängig von mehreren Teilnehmern – auch kundenübergreifend – zu autorisieren. Ein zugrunde liegender Auftrag bleibt hierbei in der VEU-Prozessverarbeitung gespeichert bis über die VEU-Aufträge entweder die erforderliche Anzahl Unterschriften mit der passenden Berechtigung eingegangen sind, ein bankrechnerseitiges Zeitlimit überschritten wurde oder ein Auftragsstorno erfolgt ist. Das VEU-Verfahren ist somit nicht nur eine Alternative zur kundeninternen nachträglichen Einreichung von EUs zu einem bestehenden Auftrag, sondern bietet überdies auch eine kundenübergreifende verteilte EU mit reichhaltigen Informationsmöglichkeiten zum VEU-Status und zum Auftrag.

Unterschriftsberechtigte eines Kunden können voneinander abweichende Signaturverfahren verwenden. Diese können verschiedene Hashverfahren unterstützen, woraus sich auch unterschiedliche Hashwerte ergeben. Im Falle des VEU-Verfahrens wird bei den administrativen Auftragsarten HVD und HVZ immer der Hashwert der Auftragsdaten zur Verfügung gestellt, der sich aus der Unterschriftsversion ergibt, die der Teilnehmer, der HVZ bzw. HVD durchführt, verwendet. Die verwendete Unterschriftsversion wird in einem Attribut mitgegeben.

Ein kompletter Auftragsvorgang zur VEU läuft generell wie folgt ab:

1. Der Auftraggeber initiiert den zugrunde liegenden Auftrag (z.B. eine SEPA-Überweisung), indem er die Auftragsdaten in einer EBICS-Transaktion mit einem Signaturflag (dies bedeutet, dass er in EBICS Unterschriften leisten will) und dem Attribut RequestEDS (dies bedeutet, dass er auch in der Lage ist, fehlende Unterschriften per VEU nachzureichen) überträgt. Für die Unterschrift kann der Auftraggeber entweder den Auftrag sofort bankfachlich signieren (Unterschriftsklasse A, B oder E) oder zunächst nur die Übertragung mittels Transportunterschrift durchführen (Unterschriftsklasse T).
2. Das Banksystem analysiert die BTF-Parameterwerte und bereits geleistete Unterschriften samt ihrer Klasse. Falls zur Ausführung des Auftrags weitere Unterschriften erforderlich sind, wird der Auftrag mitsamt dem Hashwert für das VEU-Verfahren zwischengespeichert. Den Hashwert der Auftragsdaten extrahiert das Banksystem hierbei mittels des öffentlichen Signaturschlüssels des Unterzeichners aus der EU.
3. Möchte ein anderer Teilnehmer das VEU-Verfahren für diesen Auftrag nutzen, so hat er möglicherweise bereits die zur Autorisation erforderlichen Daten – Hashwert des Auftrags, BTF-Parameterwerte und Auftragsnummer – außerhalb von EBICS (über einen dritten

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Kommunikationsweg) erhalten. In diesem Fall geht es bei Punkt 4 weiter. Benötigt er hingegen noch die Auftragsdaten, so kann er folgendermaßen vorgehen:

3i. Zunächst fragt er mittels der administrativen Auftragsart HVU oder HVZ ab, für welche Aufträge im Rahmen der VEU er unterschriftsberechtigt ist. Die Antwort enthält u.a. Informationen zu den BTF-Parameterwerten, Auftragsnummer, zur Anzahl erforderlicher und bereits geleisteter Unterschriften (einschließlich dem Hinweis, ob die eigene Unterschrift noch aussteht oder schon geleistet wurde), zum ursprünglichen Auftraggeber und zur Größe der unkomprimierten Auftragsdaten. Die HVZ-Antwort enthält zusätzliche Informationen, insbesondere den Hashwert der Auftragsdaten. Somit kann Schritt 3ii. übersprungen werden, wenn HVZ verwendet wurde.

3ii. Als nächstes ermittelt er mit der administrativen Auftragsart HVD den Status zu einem dieser Aufträge, z.B. zu dem im Rahmen der VEU eingestellten SEPA-Überweisungsauftrag. Neben dem Hashwert der Auftragsdaten, den das Banksystem aus der EU des Unterzeichners des Auftrags extrahiert hat, und einem Begleitzettel bekommt er eine Auflistung der bisherigen Unterzeichner samt ihrer jeweiligen Berechtigungsklasse.

3iii. Der Teilnehmer kann zusätzliche Auftragsdetails mit der administrativen Auftragsart HVT einholen: Je nach Anfrageparameter erhält er entweder Informationen zu den Einzeltransaktionen des Auftrags (Kontodaten, Betragsinformationen, Ausführungsdatum, Verwendungszweckdaten und weitere Beschreibungen) oder die kompletten Auftragsdaten.

4. Der Teilnehmer hat nun alle Informationen, um den Originalauftrag zu unterschreiben oder zu stornieren:

4i. Will er dem ursprünglichen Auftrag eine Unterschrift hinzufügen, so benutzt er HVE. Hierzu signiert er den Hashwert der Auftragsdaten, den er mittels HVD übermittelt bekommen oder mittels HVT aus den kompletten Auftragsdaten selbst berechnet hat. Die HVE-Steuerdaten enthalten dabei die Auftragsparameter zum ursprünglichen Auftrag (z.B. dem SEPA-Überweisungsauftrag).

4ii. Möchte er den ursprünglichen Auftrag stornieren, so verwendet er HVS. Wie bei HVE wird die Autorisation mittels der bankfachlichen Signatur über den Hashwert der Auftragsdaten bestätigt, nur dass im Falle von HVS die Unterschrift als Bestätigung des Stornos, nicht des Auftrags selbst gilt. Die HVS-Steuerdaten enthalten wie bei HVE die Auftragsparameter zum ursprünglichen (zu stornierenden) Auftrag.

Abbildung 67 dokumentiert die Abläufe bei Verwendung von VEU. Die Abbildung verdeutlicht die logische Verkettung der VEU-Auftragsarten, wobei reine Kommunikationsverbindungen (z.B. Datenversand vom Bank- zum Kundensystem bei Abruf der VEU-Details mit HVD), Fehlerfälle und die Informationsbeschaffung über alternative Kommunikationswege (z.B. Auftragshashwert per E-Mail vom Einreicher statt HVD) der Übersichtlichkeit halber nicht dargestellt werden.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

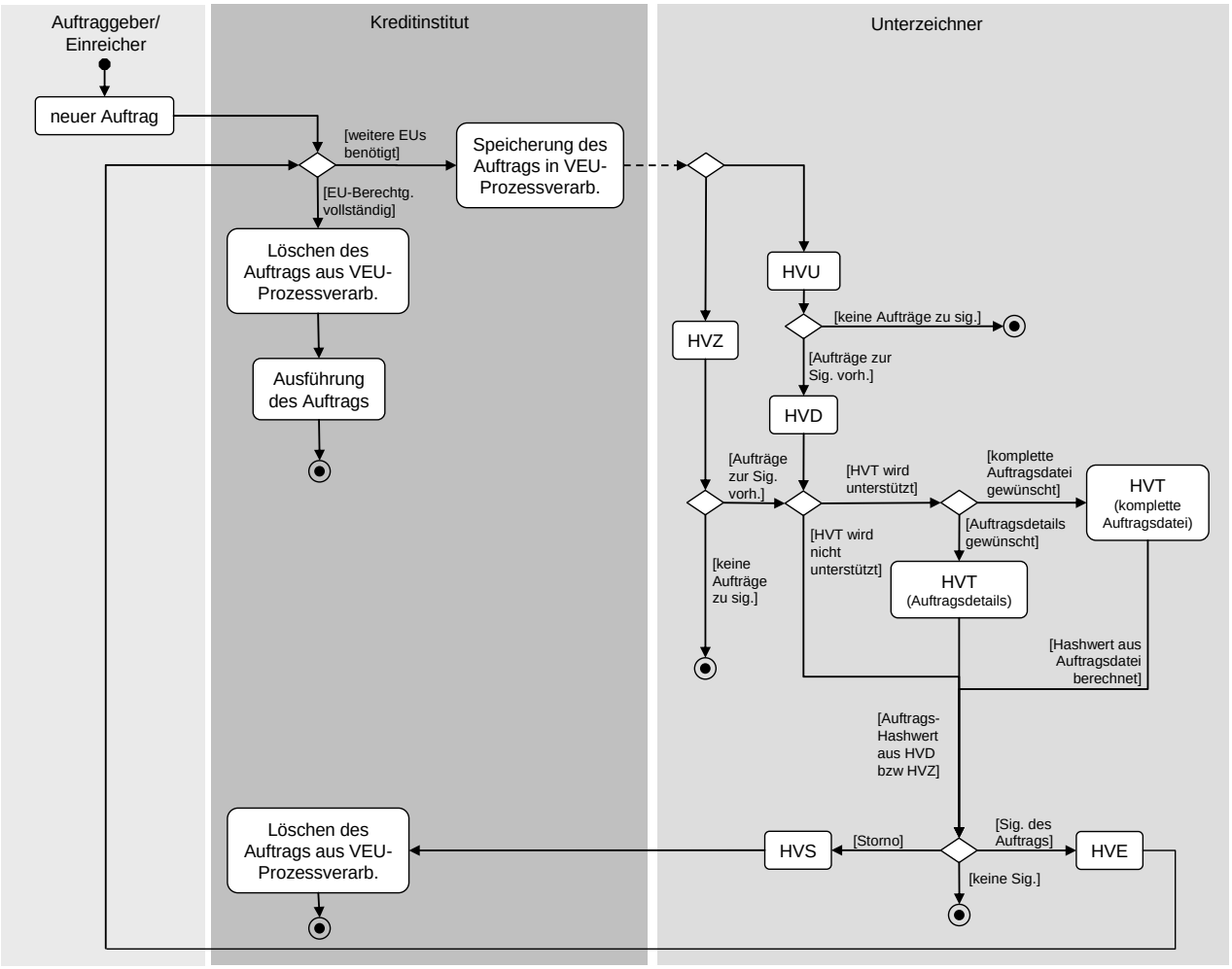


Abbildung 67: Ablaufdiagramm zur VEU

8.2 Technische Umsetzung der VEU (bzw. EDS)

- Ein Teilnehmer stößt die VEU-Prozessverarbeitung an, indem er einen Auftrag mit einer unzureichenden Anzahl bankfachlicher Unterschriften der benötigten Berechtigungsklassen einreicht. Die Einreichung des Auftrags erfolgt in einer EBICS-Transaktion mit vorhandenem Signaturflag und vorhandenem RequestEDS (VEU-)Flag . In jedem Fall muss dieser Auftrag signiert eingereicht werden (entweder mit bankfachlicher Unterschrift der Klassen „A“, „B“ oder „E“ oder mit Transportunterschrift = Unterschriftsklasse „T“).
- Das Banksystem verifiziert zunächst die mitgelieferte(n) EU(s) und die Autorisation des Teilnehmers für den gegebenen Geschäftsvorfall (BTF-Parameterwerte). Dann gleicht es die Anzahl und Unterschriftsklassen der gelieferten EU(s) mit den lokal hinterlegten EU-Anforderungen für die gegebenen BTF-Parameterwerte ab. Falls noch Unterschriften ausstehend sind, wird der Auftrag mitsamt den bereits übermittelten EUs in die VEU-Prozessverarbeitung eingestellt.
- Informationen über Aufträge, die sich derzeit in der VEU-Prozessverarbeitung befinden, können mit den administrativen VEU-Auftragsarten „HVU“, „HVZ“, „HVD“ und „HVT“ abgerufen werden. Notwendige

Parameter zur Eingrenzung der Originalaufträge werden über die zusätzlichen Auftragsparameter `HVUOrderParams`, `HVZOrderParams`, `HVDOrderParams` bzw. `HVTOrderParams` übermittelt, die Bestandteil der Steuerdaten für diese administrativen Auftragsarten sind. Bei „HVU“, „HVZ“, „HVD“ und „HVT“ handelt es sich um Download-Transaktionen, bei denen die Informationen der Antwort in Form von XML-Dokumenten transparent in das Auftragsdatenfeld eingebettet werden. „Transparent“ bedeutet: Die XML-Strukturen werden binär interpretiert und genauso wie die Auftragsdaten anderer Auftragsarten vor dem Versand komprimiert, verschlüsselt und kodiert.

- Für die Auftragsarten HVE/HVS kann ein Teilnehmer die notwendigen Daten zur Identifikation des Originalauftrags (Hashwert des Auftrags, Auftragsart, Auftragsnummer) über folgende Wege beziehen:
  - HVU & HVD: Mit HVU erfährt er BTF-Parameterwerte und Auftragsnummer, mit HVD den Hashwert des Auftrags. Der Hashwert stammt dabei aus der EU des Einreichers des Auftrags.
  - Alternativ zu HVU & HVD ist HVZ: Mit HVZ erfährt er BTF-Parameterwerte und Auftragsnummer und den Hashwert des Auftrags. Der Hashwert stammt dabei aus der EU des Einreichers des Auftrags.
  - HVU & HVT: Mit HVU erfährt der Teilnehmer wie im Fall „HVU & HVD“ BTF-Parameterwerte und Auftragsnummer. Bei HVT kann er in der Anfrage (`HVTOrderParams`) den Schalter `completeOrderData="true"` setzen und erhält damit die komplette Auftragsdatei. Zu dieser berechnet er selbst den Hashwert.
  - HVZ & HVT: Mit HVZ erfährt der Teilnehmer wie oben beschrieben BTF-Parameterwerte, Auftragsnummer und Hashwert des Auftrags. Bei HVT kann er in der Anfrage (`HVTOrderParams`) den Schalter `completeOrderData="true"` setzen und erhält damit die komplette Auftragsdatei.
  - über einen alternativen Kommunikationsweg: Dem Teilnehmer steht es frei, sich die Informationen ohne Zuhilfenahme der EBICS-Schnittstelle zu beschaffen. Falls er z.B. BTF-Parameterwerte und Auftragsnummer bereits kennt, kann er auf den Abruf mittels HVU verzichten. Ist er zusätzlich im Besitz des korrekten Hashwerts des Auftrags, so kann auch ein Aufruf von HVD, HVT bzw. HVZ entfallen.
- Neue EUs können dem Auftrag über die administrative VEU-Auftragsart HVE zugeteilt werden. Die Identifikation des Originalauftrags geschieht hierbei über die zusätzlichen Auftragsparameter `HVEOrderParams`, die Bestandteil der Steuerdaten für einen HVE-Auftrag sind. Die hinzuzufügende EU über die Auftragsdaten des Originalauftrags wird beim Initialisierungsschritt übermittelt. HVE enthält eine oder mehrere EU(s), aber keine Auftragsdaten.Sobald die geforderte Anzahl der geleisteten EUs mit den passenden Berechtigungen für die gegebene Auftragsart erreicht ist, wird der Originalauftrag aus der VEU-Prozessverarbeitung freigegeben und gelangt in die weitere Auftragsverarbeitung. Damit taucht dieser Auftrag bei erneuter Ausführung von „HVU“ (bzw „HVZ“) nicht mehr in der Rückgabeliste der zu unterschreibenden Aufträge auf.
- Ein VEU-Storno kann über HVS ausgelöst werden. Die Identifikation des Originalauftrags geschieht wie bei HVE über zusätzliche Auftragsparameter (hier `HVSOrderParams`), die Bestandteil der Steuerdaten für einen HVS-Auftrag sind. Die autorisierende EU für den Storno über die Auftragsdaten des Originalauftrags wird wie bei HVE beim Initialisierungsschritt übermittelt. Auch HVS enthält eine oder mehrere EU(s), aber keine Auftragsdaten.
  Ein Auftragsstorno ist sofort wirksam und bedarf immer nur einer einzigen berechtigten Unterschrift der Klassen „E“, „A“ oder „B“. Ein stornierter Auftrag wird aus der VEU-Prozessverarbeitung entfernt; er

gelangt nicht in die weitere Auftragsverarbeitung. Außerdem ist er bei einem erneuten Aufruf von „HVU“ (bzw „HVZ“) nicht mehr in der Liste zu unterschreibender Aufträge enthalten.

8.3 Detailbeschreibung der administrativen VEU-Auftragsarten

In diesem Kapitel werden ausschließlich die Unterschiede und Ergänzungen gegenüber EBICS-Standardaufträgen (siehe hierzu Kapitel 5) beleuchtet. Hier werden keine Prozessabläufe mehr erläutert (siehe hierzu die Kapitel 8.1 und 8.2), sondern für jede einzelne administrative VEU-Auftragsart (Request und Response) werden Syntax und Semantik der betroffenen Elemente und Attribute des XML-Schemas definiert und anhand von Beispielen erläutert.

Die Definition der VEU-Auftragselemente (VEU-Auftragsparameter und VEU-Auftragsdaten) erfolgt im XML-Schema „ebics_orders_H005.xsd“. Typdefinitionen sind z.T. im XML-Schema „ebics_types_H005.xsd“ zu finden. Bei den textuellen Darstellungen werden die jeweils relevanten Passagen aus „ebics_orders_H005.xsd“ und „ebics_types_H005.xsd“ zusammengefasst aufgeführt.

8.3.1 HVU (VEU-Übersicht abholen) und HVZ (VEU-Übersicht mit Zusatzinformationen abholen) [verpflichtend]

Mit HVU bzw. HVZ kann sich ein Teilnehmer die Aufträge auflisten lassen, für die er unterschriftsberechtigt ist. Als Filterkriterium kann er die Liste im Request auf bestimmte Geschäftsvorfälle (BTF-Parameterwerte) eingrenzen (`ServiceFilter`, 0 bis n-mal). Die Response enthält neben den Auftragskennzeichen auch Größe der Auftragsdaten, Unterschriftskonditionen und Informationen zum Einreicher des Auftrags und zu den bisherigen Unterzeichnern (`OrderDetails`).

Die Antwortnachricht von HVZ enthält neben allen Informationen aus HVU auch Daten aus HVD.HVZ ('VEU-Übersicht mit Zusatzinformationen abholen') ist somit vergleichbar mit einer Kombination von HVU mit 1 bis n HVD’s.

HVU und HVZ sind administrative Auftragsarten vom Typ „Download“.

8.3.1.1 HVU-Request

Im HVU-Request übergibt der Teilnehmer optional ein Filterkriterium. Nur Aufträge, deren BTF-Parameterwerte im übergebenen Filtern sind, werden zurückgeliefert. Falls der Teilnehmer keine Eingrenzung übergibt, bekommt er sämtliche Aufträge aufgelistet, für die er unterschriftsberechtigt ist.

Ausprägung der OrderParams (Auftragsparameter) für HVU: `HVUOrderParams`

8.3.1.1.1 XML-Schema (grafische Darstellung)

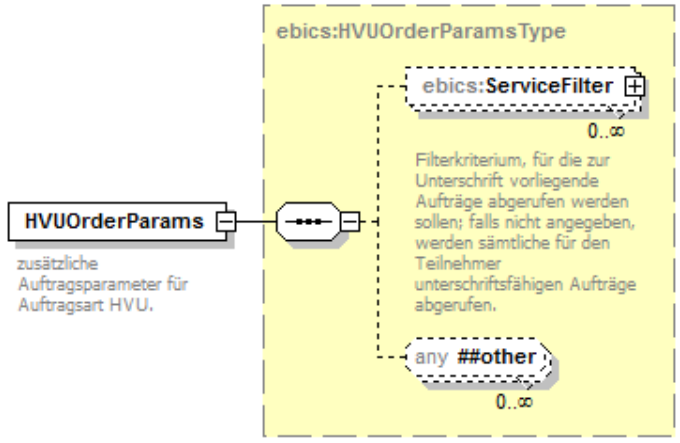


Abbildung 68: HVUOrderParams

8.3.1.1.2 XML-Schema (textuelle Darstellung)

```
<element name="HVUOrderParams" type="ebics:HVUOrderParamsType" substitutionGroup="ebics:OrderParams">
  <annotation>
    <documentation xml:lang="de">zusätzliche Auftragsparameter für Auftragsart HVU.</documentation>
  </annotation>
</element>
<complexType name="HVUOrderParamsType">
  <annotation>
    <documentation xml:lang="de">Datentyp für zusätzliche Auftragsparameter für Auftragsart
HVU.</documentation>
  </annotation>
  <sequence>
     <element name="ServiceFilter" type="ebics: ServiceType" minOccurs="0 maxOccurs="unbounded">
….(Elementgruppe ServiceFilter enthält eine Auswahl von Elementen) …
      <annotation>
        <documentation xml:lang="de"> BTF-Parameterwerte, für die zur Unterschrift vorliegende Aufträge
abgerufen werden sollen; falls nicht angegeben, werden sämtliche für den Teilnehmer unterschriftsfähigen
Aufträge abgerufen.</documentation>
      </annotation>
    </element>
    <any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
  </sequence>
</complexType>
```

8.3.1.1.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
HVUOrderParams	ebics:HVUOrderParamsType (komplex)	1	Auftragsparameter für HVU	- (komplex)
ServiceFilter	ebics:ServiceType ServiceType Zur Struktur siehe	0..nn	Filterkriterum für alle Geschäftsvorfälle, für die zur Unterschrift vorliegende	

	Hinweise in Kapitel 8.3.6		Aufträge abgerufen werden sollen; falls nicht angegeben, werden sämtliche Aufträge abgerufen, für die der Teilnehmer unterschriftsberechtigt ist	

8.3.1.1.4 Beispiel-XML (gekürzt)

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest
 xmlns=" urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation=" urn:org:ebics:H005 ebics_request_H005.xsd "
 Version="H005" Revision="1">
  <header authenticate="true">
    <static>
      <!-- […] -->
      <OrderDetails>
        <OrderType>HVU</OrderType>
        <HVUOrderParams>
          <ServiceFilter>
                <ServiceName>SCT</Servicename>
          </ServiceFilter>
        </HVUOrderParams>
      </OrderDetails>
      <!-- […] -->
    </static>
    <!-- […] -->
  </header>
  <!-- […] -->
</ebicsRequest>
```

8.3.1.2 HVU-Response

In der HVU-Response werden dem Teilnehmer Informationen zu den Aufträgen zurückgeliefert, für die er unterschriftsberechtigt ist.

Ausprägung der (dekodierten & entschlüsselten & dekomprimierten) `OrderData` (Auftragsdaten) für HVU: `HVUResponseOrderData`

8.3.1.2.1 XML-Schema (grafische Darstellung)

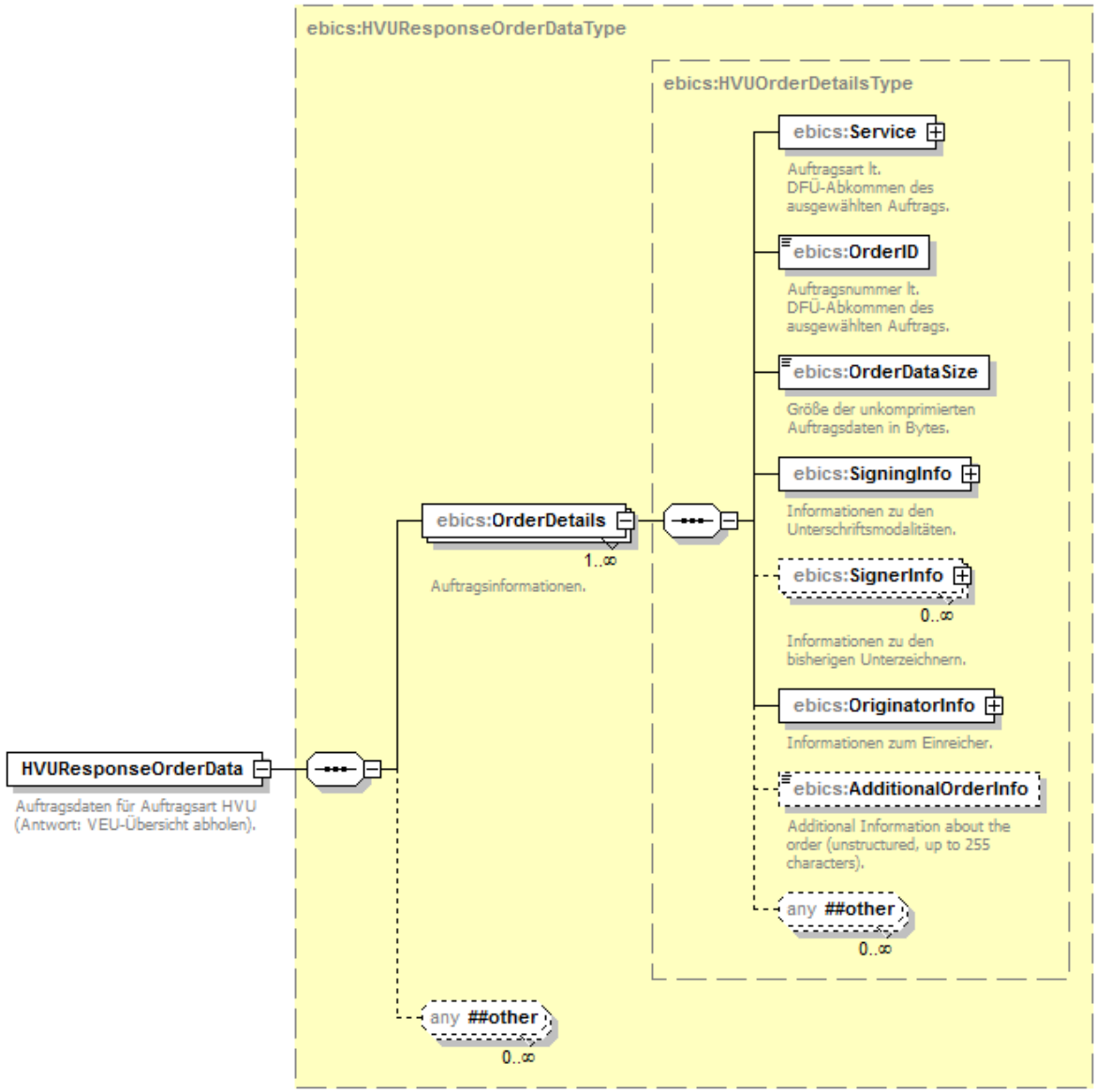


Abbildung 69: HVUResponseOrderData

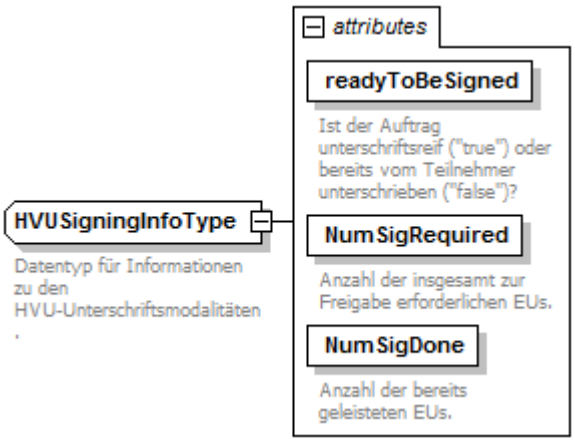


Abbildung 70: HVUSigningInfoType (zu SigningInfo)

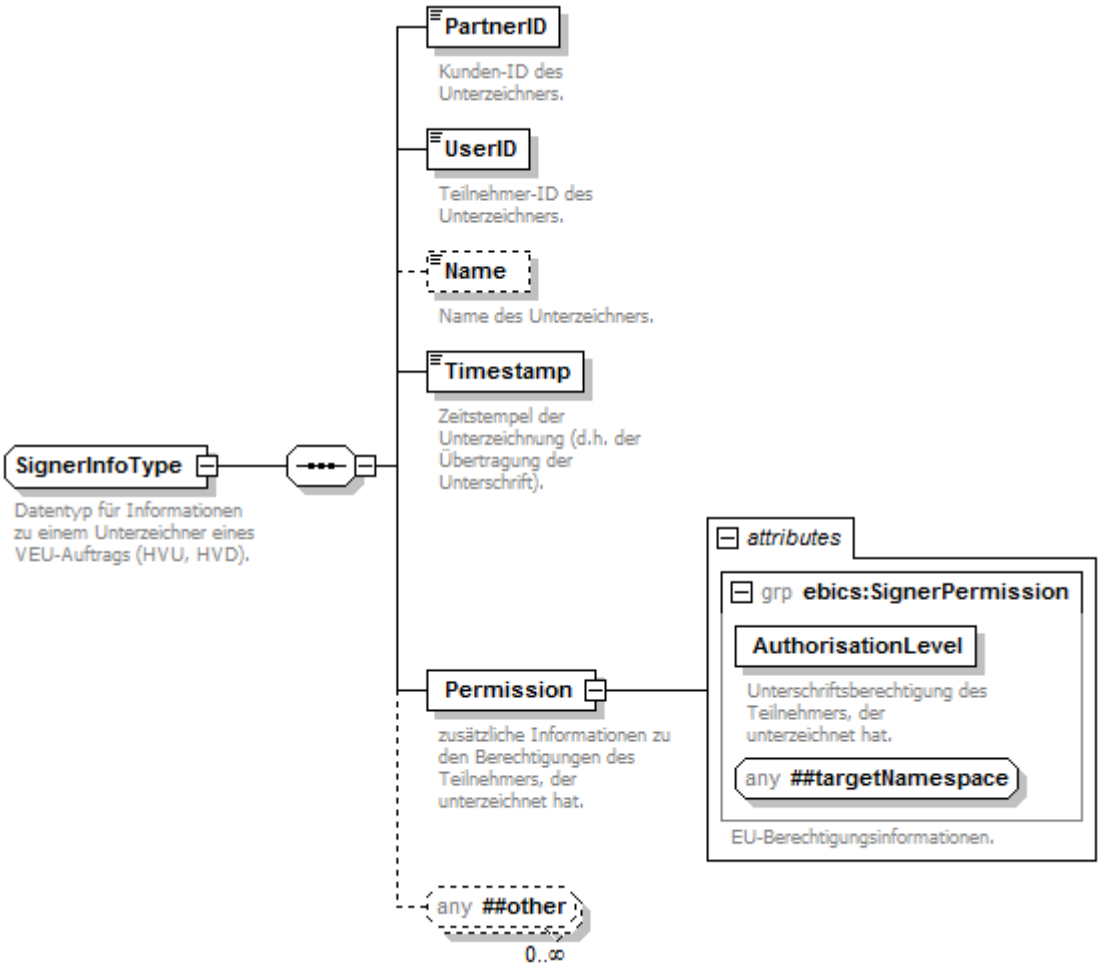


Abbildung 71: SignerInfoType (zu SignerInfo)

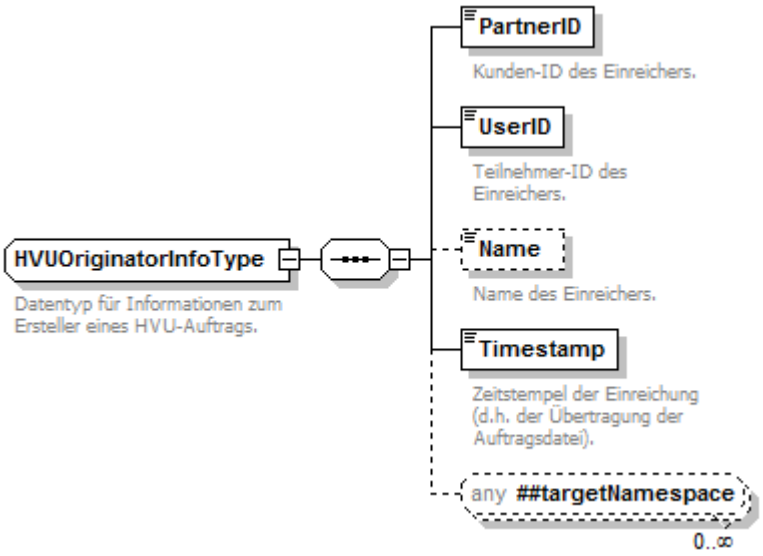


Abbildung 72: HVUOriginatorInfoType (zu OriginatorInfo)

8.3.1.2.2 XML-Schema (textuelle Darstellung)

```
 <element name="HVUResponseOrderData" type="ebics:HVUResponseOrderDataType"
substitutionGroup="ebics:EBICSOrderData">
   <annotation>
     <documentation xml:lang="de">Auftragsdaten für Auftragsart HVU (Antwort: VEU-Übersicht
abholen).</documentation>
   </annotation>
 </element>
 <complexType name="HVUResponseOrderDataType">
   <annotation>
     <documentation xml:lang="de">Datentyp für Auftragsdaten für Auftragsart HVU (Antwort: VEU-Übersicht
abholen).</documentation>
   </annotation>
   <sequence>
     <annotation>
       <documentation xml:lang="de"/>
     </annotation>
     <element name="OrderDetails" type="ebics:HVUOrderDetailsType" maxOccurs="unbounded">
       <annotation>
         <documentation xml:lang="de">Auftragsinformationen.</documentation>
       </annotation>
     </element>
     <any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
   </sequence>
 </complexType>
 <complexType name="HVUOrderDetailsType">
   <annotation>
     <documentation xml:lang="de">Datentyp für HVU-Auftragsdetails.</documentation>
   </annotation>
   <sequence>
     <element name="Service" type="ebics: RestrictedServiceType">
   … (Elementgruppe Service enthält eine Auswahl von Elementen) …
       <annotation>
         <documentation xml:lang="de">Geschäftsvorfall.</documentation>
       </annotation>
     </element>
     <element name="OrderID" type="ebics:OrderIDType">
```

```
        <annotation>
          <documentation xml:lang="de">Auftragsnummer des ausgewählten Auftrags gemäß Kapitel
10.1.</documentation>
        </annotation>
      </element>
      <element name="OrderDataSize" type="positiveInteger">
        <annotation>
          <documentation xml:lang="de">Größe der unkomprimierten Auftragsdaten in Bytes.</documentation>
        </annotation>
      </element>
      <element name="SigningInfo" type="ebics:HVUSigningInfoType">
        <annotation>
          <documentation xml:lang="de">Informationen zu den Unterschriftsmodalitäten.</documentation>
        </annotation>
      </element>
      <element name="SignerInfo" type="ebics:SignerInfoType" minOccurs="0" maxOccurs="unbounded">
        <annotation>
          <documentation xml:lang="de">Informationen zu den bisherigen Unterzeichnern.</documentation>
        </annotation>
      </element>
      <element name="OriginatorInfo" type="ebics:HVUOriginatorInfoType">
        <annotation>
          <documentation xml:lang="de">Informationen zum Einreicher.</documentation>
        </annotation>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
    </sequence>
  </complexType>
  <complexType name="HVUSigningInfoType">
    <annotation>
      <documentation xml:lang="de">Datentyp für Informationen zu den HVU-
Unterschriftsmodalitäten.</documentation>
    </annotation>
    <attribute name="readyToBeSigned" type="boolean" use="required">
      <annotation>
        <documentation xml:lang="de">Ist der Auftrag unterschriftsreif ("true") oder bereits vom Teilnehmer
unterschrieben ("false")?</documentation>
      </annotation>
    </attribute>
    <attribute name="NumSigRequired" type="positiveInteger" use="required">
      <annotation>
        <documentation xml:lang="de">Anzahl der insgesamt zur Freigabe erforderlichen EUs.</documentation>
      </annotation>
    </attribute>
    <attribute name="NumSigDone" type="nonNegativeInteger" use="required">
      <annotation>
        <documentation xml:lang="de">Anzahl der bereits geleisteten EUs.</documentation>
      </annotation>
    </attribute>
  </complexType>
  <complexType name="SignerInfoType">
    <annotation>
      <documentation xml:lang="de">Datentyp für Informationen zu einem Unterzeichner eines VEU-Auftrags
(HVU, HVD).</documentation>
    </annotation>
    <sequence>
      <element name="PartnerID" type="ebics:PartnerIDType">
        <annotation>
          <documentation xml:lang="de">Kunden-ID des Unterzeichners.</documentation>
        </annotation>
      </element>
      <element name="UserID" type="ebics:UserIDType">
        <annotation>
```

```
          <documentation xml:lang="de">Teilnehmer-ID des Unterzeichners.</documentation>
        </annotation>
      </element>
      <element name="Name" type="ebics:NameType" minOccurs="0">
        <annotation>
          <documentation xml:lang="de">Name des Unterzeichners.</documentation>
        </annotation>
      </element>
      <element name="Timestamp" type="ebics:TimestampType">
        <annotation>
          <documentation xml:lang="de">Zeitstempel der Unterzeichnung (d.h. der Übertragung der
Unterschrift).</documentation>
        </annotation>
      </element>
      <element name="Permission">
        <annotation>
          <documentation xml:lang="de">zusätzliche Informationen zu den Berechtigungen des Teilnehmers, der
unterzeichnet hat.</documentation>
        </annotation>
        <complexType>
          <attributeGroup ref="ebics:SignerPermission"/>
        </complexType>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
    </sequence>
  </complexType>
  <complexType name="HVUOriginatorInfoType">
    <annotation>
      <documentation xml:lang="de">Datentyp für Informationen zum Ersteller eines HVU-
Auftrags.</documentation>
    </annotation>
    <sequence>
      <element name="PartnerID" type="ebics:PartnerIDType">
        <annotation>
          <documentation xml:lang="de">Kunden-ID des Einreichers.</documentation>
        </annotation>
      </element>
      <element name="UserID" type="ebics:UserIDType">
        <annotation>
          <documentation xml:lang="de">Teilnehmer-ID des Einreichers.</documentation>
        </annotation>
      </element>
      <element name="Name" type="ebics:NameType" minOccurs="0">
        <annotation>
          <documentation xml:lang="de">Name des Einreichers.</documentation>
        </annotation>
      </element>
      <element name="Timestamp" type="ebics:TimestampType">
        <annotation>
          <documentation xml:lang="de">Zeitstempel der Einreichung (d.h. der Übertragung der
Auftragsdatei).</documentation>
        </annotation>
      </element>
      <any namespace="##targetNamespace" processContents="strict" minOccurs="0" maxOccurs="unbounded"/>
    </sequence>
  </complexType>
  <attributeGroup name="SignerPermission">
    <annotation>
      <documentation xml:lang="de">EU-Berechtigungsinformationen.</documentation>
    </annotation>
    <attribute name="AuthorisationLevel" type="ebics:AuthorisationLevelType" use="required">
      <annotation>
```

```
      <documentation xml:lang="de">Unterschriftsberechtigung des Teilnehmers, der unterzeichnet
hat.</documentation>
     </annotation>
   </attribute>
   <anyAttribute namespace="##targetNamespace" processContents="strict"/>
 </attributeGroup>
```

8.3.1.2.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/-Attribut	Datentyp	#	Bedeutung	Beispiel
HVUResponse» OrderData	ebics:HVUResponse» OrderDataType (komplex)	1	XML-Auftragsdaten für HVU	- (komplex)
OrderDetails	ebics:HVUOrder» DetailsType (komplex)	1..∞	Auftragsinformationen für HVU	- (komplex)
Service	ebics: RestrictedServiceType Hinweise zur Struktur siehe Kapitel 8.3.6	1	Art von Geschäftsvorfall (identifiziert durch die Servicestruktur) des zur VEU eingereichten Auftrags	
OrderID	ebics:OrderIDType (→token, fixLength=4)	1	Auftragsnummer des zur VEU eingereichten Auftrags	„OR01“
OrderDataSize	positiveInteger	1	Größe der unkomprimierten Auftragsdaten des zur VEU eingereichten Auftrags in Bytes	123456
SigningInfo	ebics:HVUSigning» InfoType (komplex)	1	Informationen zu den Unterschriftsmodalitäten	- (komplex)
SigningInfo» @readyToBeSigned	boolean	1	Ist der Auftrag unterschriftsreif (true) oder bereits vom Teilnehmer unterschrieben (false)?	„true“
SigningInfo» @NumSigRequired	positiveInteger	1	Anzahl der insgesamt zur Freigabe erforderlichen EUs	4
SigningInfo» @numSigDone	nonNegativeInteger	1	Anzahl der bereits geleisteten EUs	2
SignerInfo	ebics:SignerInfo» Type (komplex)	0..∞	Informationen zu den bisherigen Unterzeichnern	- (komplex)
PartnerID (in SignerInfo)	ebics:PartnerIDType (→token, maxLength=35) pattern="[a-zA-Z0-9,=]{1,35}	1	Kunden-ID des Unterzeichners	„PARTNER1“
UserID (in SignerInfo)	ebics:UserIDType (→token,	1	Teilnehmer-ID des Unterzeichners	„USER0001“

	maxLength=35, pattern="[a-zA-Z0-9,=]{1,35})			
Name (in SignerInfo)	ebics:NameType (→normalizedString)	0..1	Name des Unterzeichners	„Max Mustermann“
Timestamp (in SignerInfo)	ebics:TimestampType (→dateTime)	1	Zeitstempel der Unterzeichnung (d.h. der Übertragung der Unterschrift)	„2005-01-31T» 16:30:45.123Z“
Permission	- (komplex)	1	zusätzliche Berechtigungsinformationen zu dem Teilnehmer, der unterzeichnet hat	- (komplex)
Permission» @Authorisation» Level	ebics:Authorisation» LevelType (→token, length=1: "E", "A", "B", "T")	1	Unterschriftsberechtigung des Teilnehmers, der unterzeichnet hat	„A“
OriginatorInfo	ebics:HVUOriginator» InfoType (komplex)	1	Informationen zu dem Einreicher des Auftrags	- (komplex)
PartnerID (in OriginatorInfo)	ebics:PartnerIDType (→token, maxLength=35, pattern="[a-zA-Z0-9,=]{1,35})	1	Kunden-ID des Einreichers	„PARTNER2“
UserID (in OriginatorInfo)	ebics:UserIDType (→token, maxLength=35, pattern="[a-zA-Z0-9,=]{1,35})	1	Teilnehmer-ID des Einreichers	„USER0002“
Name (in OriginatorInfo)	ebics:NameType (→normalizedString)	0..1	Name des Einreichers	„Erich Einreicher“
Timestamp (in OriginatorInfo)	ebics:TimestampType (→dateTime)	1	Zeitstempel der Einreichung (d.h. der Übertragung der Auftragsdatei)	„2005-01-30T» 15:30:45.123Z“
AdditionalOrderInfo	Max255Text	0..1	Vom Kunden angegebene zusätzliche Auftragsinformationen (bis zu 255 Zeichen)	z.B. lokaler Dateiname

8.3.1.2.4 Beispiel-XML

```
<?xml version="1.0" encoding="UTF-8"?>
<HVUResponseOrderData
 xmlns="urn:org:ebics:H005"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation=" urn:org:ebics:H005 ebics_orders_H005.xsd ">
 <OrderDetails>
          <Service><ServiceName>SCT</ServiceName>
       <MsgName>pain.001</MsgName>
   </Service>
   <OrderID>OR01</OrderID>
   <OrderDataSize>123456</OrderDataSize>
   <SigningInfo NumSigRequired="4" readyToBeSigned="true" NumSigDone="2"/>
   <SignerInfo>
```

```
      <PartnerID>PARTNER1</PartnerID>
      <UserID>USER0001</UserID>
      <Name>Max Mustermann</Name>
      <Timestamp>2005-01-31T16:30:45.123Z</Timestamp>
      <Permission AuthorisationLevel="A"/>
    </SignerInfo>
    <SignerInfo>
      <PartnerID>PARTNER2</PartnerID>
      <UserID>USER0002</UserID>
      <Name>Maxime Musterfrau</Name>
      <Timestamp>2005-01-31T17:30:45.123Z</Timestamp>
      <Permission AuthorisationLevel="B"/>
    </SignerInfo>
    <OriginatorInfo>
      <PartnerID>PARTNER1</PartnerID>
      <UserID>USER0001</UserID>
      <Name>Erich Einreicher</Name>
      <Timestamp>2005-01-30T15:30:45.123Z</Timestamp>
    </OriginatorInfo>
  </OrderDetails>
</HVUResponseOrderData>
```

8.3.1.3 HVZ-Request

Im HVZ-Request übergibt der Teilnehmer optional eine Liste von BTF-Parameters als Filterkriterium. Nur Aufträge, deren Auftragsart in der übergebenen Liste enthalten ist, werden zurückgeliefert. Falls der Teilnehmer keine BTF-Parameterliste als Eingrenzung übergibt, bekommt er sämtliche Aufträge aufgelistet, für die er unterschriftsberechtigt ist.

Ausprägung der `OrderParams` (Auftragsparameter) für HVZ: `HVZOrderParams`

8.3.1.3.1 XML-Schema (grafische Darstellung)

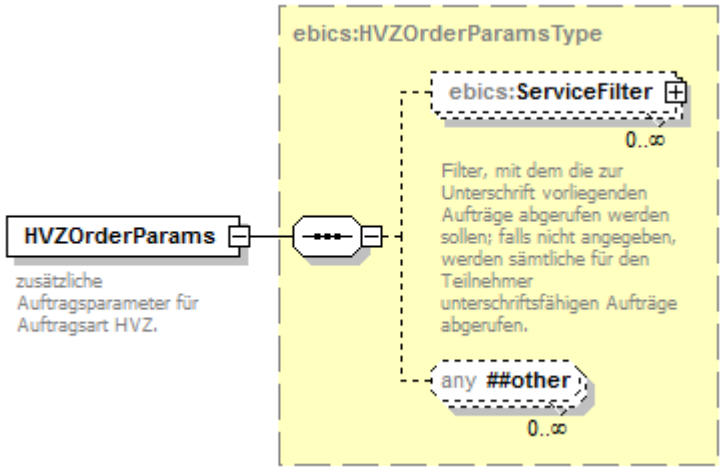


Abbildung: 73 HVZOrderParams

8.3.1.3.2 XML-Schema (textuelle Darstellung)

```
<element name="HVZOrderParams" type="ebics:HVZOrderParamsType"
   substitutionGroup="ebics:OrderParams">
   <annotation>
      <documentation xml:lang="de">
         zusätzliche Auftragsparameter für Auftragsart HVZ.
      </documentation>
   </annotation>
</element>
<complexType name="HVZOrderParamsType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für zusätzliche Auftragsparameter für Auftragsart HVZ.
      </documentation>
   </annotation>
   <sequence>
      <element name="ServiceFilter" type="ebics:ServiceType"
         minOccurs="0" maxOccurs="unbounded">
    … (Elementgruppe ServiceFilter enthält eine Auswahl an Elementen) …
         <annotation>
            <documentation xml:lang="de">
               Liste von Auftragsarten, für die zur Unterschrift
               vorliegende Aufträge abgerufen werden sollen; falls nicht
               angegeben, werden sämtliche für den Teilnehmer
               unterschriftsfähigen Aufträge abgerufen.
            </documentation>
         </annotation>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0"
         maxOccurs="unbounded" />
   </sequence>
</complexType>
```

8.3.1.3.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
`HVZOrderParams`	`ebics:HVZOrderParamsType` (komplex)	1	Auftragsparameter für Auftragsart HVZ	- (komplex)
`ServiceFilter`	`ebics:ServiceType` Details zu dieser Struktur siehe Kapitel 8.3.6	0..1	Filtert alle verfügbaren Aufträge heraus, die zum angegebenen BTF-Service-Filter passen; falls kein Filter angegeben wurde, werden alle Aufträge geliefert, für die der Teilnehmer unterschriftsberechtigt ist.	„IZV“

8.3.1.3.4 Beispiel-XML (gekürzt)

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest xmlns="urn:org:ebics:H005"
   xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
   xsi:schemaLocation="urn:org:ebics:H005 ebics_request_H005.xsd"
   Version="H005" Revision="1">
   <header authenticate="true">
      <static>
         <!-- […] -->
         <OrderDetails>
            <AdminOrderType>HVZ</AdminOrderType>
HVZOrderParams>
               <ServiceFilter>…..</ServiceFilter…..>
            </HVZOrderParams>
         </OrderDetails>
         <!-- […] -->
      </static>
      <!-- […] -->
   </header>
   <!-- […] -->
</ebicsRequest>
```

8.3.1.4 HVZ-Response

In der HVZ-Response werden dem Teilnehmer Informationen zu den Aufträgen zurückgeliefert, für die er unterschriftsberechtigt ist.

HVZResponseOrderData enthält alle Informationen aus HVUResponseOrderData und HVDResponseOrderData mit Ausnahme des Elements „DisplayFile“ mit der Dateianzeige. Wie bei HVD wird der Auftragshashwert aus der EU des ersten Unterzeichners des Auftrags extrahiert bzw. ggf. neu berechnet, wenn der Teilnehmer, der HVZ durchführt, ein anderes Signaturverfahren verwendet. Um dies transparent zu gestalten, wird dem Hashwert ein Attribut mit dem verwendeten Signaturverfahren mitgegeben.

Nur für Zahlungsaufträge werden zusätzliche Informationen aus der Dateianzeige geliefert, falls diese verfügbar sind:

- Summe aller Beträge über alle logischen Dateien
- Gesamtanzahl der Sätze über alle logischen Dateien
- Währung (nur bei sortenreinen Zahlungen, sonst keine Angabe)
- Für DTAZV: Auftraggeber, Kontonummer / IBAN und Bankleitzahl / BIC der ersten Transaktion der ersten logischen Datei

Ausprägung der (dekodierten & entschlüsselten & dekomprimierten) `OrderData` (Auftragsdaten) für HVZ: `HVZResponseOrderData`

8.3.1.4.1 XML-Schema (grafische Darstellung)

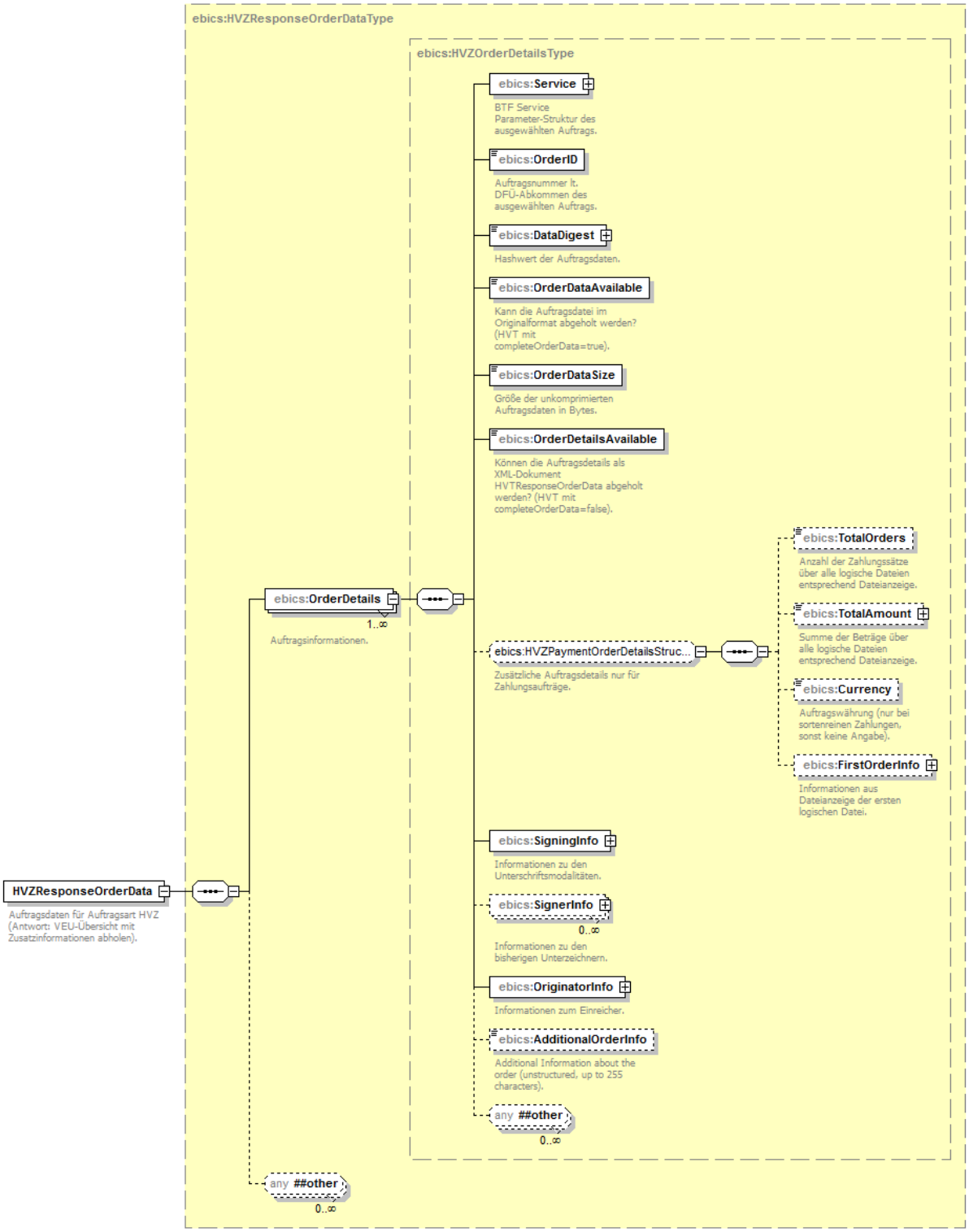


Abbildung 74: HVZResponseOrderData

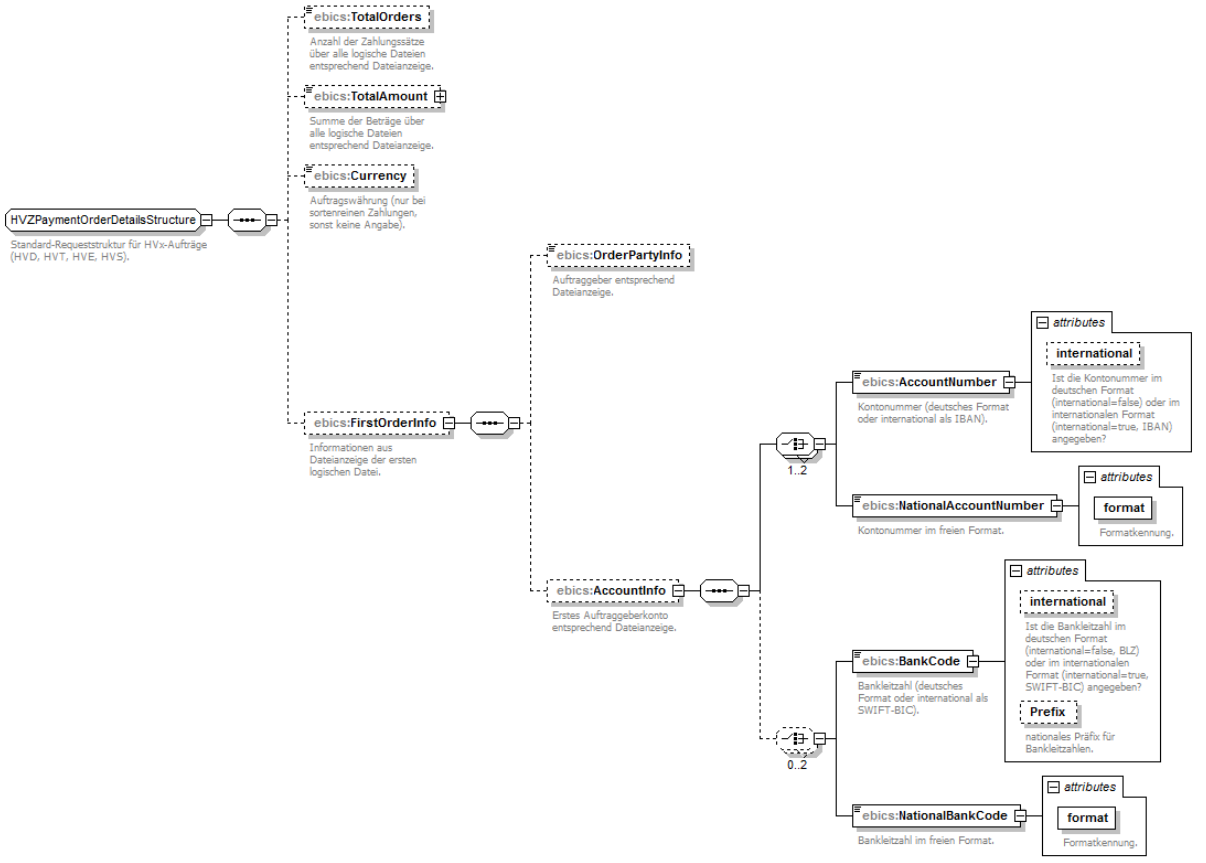


Abbildung 75 HVZPaymentOrderDetailsStructure

8.3.1.4.2 XML-Schema (textuelle Darstellung)

```
<element name="HVZResponseOrderData"
   type="ebics:HVZResponseOrderDataType"
   substitutionGroup="ebics:EBICSOrderData">
   <annotation>
      <documentation xml:lang="de">
         Auftragsdaten für Auftragsart HVZ (Antwort: VEU-Übersicht mit
         Zusatzinformationen abholen).
      </documentation>
   </annotation>
</element>
<complexType name="HVZResponseOrderDataType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für Auftragsdaten für Auftragsart HVZ (Antwort:
         VEU-Übersicht mit Zusatzinformationen abholen).
      </documentation>
   </annotation>
   <sequence>
      <annotation>
         <documentation xml:lang="de" />
      </annotation>
      <element name="OrderDetails" type="ebics:HVZOrderDetailsType"
         maxOccurs="unbounded">
         <annotation>
            <documentation xml:lang="de">
```

```
                Auftragsinformationen.
             </documentation>
          </annotation>
       </element>
       <any namespace="##other" processContents="lax" minOccurs="0"
          maxOccurs="unbounded" />
    </sequence>
</complexType>
<complexType name="HVZOrderDetailsType">
    <annotation>
       <documentation xml:lang="de">
          Datentyp für HVZ-Auftragsdetails.
       </documentation>
    </annotation>
    <sequence>
       <element name="OrderType" type="ebics:OrderTBaseType">
          <annotation>
             <documentation xml:lang="de">
                Auftragsart lt. DFÜ-Abkommen des ausgewählten Auftrags.
             </documentation>
          </annotation>
       </element>
       <element name="OrderID" type="ebics:OrderIDType">
          <annotation>
             <documentation xml:lang="de">
                Auftragsnummer lt. DFÜ-Abkommen des ausgewählten
                Auftrags.
             </documentation>
          </annotation>
       </element>
       <element name="DataDigest" type="ebics:DigestType">
          <annotation>
             <documentation xml:lang="de">
                Hashwert der Auftragsdaten.
             </documentation>
          </annotation>
       </element>
       <element name="OrderDataAvailable" type="boolean">
          <annotation>
             <documentation xml:lang="de">
                Kann die Auftragsdatei im Originalformat abgeholt werden?
                (HVT mit completeOrderData=true).
             </documentation>
          </annotation>
       </element>
       <element name="OrderDataSize" type="positiveInteger">
          <annotation>
             <documentation xml:lang="de">
                Größe der unkomprimierten Auftragsdaten in Bytes.
             </documentation>
          </annotation>
       </element>
       <element name="OrderDetailsAvailable" type="boolean">
          <annotation>
             <documentation xml:lang="de">
                Können die Auftragsdetails als XML-Dokument
                HVTResponseOrderData abgeholt werden? (HVT mit
                completeOrderData=false).
             </documentation>
          </annotation>
       </element>
       <group ref="ebics:HVZPaymentOrderDetailsStructure"
          minOccurs="0">
```

```
         <annotation>
            <documentation>
               Auftragsdetails, die nur Zahlungsaufträge betreffen.
            </documentation>
         </annotation>
      </group>
      <element name="SigningInfo" type="ebics:HVUSigningInfoType">
         <annotation>
            <documentation xml:lang="de">
               Informationen zu den Unterschriftsmodalitäten.
            </documentation>
         </annotation>
      </element>
      <element name="SignerInfo" type="ebics:SignerInfoType"
         minOccurs="0" maxOccurs="unbounded">
         <annotation>
            <documentation xml:lang="de">
               Informationen zu den bisherigen Unterzeichnern.
            </documentation>
         </annotation>
      </element>
      <element name="OriginatorInfo"
         type="ebics:HVUOriginatorInfoType">
         <annotation>
            <documentation xml:lang="de">
               Informationen zum Einreicher.
            </documentation>
         </annotation>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0"
         maxOccurs="unbounded" />
   </sequence>
</complexType>
<group name="HVZPaymentOrderDetailsStructure">
   <annotation>
      <documentation xml:lang="de">
         Struktur mit zusätzlichen Auftragsdetails in
         HVZResponseOrderData für Zahlungsaufträge.
      </documentation>
      <documentation xml:lang="en">
         Structure with additional order details in HVZResponseOrderData
         related to payment orders.
      </documentation>
   </annotation>
   <sequence>
      <element name="TotalOrders" type="nonNegativeInteger"
         minOccurs="0">
         <annotation>
            <documentation xml:lang="de">
               Anzahl der Zahlungssätze über alle logischen Dateien
               entsprechend Dateianzeige.
            </documentation>
            <documentation xml:lang="en">
               Total transaction number for all logical files (from
               dispay file).
            </documentation>
         </annotation>
      </element>
      <element name="TotalAmount" minOccurs="0">
         <annotation>
            <documentation xml:lang="de">
               Summe der Beträge über alle logische Dateien entsprechend
               Dateianzeige.
```

```
            </documentation>
            <documentation xml:lang="en">
               Total transaction amount for all logical files (from
               dispay file).
            </documentation>
         </annotation>
         <simpleType>
            <restriction base="ebics:AmountValueType" />
         </simpleType>
      </element>
      <element name="Currency" type="ebics:CurrencyBaseType"
         minOccurs="0">
         <annotation>
            <documentation xml:lang="de">
               Auftragswährung (nur bei sortenreinen Zahlungen, sonst
               keine Angabe).
            </documentation>
            <documentation xml:lang="en">
               Order currency (only if identical across all
               transactions, ship otherwise).
            </documentation>
         </annotation>
      </element>
      <element name="FirstOrderInfo" minOccurs="0">
         <annotation>
            <documentation xml:lang="de">
               Informationen aus Dateianzeige der ersten logischen
               Datei.
            </documentation>
            <documentation xml:lang="en">
               Order details from display file for first logical file.
            </documentation>
         </annotation>
         <complexType>
            <sequence>
               <element name="OrderPartyInfo" type="normalizedString"
                  minOccurs="0">
                  <annotation>
                     <documentation xml:lang="de">
                        Auftraggeber entsprechend Dateianzeige.
                     </documentation>
                     <documentation xml:lang="en">
                        Order party information (from display file).
                     </documentation>
                  </annotation>
               </element>
               <element name="AccountInfo" minOccurs="0">
                  <annotation>
                     <documentation xml:lang="de">
                        Erstes Auftraggeberkonto entsprechend
                        Dateianzeige.
                     </documentation>
                     <documentation xml:lang="en">
                        First order party account (from display file).
                     </documentation>
                  </annotation>
                  <complexType>
                     <sequence>
                        <choice maxOccurs="2">
                           <element name="AccountNumber">
                              <annotation>
                                 <documentation xml:lang="de">
                                    Kontonummer (deutsches Format oder
```

```
                international als IBAN).
            </documentation>
            <documentation xml:lang="en">
                Account number (German format or
                international as IBAN).
            </documentation>
        </annotation>
        <complexType>
            <simpleContent>
                <extension
                    base="Q1:AccountNumberType">
                    <attribute name="international"
                        type="boolean" use="optional"
                        default="false">
                        <annotation>
                            <documentation
                                xml:lang="de">
                                Ist die Kontonummer im
                                deutschen Format
                                (international=false)
                                oder im internationalen
                                Format
                                (international=true,
                                IBAN) angegeben?
                            </documentation>
                            <documentation
                                xml:lang="en">
                                Account number given in
                                German format
                                (international=false) or
                                in international format
                                (international=true,
                                IBAN)?
                            </documentation>
                        </annotation>
                    </attribute>
                </extension>
            </simpleContent>
        </complexType>
    </element>
    <element name="NationalAccountNumber">
        <annotation>
            <documentation xml:lang="de">
                Kontonummer im freien Format.
            </documentation>
            <documentation xml:lang="en">
                Account number in free format.
            </documentation>
        </annotation>
        <complexType>
            <simpleContent>
                <extension
                    base="Q1:NationalAccountNumberType">
                    <attribute name="format"
                        type="token" use="required">
                        <annotation>
                            <documentation
                                xml:lang="de">
                                Formatkennung.
                            </documentation>
                            <documentation
                                xml:lang="en">
                                Format type.
```

```
                                    </documentation>
                                </annotation>
                            </attribute>
                        </extension>
                    </simpleContent>
                </complexType>
            </element>
        </choice>
        <choice maxOccurs="2">
            <element name="BankCode">
                <annotation>
                    <documentation xml:lang="de">
                        Bankleitzahl (deutsches Format oder
                        international als SWIFT-BIC).
                    </documentation>
                    <documentation xml:lang="en">
                        Bank sort code (German format or
                        international as SWIFT-BIC).
                    </documentation>
                </annotation>
                <complexType>
                    <simpleContent>
                        <extension base="Q1:BankCodeType">
                            <attribute name="international"
                                type="boolean" use="optional"
                                default="false">
                                <annotation>
                                    <documentation
                                        xml:lang="de">
                                        Ist die Bankleitzahl im
                                        deutschen Format
                                        (international=false,
                                        BLZ) oder im
                                        internationalen Format
                                        (international=true,
                                        SWIFT-BIC) angegeben?
                                    </documentation>
                                    <documentation
                                        xml:lang="en">
                                        Bank sort code given in
                                        German format
                                        (international=false) or
                                        in international format
                                        (international=true,
                                        SWIFT-BIC)?
                                    </documentation>
                                </annotation>
                            </attribute>
                            <attribute name="Prefix"
                                type="ebics:BankCodePrefixType"
                                use="optional">
                                <annotation>
                                    <documentation
                                        xml:lang="de">
                                        nationales Präfix für
                                        Bankleitzahlen.
                                    </documentation>
                                    <documentation
                                        xml:lang="en">
                                        National prefix for bank
                                        sort code.
                                    </documentation>
                                </annotation>
```

```
                                    </attribute>
                                </extension>
                            </simpleContent>
                        </complexType>
                    </element>
                    <element name="NationalBankCode">
                        <annotation>
                            <documentation xml:lang="de">
                                Bankleitzahl im freien Format.
                            </documentation>
                            <documentation xml:lang="en">
                                Bank sort code in free format.
                            </documentation>
                        </annotation>
                        <complexType>
                            <simpleContent>
                                <extension
                                    base="Q1:NationalBankCodeType">
                                    <attribute name="format"
                                        type="token" use="required">
                                        <annotation>
                                            <documentation
                                                xml:lang="de">
                                                Formatkennung.
                                            </documentation>
                                            <documentation
                                                xml:lang="en">
                                                Format type.
                                            </documentation>
                                        </annotation>
                                    </attribute>
                                </extension>
                            </simpleContent>
                        </complexType>
                    </element>
                </choice>
            </sequence>
        </complexType>
    </element>
  </sequence>
</group>
```

8.3.1.4.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/-Attribut	Datentyp	#	Bedeutung	Beispiel
`HVZResponse» OrderData`	`ebics:HVZResponse» OrderDataType` (komplex)	1	XML-Auftragsdaten für HVZ	- (komplex)
`OrderDetails`	`ebics:HVZOrder» DetailsType` (komplex)	1..∞	Auftragsinformationen für HVZ	- (komplex)
`Service`	`ebics:RestrictedServiceType` Details zur Struktur siehe Kapitel 8.3.6		Art des Geschäftsvorfalls, identifiziert durch die Service-Struktur des zur	

			VEU eingereichten Auftrags	
OrderID	ebics:OrderIDType (→token, fixLength=4)	1	Auftragsnummer des zur VEU eingereichten Auftrags	„OR01“
DataDigest	ebics:DigestType (→dsig:DigestValue» Type →base64Binary)	1	Hashwert des Auftrags für das Signaturverfahren des Teilnehmers gemäß Request-Element UserID	- (Base64-Daten)
DataDigest» @SignatureVersion	ebics:Signature»VersionType (→token, length=4, pattern="A\d{3}"		Version des Signaturverfahrens des Teilnehmers gemäß Request-Element UserID	z.B. „A005“
OrderDataAvailable	boolean	1	Kann die Auftragsdatei im Originalformat abgeholt werden? (HVT mit completeOrderData=true)	„true“
OrderDataSize	positiveInteger	1	Größe der unkomprimierten Auftragsdaten des zur VEU eingereichten Auftrags in Bytes	123456
OrderDetails» Available	boolean	1	Können die Auftragsdetails als XML-Dokument HVTResponseOrderData abgeholt werden? (HVT mit completeOrderData=false)	„true“
TotalOrders	nonNegativeInteger	0..1	Anzahl der Zahlungssätze über alle logischen Dateien entsprechend Dateianzeige.	15
TotalAmount	ebics:AmountValue» Type (→decimal, totalDigits=24, fractionDigits=4)	0..1	Summe der Beträge über alle logischen Dateien entsprechend Dateianzeige.	129.00
TotalAmount» @isCredit	boolean	0..1	Kennzeichen zur Unterscheidung von Überweisungen (isCredit="true") und Lastschriften (isCredit="false"). (optional verwendbar, wenn dies über alle Transaktionen identisch	“false“

			ist; andernfalls auszulassen).	
Currency	ebics:CurrencyBase»Type (→token, length=3, pattern="[A-Z]{3}")	0..1	Auftragswährung (nur bei sortenreinen Zahlungen, sonst keine Angabe).	„USD“
FirstOrderInfo	(komplex)	0..1	Informationen aus Dateianzeige der ersten logischen Datei.	- (komplex)
OrderPartyInfo	normalizedString	0..1	Auftraggeber entsprechend Dateianzeige.	„Arnold Auftraggeber“
AccountInfo	komplex	0..1		- (komplex)
–	–	1..2	Angaben zur Kontonummer: AccountNumber und/oder NationalAccountNumber	-
AccountNumber	ebics:AccountNumber»Type (→token, maxLength=40, pattern="\d{3,10}|([A-Z]{2}\d{2}[A-Za-z0-9]{3,30}")	1	Kontonummer (deutsches Format oder international als IBAN)	„12345678“
AccountNumber»@international	boolean	0..1	Ist die Kontonummer im deutschen Format (international=false) oder im internationalen Format (international=true, IBAN) angegeben? Default="false"	„false“
NationalAccount»Number	ebics:National»AccountNumberType (→token, maxLength=40)	1	Kontonummer im freien Format (weder deutsches Format noch IBAN)	„1234567890123456“
NationalAccount»Number@format	token	1	Formatkennung	„other“
–	–	1..2	Angaben zur Bankleitzahl: BankCode und/oder NationalBankCode	-
BankCode	ebics:BankCodeType (→token, maxLength=11, pattern="\d{8}|([A-Z]{6}[A-Z0-9]{2}([A-Z0-9]{3})?)")	1	Bankleitzahl (deutsches Format oder international als SWIFT-BIC).	„50010060“ Hinweis: Dieses Element kann im Falle von IBAN-Only

				nicht bereitgestellt werden
BankCode» @international	boolean	0..1	Ist die Bankleitzahl im nationalen=deutschen (BankCode» @international= "false") oder internationalen=SWIFT-BIC-Format (BankCode» @international="true") angegeben? Default="false"	„false“
NationalBankCode	ebics:National» BankCodeType (→token, maxLength=30)	1	Bankleitzahl im freien Format (weder deutsches Format noch SWIFT-BIC)	„123456789012“
NationalBankCode» @format	token	1	Formatkennung	„other“
SigningInfo	ebics:HVUSigning» InfoType (komplex)	1	Informationen zu den Unterschriftsmodalitäten	- (komplex)
SigningInfo» @readyToBeSigned	Boolean	1	Ist der Auftrag unterschriftsreif (true) oder bereits vom Teilnehmer unterschrieben (false)?	„true“
SigningInfo» @NumSigRequired	positiveInteger	1	Anzahl der insgesamt zur Freigabe erforderlichen EUs	4
SigningInfo» @numSigDone	nonNegativeInteger	1	Anzahl der bereits geleisteten EUs	2
SignerInfo	ebics:SignerInfo» Type (komplex)	0..∞	Informationen zu den bisherigen Unterzeichnern	- (komplex)
PartnerID (in SignerInfo)	ebics:PartnerIDType (→token, maxLength=35, pattern="[a-zA-Z0-9,=]{1,35})	1	Kunden-ID des Unterzeichners	„PARTNER1“
UserID (in SignerInfo)	ebics:UserIDType (→token, maxLength=35, pattern="[a-zA-Z0-9,=]{1,35})	1	Teilnehmer-ID des Unterzeichners	„USER0001“
Name (in SignerInfo)	ebics:NameType (→normalizedString)	0..1	Name des Unterzeichners	„Max Mustermann“
Timestamp (in SignerInfo)	ebics:TimestampType (→dateTime)	1	Zeitstempel der Unterzeichnung (d.h. der Übertragung der Unterschrift)	„2005-01-31T» 16:30:45.123Z“
Permission	– (komplex)	1	zusätzliche Berechtigungsinformationen zu	- (komplex)

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

			dem Teilnehmer, der unterzeichnet hat	
Permission» @Authorisation» Level	ebics:Authorisation» LevelType (→token, length=1: "E", "A", "B", "T")	1	Unterschriftsberechtigung des Teilnehmers, der unterzeichnet hat	„A“
OriginatorInfo	ebics:HVUOriginator» InfoType (komplex)	1	Informationen zu dem Einreicher des Auftrags	- (komplex)
PartnerID (in OriginatorInfo)	ebics:PartnerIDType (→token, maxLength=35, pattern="[a-zA-Z0-9,=]{1,35})	1	Kunden-ID des Einreichers	„PARTNER2“
UserID (in OriginatorInfo)	ebics:UserIDType (→token, maxLength=35, pattern="[a-zA-Z0-9,=]{1,35})	1	Teilnehmer-ID des Einreichers	„USER0002“
Name (in OriginatorInfo)	ebics:NameType (→normalizedString)	0..1	Name des Einreichers	„Erich Einreicher“
Timestamp (in OriginatorInfo)	ebics:TimestampType (→dateTime)	1	Zeitstempel der Einreichung (d.h. der Übertragung der Auftragsdatei)	„2005-01-30T» 15:30:45.123Z“

8.3.1.4.4 Beispiel-XML

```
<?xml version="1.0" encoding="UTF-8"?>
<HVZResponseOrderData xmlns="urn:org:ebics:H005"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="urn:org:ebics:H005 ebics_orders_H005.xsd">
   <OrderDetails>

      <Service>
        <ServiceName>SCT</ServiceName>
        <MsgName>pain.001</MsgName>
      </Service>      <OrderID>OR01</OrderID>
      <DataDigest SignatureVersion="A006"> MTIzNDU2Nzg5MDEyMzQ1Njc4OTAxMjM0NTY3ODkwMTI6
MTIzNDU2Nzg5MDEyMzQ1Njc4OTAxMjM0NTY3ODkwMTI=</DataDigest>
      <OrderDataAvailable>true</OrderDataAvailable>
      <OrderDataSize>123456</OrderDataSize>
      <OrderDetailsAvailable>true</OrderDetailsAvailable>
      <TotalOrders>22</TotalOrders>
      <TotalAmount>500.00</TotalAmount>
      <Currency>EUR</Currency>
      <FirstOrderInfo>
         <OrderPartyInfo>Arnold Auftraggeber</OrderPartyInfo>
         <AccountInfo>
            <AccountNumber international="true">
               DE68210501700012345678
            </AccountNumber>
            <BankCode international="false" Prefix="DE">
               21050170
            </BankCode>
         </AccountInfo>
      </FirstOrderInfo>
      <SigningInfo NumSigRequired="4" readyToBeSigned="true"
```

```
         NumSigDone="2" />
      <SignerInfo>
         <PartnerID>PARTNER1</PartnerID>
         <UserID>USER0001</UserID>
         <Name>Max Mustermann</Name>
         <Timestamp>2005-01-31T16:30:45.123Z</Timestamp>
         <Permission AuthorisationLevel="A" />
      </SignerInfo>
      <SignerInfo>
         <PartnerID>PARTNER2</PartnerID>
         <UserID>USER0002</UserID>
         <Name>Maxime Musterfrau</Name>
         <Timestamp>2005-01-31T17:30:45.123Z</Timestamp>
         <Permission AuthorisationLevel="B" />
      </SignerInfo>
      <OriginatorInfo>
         <PartnerID>PARTNER1</PartnerID>
         <UserID>USER0001</UserID>
         <Name>Erich Einreicher</Name>
         <Timestamp>2016-10-11T15:30:45.123Z</Timestamp>
      </OriginatorInfo>
   </OrderDetails>
</HVZResponseOrderData>
```

8.3.2 HVD (VEU-Status abrufen)

Mit HVD kann ein Teilnehmer den Status eines Auftrags abrufen, der sich aktuell in der VEU-Prozessverarbeitung befindet und für den der Teilnehmer unterschriftsberechtigt ist. Er bekommt Informationen über den Auftrag in Form eines elektronischen Begleitzettels (`DisplayFile`) und des Auftrags-Hashwerts (`DataDigest`) sowie über die bisherigen Unterzeichner (`SignerInfo`). Den Auftragshashwert hat das Banksystem aus der EU des ersten Unterzeichners des Auftrags extrahiert oder es wird in dem Fall, dass der Teilnehmer, der HVD durchführt, ein anderes Signaturverfahren verwendet, ggf. neu berechnet. Die Daten des Begleitzettels MÜSSEN inhaltlich mit den Auftragsdaten, deren Hashwert mitgeliefert wird, übereinstimmen.
Das Banksystem muss prüfen, ob der Teilnehmer eine bankfachliche Unterschriftsberechtigung (Unterschriftsklasse E, A oder B) für den konkreten Auftrag besitzt und der Auftrag noch in der Unterschriftsmappe ist. Falls die Berechtigung fehlt, muss die Transaktion mit dem Fehlercode EBICS_DISTRIBUTED_SIGNATURE_AUTHORISATION_FAILED abgebrochen werden.

- Bei einigen zugrunde liegenden administrativen Auftragsarten / Geschäftsvorfällen ist ein Abruf von Detailinformationen zu einem bestimmten Auftrag in der VEU-Prozessverarbeitung mittels der Transaktion HVT nicht möglich. Ob das für den aktuellen Auftrag möglich ist oder nicht, wird vom Banksystem in der HVD-Response signalisiert.
- Das Banksystem prüft vor der Ausführung von HVD, dass sich der Auftrag aktuell in der VEU-Prozessverarbeitung befindet und beendet im Fehlerfall die Transaktion mit dem fachlichen Fehlercode EBICS_ORDERID_UNKNOWN.

  HVD ist eine Auftragsart vom Typ „Download“.

8.3.2.1 HVD-Request

Im HVD-Request übergibt der Teilnehmer die relevanten Daten zur Identifikation des Auftrags, für den er den VEU-Status abrufen will.

Ausprägung der `OrderParams` (Auftragsparameter) für HVD: `HVDOrderParams`

8.3.2.1.1 XML-Schema (grafische Darstellung)

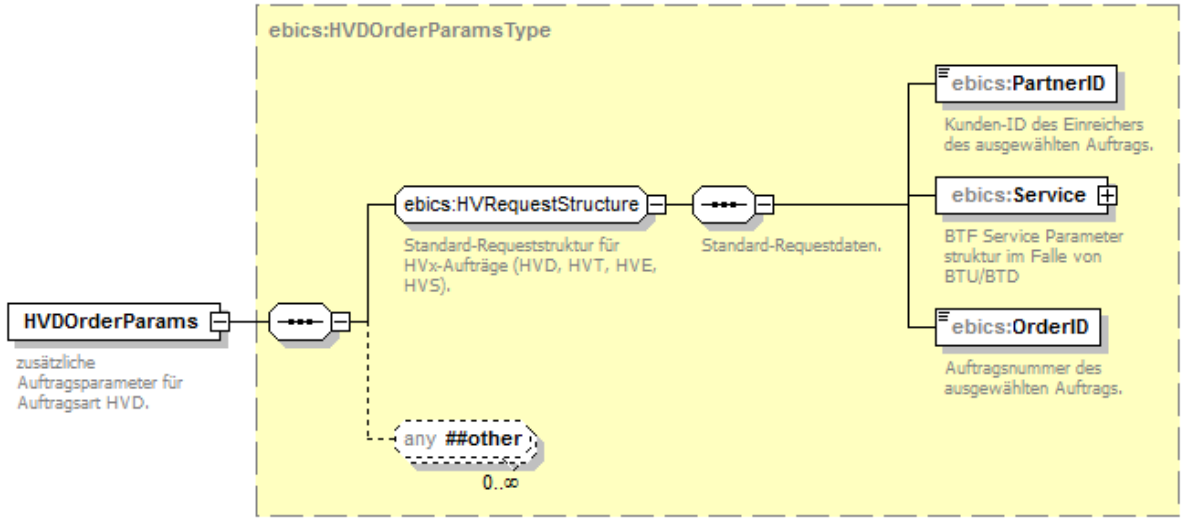


Abbildung 76: HVDOrderParams

8.3.2.1.2 XML-Schema (textuelle Darstellung)

```
<element name="HVDOrderParams" type="ebics:HVDOrderParamsType" substitutionGroup="ebics:OrderParams">
  <annotation>
    <documentation xml:lang="de">zusätzliche Auftragsparameter für Auftragsart HVD.</documentation>
  </annotation>
</element>
<complexType name="HVDOrderParamsType">
  <annotation>
    <documentation xml:lang="de">Datentyp für zusätzliche Auftragsparameter für Auftragsart
HVD.</documentation>
  </annotation>
  <sequence>
    <group ref="ebics:HVRequestStructure"/>
    <any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
  </sequence>
</complexType>
<group name="HVRequestStructure">
  <annotation>
    <documentation xml:lang="de">Standard-Requeststruktur für HVx-Aufträge (HVD, HVT, HVE,
HVS).</documentation>
  </annotation>
  <sequence>
    <annotation>
```

```
        <documentation xml:lang="de">Standard-Requestdaten.</documentation>
      </annotation>
      <element name="PartnerID" type="ebics:PartnerIDType">
        <annotation>
          <documentation xml:lang="de">Kunden-ID des Einreichers des ausgewählten Auftrags.</documentation>
        </annotation>
      </element>
      <element name="OrderType" type="ebics:OrderTBaseType">
        <annotation>
          <documentation xml:lang="de">Auftragsart des ausgewählten Auftrags gemäß Anhang (Kapitel
13).</documentation>
        </annotation>
      </element>
      <element name="OrderID" type="ebics:OrderIDType">
        <annotation>
          <documentation xml:lang="de">Auftragsnummer des ausgewählten Auftrags gemäß Kapitel
10.1.</documentation>
        </annotation>
      </element>
    </sequence>
  </group>
```

8.3.2.1.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
HVDOrderParams	ebics:HVDOrderParamsType (komplex)	1	Auftragsparameter für Auftragsart HVD	- (komplex)
PartnerID	ebics:PartnerIDType (→token, maxLength=35, pattern="[a-zA-Z0-9,=]{1,35})	1	Kunden-ID des Einreichers	„PARTNER1“
Service	ebics:RestrictedServiceType Details zu dieser Struktur siehe Kapitel 8.3.6	1	Art des Geschäftsvorfalls, identifiziert durch die Service-Struktur des zur VEU eingereichten Auftrags	
OrderID	ebics:OrderIDType (→token, fixLength=4)	1	Auftragsnummer des zur VEU eingereichten Auftrags	„OR01“

8.3.2.1.4 Beispiel-XML (gekürzt)

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest
 xmlns="urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_request_H005.xsd"
 Version="H005" Revision="1">
  <header authenticate="true">
    <static>
      <!-- […] -->
      <OrderDetails>
        <AdminOrderType>HVD</AdminOrderType>
      <HVDOrderParams>
```

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

```
          <PartnerID>PARTNER1</PartnerID>
          <Service>
              <ServiceName>SCT</ServiceName>
              <MsgName>pain.001</MsgName>
          </Service>
          <OrderID>OR01</OrderID>
        </HVDOrderParams>
      </OrderDetails>
      <!-- […] -->
    </static>
    <!-- […] -->
  </header>
  <!-- […] -->
</ebicsRequest>
```

8.3.2.2 HVD-Response

Die HVD-Response enthält VEU-Informationen zu dem Auftrag, den der Teilnehmer im HVD-Request angefragt hat. Insbesondere wird der Hashwert der Auftragsdaten aus der EU des ersten Unterzeichners samt Begleitzettel zurückgeliefert. Zusätzlich wird angegeben, ob das Banksystem die Transaktion HVT für den konkreten Auftrag unterstützt. Dabei wird unterschieden zwischen

- `OrderDataAvailable` : Abholung der kompletten Auftragsdatei mit HVT und completeOrderData=true möglich?
- `OrderDetailsAvailable` : Abholung der aufbereiteten Auftragsdetails im XML-Format mit HVT und `completeOrderData=false` möglich?

Die HVD-Response stellt dem Teilnehmer sämtliche Daten zur Verfügung, die dieser zur Bestätigung des Auftrags mittels HVE oder Stornierung mittels HVS benötigt.

Ausprägung der (dekodierten & entschlüsselten & dekomprimierten) `OrderData` (Auftragsdaten) für HVD:

Abbildung 77: HVDResponseOrderData

8.3.2.2.1 XML-Schema (textuelle Darstellung)

```
 <element name="HVDResponseOrderData" type="ebics:HVDResponseOrderDataType"
substitutionGroup="ebics:EBICSOrderData">
   <annotation>
     <documentation xml:lang="de">Auftragsdaten für Auftragsart HVD (Antwort: VEU-Status
abrufen).</documentation>
   </annotation>
 </element>
 <complexType name="HVDResponseOrderDataType">
```

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

```
    <annotation>
      <documentation xml:lang="de">Datentyp für Auftragsdaten für Auftragsart HVD (Antwort: VEU-Status
abrufen).</documentation>
    </annotation>
    <sequence>
      <element name="DataDigest" type="ebics:DigestType">
        <annotation>
          <documentation xml:lang="de">Hashwert der Auftragsdaten.</documentation>
        </annotation>
      </element>
      <element name="DisplayFile" type="base64Binary">
        <annotation>
          <documentation xml:lang="de">Begleitzettel/"Displaydatei" (entspricht der Dateianzeige im
Kundenprotokoll) </documentation>
        </annotation>
      </element>
      <element name="OrderDataAvailable" type="boolean">
        <annotation>
          <documentation xml:lang="de">Kann die Auftragsdatei im Originalformat abgeholt werden? (HVT mit
completeOrderData=true)</documentation>
        </annotation>
      </element>
      <element name="OrderDataSize" type="positiveInteger">
        <annotation>
          <documentation xml:lang="de">Größe der unkomprimierten Auftragsdaten in Bytes.</documentation>
        </annotation>
      </element>
      <element name="OrderDetailsAvailable" type="boolean">
        <annotation>
          <documentation xml:lang="de">Können die Auftragsdetails als XML-Dokument HVTResponseOrderData
abgeholt werden?  (HVT mit completeOrderData=false)</documentation>
        </annotation>
      </element>
      <element name="BankSignature" type="ebics:SignatureType" minOccurs="0" maxOccurs="0">
        <annotation>
          <documentation xml:lang="de">Elektronische Unterschrift des Kreditinstituts über Hashwert und
Displaydatei.</documentation>
        </annotation>
      </element>
      <element name="SignerInfo" type="ebics:SignerInfoType" minOccurs="0" maxOccurs="unbounded">
        <annotation>
          <documentation xml:lang="de">Informationen zu den bisherigen Unterzeichnern.</documentation>
        </annotation>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
    </sequence>
  </complexType>
```

8.3.2.2.2 Bedeutung der XML-Elemente/-Attribute

XML-Element/-Attribut	Datentyp	#	Bedeutung	Beispiel
HVDResponseOrderData	ebics:HVDResponse»OrderDataType (komplex)	1	XML-Auftragsdaten für Auftragsart HVD	- (komplex)
DataDigest	ebics:DigestType (→dsig:DigestValue»Type →base64Binary)	1	Hashwert des Auftrags für das Signaturverfahren des Teilnehmers gemäß Request-Element UserID	- (Base64-Daten)

DataDigest» @SignatureVersion	ebics:Signature»VersionType (→token, length=4, pattern="A\d{3}"		Version des Signaturverfahrens des Teilnehmers gemäß Request-Element UserID	z.B. „A006“
DisplayFile	base64Binary	1	Begleitzettel/ „Display-Datei“ zum eingereichten Auftrag	- (Base64-Daten)
OrderDataAvailable	boolean	1	Kann die Auftragsdatei im Originalformat abgeholt werden? (HVT mit completeOrderData=true)	true
OrderDataSize	positiveInteger	1	Größe der unkomprimierten Auftragsdaten in Bytes.	1280
OrderDetailsAvailable	boolean	1	Können die Auftragsdetails als XML-Dokument HVTResponseOrderData abgeholt werden? (HVT mit completeOrderData=false)	true
BankSignature	ebics:SignatureType (→base64Binary)	0	EU des Kreditinstituts über Hashwert und Begleitzettel; vorgesehenes Feature	- (Base64-Daten)
SignerInfo	ebics:SignerInfo»Type (komplex)	0..∞	Informationen zu den bisherigen Unterzeichnern	- (komplex)

Für die restlichen XML-Elemente und -Attribute: Siehe Auftragsart HVU (Kapitel 8.3.1.2).

8.3.2.2.3 Beispiel-XML

```
<?xml version="1.0" encoding="UTF-8"?>
<HVDResponseOrderData
 xmlns="urn:org:ebics:H005"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_orders_H005.xsd">
 <DataDigest SignatureVersion="A006">MTIzNDU2Nzg5MDEyMzQ1Njc4OTAxMjM0NTY3ODkwMTI=</DataDigest>
 <DisplayFile>…</DisplayFile>
 <OrderDataAvailable>true</OrderDataAvailable>
 <OrderDataSize>1280</OrderDataSize>
 <OrderDetailsAvailable>true</OrderDetailsAvailable>
 <SignerInfo>
   <PartnerID>PARTNER1</PartnerID>
   <UserID>USER0001</UserID>
   <Name>Max Mustermann</Name>
   <Timestamp>2005-01-31T16:30:45.123Z</Timestamp>
   <Permission AuthorisationLevel="A"/>
 </SignerInfo>
 <SignerInfo>
   <PartnerID>PARTNER2</PartnerID>
   <UserID>USER0002</UserID>
   <Name>Maxime Musterfrau</Name>
   <Timestamp>2005-01-31T17:30:45.123Z</Timestamp>
   <Permission AuthorisationLevel="B"/>
 </SignerInfo>
</HVDResponseOrderData>
```

8.3.3 HVT (VEU-Transaktionsdetails abrufen)

HVT liefert dem Teilnehmer detaillierte Informationen über einen Auftrag aus der VEU-Prozessverarbeitung zurück, für den der Teilnehmer unterschriftsberechtigt ist. Je nach Request (`OrderFlags@completeOrderData`) bekommt er entweder die komplette Auftragsdatei oder Details zu Konten, Ausführungstermin, Beträgen und weitere Beschreibungen (`OrderInfo`).

Weitere Filterkriterien (z.B. zur Auswahl von Einzelaufträgen innerhalb eines Gesamtauftrags) kann der Teilnehmer beim Request in der generischen Schlüssel-Wert-Struktur (`Parameter`) übermitteln.

Bei einigen zugrunde liegenden administrativen Auftragsarten / Geschäftsvorfällen ist ein Abruf von Detailinformationen mittels `OrderFlags@completeOrderData="false"` nicht möglich. In diesem Fall wird vom Banksystem der fachliche Fehlercode EBICS_UNSUPPORTED_REQUEST_FOR_ORDER_INSTANCE zurückgeliefert. Das Banksystem signalisiert in der HVD-Response mit `OrderDataAvailable` und `OrderDetailsAvailable`, ob für einen bestimmten Auftrag in der VEU-Verwaltung die Durchführung der HVT Transaktion möglich ist.

Das Banksystem prüft vor der Ausführung von HVT, dass sich der Auftrag aktuell in der VEU-Prozessverarbeitung befindet und beendet im Fehlerfall die Transaktion mit dem fachlichen Fehlercode EBICS_ORDERID_UNKNOWN.

Das Banksystem muss prüfen, ob der Teilnehmer eine bankfachliche Unterschriftsberechtigung (Unterschriftsklasse E, A oder B) für den konkreten Auftrag besitzt und der Auftrag noch in der Unterschriftsmappe ist. Falls die Berechtigung fehlt, muss die Transaktion mit dem Fehlercode EBICS_DISTRIBUTED_SIGNATURE_AUTHORISATION_FAILED abgebrochen werden.

HVT ist eine administrative Auftragsart vom Typ „Download“.

8.3.3.1 HVT-Request

Im HVT-Request spezifiziert der Teilnehmer den Auftrag, für den er VEU-Transaktionsdetails abrufen will. Außerdem entscheidet er durch Setzen des OrderFlags `completeOrderData`, ob er als Rückmeldung Auftragsdetails (`completeOrderData="false"`) oder die komplette Auftragsdatei (`completeOrderData="true"`) wünscht.

Das Kundensystem kann bei `completeOrderData="false"` die Anzahl der vom Banksystem zu liefernden Auftragsdetails begrenzen. Mit dem Attribut `fetchLimit` am Element `OrderFlags` kann die maximale Anzahl der zu liefernden Auftragsdetails angegeben werden (Ein Vorschlag ist `fetchLimit` = 100). Die Angabe `fetchLimit`=0 bedeutet, dass alle Auftragsdetails des Auftrags angefordert werden. Mit dem Attribut `fetchOffset` kann das Kundensystem festlegen, ab welcher Offset-Position in der Originalauftragsdatei die Auftragsdetails zurückgeliefert werden. `fetchOffset` = 0 bedeutet, dass die Auftragsdetails vom Anfang der Auftragsdatei angefordert werden.
Wenn `fetchOffset` größer als die Gesamtanzahl der Auftragsdetails ist, wird der fachliche Returncode EBICS_INVALID_ORDER_PARAMS geliefert.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Für weitere Filterkriterien steht die generische Schlüssel-Wert-Struktur (`Parameter`) bereit.

Ausprägung der `OrderParams` (Auftragsparameter) für HVT: `HVTOrderParams`

8.3.3.1.1 XML-Schema (grafische Darstellung)

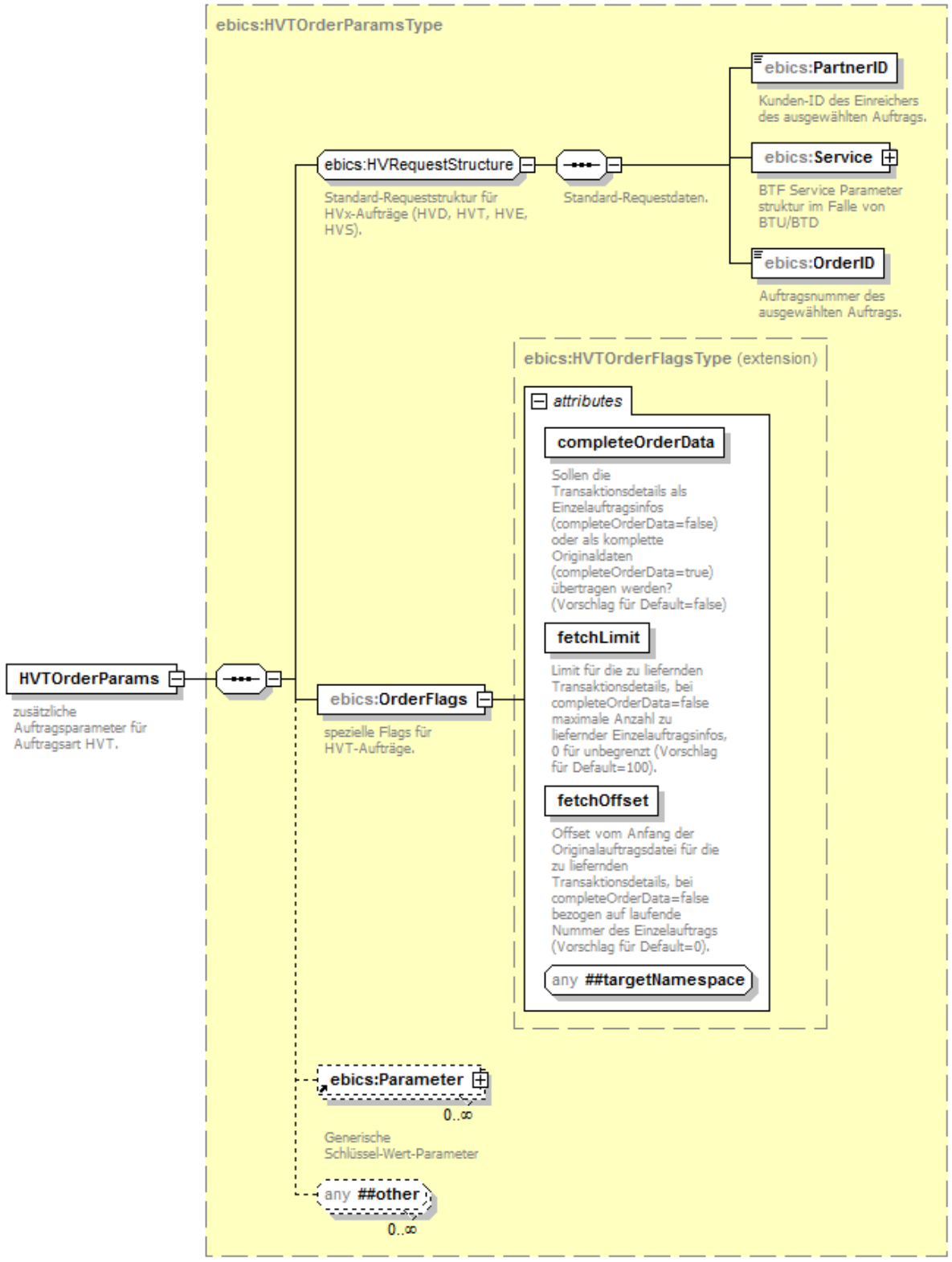


Abbildung 78: HVTOrderParams

8.3.3.1.2 XML-Schema (textuelle Darstellung)

```
<element name="HVTOrderParams" type="ebics:HVTOrderParamsType"
   substitutionGroup="ebics:OrderParams">
   <annotation>
      <documentation xml:lang="de">
         zusätzliche Auftragsparameter für Auftragsart HVT.
      </documentation>
   </annotation>
</element>
<complexType name="HVTOrderParamsType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für zusätzliche Auftragsparameter für Auftragsart HVT.
      </documentation>
   </annotation>
   <sequence>
      <group ref="ebics:HVRequestStructure" />
      <element name="OrderFlags" type="ebics:HVTOrderFlagsType">
         <annotation>
            <documentation xml:lang="de">
               spezielle Flags für HVT-Aufträge.
            </documentation>
         </annotation>
      </element>
      <element ref="ebics:Parameter" minOccurs="0"
         maxOccurs="unbounded" />
      <any namespace="##other" processContents="lax" minOccurs="0"
         maxOccurs="unbounded" />
   </sequence>
</complexType>
<group name="HVRequestStructure">
   <annotation>
      <documentation xml:lang="de">
         Standard-Requeststruktur für HVx-Aufträge (HVD, HVT, HVE, HVS).
      </documentation>
   </annotation>
   <sequence>
      <annotation>
         <documentation xml:lang="de">
            Standard-Requestdaten.
         </documentation>
      </annotation>
      <element name="PartnerID" type="ebics:PartnerIDType">
         <annotation>
            <documentation xml:lang="de">
               Kunden-ID des Einreichers des ausgewählten Auftrags.
            </documentation>
         </annotation>
      </element>
      <element name="OrderType" type="ebics:OrderTBaseType">
               </element>
      <element name="OrderID" type="ebics:OrderIDType">
         <annotation>
            <documentation xml:lang="de">
               Auftragsnummer des ausgewählten Auftrags</documentation>
         </annotation>
      </element>
   </sequence>
</group>
<complexType name="HVTOrderFlagsType">
   <annotation>
```

```
      <documentation xml:lang="de">
         Datentyp für HVT-Auftragsnummern.
      </documentation>
   </annotation>
   <simpleContent>
      <extension base="ebics:OrderIDType">
         <attribute name="completeOrderData" type="boolean"
            use="required">
            <annotation>
               <documentation xml:lang="de">
                  Sollen die Transaktionsdetails als Einzelauftragsinfos
                  (completeOrderData=false) oder als komplette
                  Originaldaten (completeOrderData=true) übertragen
                  werden?
               </documentation>
            </annotation>
         </attribute>
         <attribute name="fetchLimit" use="required">
            <annotation>
               <documentation xml:lang="de">
                  Limit für die zu liefernden Transaktionsdetails, bei
                  completeOrderData=false maximale Anzahl zu liefernder
                  Einzelauftragsinfos, 0 für unbegrenzt.
               </documentation>
            </annotation>
            <simpleType>
               <restriction base="nonNegativeInteger">
                  <totalDigits value="10" />
               </restriction>
            </simpleType>
         </attribute>
         <attribute name="fetchOffset" use="required">
            <annotation>
               <documentation xml:lang="de">
                  Offset vom Anfang der Originalauftragsdatei für die zu
                  liefernden Transaktionsdetails, bei
                  completeOrderData=false bezogen auf laufende Nummer
                  des Einzelauftrags.
               </documentation>
            </annotation>
            <simpleType>
               <restriction base="nonNegativeInteger">
                  <totalDigits value="10" />
               </restriction>
            </simpleType>
         </attribute>
         <anyAttribute namespace="##targetNamespace"
            processContents="strict" />
      </extension>
   </simpleContent>
</complexType>
<element name="Parameter">
   <annotation>
      <documentation xml:lang="de">
         generische Schlüssel-Wert-Parameter.
      </documentation>
   </annotation>
   <complexType>
      <sequence>
         <element name="Name" type="token">
            <annotation>
               <documentation xml:lang="de">
                  Name des Parameters.
```

```
            </documentation>
         </annotation>
      </element>
      <element name="Value">
         <annotation>
            <documentation xml:lang="de">
               Wert des Parameters.
            </documentation>
         </annotation>
         <complexType>
            <simpleContent>
               <extension base="anySimpleType">
                  <attribute name="Type" type="NCName" use="required">
                     <annotation>
                        <documentation xml:lang="de">
                           XML-Typ des Parameterwerts
                           (Vorschlag für Default=string).
                        </documentation>
                     </annotation>
                  </attribute>
               </extension>
            </simpleContent>
         </complexType>
      </element>
   </sequence>
</complexType>
</element>
```

8.3.3.1.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
`HVTOrderParams`	`ebics:HVTOrderParams»Type` (komplex)	1	Auftragsparameter für Auftragsart HVT	- (komplex)
`PartnerID`	`ebics:PartnerIDType (→token, maxLength=35, pattern="[a-zA-Z0-9,=]{1,35})`	1	Kunden-ID des Einreichers	„PARTNER1“
`Service`	`ebics:RestrictedServiceType` Details zu dieser Struktur siehe Kapitel 8.3.6	1	Art des Geschäftsvorfalls, identifiziert durch die Servicestruktur	
`OrderID`	`ebics:OrderIDType (→token, fixLength=4)`	1	Auftragsnummer des zur VEU eingereichten Auftrags	„OR01“
`OrderFlags`	`ebics:HVTOrderFlags»Type` (komplex)	1	spezifische „Schalter“ für HVT-Aufträge	- (komplex)
`OrderFlags»@complete»OrderData`	`boolean`	1	Sollen die Transaktions-details als einzelne Auftrags-detailinformationen (`@completeOrderData="false"`) oder als komplette Auftragsdatei (`@completeOrderData="true"`) übertragen werden? (Vorschlag für	„false“

			Default="false")	
OrderFlags» @fetchLimit	nonNegativeInteger	1	Maximale Anzahl zu liefernder Auftragsdetails bei @completeOrderData= "false", „0“ für unbegrenzt (Vorschlag für Default=“100“)	10
OrderFlags» @fetchOffset	nonNegativeInteger	1	Offset vom Anfang der Auftragsdatei für zu liefernde Auftragsdetails bezogen auf laufende Nummer der Auftragsdetails bei @completeOrderData= "false" (Vorschlag für Default=“0“)	20
Parameter	Referenz auf globales Element (komplex)	0..∞	Struktur für generische Schlüssel-Wert-Parameter mit optionaler Typangabe	- (komplex)

8.3.3.1.4 Beispiel-XML (gekürzt)

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest xmlns="urn:org:ebics:H005"
   xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="urn:org:ebics:H005 ebics_request_H005.xsd"
   Version="H005" Revision="1">
   <header authenticate="true">
      <static>
         <!-- […] -->
         <OrderDetails>
            <AdminOrderType>HVT</AdminOrderType>
HVTOrderParams>
               <PartnerID>PARTNER1</PartnerID>
              <Service>
               <ServiceName>SCT</ServiceName>
               <MsgName>pain.001</MsgName>
          </Service>              <OrderID>OR01</OrderID>
               <OrderFlags completeOrderData="false" fetchLimit="50"
                  fetchOffset="0" />
            </HVTOrderParams>
         </OrderDetails>
         <!-- […] -->
      </static>
      <!-- […] -->
   </header>
   <!-- […] -->
</ebicsRequest>
```

8.3.3.2 HVT-Response

Die HVT-Response enthält je nach Wahl des Attributes `completeOrderData` am Element `OrderFlags` zu dem im HVT-Request spezifizierten Auftrag zwei unterschiedliche Formate.

Mit dem Flag `completeOrderData=true` fordert das Kundensystem die Abholung der Auftragsdaten im Originalformat an. Dabei handelt es sich um einen Standard-Download ohne zusätzliche Einbettung der Auftragsdaten in ein XML-Dokument, d.h. die Auftragsdaten werden komprimiert, verschlüsselt und wenn erforderlich segmentiert zum Kundensystem übertragen.

Mit dem `Flag completeOrderData=false` fordert das Kundensystem die Abholung der Auftragsdetails im aufbereiteten XML-Format an. Dabei handelt es sich um ein XML-Dokument mit dem Wurzelelement `HVTResponseOrderData`, das komprimiert, verschlüsselt und wenn erforderlich segmentiert zum Kundensystem übertragen wird. In diesem Fall enthält die Response im Element `NumOrderInfos` die Gesamtanzahl der Auftragsdetails in der Originalauftragsdatei.

Ausprägung der (dekodierten & entschlüsselten & dekomprimierten) `OrderData` (Auftragsdaten) für HVT: `HVTResponseOrderData`

Wenn ein Teilnehmer HVT durchführt, obwohl die Bank HVT für diesen konkreten Auftrag nicht unterstützt, muss die Transaktion mit einem Returncode abgebrochen werden. Dazu ist EBICS_UNSUPPORTED_REQUEST_FOR_ORDER_INSTANCE anzuwenden.

8.3.3.2.1 XML-Schema (grafische Darstellung)

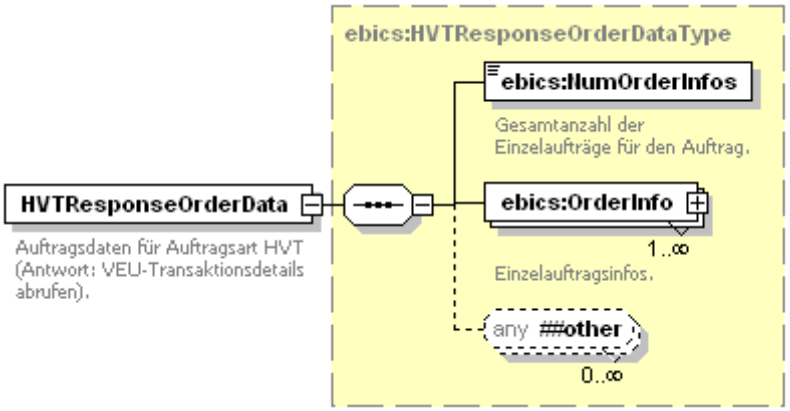


Abbildung 79: HVTResponseOrderData

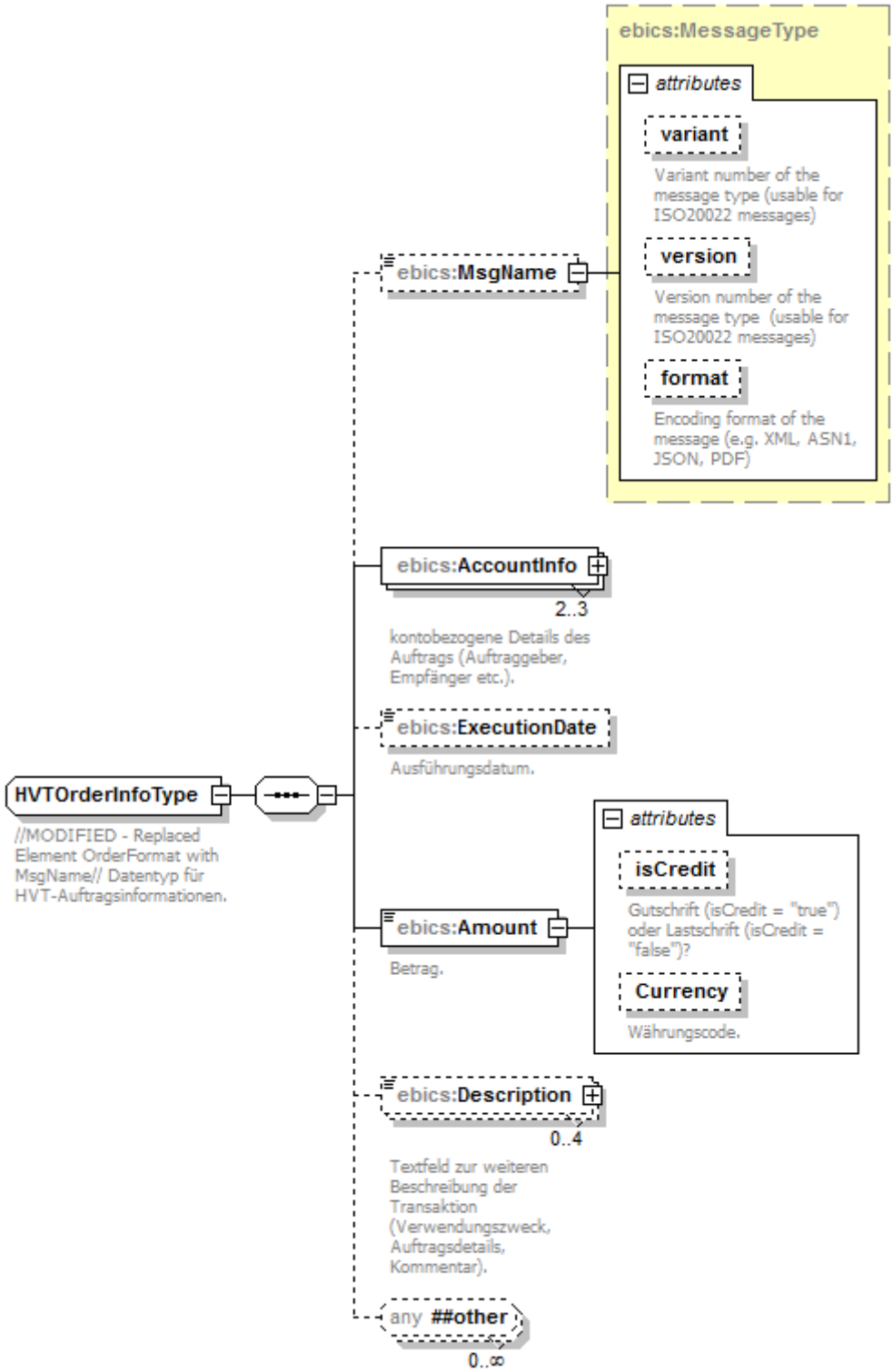


Abbildung 80: HVTOrderInfoType (zu OrderInfo)

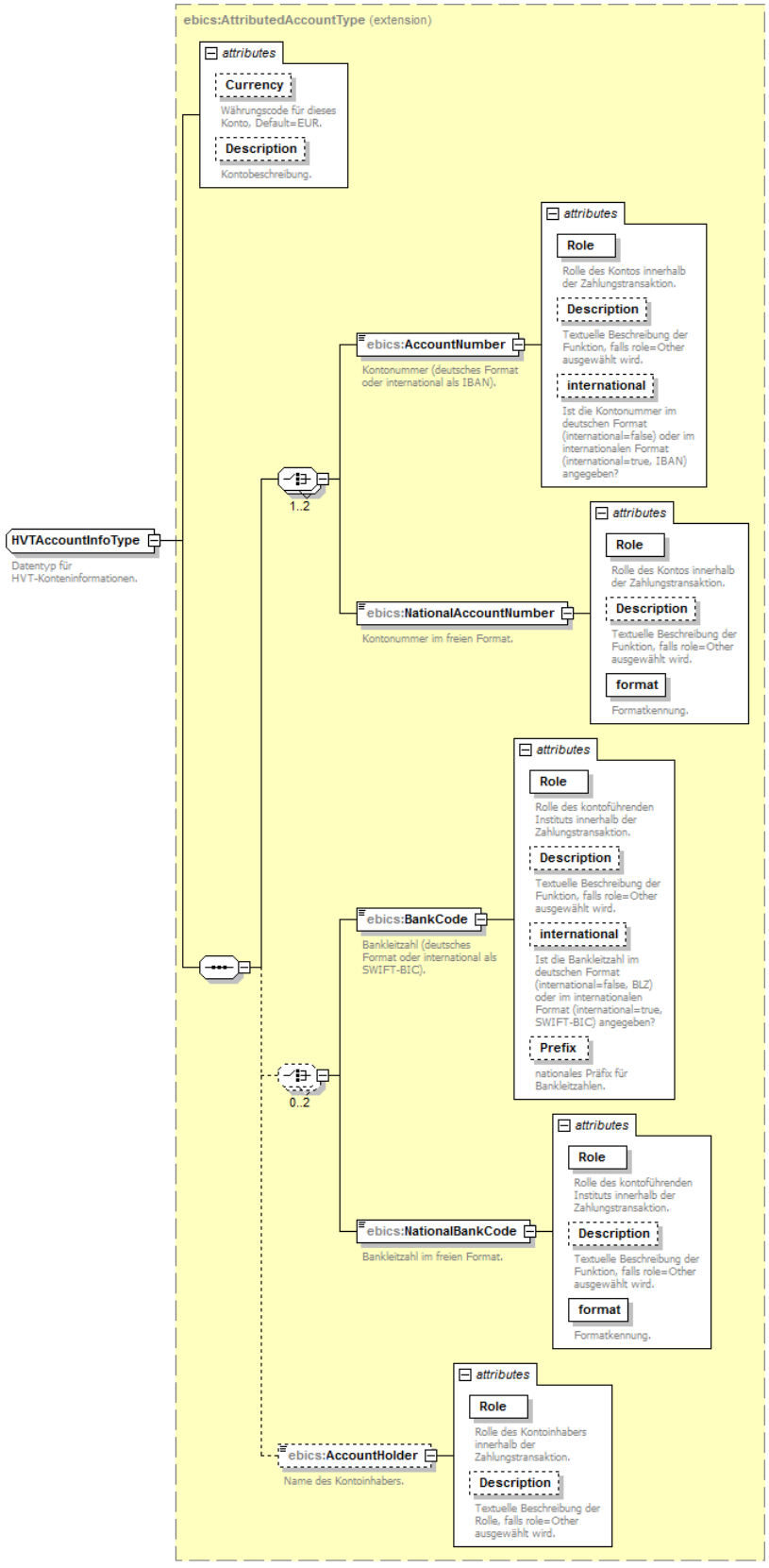


Abbildung 81: HVTAccountInfoType (zu AccountInfo)

8.3.3.2.2 XML-Schema (textuelle Darstellung)

```
<element name="HVTResponseOrderData"
   type="ebics:HVTResponseOrderDataType"
   substitutionGroup="ebics:EBICSOrderData">
   <annotation>
      <documentation xml:lang="de">
         Auftragsdaten für Auftragsart HVT (Antwort:
         VEU-Transaktionsdetails abrufen).
      </documentation>
   </annotation>
</element>
<complexType name="HVTResponseOrderDataType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für Antwort mit Einzelauftraginfos für Auftragsart HVT
         (Antwort VEU-Transaktionsdetails abrufen mit
         completeOrderData="false").
      </documentation>
   </annotation>
   <sequence>
      <element name="NumOrderInfos" type="ebics:NumOrderInfosType">
         <annotation>
            <documentation xml:lang="de">
               Gesamtanzahl der Einzelaufträge für den Auftrag.
            </documentation>
         </annotation>
      </element>
      <element name="OrderInfo" type="ebics:HVTOrderInfoType"
         maxOccurs="unbounded">
         <annotation>
            <documentation xml:lang="de">
               Einzelauftragsinfos.
            </documentation>
         </annotation>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0"
         maxOccurs="unbounded" />
   </sequence>
</complexType>
<complexType name="HVTOrderInfoType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für HVT-Auftragsinformationen.
      </documentation>
   </annotation>
   <sequence>
      <element name="OrderFormat" type="ebics:OrderFormatType"
         minOccurs="0">
         <annotation>
            <documentation xml:lang="de">
               Auftragsformat (z.B. DTAZV).
            </documentation>
         </annotation>
      </element>
      <element name="AccountInfo" type="ebics:HVTAccountInfoType"
         minOccurs="2" maxOccurs="3">
         <annotation>
            <documentation xml:lang="de">
               kontobezogene Details des Auftrags (Auftraggeber,
               Empfänger etc.).
            </documentation>
```

```
      </annotation>
   </element>
   <element name="ExecutionDate" minOccurs="0">
      <annotation>
         <documentation xml:lang="de">
            Ausführungsdatum.
         </documentation>
      </annotation>
      <complexType>
         <simpleContent>
            <extension base="date" />
         </simpleContent>
      </complexType>
   </element>
   <element name="Amount">
      <annotation>
         <documentation xml:lang="de">Betrag.</documentation>
      </annotation>
      <complexType>
         <simpleContent>
            <extension base="ebics:AmountValueType">
               <attribute name="isCredit" type="boolean"
                  use="optional">
                  <annotation>
                     <documentation xml:lang="de">
                        Gutschrift (isCredit = "true") oder
                        Lastschrift (isCredit = "false")?
                     </documentation>
                  </annotation>
               </attribute>
               <attribute name="Currency"
                  type="ebics:CurrencyBaseType" use="optional">
                  <annotation>
                     <documentation xml:lang="de">
                        Währungscode.
                     </documentation>
                  </annotation>
               </attribute>
            </extension>
         </simpleContent>
      </complexType>
   </element>
   <element name="Description" minOccurs="0" maxOccurs="4">
      <annotation>
         <documentation xml:lang="de">
            Textfeld zur weiteren Beschreibung der Transaktion
            (Verwendungszweck, Auftragsdetails, Kommentar).
         </documentation>
      </annotation>
      <complexType>
         <simpleContent>
            <extension base="string">
               <attribute name="Type" use="required">
                  <annotation>
                     <documentation xml:lang="de">
                        Beschreibungstyp.
                     </documentation>
                  </annotation>
                  <simpleType>
                     <restriction base="token">
                        <enumeration value="Purpose">
                           <annotation>
                              <documentation xml:lang="de">
```

```
                                    Verwendungszweck
                                </documentation>
                            </annotation>
                        </enumeration>
                        <enumeration value="Details">
                            <annotation>
                                <documentation xml:lang="de">
                                    Auftragsdetails
                                </documentation>
                            </annotation>
                        </enumeration>
                        <enumeration value="Comment">
                            <annotation>
                                <documentation xml:lang="de">
                                    Kommentar
                                </documentation>
                            </annotation>
                        </enumeration>
                    </restriction>
                </simpleType>
            </attribute>
        </extension>
    </simpleContent>
</complexType>
</element>
<any namespace="##other" processContents="lax" minOccurs="0"
    maxOccurs="unbounded" />
</sequence>
</complexType>
<complexType name="HVTAccountInfoType">
<annotation>
<documentation xml:lang="de">
    Datentyp für HVT-Konteninformationen.
</documentation>
</annotation>
<complexContent>
<extension base="ebics:AttributedAccountType" />
</complexContent>
</complexType>
<complexType name="AttributedAccountType">
<annotation>
<documentation xml:lang="de">
    Datentyp für Kontoinformationen inkl. der Eigenschaftszuordnung
    innerhalb einer Zahlungstransaktion.
</documentation>
</annotation>
<sequence>
<choice maxOccurs="2">
<element name="AccountNumber">
<annotation>
<documentation xml:lang="de">
    Kontonummer (deutsches Format oder international als
    IBAN).
</documentation>
</annotation>
<complexType>
<simpleContent>
<extension base="ebics:AccountNumberType">
<attribute name="Role"
    type="ebics:AccountNumberRoleType"
    use="required">
<annotation>
<documentation xml:lang="de">
```

```
                        Rolle des Kontos innerhalb der
                        Zahlungstransaktion.
                     </documentation>
                  </annotation>
               </attribute>
               <attribute name="Description"
                  type="normalizedString">
                  <annotation>
                     <documentation xml:lang="de">
                        Textuelle Beschreibung der Funktion, falls
                        role=Other ausgewählt wird.
                     </documentation>
                  </annotation>
               </attribute>
               <attribute name="international" type="boolean"
                  use="optional" default="false">
                  <annotation>
                     <documentation xml:lang="de">
                        Ist die Kontonummer im deutschen Format
                        (international=false) oder im
                        internationalen Format
                        (international=true, IBAN) angegeben?
                     </documentation>
                  </annotation>
               </attribute>
            </extension>
         </simpleContent>
      </complexType>
   </element>
   <element name="NationalAccountNumber">
      <annotation>
         <documentation xml:lang="de">
            Kontonummer im freien Format.
         </documentation>
      </annotation>
      <complexType>
         <simpleContent>
            <extension base="ebics:NationalAccountNumberType">
               <attribute name="Role"
                  type="ebics:AccountNumberRoleType"
                  use="required">
                  <annotation>
                     <documentation xml:lang="de">
                        Rolle des Kontos innerhalb der
                        Zahlungstransaktion.
                     </documentation>
                  </annotation>
               </attribute>
               <attribute name="Description"
                  type="normalizedString">
                  <annotation>
                     <documentation xml:lang="de">
                        Textuelle Beschreibung der Funktion, falls
                        role=Other ausgewählt wird.
                     </documentation>
                  </annotation>
               </attribute>
               <attribute name="format" type="token"
                  use="required">
                  <annotation>
                     <documentation xml:lang="de">
                        Formatkennung.
                     </documentation>
```

```
                    </annotation>
                </attribute>
            </extension>
        </simpleContent>
    </complexType>
</element>
</choice>
<choice maxOccurs="2">
    <element name="BankCode">
        <annotation>
            <documentation xml:lang="de">
                Bankleitzahl (deutsches Format oder international als
                SWIFT-BIC).
            </documentation>
        </annotation>
        <complexType>
            <simpleContent>
                <extension base="ebics:BankCodeType">
                    <attribute name="Role"
                        type="ebics:BankCodeRoleType" use="required">
                        <annotation>
                            <documentation xml:lang="de">
                                Rolle des kontoführenden Instituts
                                innerhalb der Zahlungstransaktion.
                            </documentation>
                        </annotation>
                    </attribute>
                    <attribute name="Description"
                        type="normalizedString">
                        <annotation>
                            <documentation xml:lang="de">
                                Textuelle Beschreibung der Funktion, falls
                                role=Other ausgewählt wird.
                            </documentation>
                        </annotation>
                    </attribute>
                    <attribute name="international" type="boolean"
                        use="optional" default="false">
                        <annotation>
                            <documentation xml:lang="de">
                                Ist die Bankleitzahl im deutschen Format
                                (international=false, BLZ) oder im
                                internationalen Format
                                (international=true, SWIFT-BIC) angegeben?
                            </documentation>
                        </annotation>
                    </attribute>
                    <attribute name="Prefix"
                        type="ebics:BankCodePrefixType"
                        use="optional">
                        <annotation>
                            <documentation xml:lang="de">
                                nationales Präfix für Bankleitzahlen.
                            </documentation>
                        </annotation>
                    </attribute>
                </extension>
            </simpleContent>
        </complexType>
    </element>
    <element name="NationalBankCode">
        <annotation>
            <documentation xml:lang="de">
```

```
                Bankleitzahl im freien Format.
            </documentation>
         </annotation>
         <complexType>
            <simpleContent>
               <extension base="ebics:NationalBankCodeType">
                  <attribute name="Role"
                     type="ebics:BankCodeRoleType" use="required">
                     <annotation>
                        <documentation xml:lang="de">
                           Rolle des kontoführenden Instituts
                           innerhalb der Zahlungstransaktion.
                        </documentation>
                     </annotation>
                  </attribute>
                  <attribute name="Description"
                     type="normalizedString">
                     <annotation>
                        <documentation xml:lang="de">
                           Textuelle Beschreibung der Funktion, falls
                           role=Other ausgewählt wird.
                        </documentation>
                     </annotation>
                  </attribute>
                  <attribute name="format" type="token"
                     use="required">
                     <annotation>
                        <documentation xml:lang="de">
                           Formatkennung.
                        </documentation>
                     </annotation>
                  </attribute>
               </extension>
            </simpleContent>
         </complexType>
      </element>
   </choice>
   <element name="AccountHolder" minOccurs="0">
      <annotation>
         <documentation xml:lang="de">
            Name des Kontoinhabers.
         </documentation>
      </annotation>
      <complexType>
         <simpleContent>
            <extension base="ebics:AccountHolderType">
               <attribute name="role"
                  type="ebics:AccountHolderRoleType"
                  use="required">
                  <annotation>
                     <documentation xml:lang="de">
                        Rolle des Kontoinhabers innerhalb der
                        Zahlungstransaktion.
                     </documentation>
                  </annotation>
               </attribute>
               <attribute name="description"
                  type="normalizedString">
                  <annotation>
                     <documentation xml:lang="de">
                        Textuelle Beschreibung der Rolle, falls
                        role=Other ausgewählt wird.
                     </documentation>
```

```
                    </annotation>
                 </attribute>
              </extension>
           </simpleContent>
        </complexType>
     </element>
  </sequence>
  <attribute name="Currency" type="ebics:CurrencyBaseType"
     use="optional" default="EUR">
     <annotation>
        <documentation xml:lang="de">
           Währungscode für dieses Konto, Default=EUR.
        </documentation>
     </annotation>
  </attribute>
  <attribute name="Description" type="ebics:AccountDescriptionType"
     use="optional">
     <annotation>
        <documentation xml:lang="de">Kontobeschreibung.</documentation>
     </annotation>
  </attribute>
</complexType>
<complexType name="AmountType">
  <annotation>
     <documentation xml:lang="de">
        Datentyp für einen Betrag inkl. Währungscode-Attribut (Default
        = "EUR").
     </documentation>
  </annotation>
  <simpleContent>
     <extension base="ebics:AmountValueType">
        <attribute name="Currency" type="ebics:CurrencyBaseType"
           use="optional" default="EUR">
           <annotation>
              <documentation xml:lang="de">
                 Währungscode, Default="EUR".
              </documentation>
           </annotation>
        </attribute>
     </extension>
  </simpleContent>
</complexType>
```

8.3.3.2.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
NumOrderInfos	ebics:NumOrderInfosType	1	Gesamtanzahl der Einzelaufträge in der Originaldatei	42
MsgName	ebics: MessageNameStringType (simple) restriction base: minLength value="1" maxLength value="10"	0..1	Nachrichtenformat, beginnend mit dem BA-Code (ISO) oder MT (FIN) oder Zeichenkette In der HVT-Response ist MsgName eine optionale Information	"pain.001", "mt103" Von Märkten spezifizierte Datei-

	pattern = [a-z\.0-9]			formate (Markt erkennbar über “scope”) sind auch zulässig
MsgName@version	ebics:NumString (simple) restriction base: minLength value="2" maxLength value="2" pattern = [0-9]	0..1	Verwendete ISO-Version der Nachricht (wenn es sich um kein ISO-Format handelt, wird dieses Attribut ignoriert)	“03”
MsgName@variant	ebics:NumString (simple) restriction base: minLength value="3" maxLength value="3" pattern = [0-9]	0..1	Wird zusammen mit <MsgName> betrachtet, (wenn es sich um kein ISO-Format handelt, wird dieses Attribut ignoriert)	“001”
MsgName@format	ebics:CodeString (simple) restriction base: minLength value="1" maxLength value="4" pattern = [A-Z0-9]	0..1	Wird zusammen mit <MsgName> betrachtet, ist für jede Art von Format zulässig, sollte aber nur verwendet werden, wenn diese Angabe eine Abweichung zum Standard darstellt (insbesondere, wenn kein XML für ISO 20022 verwendet wird).	„XML“, „ASN1“, „JSON“, „PDF“
AccountInfo	ebics:HVTAccount» InfoType (komplex)	2..3	kontobezogene Detailinformationen des Einzelauftrags (Auftraggeber, Empfänger, opt. Einreicher)	- (komplex)
AccountInfo» @Currency	ebics:CurrencyBase» Type (→token, length=3, pattern="[A-Z]{3}")	0..1	Währungscode gemäß ISO 4217 des Kontos; Default= „EUR”	„EUR“
AccountInfo» @Description	ebics:Account» DescriptionType (→normalizedString)	0..1	textuelle Beschreibung des Kontos	„Savings“
ExecutionDate	date	0..1	Ausführungsdatum gemäß ISO 8601 des Einzelauftrags	2005-01-31
Amount	ebics:AmountValue» Type (→decimal, totalDigits=24, fractionDigits=4)	1	Betrag des Einzelauftrags	1234.567
Amount» @Currency	ebics:CurrencyBase» Type (→token, length=3, pattern="[A-Z]{3}")	0..1	Währungscode gemäß ISO 4217 des Einzelauftragsbetrags	„EUR“
Amount» @isCredit	boolean	0..1	Flag zur Unterscheidung	„false“

			zwischen Gutschrift (`isCredit="true"`) und Lastschrift (`isCredit="false"`)	
`Description`	`string`	0..4	Textfelder zur weiteren Beschreibung der Auftragstransaktion (Verwendungszweck, Auftragsdetails, Kommentar)	„Rechnung Nr. 2345“
`Description» @Type`	`token: "Purpose", "Details", "Comment")`	1	Typ der Beschreibung: „Purpose“=Verwendungszweck, „Details“=Auftragsdetails, „Comment“=Kommentar	„Purpose“
-	-	1..2	Angaben zur Kontonummer: AccountNumber und/oder NationalAccountNumber	-
`AccountNumber`	`ebics:AccountNumber» Type (→token, maxLength=40, pattern="\d{3,10}|([A-Z]{2}\d{2}[A-Za-z0-9]{3,30}")`	1	Kontonummer, wahlweise im nationalen (=deutschen) oder internationalen Format (IBAN)	„12345678“
`AccountNumber» @Role`	`ebics:AccountNumber» RoleType (→token: "Originator", "Recipient", "Charges", "Other")`	1	Rolle des Kontos innerhalb der Zahlungstransaktion: “Originator”=Auftraggeberkonto, “Recipient”=Empfängerkonto, “Charges”=Entgeltkonto, “Other”= andere Rolle (siehe `AccountNumber» @Description`)	„Originator“
`AccountNumber» @Description`	`normalizedString`	0..1	textuelle Beschreibung der Rolle des Kontos innerhalb der Zahlungstransaktion, falls `AccountNumber@Role= "Other"` ausgewählt wird	„Nostro“
`AccountNumber» @international`	`boolean`	0..1	Ist die Kontonummer im nationalen=deutschen (`AccountNumber» @international="false"`) oder im internationalen= IBAN-Format (`AccountNumber» @international="true"`) angegeben? Default=`"false"`	„false“
`NationalAccount Number`		1	Kontonummer im freien Format (für nationale Kontonummer, die weder den deutschen noch den internationalen Vorgaben entsprechen)	„12345678 90123456“
`NationalAccount Number»`	`ebics:AccountNumber» RoleType`	1	Rolle des Kontos innerhalb der	„Originator“

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

@Role	(→token: "Originator", "Recipient", "Charges", "Other")		Zahlungstransaktion: “Originator”=Auftraggeberkonto, “Recipient”=Empfängerkonto, “Charges”=Entgeltkonto, “Other”= andere Rolle (siehe AccountNumber» @Description)	
NationalAccount Number» @Description	normalizedString	0..1	textuelle Beschreibung der Rolle des Kontos innerhalb der Zahlungstransaktion, falls AccountNumber@Role= "Other" ausgewählt wird	„Nostro“
National» AccountNumber» @format	token	1	Beschreibung für Format der Kontonummer	„other“
-	-	1..2	Angaben zur Bankleitzahl: BankCode und/oder NationalBankCode	-
BankCode	ebics:BankCodeType (→token, maxLength=11, pattern="\d{8}|([A-Z]{6}[A-Z0-9]{2}([A-Z0-9]{3})?)")	0..1	Bankleitzahl, wahlweise im nationalen (=deutschen) oder internationalen Format (SWIFT-BIC)	„50010060“
BankCode@Role	ebics:BankCodeRole» Type (→token: "Originator", "Recipient", "Correspondent", "Other")	1	Rolle des Kreditinstituts innerhalb der Zahlungstransaktion: „Originator”=Auftraggeberbank, „Recipient”=Empfängerbank, „Correspondent”=Korrespondenzbank, „Other“=andere Rolle (siehe BankCode@Description)	„Originator“
BankCode» @Description	normalizedString	0..1	textuelle Beschreibung der Rolle des Kreditinstituts innerhalb der Zahlungstransaktion, falls BankCode@Role="Other" ausgewählt wird	„Clearing“
BankCode» @international	boolean	0..1	Ist die Bankleitzahl im nationalen=deutschen (BankCode» @international= "false") oder internationalen=SWIFT-BIC-Format (BankCode» @international="true") angegeben? Default="false"	„false“
BankCode@Prefix	token, maxLength=2	0..1	nationales Präfix für die Bankleitzahl	„DE“
NationalBank» Code	ebics:National» BankCodeType	1	Bankleitzahl im freien Format. (weder deutsches Format noch	„12345678 9012“

	(→token, maxLength=30)		SWIFT-BIC)	
NationalBank» Code@Role	ebics:BankCodeRole» Type (→token: "Originator", "Recipient", "Correspondent", "Other")	1	Rolle des Kreditinstituts innerhalb der Zahlungstransaktion: „Originator"=Auftraggeberbank, „Recipient"=Empfängerbank, „Correspondent"=Korrespondenzbank, „Other"=andere Rolle (siehe BankCode@Description)	„Originator"
BankCode» @Description	normalizedString	0..1	textuelle Beschreibung der Rolle des Kreditinstituts innerhalb der Zahlungstransaktion, falls BankCode@Role="Other" ausgewählt wird	„Clearing"
NationalBank» Code@format	token	1	Formatkennung	"other"
AccountHolder	ebics:AccountHolder» Type (→normalizedString)	0..1	Name des Kontoinhabers	„Hans Mustermann"
AccountHolder» @Role	ebics:AccountHolder» RoleType (→token: "Originator", "Recipient", "Presenter", "Other")	0..1	Rolle des Kontoinhabers innerhalb der Zahlungstransaktion: „Originator"=Auftraggeber, „Recipient"=Empfänger, „Presenter"=Einreicher, „Other"=andere Rolle (siehe AccountHolder» @Description)	„Originator"
AccountHolder» @Description	normalizedString	0..1	textuelle Beschreibung der Rolle des Kontoinhabers innerhalb der Zahlungstransaktion, falls AccountHolder@Role= "Other" ausgewählt wird	„Trustee"

8.3.3.2.4 Beispiel-XML

```
<?xml version="1.0" encoding="UTF-8"?>
<HVTResponseOrderData
 xmlns="urn:org:ebics:H005"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_orders_H005.xsd">
 <NumOrderInfos>42</NumOrderInfos>
 <OrderInfo>
   <AccountInfo Currency="EUR">
     <AccountNumber Role="Originator" international="false">1234567890</AccountNumber>
     <BankCode Role="Originator" international="false" Prefix="DE">50010060</BankCode>
     <AccountHolder Role="Originator">Arnold Auftraggeber</AccountHolder>
   </AccountInfo>
   <AccountInfo Currency="EUR">
     <AccountNumber Role="Recipient" international="false">1122334455</AccountNumber>
     <BankCode Role="Recipient" international="false">50070010</BankCode>
     <AccountHolder Role="Recipient">Erich Empfänger</AccountHolder>
```

```
    </AccountInfo>
    <ExecutionDate>2005-01-31</ExecutionDate>
    <Amount isCredit="true" Currency="EUR">500.00</Amount>
    <Description Type="Purpose">Testüberweisung</Description>
  </OrderInfo>
</HVTResponseOrderData>
```

8.3.4 HVE (Elektronische Unterschrift hinzufügen)

Mit HVE fügt der Teilnehmer einem Auftrag aus der VEU-Prozessverarbeitung eine weitere bankfachliche Unterschrift zur Autorisation hinzu.

Das Banksystem muss prüfen, ob der Teilnehmer eine bankfachliche Unterschriftsberechtigung (nicht Unterschriftsklasse T) für den referenzierten Auftrag besitzt. Falls die Berechtigung fehlt, wird die Transaktion mit dem existierenden Returncode EBICS_AUTHORISATION_ORDER_IDENTIFIER_FAILED abgebrochen.

Das Banksystem prüft vor der Ausführung von HVE, dass sich der Auftrag aktuell in der VEU-Prozessverarbeitung befindet und beendet im Fehlerfall die Transaktion mit dem fachlichen Fehlercode EBICS_ORDERID_UNKNOWN.

HVE ist eine Auftragsart vom Typ „Upload“. Es wird nur die EU über den Hashwert des Auftrags aus der VEU-Prozessverarbeitung übermittelt (keine Auftragsdaten, keine EU für die administrative Auftragsart HVE selbst), wobei lediglich der Hashwert aus der VEU-Prozessverarbeitung signiert wird.

8.3.4.1 HVE-Request

Mit dem HVE-Request spezifiziert der Teilnehmer den Auftrag, zu dem er eine bankfachliche Unterschrift hinzufügen will, und liefert diese Unterschrift im gleichen Request im XML-Rumpfelement `ebicsRequest/body/DataTransfer/SignatureData` in komprimierter, verschlüsselter und base64-kodierter Form mit. Ein HVE-Request enthält keine Auftragsdaten, d.h. das XML-Rumpfelement `ebicsRequest/body/DataTransfer/OrderData` bleibt unbefüllt.
Zur Leistung der bankfachlichen Unterschrift benötigt der Teilnehmer entweder den Hashwert der ursprünglichen Auftragsdaten (z.B. mittels HVD oder HVZ abrufbar) oder die Auftragsdaten selbst (z.B. mittels HVT mit `completeOrderData="true"`).

Ausprägung der `OrderParams` (Auftragsparameter) für HVE: `HVEOrderParams`

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

8.3.4.1.1 XML-Schema (grafische Darstellung)

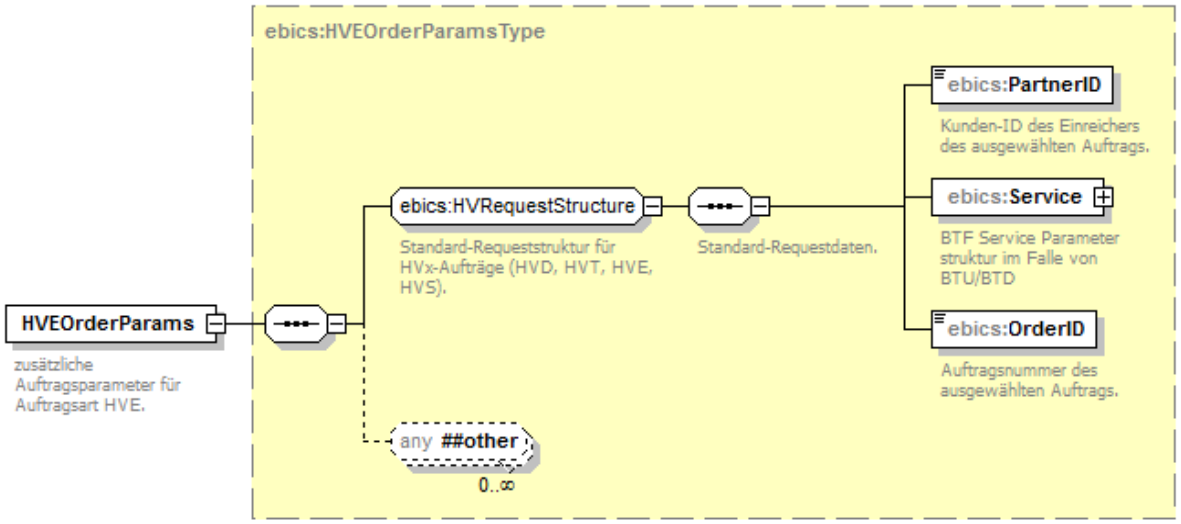


Abbildung 82: HVEOrderParams

8.3.4.1.2 XML-Schema (textuelle Darstellung)

```
 <element name="HVEOrderParams" type="ebics:HVEOrderParamsType" substitutionGroup="ebics:OrderParams">
   <annotation>
     <documentation xml:lang="de">zusätzliche Auftragsparameter für Auftragsart HVE.</documentation>
   </annotation>
 </element>
 <complexType name="HVEOrderParamsType">
   <annotation>
     <documentation xml:lang="de">Datentyp für zusätzliche Auftragsparameter für Auftragsart
HVE.</documentation>
   </annotation>
   <sequence>
     <group ref="ebics:HVRequestStructure"/>
     <any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
   </sequence>
 </complexType>
 <group name="HVRequestStructure">
   <annotation>
     <documentation xml:lang="de">Standard-Requeststruktur für HVx-Aufträge (HVD, HVT, HVE,
HVS).</documentation>
   </annotation>
   <sequence>
     <annotation>
       <documentation xml:lang="de">Standard-Requestdaten.</documentation>
     </annotation>
     <element name="PartnerID" type="ebics:PartnerIDType">
       <annotation>
         <documentation xml:lang="de">Kunden-ID des Einreichers des ausgewählten Auftrags.</documentation>
       </annotation>
     </element>
     <element name="OrderType" type="ebics:OrderTBaseType">
       <annotation>
```

```
        <documentation xml:lang="de">Auftragsart des ausgewählten Auftrags gemäß Anhang (Kapitel
13).</documentation>
      </annotation>
    </element>
    <element name="OrderID" type="ebics:OrderIDType">
      <annotation>
        <documentation xml:lang="de">Auftragsnummer des ausgewählten Auftrags</documentation>
      </annotation>
    </element>
  </sequence>
 </group>
```

8.3.4.1.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
HVEOrderParams	ebics:HVEOrderParamsType (komplex)	1	Auftragsparameter für Auftragsart HVE	- (komplex)
PartnerID	ebics:PartnerIDType (→token, maxLength=35, pattern="[a-zA-Z0-9,=]{1,35})	1	Kunden-ID des Einreichers	„PARTNER1“
Service	ebics:RestrictedServiceType Details zu dieser Struktur siehe Kapitel 8.3.6	1	Art des Geschäftsvorfalls, identifiziert durch die Servicestruktur	
OrderID	ebics:OrderIDType (→token, fixLength=4)	1	Auftragsnummer des Auftrags in der VEU-Prozessverarbeitung	„OR01“

8.3.4.1.4 Beispiel-XML (gekürzt)

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest
 xmlns="urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_request_H005.xsd"
 Version="H005" Revision="1">
  <header authenticate="true">
    <static>
      <!-- […] -->
      <OrderDetails>
        <AdminOrderType>HVE</AdminOrderType>
        <OrderID>H005</OrderID>
        <HVEOrderParams>
          <PartnerID>PARTNER1</PartnerID>
              <Service>
               <ServiceName>SCT</ServiceName>
               <MsgName>pain.001</MsgName>
          </Service>
          <OrderID>OR01</OrderID>
        </HVEOrderParams>
      </OrderDetails>
      <!-- […] -->
    </static>
```

```
    <!-- […] -->
  </header>
  <!-- […] -->
</ebicsRequest>
```

8.3.4.2 HVE-Response

Die HVE-Response enthält keine VEU-spezifischen Daten.

8.3.5 HVS (VEU-Storno)

Mit HVS storniert der Teilnehmer einen bestehenden Auftrag aus der VEU-Prozessverarbeitung unwiderruflich.

Das Banksystem prüft vor der Ausführung von HVS, dass sich der Auftrag aktuell in der VEU-Prozessverarbeitung befindet und beendet im Fehlerfall die Transaktion mit dem fachlichen Fehlercode EBICS_ORDERID_UNKNOWN.

HVS ist eine administrative Auftragsart vom Typ „Upload“. Als Stornoautorisation wird die EU über den Hashwert des zu stornierenden Auftrags übermittelt (keine Auftragsdaten, keine EU für HVS selbst).

8.3.5.1 HVS-Request

Mit dem HVS-Request spezifiziert der Teilnehmer den zu stornierenden Auftrag und liefert die zum Storno notwendige bankfachliche Signatur über den Hashwert der Auftragsdaten mit.

Das Banksystem muss prüfen, ob der Teilnehmer eine bankfachliche Unterschriftsberechtigung (nicht Unterschriftsklasse T) für den referenzierten Auftrag besitzt. Falls die Berechtigung fehlt, wird die Transaktion mit dem existierenden Returncode EBICS_AUTHORISATION_ORDER_IDENTIFIER_FAILED abgebrochen.

Die Signatur wird im XML-Rumpfelement `ebicsRequest/body/DataTransfer/SignatureData` in komprimierter, verschlüsselter und base64-kodierter Form transportiert. Die Stornierung ist endgültig und bedarf immer nur einer berechtigten bankfachlichen Unterschrift der Unterschriftsklassen „E“, „A“ oder „B“.

Zur Leistung der bankfachlichen Unterschrift benötigt der Teilnehmer entweder den Hashwert der ursprünglichen Auftragsdaten (z.B. mittels HVD oder HVZ abrufbar) oder die Auftragsdaten selbst (z.B. mittels HVT mit `completeOrderData="true"`).

Ausprägung der `OrderParams` für HVS: `HVSOrderParams`

8.3.5.1.1 XML-Schema (grafische Darstellung)

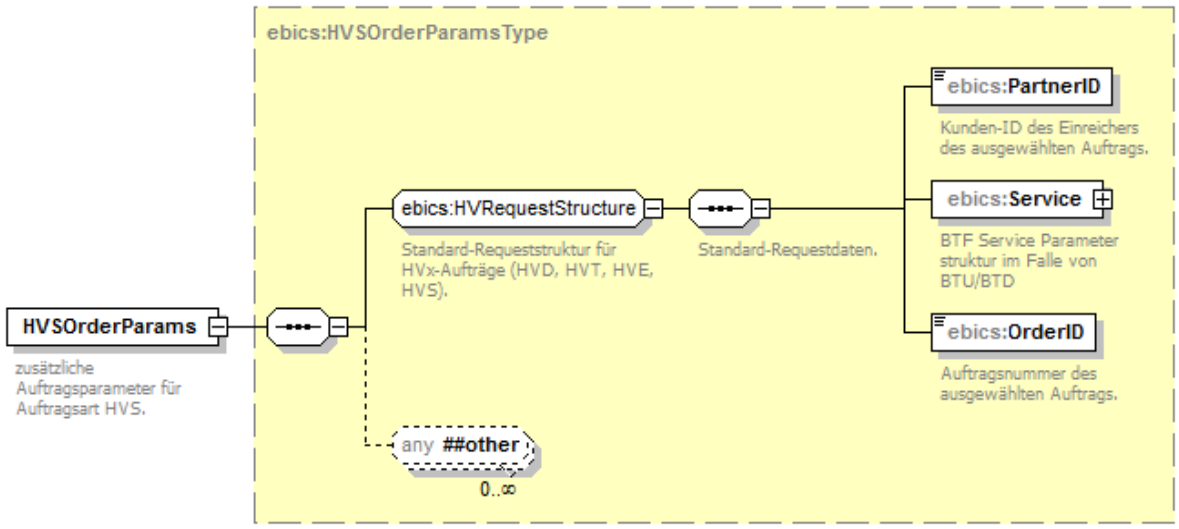


Abbildung 83: HVSOrderParams

8.3.5.1.2 XML-Schema (textuelle Darstellung)

```
 <element name="HVSOrderParams" type="ebics:HVSOrderParamsType" substitutionGroup="ebics:OrderParams">
   <annotation>
     <documentation xml:lang="de">zusätzliche Auftragsparameter für Auftragsart HVS.</documentation>
   </annotation>
 </element>
 <complexType name="HVSOrderParamsType">
   <annotation>
     <documentation xml:lang="de">Datentyp für zusätzliche Auftragsparameter für Auftragsart
HVS.</documentation>
   </annotation>
   <sequence>
     <group ref="ebics:HVRequestStructure"/>
     <any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
   </sequence>
 </complexType>
 <group name="HVRequestStructure">
   <annotation>
     <documentation xml:lang="de">Standard-Requeststruktur für HVx-Aufträge (HVD, HVT, HVE,
HVS).</documentation>
   </annotation>
   <sequence>
     <annotation>
       <documentation xml:lang="de">Standard-Requestdaten.</documentation>
     </annotation>
     <element name="PartnerID" type="ebics:PartnerIDType">
       <annotation>
         <documentation xml:lang="de">Kunden-ID des Einreichers des ausgewählten Auftrags.</documentation>
       </annotation>
     </element>
     <element name="OrderType" type="ebics:OrderTBaseType">
       <annotation>
         <documentation xml:lang="de">Auftragsart des ausgewählten Auftrags gemäß Anhang (Kapitel
13).</documentation>
```

```
      </annotation>
    </element>
    <element name="OrderID" type="ebics:OrderIDType">
      <annotation>
        <documentation xml:lang="de">Auftragsnummer des ausgewählten Auftrags</documentation>
      </annotation>
    </element>
  </sequence>
</group>
```

8.3.5.1.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
HVSOrderParams	ebics:HVSOrderParamsType (komplex)	1	Auftragsparameter für Auftragsart HVS	- (komplex)
PartnerID	ebics:PartnerIDType (→token, maxLength=35, pattern="[a-zA-Z0-9,=]{1,35})	1	Kunden-ID des Einreichers.	„PARTNER1“
Service	ebics:RestrictedServiceType Details zu dieser Struktur siehe Kapitel 8.3.6	1	Art des Geschäftsvorfalls, identifiziert durch die Servicestruktur	
OrderID	ebics:OrderIDType (→token, fixLength=4)	1	Auftragsnummer des zu stornierenden Auftrags in der VEU-Prozessverarbeitung	„OR01“

8.3.5.1.4 Beispiel-XML (gekürzt)

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest
 xmlns="urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_request_H005.xsd"
 Version="H005" Revision="1">
  <header authenticate="true">
    <static>
      <!-- […] -->
      <OrderDetails>
        <AdminOrderType>HVS</AdminOrderType>
        <OrderID>H005</OrderID>
        <HVSOrderParams>
          <PartnerID>PARTNER1</PartnerID>
            <Service>
             <ServiceName>SCT</ServiceName>
             <MsgName>pain.001</MsgName>
          </Service>
          <OrderID>OR01</OrderID>
        </HVSOrderParams>
      </OrderDetails>
      <!-- […] -->
    </static>
```

```
    <!-- […] -->
  </header>
  <!-- […] -->
</ebicsRequest>
```

8.3.5.2 HVS-Response

Die HVS-Response enthält keine VEU-spezifischen Daten.

8.3.6 Verwendete Service-Strukturen (eingeschränkt/nicht eingeschränkt)

Die Service-Struktur vom Typ `RestrictedServiceType` mit optionalen und verpflichtenden Elementen wird für nahezu alle Up- und Download-Requests verwendet, wo BTF-Informationen benötigt werden. Nur für den HVU- und den HVZ-Request müssen ALLE Elemente in der Struktur optional sein (Typ der Struktur dann `ServiceType`). Folglich kann der Teilnehmer mit einem speziellen Filter Aufträge mit bestimmten BTF-Parameterwerten heraussuchen. Wenn die Struktur nicht angegeben ist, werden alle Aufträge geliefert, für die der Teilnehmer unterschriftsberechtigt ist.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

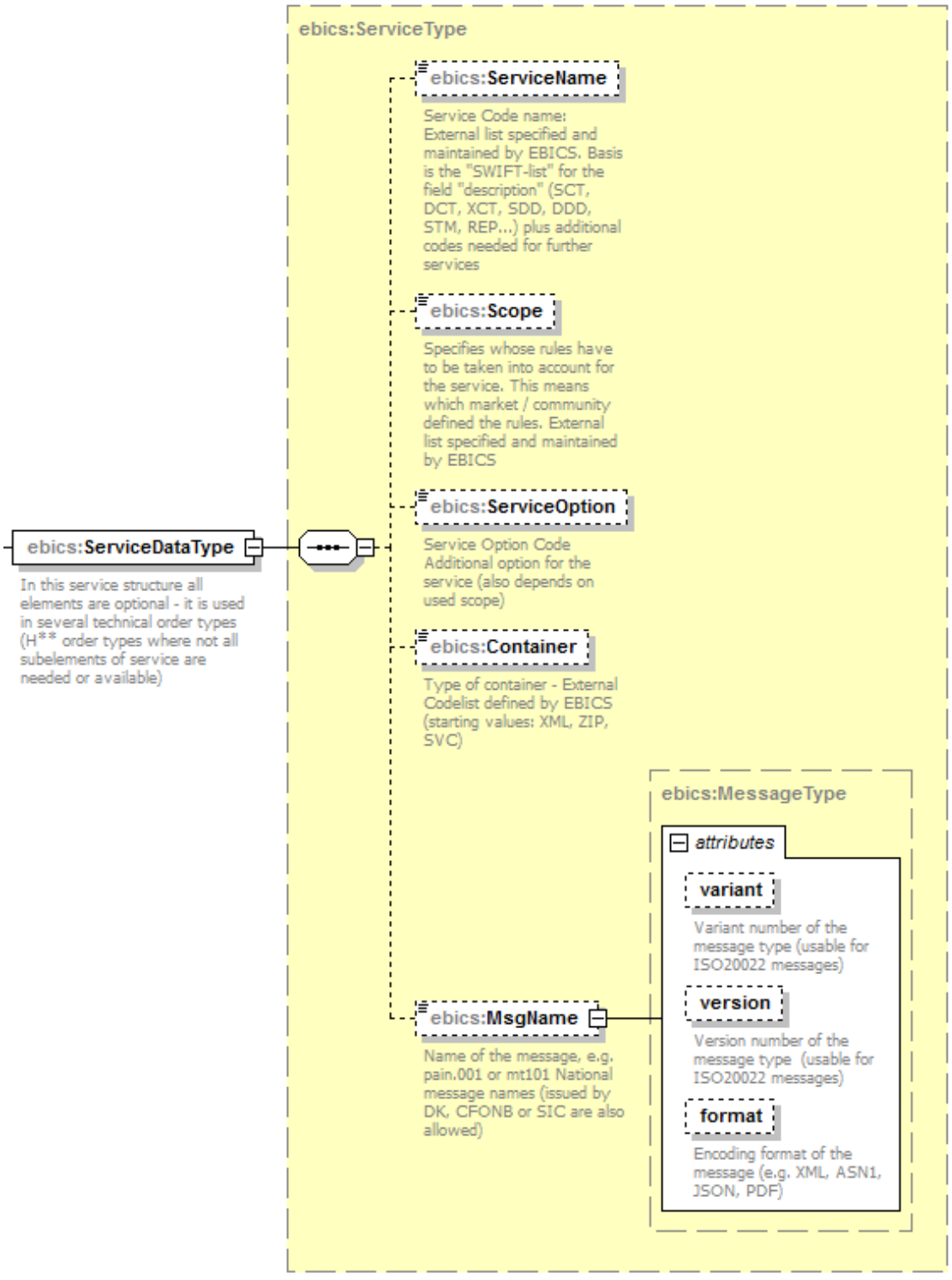


Abbildung 84: Nicht-eingeschränkte BTF-Service-Struktur nur für HVU und HVZ-Requests

Für alle anderen Requests sowie für alle Responses, die Informationen zu BTF enthalten, wird die eingeschränkte Service-Struktur verwendet:

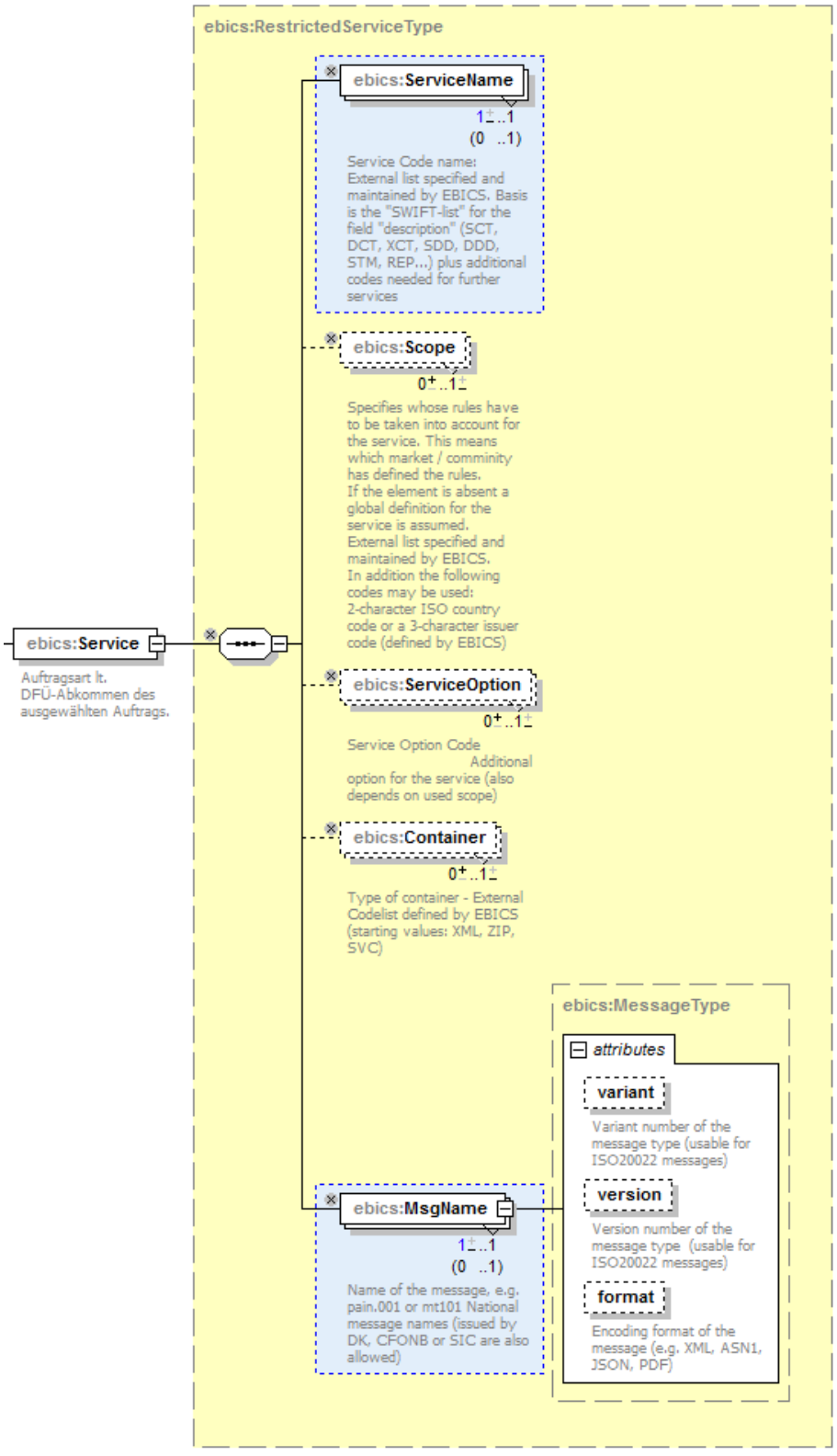


Abbildung 85: „Standard“-BTF-Servicestruktur für alle anderen Fälle

Hinsichtlich der Bedeutung der Elemente/Attribute wird auf Kapitel 5.5.1.1.3 verwiesen.

9 „Sonstige“ administrative EBICS-Auftragsarten

In den folgenden Abschnitten werden die folgenden Auftragsarten beschrieben:

- HAA (abrufbare Auftragsarten abholen)
- HPD (Bankparameter abholen)
- HKD (Kunden- und Teilnehmerinformationen des Kunden abrufen)
- HTD (Kunden- und Teilnehmerinformationen des Teilnehmers abrufen)
- HEV (Unterstützte EBICS-Versionen abrufen).

Informationen zur bankseitigen Unterstützung (verpflichtend, optional) siehe Anhang Kapitel 13.

9.1 HAA (abrufbare BTF-Parameter abholen)

Mit HAA kann sich der Teilnehmer eine Liste aller BTF-Parameter liefern lassen, für die im Banksystem aktuelle Kundendaten zur Abholung bereit stehen.

HAA ist eine administrative Auftragsart vom Typ „Download“.

9.1.1 HAA-Request

Der HAA-Request enthält keine spezifischen Daten, die über die in der allgemeinen Transaktionsbeschreibung genannten hinausgehen (siehe Kapitel 5.6.1.1).

9.1.2 HAA-Response

Ausprägung der (dekodierten & entschlüsselten & dekomprimierten) `OrderData` (Auftragsdaten) für HAA: `HAAResponseOrderData`

9.1.2.1.1 XML-Schema (grafische Darstellung)

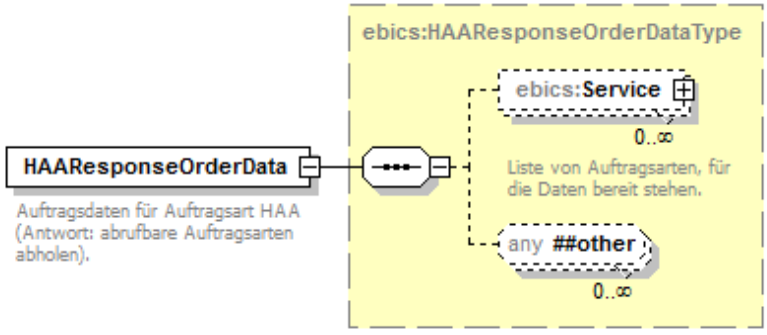


Abbildung 86: HAAResponseOrderData

9.1.2.1.2 XML-Schema (textuelle Darstellung)

```
 <element name="HAAResponseOrderData" type="ebics:HAAResponseOrderDataType"
substitutionGroup="ebics:EBICSOrderData">
   <annotation>
     <documentation xml:lang="de">Auftragsdaten für Auftragsart HAA (Antwort: abrufbare Auftragsarten
abholen).</documentation>
   </annotation>
 </element>
 <complexType name="HAAResponseOrderDataType">
   <annotation>
     <documentation xml:lang="de">Datentyp für Auftragsdaten für Auftragsart HAA (Antwort: abrufbare
Auftragsarten abholen).</documentation>
   </annotation>
   <sequence>
     <element name="OrderTypes" type="ebics:OrderTListType">
       <annotation>
         <documentation xml:lang="de">Liste von Auftragsarten, für die Daten bereit
stehen.</documentation>
       </annotation>
     </element>
     <any namespace="##other" minOccurs="0" maxOccurs="unbounded"/>
   </sequence>
 </complexType>
```

9.1.2.1.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
`HAAResponse» OrderData`	`ebics:HAAResponse» OrderDataType` (komplex)	1	Auftragsdaten für Auftragsart HAA	- (komplex)
`Service`	`ebics:RestrictedServiceType` Details zu dieser Struktur siehe Kapitel 8.3.6	1	Art des Geschäftsvorfalls, identifiziert durch die Service-Struktur	

9.2 HPD (Bankparameter abholen)

Mit HPD kann sich der Teilnehmer über die spezifischen Zugangs- (`AccessParams`) und Protokollparameter (`ProtocolParams`) des Kreditinstituts informieren.

Zu den Zugangsparametern gehören:

- `URL`: URL oder IP-Adresse für den elektronischen Zugang zum Kreditinstitut. Das optionale Attribut `valid_from` gibt den Gültigkeitsbeginn (Zeitstempel) der Angabe vor

- `Institute`: Bezeichnung des Kreditinstituts
- `HostID` (optional): EBICS Host-ID des Banksystems.

Bei den Protokollparametern werden folgende Informationen übermittelt:

- `Version`: zulässige Versionen (jeweils als Auflistung) für EBICS-Protokoll (`Protocol`), Authentifikation (`Authentication`), Verschlüsselung (`Encryption`) und Signatur (`Signature`)
- `Recovery` (optional): Unterstützung für das Wiederaufsetzen von Transaktionen (`@supported`)
- `PreValidation` (optional): Unterstützung der Vorabprüfung (`@supported`). Falls dieser Parameter gesetzt ist, sichert das Kreditinstitut damit lediglich zu, dass der Teilnehmer im Rahmen der Vorabprüfung Daten an das Kreditinstitut übermitteln kann. Das Kreditinstitut ist aber nicht verpflichtet, diese Daten in vollem Umfang zu prüfen.
- `ClientDataDownload` (optional): Unterstützung der Auftragsarten HKD (Download von Kundendaten) und HTD (Download von Teilnehmerdaten) (`@supported`). Siehe hierzu Kapitel 9.3 (HKD) bzw. 9.4 (HTD)
- `DownloadableOrderData` (optional): Unterstützung der Auftragsart HAA (abrufbare Auftragsarten abholen) (`@supported`). Siehe hierzu Kapitel 9.1

Für alle optionalen Elemente der Protokollparameter ist – sofern nicht explizit anders angegeben – folgendes Standardverhalten definiert:

- Falls der Parameter fehlt, MUSS der Teilnehmer dies als fehlende Unterstützung der entsprechenden Funktionalität werten, d.h. das Resultat entspricht *`Parameter`*`@supported="false"`
- Falls der Parameter angegeben ist, aber das Attribut `@supported` fehlt, so MUSS der Teilnehmer dies als Unterstützung der entsprechenden Funktionalität werten, d.h. das Resultat entspricht *`Parameter`*`@supported="true"`.

Diese Festlegung erleichtert die Interoperabilität von Kundenprodukt und Banksystem: Einerseits ist gewährleistet, dass ein Kreditinstitut, welches eine Funktionalität nicht unterstützt, diese in den Bankparametern auch nicht explizit als „nicht unterstützt“ aufführen muss. Andererseits wird davon ausgegangen, dass bei Nennung einer Funktionalität diese auch unterstützt wird, so dass in diesem Fall das `@supported`-Flag entfallen kann.

HPD ist eine administrative Auftragsart vom Typ „Download“.

9.2.1 HPD-Request

Der HPD-Request enthält keine spezifischen Daten, die über die in der allgemeinen Transaktionsbeschreibung genannten hinausgehen.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

9.2.2 HPD-Response

Die HPD-Response enthält die Bankparameter, aufgeteilt auf Zugangsparameter (`AccessParams`) und Protokollparameter (`ProtocolParams`).

Ausprägung der (dekodierten & entschlüsselten & dekomprimierten) `OrderData` (Auftragsdaten) für HPD: `HPDResponseOrderData`

9.2.2.1.1 XML-Schema (grafische Darstellung)

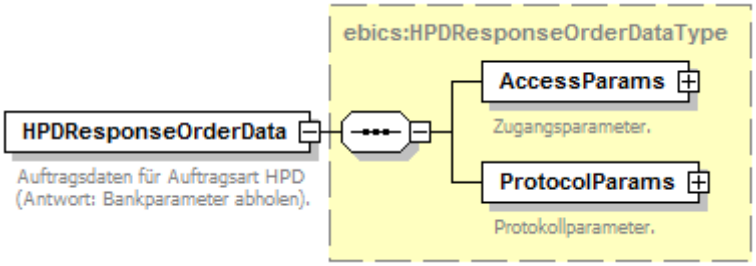


Abbildung 87: HPDResponseOrderData

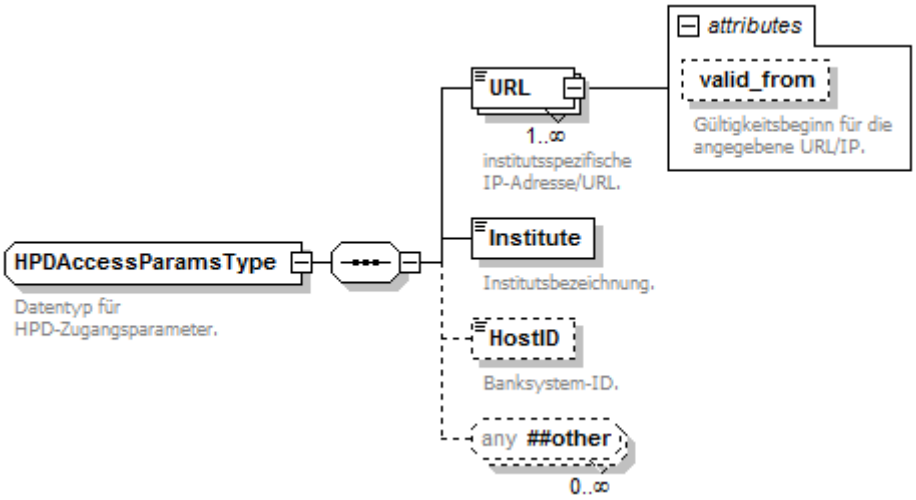


Abbildung 88: HPDAccessParamsType (zu AccessParams)

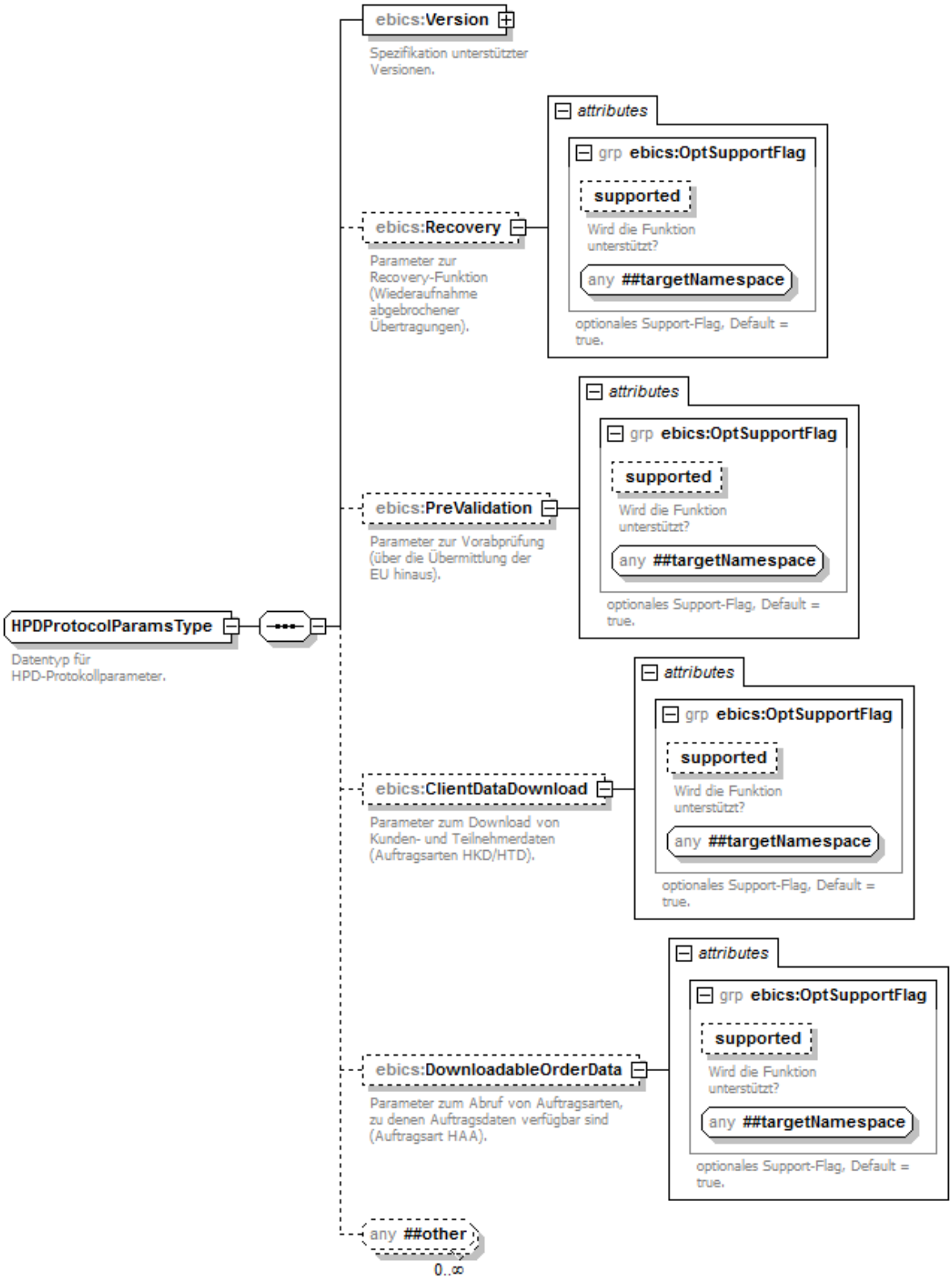


Abbildung 89: HPDProtocolParamsType (zu ProtocolParams)

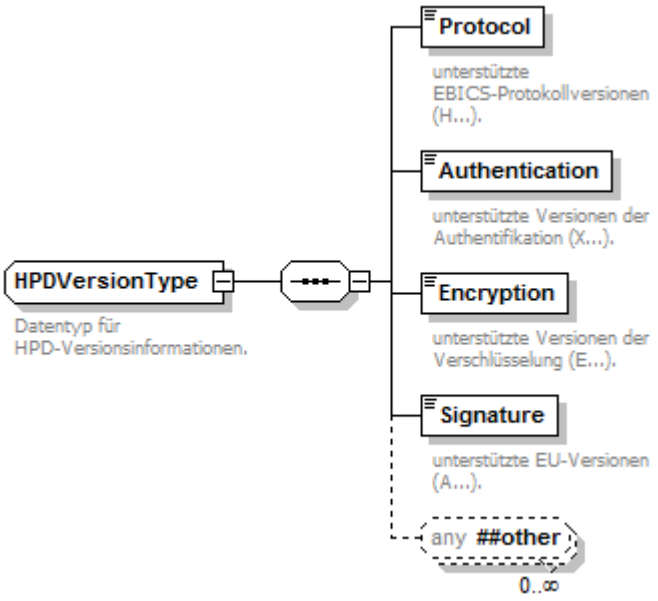


Abbildung 90: HPDVersionType (zu Version)

9.2.2.1.2 XML-Schema (textuelle Darstellung)

```
 <element name="HPDResponseOrderData" type="ebics:HPDResponseOrderDataType"
substitutionGroup="ebics:EBICSOrderData">
   <annotation>
     <documentation xml:lang="de">Auftragsdaten für Auftragsart HPD (Antwort: Bankparameter
abholen).</documentation>
   </annotation>
 </element>
 <complexType name="HPDResponseOrderDataType">
   <annotation>
     <documentation xml:lang="de">Datentyp für Auftragsdaten für Auftragsart HPD (Antwort: Bankparameter
abholen).</documentation>
   </annotation>
   <sequence>
     <element name="AccessParams" type="ebics:HPDAccessParamsType">
       <annotation>
         <documentation xml:lang="de">Zugangsparameter.</documentation>
       </annotation>
     </element>
     <element name="ProtocolParams" type="ebics:HPDProtocolParamsType">
       <annotation>
         <documentation xml:lang="de">Protokollparameter.</documentation>
       </annotation>
     </element>
   </sequence>
 </complexType>
 <complexType name="HPDAccessParamsType">
   <annotation>
     <documentation xml:lang="de">Datentyp für HPD-Zugangsparameter.</documentation>
   </annotation>
   <sequence>
     <element name="URL" maxOccurs="unbounded">
       <annotation>
         <documentation xml:lang="de">institutsspezifische IP-Adresse/URL.</documentation>
```

```
        </annotation>
        <complexType>
          <simpleContent>
            <extension base="anyURI">
              <attribute name="valid_from" type="ebics:TimestampType">
                <annotation>
                  <documentation xml:lang="de">Gültigkeitsbeginn für die angegebene URL/IP.</documentation>
                </annotation>
              </attribute>
            </extension>
          </simpleContent>
        </complexType>
      </element>
      <element name="Institute">
        <annotation>
          <documentation xml:lang="de">Institutsbezeichnung.</documentation>
        </annotation>
        <simpleType>
          <restriction base="normalizedString">
            <maxLength value="80"/>
          </restriction>
        </simpleType>
      </element>
      <element name="HostID" type="ebics:HostIDType" minOccurs="0">
        <annotation>
          <documentation xml:lang="de">Banksystem-ID.</documentation>
        </annotation>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
    </sequence>
  </complexType>
  <complexType name="HPDProtocolParamsType">
    <annotation>
      <documentation xml:lang="de">Datentyp für HPD-Protokollparameter.</documentation>
    </annotation>
    <sequence>
      <element name="Version" type="ebics:HPDVersionType">
        <annotation>
          <documentation xml:lang="de">Spezifikation unterstützter Versionen.</documentation>
        </annotation>
      </element>
      <element name="Recovery" minOccurs="0">
        <annotation>
          <documentation xml:lang="de">Parameter zur Recovery-Funktion (Wiederaufnahme abgebrochener
Übertragungen).</documentation>
        </annotation>
        <complexType>
          <attributeGroup ref="ebics:OptSupportFlag"/>
        </complexType>
      </element>
      <element name="PreValidation" minOccurs="0">
        <annotation>
          <documentation xml:lang="de">Parameter zur Vorabprüfung (über die Übermittlung der EU
hinaus).</documentation>
        </annotation>
        <complexType>
          <attributeGroup ref="ebics:OptSupportFlag"/>
        </complexType>
      </element>
      <element name="X509Data" minOccurs="0">
        <annotation>
          <documentation xml:lang="de">Parameter zur X.509-Funktionalität.</documentation>
```

```
        </annotation>
        <complexType>
          <attributeGroup ref="ebics:OptSupportFlag"/>
          <attribute name="persistent" type="boolean" use="optional" default="false">
            <annotation>
              <documentation xml:lang="de">Sind die X.509-Daten der Teilnehmer serverseitig persistent
gespeichert?</documentation>
            </annotation>
          </attribute>
        </complexType>
      </element>
      <element name="ClientDataDownload" minOccurs="0">
        <annotation>
          <documentation xml:lang="de">Parameter zum Download von Kunden- und Teilnehmerdaten
(Auftragsarten HKD/HTD).</documentation>
        </annotation>
        <complexType>
          <attributeGroup ref="ebics:OptSupportFlag"/>
        </complexType>
      </element>
      <element name="DownloadableOrderData" minOccurs="0">
        <annotation>
          <documentation xml:lang="de">Parameter zum Abruf von Auftragsarten, zu denen Auftragsdaten
verfügbar sind (Auftragsart HAA).</documentation>
        </annotation>
        <complexType>
          <attributeGroup ref="ebics:OptSupportFlag"/>
        </complexType>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
    </sequence>
  </complexType>
  <complexType name="HPDVersionType">
    <annotation>
      <documentation xml:lang="de">Datentyp für HPD-Versionsinformationen.</documentation>
    </annotation>
    <sequence>
      <element name="Protocol">
        <annotation>
          <documentation xml:lang="de">unterstützte EBICS-Protokollversionen (H...).</documentation>
        </annotation>
        <simpleType>
          <list itemType="ebics:ProtocolVersionType"/>
        </simpleType>
      </element>
      <element name="Authentication">
        <annotation>
          <documentation xml:lang="de">unterstützte Versionen der Authentifikation (X...).</documentation>
        </annotation>
        <simpleType>
          <list itemType="ebics:AuthenticationVersionType"/>
        </simpleType>
      </element>
      <element name="Encryption">
        <annotation>
          <documentation xml:lang="de">unterstützte Versionen der Verschlüsselung (E...).</documentation>
        </annotation>
        <simpleType>
          <list itemType="ebics:EncryptionVersionType"/>
        </simpleType>
      </element>
      <element name="Signature">
```

```
        <annotation>
          <documentation xml:lang="de">unterstützte EU-Versionen (A...).</documentation>
        </annotation>
        <simpleType>
          <list itemType="ebics:SignatureVersionType"/>
        </simpleType>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
    </sequence>
  </complexType>
  <attributeGroup name="OptSupportFlag">
    <annotation>
      <documentation xml:lang="de">optionales Support-Flag, Default = true.</documentation>
    </annotation>
    <attribute name="supported" type="boolean" use="optional" default="true">
      <annotation>
        <documentation xml:lang="de">Wird die Funktion unterstützt?</documentation>
      </annotation>
    </attribute>
    <anyAttribute namespace="##targetNamespace" processContents="strict"/>
  </attributeGroup>
```

9.2.2.1.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
HPDResponse»OrderData	ebics:HPDResponse»OrderDataType (komplex)	1	Auftragsdaten für Auftragsart HPD	- (komplex)
AccessParams	ebics:HPDAccessParams»Type (komplex)	1	Zugangsparameter	- (komplex)
ProtocolParams	ebics:HPDProtocol»ParamsType (komplex)	1	Protokollparameter	- (komplex)
URL	anyURI	1..∞	institutsspezifische IP-Adresse/ URL	„https://www.die-bank.de“
URL@valid_from	ebics:TimestampType (→dateTime)	0..1	Gültigkeitsbeginn für die angegebene URL/IP; falls nicht angegeben, ist die URL/IP mit sofortiger Wirkung gültig	„2005-02-28T»15:30:45.123Z“
Institute	normalizedString, maxLength=80	1	Kreditinstitutsbezeichnung	„Die Bank“
HostID	ebics:HostIDType (→token, maxLength=35)	0..1	EBICS Banksystem-ID	„EBIXHOST“
Version	ebics:HPDVersionType (komplex)	1	Spezifikation unterstützter Versionen	- (komplex)
Protocol	list<ebics:Protocol»VersionType> (→list<token, length=4, pattern="H\d{3}">)	1	Liste unterstützter EBICS-Protokollversionen	„H005“
Authentication	list<ebics:Authentica»	1	Liste unterstützter	„X002“

	tionVersionType> (→list<token, length=4, pattern= "X\d{3}">)		Versionen der Authentifikation	
Encryption	list<ebics:Encryption» VersionType> (→list<token, length=4, pattern="E\d{3}">)	1	Liste unterstützter Versionen der Verschlüsselung	„E002“
Signature	list<ebics:Signature» VersionType> (→list<token, length=4, pattern="A\d{3}">)	1	Liste unterstützter EU-Versionen	„A005 A006”
Recovery	– (komplex)	0..1	Parameter zur Recovery-Funktion (Wiederaufnahme abgebrochener Verbindungen); falls nicht angegeben, wird die Funktion nicht unterstützt	- (komplex)
Recovery» @supported	boolean	0..1	Wird Recovery unterstützt? (Default=true)	„true“
PreValidation	– (komplex)	0..1	Parameter zur Vorabprüfung; falls nicht angegeben, wird die Funktion nicht unterstützt	- (komplex)
PreValidation» @supported	boolean	0..1	Wird die Vorabprüfung unterstützt? (Default=true)	„true“
X509Data	– (komplex)	0..1	Parameter zu X.509-Daten; falls nicht angegeben, wird die Funktion nicht unterstützt	- (komplex)
X509Data» @supported	boolean	0..1	Werden X.509-Daten unterstützt? (Default=true)	„false“
X509Data» @persistent	boolean	0..1	Werden die X.509-Daten des Teilnehmers serverseitig persistent gespeichert? (Default=false)	„false“
ClientData» Download	– (komplex)	0..1	Parameter zum Download von Kunden- und Teilnehmerdaten (HKD/HTD); falls nicht angegeben, wird die Funktion nicht unterstützt	„true“
ClientData» Download» @supported	boolean	0..1	Werden die Auftragsarten HKD/HTD unterstützt? (Default=true)	„true“
Downloadable» OrderData	– (komplex)	0..1	Parameter zum Abruf von Auftragsarten, zu denen Auftragsdaten verfügbar	- (komplex)

			sind (HAA); falls nicht angegeben, wird die Funktion nicht unterstützt	
Downloadable» OrderData» @supported	boolean	0..1	Wird die Auftragsart HAA unterstützt? (Default=true)	„true“

9.2.2.1.4 Beispiel-XML

```
<?xml version="1.0" encoding="UTF-8"?>
<HPDResponseOrderData
 xmlns="urn:org:ebics:H005"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_orders_H005.xsd">
 <AccessParams>
   <URL>https://www.die-bank.de</URL>
   <URL valid_from="2005-02-15T15:30:45.123Z">192.168.0.1</URL>
   <Institute>Die Bank</Institute>
   <HostID>EBIXHOST</HostID>
 </AccessParams>
 <ProtocolParams>
   <Version>
     <Protocol>H005</Protocol>
     <Authentication>X002</Authentication>
     <Encryption>E002</Encryption>
     <Signature>A005 A006</Signature>
   </Version>
   <Recovery supported="true"/>
   <PreValidation supported="true"/>
   <X509Data supported="false"/>
   <ClientDataDownload supported="true"/>
   <DownloadableOrderData supported="true"/>
 </ProtocolParams>
</HPDResponseOrderData>
```

9.3 HKD (Kunden- und Teilnehmerinformationen des Kunden abrufen)

Mit HKD kann der Teilnehmer bankseitig gespeicherte Informationen zu seinem Unternehmen und zu allen zugehörigen Teilnehmern (auch sich selbst) abrufen.

Die Antwort der Bank enthält eine Liste mit Konten des Kunden.
Ein Konto wird in der HKD Response nur dann aufgeführt, wenn mindestens eine dieser Bedingungen erfüllt ist:

1. Der Kunde hat für das Konto eine Bereitstellungsvereinbarung für Kontoauszüge.
2. Mindestens ein Teilnehmer des Kunden hat eine Unterschriftsberechtigung für das Konto.

Es ist nicht relevant, ob der Inhaber eines Kontos derselbe Kunde ist, für den HKD abgerufen wird.

HKD ist eine administrative Auftragsart vom Typ „Download“.

9.3.1 HKD-Request

Der HKD-Request enthält keine spezifischen Daten, die über die in der allgemeinen Transaktionsbeschreibung genannten hinausgehen.

9.3.2 HKD-Response

Ausprägung der (dekodierten & entschlüsselten & dekomprimierten) `OrderData` (Auftragsdaten) für HKD: `HKDResponseOrderData`

9.3.2.1.1 XML-Schema (grafische Darstellung)

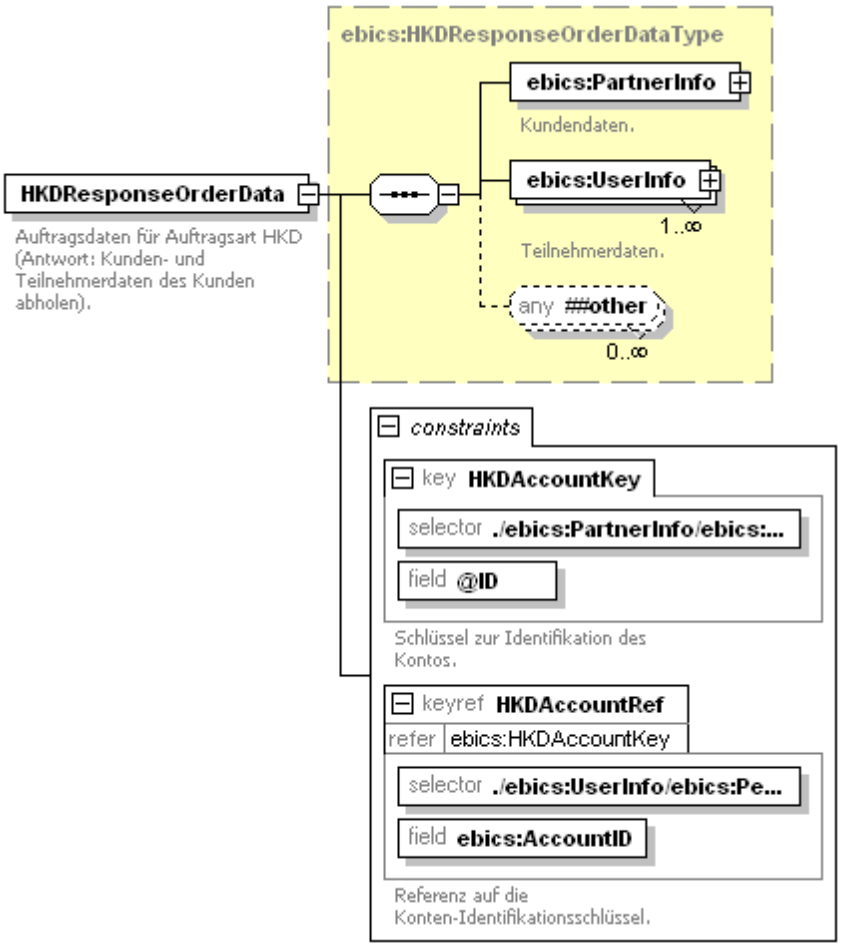


Abbildung 91: HKDResponseOrderData

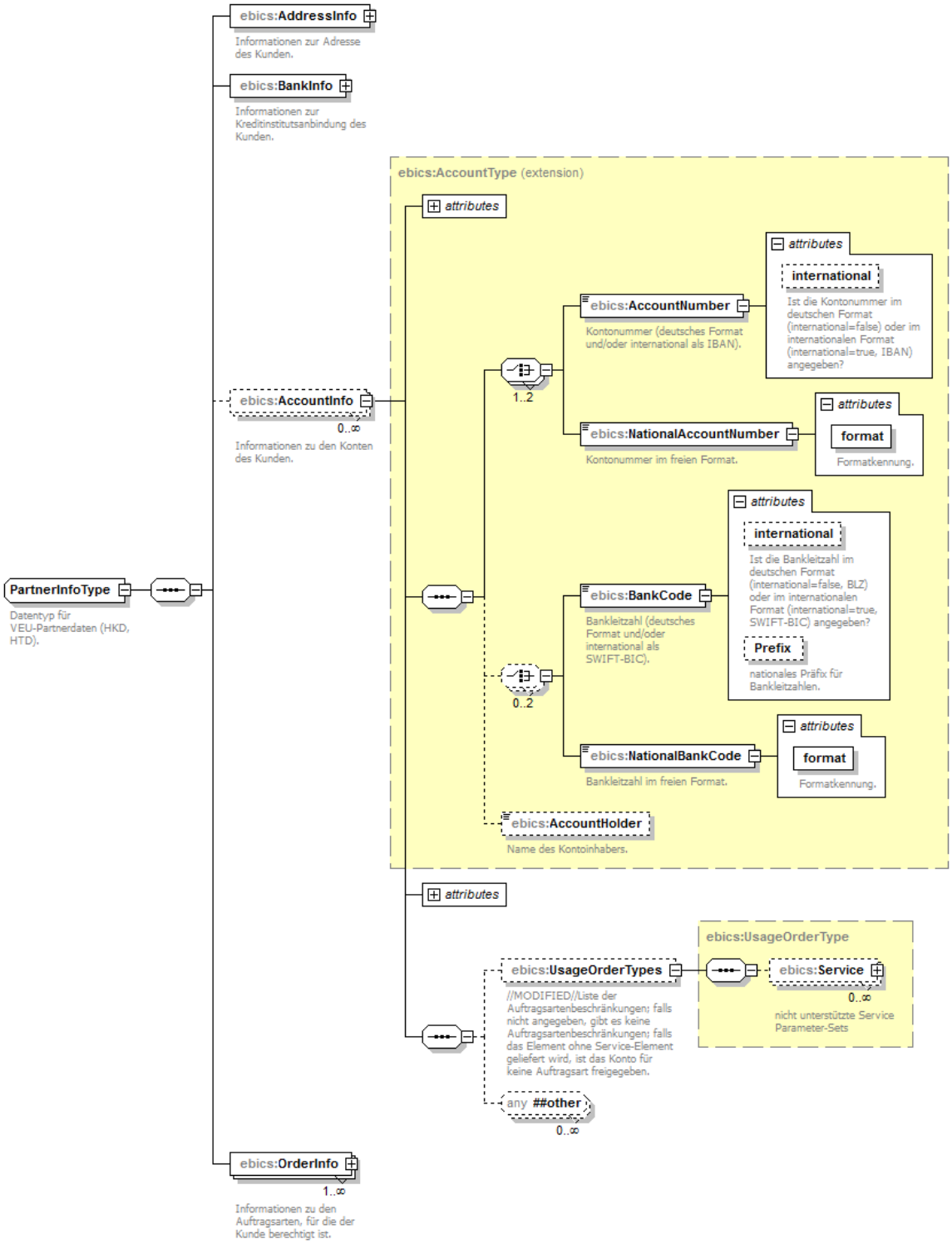


Abbildung 92: PartnerInfoType (zu PartnerInfo)

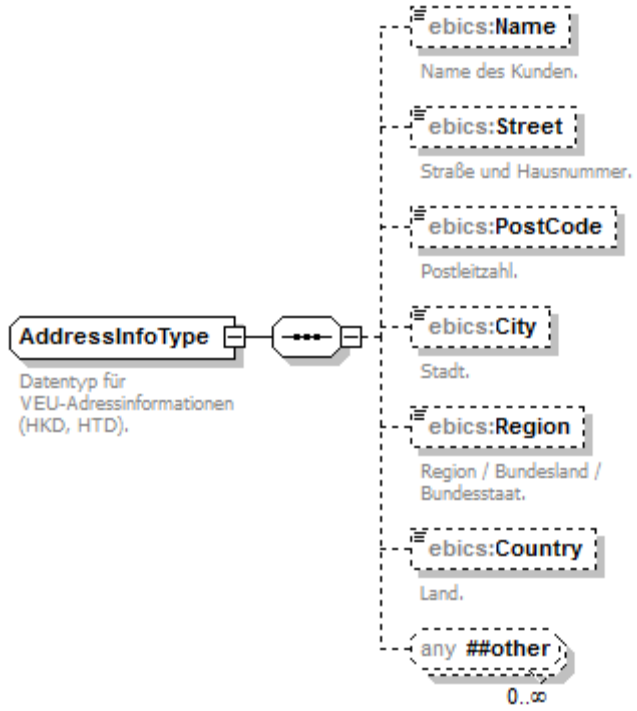


Abbildung 93: AddressInfoType (zu AddressInfo)

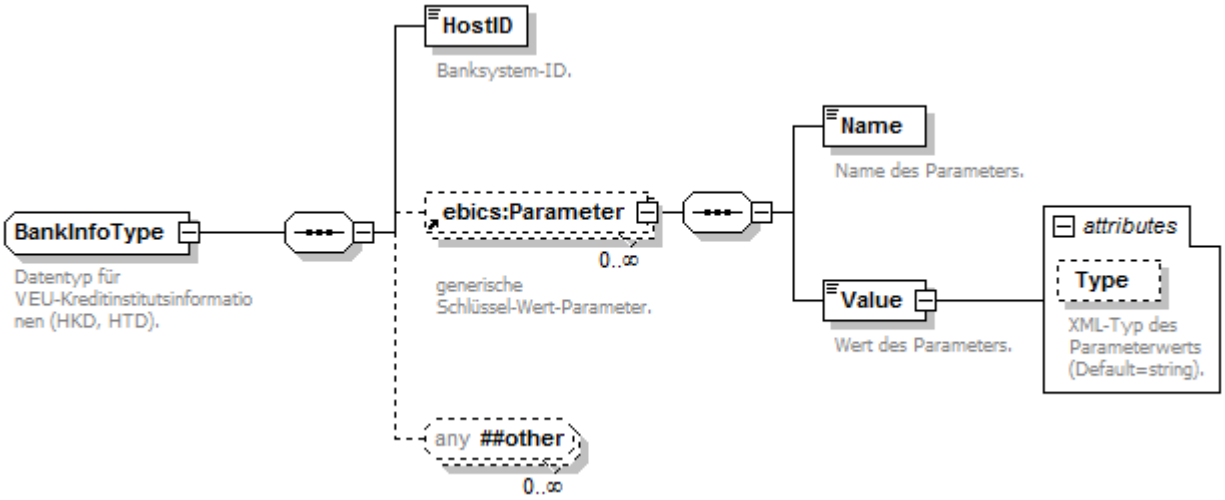


Abbildung 94: BankInfoType (zu BankInfo)

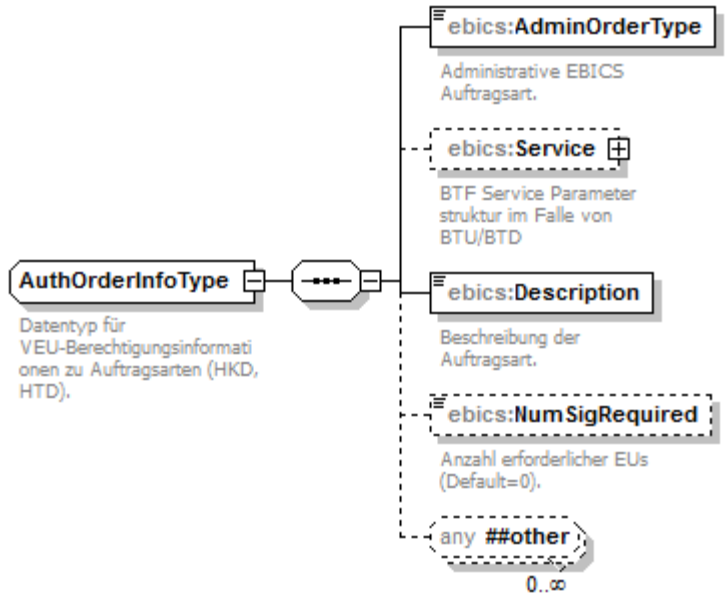


Abbildung 95: AuthOrderInfoType (zu OrderInfo)

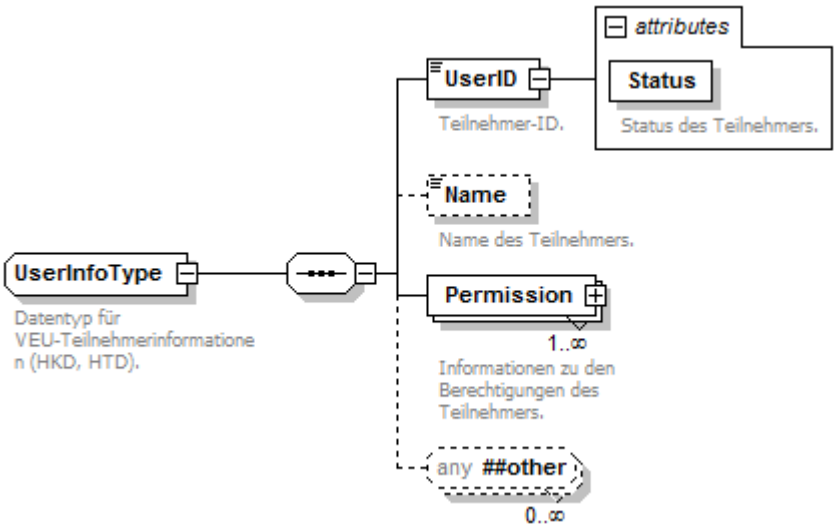


Abbildung 96: UserInfoType (zu UserInfo)

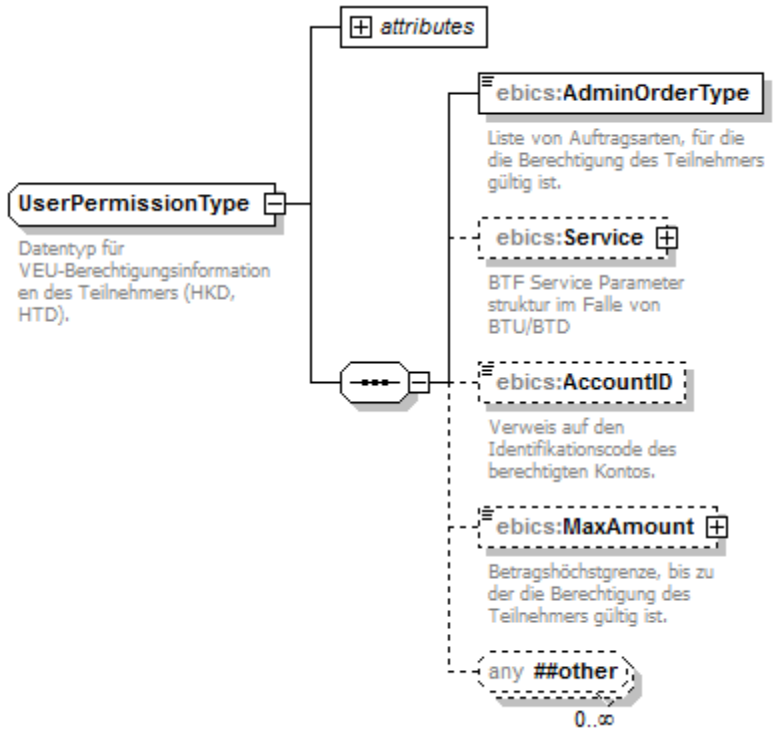


Abbildung 97: UserPermissionType (zu Permission)

9.3.2.1.2 XML-Schema (textuelle Darstellung)

```
<element name="HKDResponseOrderData"
   type="ebics:HKDResponseOrderDataType"
   substitutionGroup="ebics:EBICSOrderData">
   <annotation>
      <documentation xml:lang="de">
         Auftragsdaten für Auftragsart HKD (Antwort: Kunden- und
         Teilnehmerdaten des Kunden abholen).
      </documentation>
   </annotation>
   <key name="HKDAccountKey">
      <annotation>
         <documentation xml:lang="de">
            Schlüssel zur Identifikation des Kontos.
         </documentation>
      </annotation>
      <selector xpath="./ebics:PartnerInfo/ebics:AccountInfo" />
      <field xpath="@ID" />
   </key>
   <keyref name="HKDAccountRef" refer="ebics:HKDAccountKey">
      <annotation>
         <documentation xml:lang="de">
            Referenz auf die Konten-Identifikationsschlüssel.
         </documentation>
      </annotation>
```

```
      <selector xpath="./ebics:UserInfo/ebics:Permission" />
      <field xpath="AccountID" />
   </keyref>
</element>
<complexType name="HKDResponseOrderDataType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für Auftragsdaten für Auftragsart HKD (Antwort:
         Kunden- und Teilnehmerdaten des Kunden abholen).
      </documentation>
   </annotation>
   <sequence>
      <element name="PartnerInfo" type="ebics:PartnerInfoType">
         <annotation>
            <documentation xml:lang="de-DE">Kundendaten.</documentation>
         </annotation>
      </element>
      <element name="UserInfo" type="ebics:UserInfoType"
         maxOccurs="unbounded">
         <annotation>
            <documentation xml:lang="de">
               Teilnehmerdaten.
            </documentation>
         </annotation>
      </element>
      <any namespace="##other" minOccurs="0" maxOccurs="unbounded" />
   </sequence>
</complexType>
<complexType name="PartnerInfoType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für VEU-Partnerdaten (HKD, HTD).
      </documentation>
   </annotation>
   <sequence>
      <element name="AddressInfo" type="ebics:AddressInfoType">
         <annotation>
            <documentation xml:lang="de">
               Informationen zur Adresse des Kunden.
            </documentation>
         </annotation>
      </element>
      <element name="BankInfo" type="ebics:BankInfoType">
         <annotation>
            <documentation xml:lang="de">
               Informationen zur Kreditinstitutsanbindung des Kunden.
            </documentation>
         </annotation>
      </element>
      <element name="AccountInfo" minOccurs="0" maxOccurs="unbounded">
         <annotation>
            <documentation xml:lang="de">
               Informationen zu den Konten des Kunden.
            </documentation>
         </annotation>
         <complexType>
            <complexContent>
               <extension base="ebics:AccountType">
                  <sequence>
                     <element name="UsageOrderTypes"
                        type="ebics:OrderTListType" minOccurs="0">
                        <annotation>
                           <documentation xml:lang="de">
```

```
                              Liste der Auftragsartenbeschränkungen;
                              falls nicht angegeben, gibt es keine
                              Auftragsartenbeschränkungen; falls Liste
                              leer, ist das Konto für keine Auftragsart
                              freigegeben.
                           </documentation>
                        </annotation>
                     </element>
                     <any namespace="##other" processContents="lax"
                        minOccurs="0" maxOccurs="unbounded" />
                  </sequence>
                  <attribute name="ID" type="ebics:AccountIDType"
                     use="required">
                     <annotation>
                        <documentation xml:lang="de">
                           Identifikationscode des Kontos.
                        </documentation>
                     </annotation>
                  </attribute>
               </extension>
            </complexContent>
         </complexType>
      </element>
      <element name="OrderInfo" type="ebics:OrderInfoType"
         maxOccurs="unbounded">
         <annotation>
            <documentation xml:lang="de">
               Informationen zu den Auftragsarten, für die der Kunde
               berechtigt ist.
            </documentation>
         </annotation>
      </element>
   </sequence>
</complexType>
<complexType name="AddressInfoType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für VEU-Adressinformationen (HKD, HTD).
      </documentation>
   </annotation>
   <sequence>
      <element name="Name" type="ebics:NameType" minOccurs="0">
         <annotation>
            <documentation xml:lang="de">
               Name des Kunden.
            </documentation>
         </annotation>
      </element>
      <element name="Street" type="ebics:NameType" minOccurs="0">
         <annotation>
            <documentation xml:lang="de">
               Straße und Hausnummer.
            </documentation>
         </annotation>
      </element>
      <element name="PostCode" type="token" minOccurs="0">
         <annotation>
            <documentation xml:lang="de">Postleitzahl.</documentation>
         </annotation>
      </element>
      <element name="City" type="ebics:NameType" minOccurs="0">
         <annotation>
            <documentation xml:lang="de">Stadt.</documentation>
```

```
         </annotation>
      </element>
      <element name="Region" type="ebics:NameType" minOccurs="0">
         <annotation>
            <documentation xml:lang="de-DE">
               Region / Bundesland / Bundesstaat.
            </documentation>
         </annotation>
      </element>
      <element name="Country" type="ebics:NameType" minOccurs="0">
         <annotation>
            <documentation xml:lang="de-DE">Land.</documentation>
         </annotation>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0"
         maxOccurs="unbounded" />
   </sequence>
</complexType>
<complexType name="BankInfoType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für VEU-Kreditinstitutsinformationen (HKD, HTD).
      </documentation>
   </annotation>
   <sequence>
      <element name="HostID" type="ebics:HostIDType">
         <annotation>
            <documentation xml:lang="de">Banksystem-ID.</documentation>
         </annotation>
      </element>
      <element ref="ebics:Parameter" minOccurs="0"
         maxOccurs="unbounded" />
      <any namespace="##other" processContents="lax" minOccurs="0"
         maxOccurs="unbounded" />
   </sequence>
</complexType>
<complexType name="AuthOrderInfoType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für VEU-Berechtigungsinformationen zu Auftragsarten
         (HKD, HTD).
      </documentation>
   </annotation>
   <sequence>
      <element name="OrderType" type="ebics:OrderTBaseType">
         <annotation>
            <documentation xml:lang="de">Auftragsart.</documentation>
         </annotation>
      </element>
      <element name="FileFormat" type="ebics:FileFormatType" minOccurs="0">
       <annotation>
          <documentation xml:lang="en">File format parameter.</documentation>
       </annotation>
      </element>
      <element name="TransferType" type="ebics:TransferType">
         <annotation>
            <documentation xml:lang="de">
               Transfertyp (Upload/Download).
            </documentation>
         </annotation>
      </element>
      <element name="OrderFormat" type="ebics:OrderFormatType"
         minOccurs="0">
```

```
         <annotation>
            <documentation xml:lang="de">Auftragsformat.</documentation>
         </annotation>
      </element>
      <element name="Description" type="ebics:OrderDescriptionType">
         <annotation>
            <documentation xml:lang="de">
               Beschreibung der Auftragsart.
            </documentation>
         </annotation>
      </element>
      <element name="NumSigRequired" type="nonNegativeInteger"
         minOccurs="0">
         <annotation>
            <documentation xml:lang="de">
               Anzahl erforderlicher EUs (Default=0).
            </documentation>
         </annotation>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0"
         maxOccurs="unbounded" />
   </sequence>
</complexType>
<complexType name="UserInfoType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für VEU-Teilnehmerinformationen (HKD, HTD).
      </documentation>
   </annotation>
   <sequence>
      <element name="UserID">
         <annotation>
            <documentation xml:lang="de">Teilnehmer-ID.</documentation>
         </annotation>
         <complexType>
            <simpleContent>
               <extension base="ebics:UserIDType">
                  <attribute name="Status" type="ebics:UserStatusType"
                     use="required">
                     <annotation>
                        <documentation xml:lang="de">
                           Status des Teilnehmers.
                        </documentation>
                     </annotation>
                  </attribute>
               </extension>
            </simpleContent>
         </complexType>
      </element>
      <element name="Name" type="ebics:NameType" minOccurs="0">
         <annotation>
            <documentation xml:lang="de">
               Name des Teilnehmers.
            </documentation>
         </annotation>
      </element>
      <element name="Permission" type="ebics:UserPermissionType"
         maxOccurs="unbounded">
         <annotation>
            <documentation xml:lang="de">
               Informationen zu den Berechtigungen des Teilnehmers.
            </documentation>
         </annotation>
```

```
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0"
         maxOccurs="unbounded" />
   </sequence>
</complexType>
<complexType name="UserPermissionType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für VEU-Berechtigungsinformationen des Teilnehmers
         (HKD, HTD).
      </documentation>
   </annotation>
   <sequence>
      <element name="OrderTypes" type="ebics:OrderTListType">
         <annotation>
            <documentation xml:lang="de">
               Liste von Auftragsarten, für die die Berechtigung des
               Teilnehmers gültig ist.
            </documentation>
         </annotation>
      </element>
      <element name="FileFormat" type="ebics:FileFormatType" minOccurs="0">
       <annotation>
         <documentation xml:lang="en">File format parameters which the user's permission belongs
to.</documentation>
       </annotation>
      </element>
      <element name="AccountID" type="ebics:AccountIDType"
         minOccurs="0">
         <annotation>
            <documentation xml:lang="de">
               Verweis auf die Identifikationscodes der berechtigten
               Konten.
            </documentation>
         </annotation>
      </element>
      <element name="MaxAmount" type="ebics:AmountType" minOccurs="0">
         <annotation>
            <documentation xml:lang="de">
               Betragshöchstgrenze, bis zu der die Berechtigung des
               Teilnehmers gültig ist.
            </documentation>
         </annotation>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0"
         maxOccurs="unbounded" />
   </sequence>
   <attribute name="AuthorisationLevel"
      type="ebics:AuthorisationLevelType" use="optional">
      <annotation>
         <documentation xml:lang="de">
            Unterschriftsklasse, für die der Teilnehmer berechtigt ist;
            nicht anzugeben bei Download-Auftragsarten.
         </documentation>
      </annotation>
   </attribute>
   <anyAttribute namespace="##targetNamespace" processContents="strict" />
</complexType>
<complexType name="AccountType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für Kontoinformationen.
      </documentation>
```

```
</annotation>
<sequence>
   <choice maxOccurs="2">
      <element name="AccountNumber">
         <annotation>
            <documentation xml:lang="de">
               Kontonummer (deutsches Format und/oder international
               als IBAN).
            </documentation>
         </annotation>
         <complexType>
            <simpleContent>
               <extension base="ebics:AccountNumberType">
                  <attribute name="international" type="boolean"
                     use="optional" default="false">
                     <annotation>
                        <documentation xml:lang="de">
                           Ist die Kontonummer im deutschen Format
                           (international=false) oder im
                           internationalen Format
                           (international=true, IBAN) angegeben?
                        </documentation>
                     </annotation>
                  </attribute>
               </extension>
            </simpleContent>
         </complexType>
      </element>
      <element name="NationalAccountNumber">
         <annotation>
            <documentation xml:lang="de">
               Kontonummer im freien Format.
            </documentation>
         </annotation>
         <complexType>
            <simpleContent>
               <extension base="ebics:NationalAccountNumberType">
                  <attribute name="format" type="token"
                     use="required">
                     <annotation>
                        <documentation xml:lang="de">
                           Formatkennung.
                        </documentation>
                     </annotation>
                  </attribute>
               </extension>
            </simpleContent>
         </complexType>
      </element>
   </choice>
   <choice maxOccurs="2">
      <element name="BankCode">
         <annotation>
            <documentation xml:lang="de">
               Bankleitzahl (deutsches Format und/oder international
               als SWIFT-BIC).
            </documentation>
         </annotation>
         <complexType>
            <simpleContent>
               <extension base="ebics:BankCodeType">
                  <attribute name="international" type="boolean"
                     use="optional" default="false">
```

```
                        <annotation>
                           <documentation xml:lang="de">
                              Ist die Bankleitzahl im deutschen Format
                              (international=false, BLZ) oder im
                              internationalen Format
                              (international=true, SWIFT-BIC) angegeben?
                           </documentation>
                        </annotation>
                     </attribute>
                     <attribute name="Prefix"
                        type="ebics:BankCodePrefixType"
                        use="optional">
                        <annotation>
                           <documentation xml:lang="de">
                              nationales Präfix für Bankleitzahlen.
                           </documentation>
                        </annotation>
                     </attribute>
                  </extension>
               </simpleContent>
            </complexType>
         </element>
         <element name="NationalBankCode">
            <annotation>
               <documentation xml:lang="de">
                  Bankleitzahl im freien Format.
               </documentation>
            </annotation>
            <complexType>
               <simpleContent>
                  <extension base="ebics:NationalBankCodeType">
                     <attribute name="format" type="token"
                        use="required">
                        <annotation>
                           <documentation xml:lang="de">
                              Formatkennung.
                           </documentation>
                        </annotation>
                     </attribute>
                  </extension>
               </simpleContent>
            </complexType>
         </element>
      </choice>
      <element name="AccountHolder" type="ebics:AccountHolderType"
         minOccurs="0">
         <annotation>
            <documentation xml:lang="de">
               Name des Kontoinhabers.
            </documentation>
         </annotation>
      </element>
   </sequence>
   <attribute name="Currency" type="ebics:CurrencyBaseType"
      use="optional" default="EUR">
      <annotation>
         <documentation xml:lang="de">
            Währungscode für dieses Konto, Default=EUR.
         </documentation>
      </annotation>
   </attribute>
   <attribute name="Description" type="ebics:AccountDescriptionType"
      use="optional">
```

```
      <annotation>
         <documentation xml:lang="de">Kontobeschreibung.</documentation>
      </annotation>
   </attribute>
</complexType>
<complexType name="AmountType">
   <annotation>
      <documentation xml:lang="de">
         Datentyp für einen Betrag inkl. Währungscode-Attribut (Default
         = "EUR").
      </documentation>
   </annotation>
   <simpleContent>
      <extension base="ebics:AmountValueType">
         <attribute name="Currency" type="ebics:CurrencyBaseType"
            use="optional" default="EUR">
            <annotation>
               <documentation xml:lang="de">
                  Währungscode, Default="EUR".
               </documentation>
            </annotation>
         </attribute>
      </extension>
   </simpleContent>
</complexType>
<element name="Parameter">
   <annotation>
      <documentation xml:lang="de">
         generische Schlüssel-Wert-Parameter.
      </documentation>
   </annotation>
   <complexType>
      <sequence>
         <element name="Name" type="token">
            <annotation>
               <documentation xml:lang="de">
                  Name des Parameters.
               </documentation>
            </annotation>
         </element>
         <element name="Value">
            <annotation>
               <documentation xml:lang="de">
                  Wert des Parameters.
               </documentation>
            </annotation>
            <complexType>
               <simpleContent>
                  <extension base="anySimpleType">
                     <attribute name="Type" type="NCName" use="optional"
                        default="string">
                        <annotation>
                           <documentation xml:lang="de">
                              XML-Typ des Parameterwerts
                              (Default=string).
                           </documentation>
                        </annotation>
                     </attribute>
                  </extension>
               </simpleContent>
            </complexType>
         </element>
      </sequence>
```

```
    </complexType>
</element>
```

9.3.2.1.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
HKDResponse» OrderData	ebics:HKDResponse» OrderDataType (komplex)	1	Auftragsdaten für Auftragsart HKD	- (komplex)
PartnerInfo	ebics:PartnerInfoType (komplex)	1	Kundendaten	- (komplex)
AddressInfo	ebics:AddressInfoType	1	Adressinformationen des Kunden	- (komplex)
Name (in AddressInfo)	ebics:NameType (→normalizedString)	0..1	Name des Kunden	„Hans Mustermann“
Street	ebics:NameType (→normalizedString)	0..1	Straße und Hausnummer des Kunden	„Musterstr. 1“
PostCode	token	0..1	Postleitzahl des Kunden	„D-12345“
City	ebics:NameType (→normalizedString)	0..1	Stadt des Kunden	„Musterstadt“
Region	ebics:NameType (→normalizedString)	0..1	Region/Bundesland/ Bundesstaat des Kunden	„Musterregion“
Country	ebics:NameType (→normalizedString)	0..1	Land des Kunden	„Musterland“
BankInfo	ebics:BankInfoType (komplex)	1	Informationen zur Kreditinstitutsanbindung des Kunden	- (komplex)
HostID	ebics:HostIDType (→token, maxLength=35	1	EBICS Banksystem-ID	„EBIXHOST“ EBICS-HOST4711” “BANKFRPP
Parameter	Referenz auf globales Element (komplex)	0..∞	Struktur für generische Schlüssel-Wert-Parameter mit optionaler Typangabe	- (komplex)
AccountInfo	ebics:AccountType (komplex)	0..∞	Informationen zu den Konten des Kunden. Ein Konto wird in der HKD Response nur dann aufgeführt, wenn der Kunde dafür eine Bereitstellungs-vereinbarung für Kontoauzüge hat ODER wenn mindestens ein Teilnehmer des Kunden dafür eine Unterschrifts-berechtigung hat. Der Inhaber des Kontos kann	- (komplex)

			dabei ein anderer Kunde sein als der Kunde, für den HKD abgerufen wird.	
AccountInfo» @Currency	ebics:CurrencyBaseType (→token, length=3)	0..1	Währungscode gemäß ISO 4217 für das angegebene Konto; falls nicht angegeben, wird „EUR“ angenommen	„EUR“
Description	ebics:Account» DescriptionType (→normalizedString)	0..1	textuelle Beschreibung des Kontos	„Girokonto“
AccountInfo@ID	ebics:AccountIDType (→token, maxLength=64)	1	eindeutiger Identifikations-code des Kontos	„ABCDEFG» abcdefg» 1234567890“
-	-	1..2	Angaben zur Kontonummer: AccountNumber und/oder NationalAccountNumber	-
AccountNumber	ebics:AccountNumber» Type (→token, maxLength=40, pattern="\d{3,10}| ([A-Z]{2}\d{2} [A-Za-z0-9]{3,30})")	1	Kontonummer (deutsches Format oder international als IBAN)	„123456789“
AccountNumber» @international	boolean	0..1	Ist die Kontonummer im nationalen=deutschen (false, Default) oder im internationalen=IBAN-Format (true) angegeben?	„false“
National» AccountNumber	ebics:National» AccountNumberType (→token, maxLength=40)	1	Kontonummer im freien Format (für nationale Kontonummer, die weder den deutschen noch den internationalen Vorgaben entsprechen)	„12345678901 23456“
National» Account» Number@format	token	1	Beschreibung für Format der Kontonummer	„other“
-	-	0..2	Angaben zur Bankleitzahl: BankCode und/oder NationalBankCode	-
BankCode	ebics:BankCodeType (→token, maxLength=11, pattern="\d{8}| ([A-Z]{6}[A-Z0-9]{2} ([A-Z0-9]{3})?)")	1	Bankleitzahl (deutsches Format oder international als SWIFT-BIC)	„50010070“
BankCode» @international	boolean	0..1	Ist die Bankleitzahl im nationalen=deutschen (false, Default) oder im internationalen=SWIFT-	„false“

			BIC-Format (true) angegeben?	
BankCode» @Prefix	ebics:BankCodePrefix» Type (→token, length=2)	0..1	nationales Bankleitzahl-Präfix	„DE“
NationalBank» Code	ebics:National» BankCodeType (→token, maxLength=30)	1	Bankleitzahl im freien Format. (weder deutsches Format noch SWIFT-BIC)	„123456789012“
NationalBank» Code@format	token	1	Beschreibung für Format der Bankleitzahl	“other”
AccountHolder	ebics:AccountHolder» Type (→normalizedString)	0..1	Name des Kontoinhabers	„Hans Mustermann“
UsageOrder» Types	ebics: UsageOrderType	0..1	Beschränkungen hinsichtlich ausführbarer Geschäftsvorfälle (BTF) für das angegebene Konto; falls nicht angegeben, gibt es keine Beschränkungen für das angegebene Konto; falls die Liste leer ist, ist das angegebene Konto für keinen Geschäftsvorfall freigegeben	
Service	ebics:RestrictedServiceType Details zu dieser Struktur siehe Kapitel 8.3.6	0.. ∞	Art von Geschäftsvorfall, identifiziert durch die Service-Struktur	
OrderInfo	ebics:OrderInfoType (komplex)	1..∞	Informationen zu den dem Kunden zugewiesenen administrativen Auftragsarten	- (komplex)
AdminOrderType	ebics:OrderTBaseType (→token, length=3, pattern="[A-Z0-9]{3}")	1	dem Kunden zugewiesene administrative Auftragsart	„“BTD”, BTU” (Senden bzw. Abholen)
Description	ebics:Order» DescriptionType (→normalizedString, maxLength=128)	1	textuelle Beschreibung des Geschäftsvorfalls	““SEPA-Überweisung“
NumSig» Required	nonNegativeInteger	0..1	Anzahl erforderlicher EUs für den Auftrag; Default=0, falls nicht angegeben	2
UserInfo	ebics:UserInfoType (komplex)	1..∞	Teilnehmerinformationen	- (komplex)
UserID	ebics:UserIDType (→token, maxLength=35, pattern="[a-zA-Z0-9,=]{1,35})	1	Teilnehmer-ID	„USER01“
UserID@Status	ebics:UserStatusType (→nonNegativeInteger,	1	Status des Teilnehmers: 1: Bereit: Teilnehmer ist	1

	maxInclusive=99)		freigeschaltet 2: Neu: Initialzustand nach Einrichtung des Teilnehmers für EBICS („eingerichtet“) 3: Teilweise Initialisiert(INI): Teilnehmer hat INI-Datei gesendet, aber noch kein HIA 4: Teilweise Initialisiert(HIA): Teilnehmer hat HIA-Auftrag gesendet, aber noch kein INI 5: Initialisiert: Teilnehmer hat HIA-Auftrag und INI-Datei gesendet 6: Gesperrt (mehrmalige Fehlversuche), neue Initialisierung über INI und HIA ist möglich 7: Neu_FTAM: wird nicht mehr verwendet 8: Gesperrt (durch SPR-Auftrag des Kunden) , neue Initialisierung über INI und HIA ist möglich 9: Gesperrt (durch Bank) , neue Initialisierung über INI und HIA ist nicht möglich, Entsperrung nur durch die Bank	
Name (in UserInfo)	ebics:NameType (→normalizedString)	0..1	Name des Teilnehmers	„Hans Mustermann“
Permission	ebics:PermissionType (komplex)	1..∞	Informationen zu den Berechtigungen des Teilnehmers	- (komplex)
Permission» @Authorisa» tionLevel	ebics:Authorisation» LevelType (→token, length=1: "E", "A", "B", "T")	0..1	Unterschriftsklasse, für die der Teilnehmer berechtigt ist: „E“=Einzelunterschrift, „A“=Erstunterschrift, „B“=Zweitunterschrift, „T“=Transportunterschrift. Nicht anzugeben bei Download-Auftragsarten	„A“
Admin	ebics:OrderTBaseType (→token, length=3, pattern="[A-Z0-9]{3}")	1	Administrative Auftragsart, die dem Teilnehmer zugeordnet ist.	””BTD”, “BTU” (für Abholen, Senden)
Service	ebics:RestrictedServic eType Details zu dieser Struktur siehe Kapitel 8.3.6	0.. ∞	Art von Geschäftsvorfall, identifiziert durch die Service-Struktur	
AccountID	ebics:AccountIDType (→token, maxLength=64)	0.. 1	Verweis auf den Identifikationscode eines berechtigten Kontos (Gültigkeit des Verweises wird vom EBICS	„ABCDEFG» abcdefg» 1234567890“

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

			XML-Schema erzwungen)	
MaxAmount	ebics:AmountType (→ebics:AmountValue» Type →decimal, totalDigits=24, fractionDigits=4)	0..1	Betragshöchstgrenze, bis zu der die Unterschrifts-berechtigung des Teil-nehmers gültig ist	5000.00
MaxAmount» @Currency	ebics:CurrencyBaseType (→token, length=3)	0..1	Währung gemäß ISO 4217 des Maximalbetrags; falls nicht angegeben, wird „EUR“ angenommen	„EUR“

Bemerkung zur Klarstellung:

Die Vergabe der Kontoberechtigungen für den einzelnen Teilnehmer erfolgt über das Element `UserInfo/Permission` in folgender Weise:

Falls innerhalb von `UserInfo/Permission` kein Element `AccountID` übergeben wird, gelten die in `UserInfo/Permission` übergebenen Auftragsarten automatisch für *alle* Konten des zugehörigen Kunden.

Wird hingegen in `UserInfo/Permission` ein Element `AccountID` übergeben, so gelten die im zugehörigen Element `UserInfo/Permission/OrderTypes` übergebenenBerechtigungen ausschließlich für die über `AccountID` referenzierten Account-IDs.

9.3.2.1.4 Beispiel-XML

```
<?xml version="1.0" encoding="UTF-8"?>
<HKDResponseOrderData
 xmlns="urn:org:ebics:H005"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_orders_H005.xsd">
 <PartnerInfo>
   <AddressInfo>
     <Name>Hans Mustermann</Name>
     <Street>Musterstr. 1</Street>
     <PostCode>D-12345</PostCode>
     <City>Musterstadt</City>
     <Region>Musterregion</Region>
     <Country>Musterland</Country>
   </AddressInfo>
   <BankInfo>
     <HostID>EBIXHOST</HostID>
   </BankInfo>
   <AccountInfo ID="accid01" Currency="EUR" Description="Girokonto">
     <AccountNumber international="false">123456789</AccountNumber>
     <BankCode international="false" Prefix="DE">50010070</BankCode>
     <AccountHolder>Hans Mustermann</AccountHolder>
   </AccountInfo>
   <OrderInfo>
     <AdminOrderType>BTD</AdminOrderType>
     <ServiceName>
         <ServiceName>EOP</ServiceName>
         <MsgName>camt.053</MsgName>
     </ServiceName>
     <Description>ISO20022ISO20022 camt-Kontoauszug</Description>
   </OrderInfo>
   <OrderInfo>
     <AdminOrderType>BTU</AdminOrderType>
     <ServiceName>
```

```
        <ServiceName>SCT</ServiceName>
        <MsgName>pain.001</MsgName>
    </ServiceName>
    <Description>SEPA-Überweisung</Description>     <NumSigRequired>2</NumSigRequired>   </OrderInfo>
 </PartnerInfo>
 <UserInfo>
   <UserID Status="1">USER01</UserID>
   <Permission>
    <AdminOrderType>BTD</AdminOrderType>
     <ServiceName>
       <ServiceName>EOP</ServiceName>
       <MsgName>camt.053</MsgName>
     </ServiceName>
   </Permission>
 </UserInfo>
 <UserInfo>
   <UserID Status="1">USER02</UserID>
   <Permission AuthorisationLevel="A">
   <AdminOrderType>BTD</AdminOrderType>
     <ServiceName>
       <ServiceName>SDD</ServiceName>
       <MsgName>pain.008</MsgName>
     </ServiceName>
     <AccountID>accid01</AccountID>
     <MaxAmount Currency="EUR">6000.00</MaxAmount>
   </Permission>
   <Permission>
   <AdminOrderType>BTD</AdminOrderType>
     <ServiceName>
       <ServiceName>EOP</ServiceName>
       <MsgName>camt.053</MsgName>
     </ServiceName>
   </Permission>
 </UserInfo>
</HKDResponseOrderData>
```

9.4 HTD (Kunden- und Teilnehmerinformationen des Teilnehmers abrufen)

Mit HTD kann der Teilnehmer bankseitig gespeicherte Informationen zu seinem Unternehmen und zu sich selbst als Teilnehmer abrufen; im Unterschied zu HKD erfährt er aber nichts über die restlichen Teilnehmer des Unternehmens.

HTD ist eine administrative Auftragsart vom Typ „Download“.

9.4.1 HTD-Request

Der HTD-Request enthält keine spezifischen Daten, die über die in der allgemeinen Transaktionsbeschreibung genannten hinausgehen.

9.4.2 HTD-Response

Ausprägung der (dekodierten & entschlüsselten & dekomprimierten) `OrderData` (Auftragsdaten) für HTD: `HTDResponseOrderData`

9.4.2.1.1 XML-Schema (grafische Darstellung)

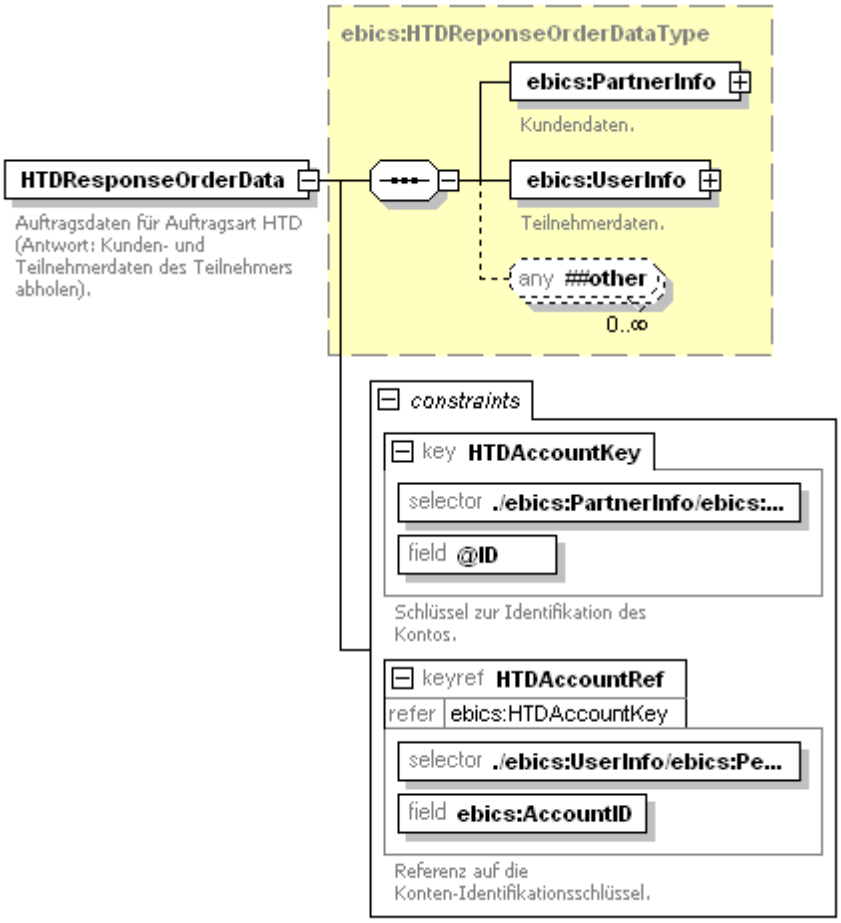


Abbildung 98: HTDResponseOrderData

9.4.2.1.2 XML-Schema (textuelle Darstellung)

```
 <element name="HTDResponseOrderData" type="ebics:HTDReponseOrderDataType"
substitutionGroup="ebics:EBICSOrderData">
   <annotation>
     <documentation>Auftragsdaten für Auftragsart HTD (Antwort: Kunden- und Teilnehmerdaten des
Teilnehmers abholen).</documentation>
   </annotation>
   <key name="HTDAccountKey">
     <annotation>
       <documentation xml:lang="de">Schlüssel zur Identifikation des Kontos.</documentation>
     </annotation>
     <selector xpath="./ebics:PartnerInfo/ebics:AccountInfo"/>
     <field xpath="@ID"/>
   </key>
   <keyref name="HTDAccountRef" refer="ebics:HTDAccountKey">
     <annotation>
       <documentation xml:lang="de">Referenz auf die Konten-Identifikationsschlüssel.</documentation>
     </annotation>
     <selector xpath="./ebics:UserInfo/ebics:Permission"/>
```

```
      <field xpath="AccountID"/>
    </keyref>
  </element>
  <complexType name="HTDReponseOrderDataType">
    <annotation>
      <documentation>Datentyp für Auftragsdaten für Auftragsart HTD (Antwort: Kunden- und Teilnehmerdaten
des Teilnehmers abholen).</documentation>
    </annotation>
    <sequence>
      <element name="PartnerInfo" type="ebics:PartnerInfoType">
        <annotation>
          <documentation xml:lang="de">Kundendaten.</documentation>
        </annotation>
      </element>
      <element name="UserInfo" type="ebics:UserInfoType">
        <annotation>
          <documentation xml:lang="de">Teilnehmerdaten.</documentation>
        </annotation>
      </element>
      <any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
    </sequence>
  </complexType>
```

9.4.2.1.3 Bedeutung der XML-Elemente/-Attribute

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
HTDResponse» OrderData	ebics:HTDResponse» OrderDataType (komplex)	1	Auftragsdaten für Auftragsart HTD	- (komplex)
PartnerInfo	ebics:PartnerInfoType (komplex)	1	Kundendaten	- (komplex)
UserInfo	ebics:UserInfo (komplex)	1	Teilnehmerinformationen	- (komplex)

Für die restlichen XML-Elemente & -Attribute: siehe Auftragsart HKD (Kapitel 9.3.2.1.3).
Ebenso gilt die dort formulierte Klarstellung zur Vergabe der Kontoberechtigungen.

9.4.2.1.4 Beispiel-XML

```
<?xml version="1.0" encoding="UTF-8"?>
<HTDResponseOrderData
 xmlns="urn:org:ebics:H005"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_orders_H005.xsd">
 <PartnerInfo>
   <AddressInfo>
     <Name>Hans Mustermann</Name>
     <Street>Musterstr. 1</Street>
     <PostCode>D-12345</PostCode>
     <City>Musterstadt</City>
     <Region>Musterregion</Region>
     <Country>Musterland</Country>
   </AddressInfo>
   <BankInfo>
     <HostID>EBIXHOST</HostID>
```

```
    </BankInfo>
    <AccountInfo ID="accid01" Currency="EUR" Description="Girokonto">
      <AccountNumber international="false">123456789</AccountNumber>
      <BankCode international="false" Prefix="DE">50010070</BankCode>
      <AccountHolder>Hans Mustermann</AccountHolder>
    </AccountInfo>
    <OrderInfo>
      <AdminOrderType>BTD</AdminOrderType>
        <Service>
             <ServiceName>EOP</ServiceName>
             <Scope>DE</Scope>
             <MsgName>mt940</MsgName>
        </Service>
      <Description>Abholen SWIFT-Tagesauszüge</Description>
    </OrderInfo>
    <OrderInfo>
      <AdminOrderType>BTU</AdminOrderType>
        <Service>
             <ServiceName>SCT</ServiceName>
             <MsgName>pain.001</MsgName>
        </Service>
      <Description>SEPA-Überweisung senden</Description>
      <NumSigRequired>2</NumSigRequired>
    </OrderInfo>
  </PartnerInfo>
  <UserInfo>
    <UserID Status="1">USER01</UserID>
    <Permission>
      <AdminOrderType>BTD</AdminOrderTypeAdmin>
        <Service>
             <ServiceName>EOP</ServiceName>
             <Scope>DE</Scope>
             <MsgName>camt.053</MsgName>
        </Service>
    </Permission>
    <Permission AuthorisationLevel="A">
      <AdminOrderTypeAdmin>BTU</AdminOrderTypeAdmin>
        <Service>
             <ServiceName>DCT</ServiceName>
             <Scope>BIL</Scope>
             <MsgName>abcd1123</MsgName>
        </Service>
      <AccountID>accid01</AccountID>
      <MaxAmount Currency="EUR">6000.00</MaxAmount>
    </Permission>
  </UserInfo>
</HTDResponseOrderData>
```

9.5 HEV (Unterstützte EBICS-Versionen abrufen)

Mit HEV kann sich der Teilnehmer über die bankseitig unterstützen EBICS-Versionen informieren. Die Antwort der Bank enthält eine Liste der unterstützten EBICS-Versionen und der Version des dazu gültigen Schemas.

HEV ist eine administrative Auftragsart vom Typ „Download“.

9.5.1 HEV-Request

Der HEV-Request ruft lediglich die EBICS-Versionen, die von der Bank unterstützt werden, ab. Dieser Request kann auch von noch nicht initialisierten Teilnehmern durchgeführt werden und ist daher ohne Authentifikationssignatur möglich.
Es wird nur folgende Information verpflichtend mit dem HEV-Request übertragen:

- Host-ID des EBICS Bankrechners

Der Request wird abgewiesen und der Return Code EBICS_INVALID_HOST_ID zurückgegeben, wenn die übertragene HostID bankseitig unbekannt ist.
Hinweis: Dieser Return Code ist nur für den HEV-Request zulässig.

9.5.2 HEV-Response

Die Response liefert:

- technischer Returncode
- technischer Reporttext

Der Wertebereich dieser beiden Felder ist dem Dokument „EBICS Anhang 1 Return Codes“ zu entnehmen. Da dem Banksystem zum Zeitpunkt des HEV-Requests die EBICS-Version des Kundensystems noch nicht bekannt ist, befüllt es die Felder mit den Werten, die in der höchsten bankseitig unterstützten EBICS-Version definiert sind. Da in diesem Request kein Sprach-Attribut verfügbar ist, wird der Reporttext stets in Englisch übermittelt.

Liste der vom Banksystem unterstützten EBICS-Versionen und der Name der dazu gültigen Schema-Version

9.5.3 Schema für HEV-Request / HEV-Response

Für HEV-Request und HEV-Response wird ein neutrales -von den aktuell von der Bank unterstützten EBICS-Versionen unabhängiges - Schema ebics_hev.xsd, abrufbar unter http://www.ebics.org (Rubrik „technical-information - verwendet. Das Schema enthält Request und Response. Beim Request ist ebicsHEVRequest und bei der Response ebicsHEVResponse zu füllen.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

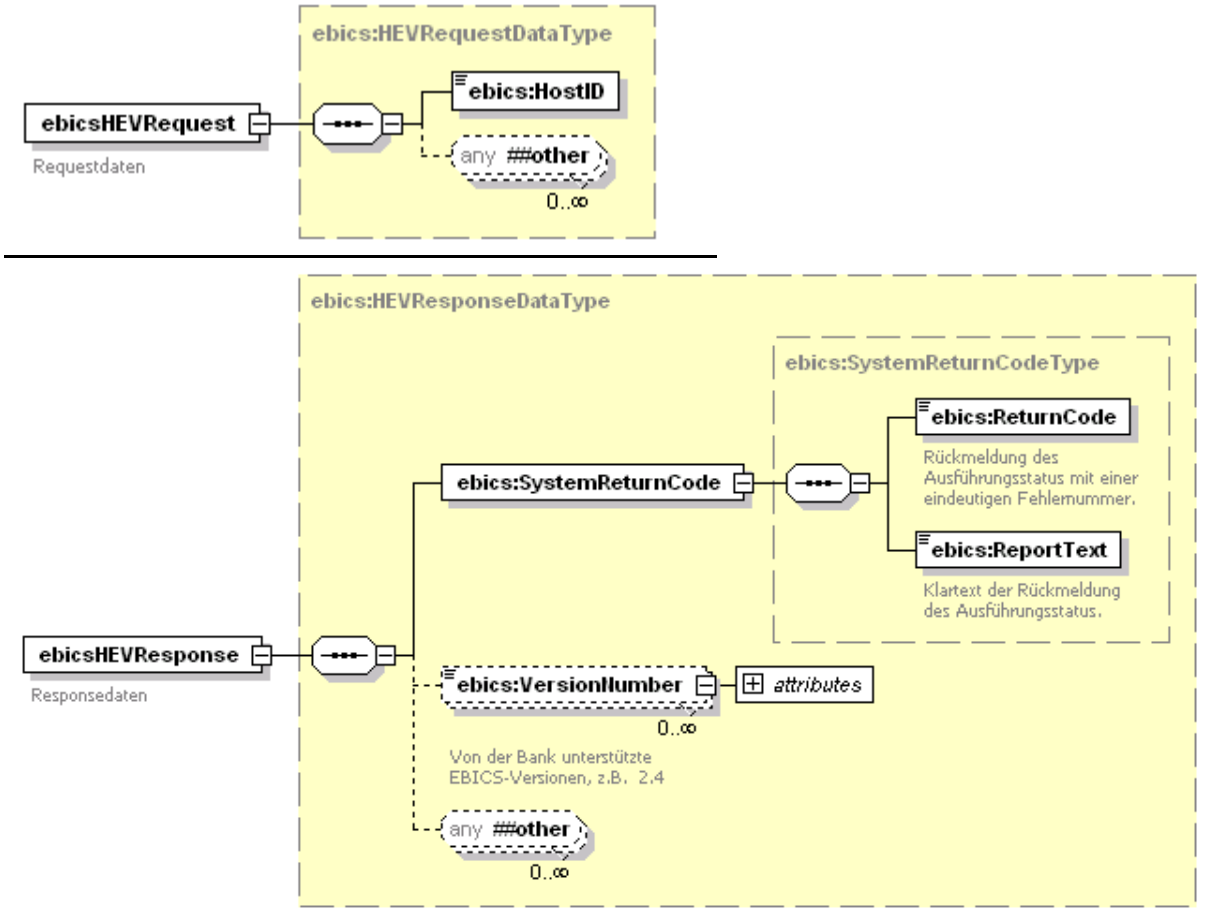


Abbildung 99: HEVRequest / HEVResponse

Die textuelle Darstellung des Schemas ebics_hev.xsd findet sich unter www.ebics.org oder www.ebics.de.

9.5.3.1 Bedeutung der XML-Elemente /-Attribute der HEV-Response

XML-Element/ -Attribut	Datentyp	#	Bedeutung	Beispiel
`System ReturnCode`	`ebics:SystemReturnCodeType` (komplex)	1	Technischer Return-Code und Fehlertext (in Englisch)	Wertebereich für Code gemäß Dokument „Anhang 1 der EBICS-Spezifikation

				ReturnCodes“
VersionNumber	ebics:VersionNumberType (komplex) (→token, length=5, pattern="[0-9]{2}[.][0-9]{2}"	0..∞	Vom Banksystem unterstützte EBICS-Version	03.00 (entspricht also 3.0)
ProtocolVersion	ebics:ProtocolVersionType (→token, length=4)	1	Die zur unterstützenden EBICS-Version gültige Schemaversion	H005

9.5.3.2 Beispiel-XML für eine HEV-Response

```
<?xml version="1.0" encoding="UTF-8"?>
<ebics:ebicsHEVResponse xsi:schemaLocation="http://www.ebics.org/H000 ebics_hev.xsd"
xmlns:ebics="http://www.ebics.org/H000" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
	<ebics:SystemReturnCode>
		<ebics:ReturnCode>000000</ebics:ReturnCode>
		<ebics:ReportText>EBICS_OK</ebics:ReportText>
	</ebics:SystemReturnCode>
	<ebics:VersionNumber ProtocolVersion="H004">02.50</ebics:VersionNumber>
	<ebics:VersionNumber ProtocolVersion="H005">03.00</ebics:VersionNumber>
</ebics:ebicsHEVResponse>
```

10 EBICS-Kundenprotokoll (HAC)

10.1 Vorbemerkungen

1. Die folgenden Festlegungen für die Belegung der pain.002-Nachricht (ISO Edition 2009) beziehen sich nur auf das EBICS-Kundenprotokoll (HAC: H Kennzeichen für eine administrative Auftragsart; A = Acknowledgement; C = Customer).
   HAC beschreibt alle Aktionen und Ergebnisse, die beim Senden, Abholen und Unterschreiben von Dateien vorkommen und kann zusätzlich Informationen über den Inhalt des Auftrags / der Datei geben (Dateianzeige).

2. Die Zielvorgabe war, dafür einen internationalen Standard (Schema) zu nutzen; man entschied sich für die ISO20022-Nachricht pain.002.

   - Denn zurzeit ist die pain.002-Nachricht die beste Alternative, auch wenn sie nicht ideal ist.
   - Als langfristige Lösung sollte eine eigens für diesen Zweck spezifizierte ISO-Nachricht angestrebt werden.

3. Beim Abholen von HAC erhält der Kunde alle Statusinformationen seit letzter Abholung von HAC. Das Kundenprotokoll enthält alle Aktionen und Statusinformationen der PartnerID (Kunden-ID). Dafür kommt die Elementgruppe <OrgnlPmtInfAndSts> 1 bis n Mal vor. Jede Wiederholung ist ein Protokollschritt.

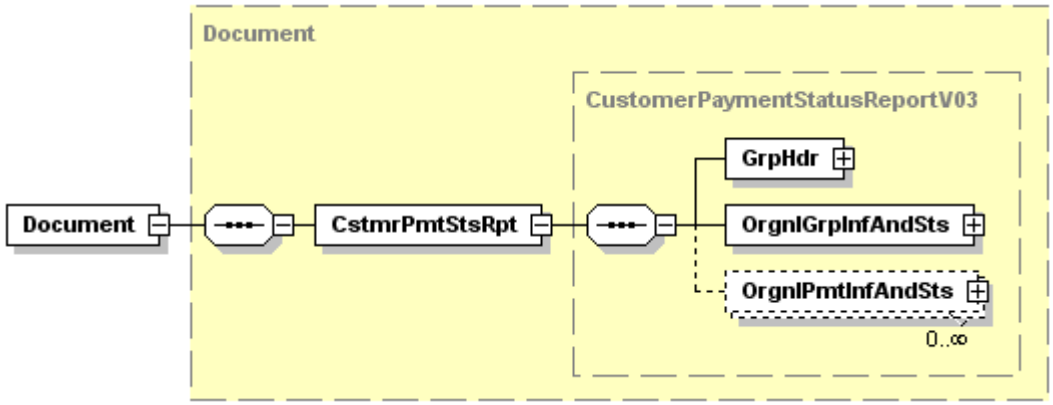
   Hinweis: Eigentlich ist diese Elementgruppe optional. Eine wesentliche Regel für das EBICS-Kundenprotokoll ist, dass die Elementgruppe <OrgnlPmtInfAndSts> mindestens einmal vorkommt.

4. Im Gegensatz zum “menschenlesbaren” alten deutschen Kundenprotokoll PTK liegt der Schwerpunkt beim XML-basierten HAC auf der automatischen Auswertung (geeignete Aufbereitung durch das Kundensystem ist notwendig)

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

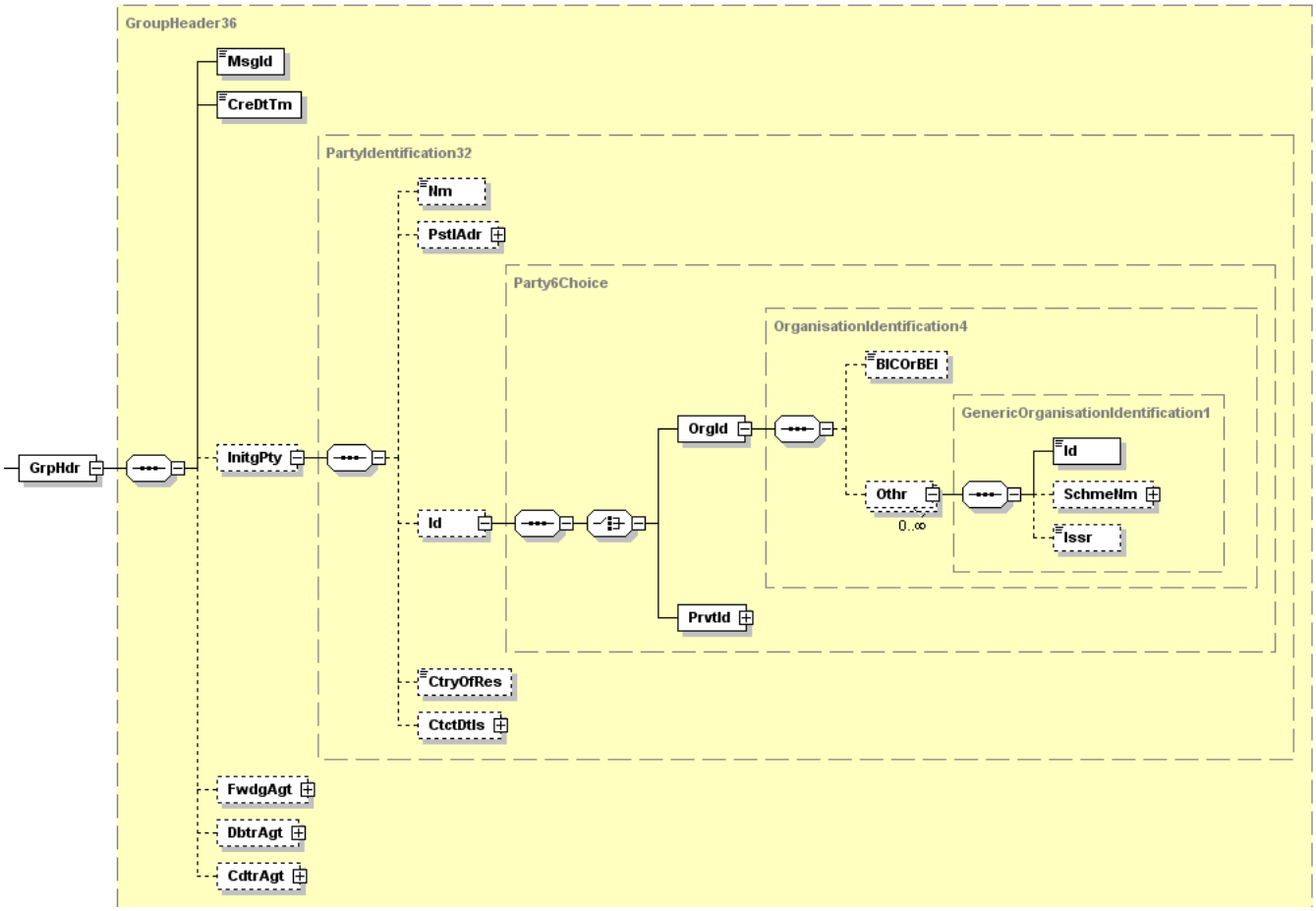
10.2 Belegung von pain.002 für HAC

Gesamte Nachricht – Allgemeiner Überblick:



10.2.1 Belegung der Elementgruppe Group Header

Elementgruppe <GrpHdr> - Überblick:



Festlegungen für die Belegung:

Diese Elementgruppe kommt genau einmal vor.

Alle Elemente in <GrpHdr>, die in dieser Liste nicht erwähnt werden, werden nie in HAC verwendet!

Name	XML Tag	Regeln für HAC
MessageIdentification	<MsgId>	Verpflichtend gemäß ISO-Schema (und somit auch in HAC)
CreationDateTime	<CreDtTm>	Verpflichtend gemäß ISO-Schema (und somit auch in HAC): Erstellungsdatum/-uhrzeit der pain.002-Nachricht Belegungsregel: Die Darstellung von <CreDtTm> muss der im EBICS-Schema entsprechen (siehe EBICS-Spezifikation Kapitel 2.3). Beispiel: 2019-05-13T10:00:00Z
InitiatingParty	<InitPty><Id><OrgId>	Elementgruppe für die Übertragung der HostID (optional im ISO-Schema, jedoch verpflichtend in HAC) HostId (technische ID des EBICS-Bankrechners) ist in <Othr> zu hinterlegen.

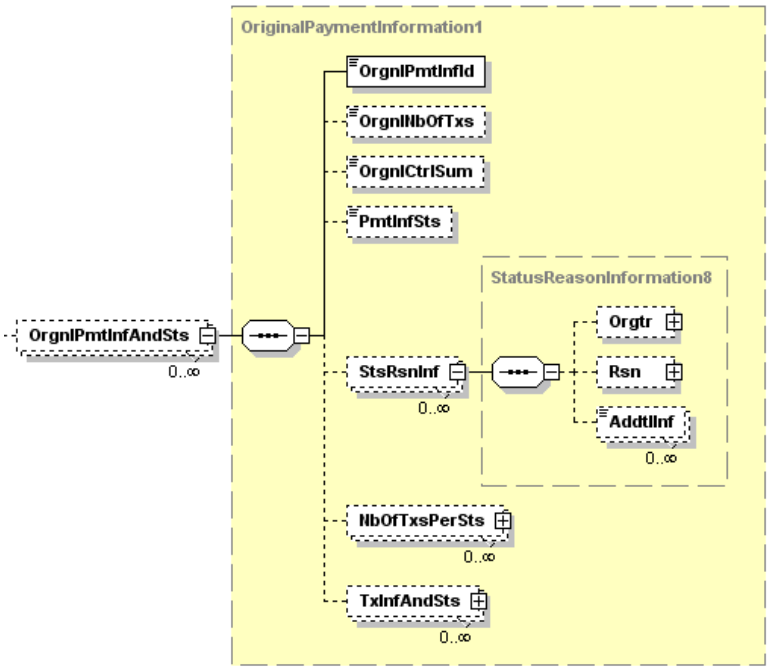
10.2.2 Belegung der Elementgruppe Original Group Information and Status

Wie in ISO kommt diese Elementgruppe genau einmal vor.
Es gibt zwei Pflichtfelder: <OrgnlMsgId> und <OrgnlMsgNmId> (beide vom Typ Max35Text). Sie legen Gruppierungsinformationen der Originalnachricht dar. Da HAC Informationen über eine Sammlung von verschiedenen Aufträgen und Aktionen eines Kunden (einer PartnerID) darstellt, werden diese Elemente mit der Konstante „EBICS“ belegt.

10.2.3 Belegung der Elementgruppe Original Payment Information and Status

Diese Elementgruppe ist in ISO optional, in HAC muss sie jedoch mindestens einmal vorkommen!

Elementgruppe <OrgnlPmtInfAndSts> - Überblick:



Festlegungen für die Belegung:
Alle Elemente in <OrgnlPmtInfAndSts>, die in dieser Liste nicht erwähnt werden, werden nie in HAC verwendet!

Name	XML Tag	Regeln für HAC
OriginalPaymentInformationIdentification	<OrgnlPmtInfId>	Verpflichtend im ISO-Schema (und somit auch in HAC). Informationen über die Art der Aktion; siehe Kapitel 10.2.3.1
StatusReasonInformation	<StsRsnInf>	[0..unbounded] im ISO-Schema, in HAC kommt diese Elementgruppe genau einmal vor! Informationen über den Auftrag (dazu gehören alle beteiligten Teilnehmer und der Timestamp), Ergebnis der Aktion und Daten für die Dateianzeige; siehe Kapitel 10.2.3.2

10.2.3.1 Art der Aktion

Die Art der Aktion wird im Element <OrgnlPmtInfId> hinterlegt.
Der folgende Wertebereich ist dafür definiert:

HAC – Art der Aktion	Bedeutung	Wertebereich (Max35Text); für EBICS definierter Wert
„Art der Aktion“ *Übertragung*		
Datei wurde zur Bank übertragen	Jede Art von Upload außer dem Upload einer EU-Datei	FILE_UPLOAD
Datei wurde von der Bank abgeholt	Jede Art von Download außer Download einer EU-Datei	FILE_DOWNLOAD

HAC – Art der Aktion	Bedeutung	Wertebereich (Max35Text); für EBICS definierter Wert
Elektronische Unterschrift wurde zur Bank übertragen	Upload einer EU-Datei innerhalb des VEU-Verfahrens (administrative Auftragsart HVE)	ES_UPLOAD
Elektronische Unterschrift wurde von der Bank abgeholt	Download einer EU-Datei (für eine spätere Version reserviert)	ES_DOWNLOAD
„Art der Aktion“ *Nachverarbeitung (VEU etc.)*		
Unterschriftsprüfung	Der Bankrechner überprüft die übertragene EU	ES_VERIFICATION Code, wenn **keine** VEU eingesetzt wird.
Weiterleitung zur VEU	Die Datei wird im VEU-Verfahrengespeichert und wartet auf die erforderlichen EU’s	VEU_FORWARDING
VEU Unterschriftsprüfung	Der Bankrechner überprüft die übertragenen EUs innerhalb des VEU-Verfahrens	VEU_VERIFICATION Code, wenn die VEU eingesetzt wird.
Abschluss der VEU Unterschriftsprüfung	Der Prüfprozess im VEU-Verfahren wurde abgeschlossen, da die letzte für die Autorisierung einer Zahlung erforderliche Unterschrift nun erfolgreich geprüft wurde **oder** die übertragene Unterschrift nicht korrekt war.	VEU_VERIFICATION_END
Stornierung eines VEU-Auftrags	Der Auftrag wurde durch einen autorisierten Teilnehmer innerhalb des VEU-Verfahrens storniert	VEU_CANCEL_ORDER
„Art der Aktion“ *Zusatzinformationen*	Bereitstellung von zusätzlichen Informationen der Bank an den Kunden mittels HAC (in <AddtlInf>)	ADDITIONAL

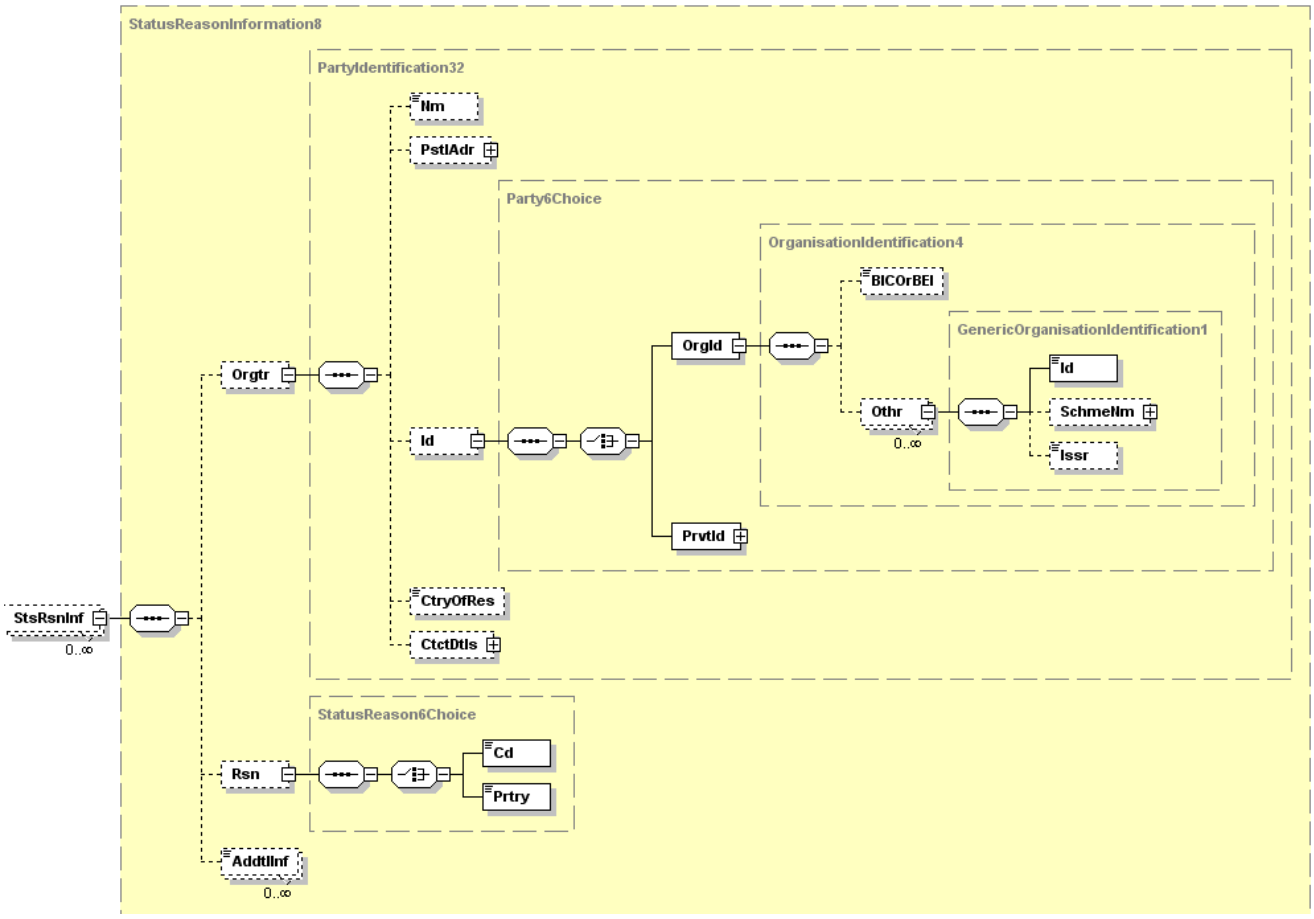
Eines der beiden Ende-Kennzeichen ist anzugeben um kenntlich zu machen, dass kein weiterer EBICS-Protokollschritt für den angegebenem Auftrag mehr folgt. Für diese Arten der Aktion ist kein Reason-Code (im „Ende der Aktion“) zulässig. Es ist lediglich ein Kennzeichen.

„Art der Aktion“ *Ende-Kennzeichen*		
HAC Ende des Auftrags	Mit Referenzierung auf eine Auftragsnummer dient dies als Ende-Kennzeichen für den Auftrag (kein weiterer Protokollschritt zu diesem Auftrag folgt)	ORDER_HAC_FINAL

10.2.3.2 Ergebnis der Aktion

Das Ergebnis der Aktion wird in der Elementgruppe Status Reason Information hinterlegt.

Elementgruppe <StsRsnInf> - Überblick:



Festlegungen für die Belegung:
Alle Elemente in <StsRsnInf>, die in dieser Liste nicht erwähnt werden, werden nie in HAC verwendet!

Name	XML Tag	Regeln für HAC
Originator / Name	<Orgtr><Nm>	Name des Kunden

Name	XML Tag	Regeln für HAC
Originator / Identification / OrganisationId entification /Other	<Orgtr><Id><OrgId><Othr> 1) <Id> 2)<SchmeNm><Prtry> Hinweis: Länge der Id Hinweis: Länge der ID-Namen ist Max35Text	Elementgruppe (kommt 0 bis n-mal vor) für verschiedene Identifikationscodes mit folgender Bedeutung: Identifikations-ID Über das Kennzeichen in <SchmeNm><Prtry> ist die Art von ID im Element <Id> zu identifizieren (die meisten der Kennzeichen sind bereits für EBICS definiert; in diesem Fall werden sie auch in dieser Notation verwendet): ➢ UserID (Teilnehmer-ID) ➢ PartnerID (Kunden-ID) ➢ SystemID (ID des technischen Teilnehmers) ➢ OrderID (Auftragsnummer) ➢ AdminOrderTypeServiceName ➢ Scope ➢ ServiceOption ➢ ContainerType ➢ MsgName ➢ OrderIDRef (Wenn die Aktion sich auf einen anderen Auftrag bezieht, wird die Auftragsnummer des referenzierten Auftrags hiermit geliefert) ➢ AdminOrderTypeRef (Wenn die Aktion sich auf einen anderen Auftrag bezieht, wird die administrative Auftragsart des referenzierten Auftrags hiermit geliefert) – zudem auch die BTF-Parameterwerte (Service) des Auftrags: ➢ ServiceNameRef ➢ ScopeRef ➢ ServiceOptionRef ➢ ContainerTypeRef ➢ MsgNameRef ➢ PartnerIDRef (Wenn die Aktion sich auf einen anderen Auftrag bezieht, der ZUDEM einem anderen Kunden zugeordnet ist, wird dessen Kunden-ID hiermit geliefert) ➢ TimeStamp (Timestamp der Aktion im ISO 8601 Format, analog Kapitel 2.3 der EBICS-Spezifikation) ➢ DataDigest (Hashwert) Hinweis: 1. Alle IDs, die hier beschrieben sind, sind in HAC zu liefern, wenn die entsprechenden Daten auf dem Bankrechner verfügbar sind. 2. Wenn die IDs für die “Ref”-Serviceelemente vorhanden sind (z.B. ServiceNameRef) kann die aktuelle AdminOrderType” nur eine H*-Auftragsart sein. Dann sind die IDs “ServiceName” bis “MsgName” nicht vorhanden.

Name	XML Tag	Regeln für HAC
Reason	<Rsn>	Ergebnis der Aktion, Tabelle siehe Kapitel 10.3. Alle möglichen Ergebnisse der Aktion sind als ISO Reasoncodes definiert (zu belegen in <Cd>). Dieses Element ist für HAC verpflichtend, es sei denn die Art der Aktion <OrgnlPmtInfAndSts><OrgnlPmtInfId> enthält 1) eines der beiden Ende-Kennzeichen (siehe Kapitel 10.2.3.1). In diesem Fall ist die Belegung von <Cd> nicht zulässig. 2) „ADDITIONAL“. In diesem Fall ist die Belegung optional
AdditionalInformation	<AddtlInf>	Wenn eine Dateianzeige erfolgt (Anzeige eines Auszugs/Zusammenfassung des Dateiinhalts) kommt dieses Element 1 bis n-mal vor (siehe Kapitel 10.2.3.3) Weiterer freier Text (Max105Text) ist immer zulässig. Die Nutzung ist optional und beliebig oft wiederholbar (z.B. Nutzung für Freitextinformationen für den Kunden) – da das Element <AdditionalOrderInfo> (Informationen des Kunden zum Auftrag) bis zu 255 Zeichen groß sein kann, sind auch Wiederholungen von <AddtlInf> möglich.

10.2.3.3 Dateianzeige (Verwendung in Deutschland)

Die Daten für die Dateianzeige (Informationen über den Dateiinhalt) werden in <StsRsnInf><AddtlInf> bereitgestellt.
Die Dateianzeige wird mit einem Ende-Kennzeichen geliefert (ORDER_HAC_FINAL); siehe Kapitel 10.2.3.1).
Hinweis: Dies findet auch bei Dateien mit fehlendem Signatur-Flag Anwendung, was bedeutet, dass die Datei nicht per EU, sondern per Datenträgerbegleitzettel autorisiert wird.
Die Eigenart der bestehenden Dateianzeige kann weiterverwendet werden: Pro Zeile der bekannten Dateianzeige gemäß PTK kommt das Element <AddtlInf> einmal vor.

Name	XML Tag	Rules for HAC
StatusReasonInformation / AdditionalInformation	<StsRsnInf><AddtlInf>	Wir nur für bestimmte Daten in Freitextform verwendet: 1. Beispiele für übliche Formate siehe Kapitel , 10.2.3.3.1 und 10.2.3.3.2 2. Hashwert (nur benötigt für die SEPA Containerdatei) 3. Dateianzeige für Dateien ohne besonderes Format

10.2.3.3.1 Beispiel für SEPA

Dateianzeige in HAC für eine pain.001-Nachricht mit drei Payment Information Blöcken:

<StsRsnInf>
<AddtlInf>G U T S C H R I F T E N</AddtlInf>
<AddtlInf>Datei-ID: 4782647268346</AddtlInf>

<AddtlInf>Datum/Zeit: 28.11.2018/09:30:47</AddtlInf>
<AddtlInf>----------</AddtlInf>
<AddtlInf>Sammlerreferenz : 46573264784</AddtlInf>
<AddtlInf>Bank-Code : WELADEDD</AddtlInf>
<AddtlInf>Kontonummer : DE44300500000054627452</AddtlInf>
<AddtlInf>Auftraggeberdaten : XXX</AddtlInf>
<AddtlInf>Anzahl der Zahlungssaetze : 187</AddtlInf>
<AddtlInf>Summe der Betraege (EUR) : 68.672,00</AddtlInf>
<AddtlInf>Ausfuehrungstermin : 01.12.2018</AddtlInf>
<AddtlInf>----------</AddtlInf>
<AddtlInf>Sammlerreferenz : 46573264783</AddtlInf>
<AddtlInf>Bank-Code : WELADEDD</AddtlInf>
<AddtlInf>Kontonummer : DE44300500000054627452</AddtlInf>
<AddtlInf>Auftraggeberdaten : YYY</AddtlInf>
<AddtlInf>Anzahl der Zahlungssaetze : 165</AddtlInf>
<AddtlInf>Summe der Betraege (EUR) : 354.378,00</AddtlInf>
<AddtlInf>Ausfuehrungstermin : 03.12.2018</AddtlInf>
<AddtlInf>----------</AddtlInf>
<AddtlInf>Sammlerreferenz : 46573264782</AddtlInf>
<AddtlInf>Bank-Code : WELADEDD</AddtlInf>
<AddtlInf>Kontonummer : DE30300500000035351767</AddtlInf>
<AddtlInf>Auftraggeberdaten : XXX</AddtlInf>
<AddtlInf>Anzahl der Zahlungssaetze : 34</AddtlInf>
<AddtlInf>Summe der Betraege (EUR) : 45.100,20</AddtlInf>
<AddtlInf>Ausfuehrungstermin : 01.12.2018</AddtlInf>
</StsRsnInf>
….

10.2.3.3.2 Beispiel für den SEPA-Container

Dateianzeige in HAC für einen Container mit zwei pain.001-Nachrichten:

<StsRsnInf>
<AddtlInf>G U T S C H R I F T E N</AddtlInf>
<AddtlInf>Datei-ID: 4782647268346</AddtlInf>
<AddtlInf>Datum/Zeit: 28.11.2018/09:30:47</AddtlInf>
<AddtlInf>----------</AddtlInf>
<AddtlInf>Sammlerreferenz : 46573264784</AddtlInf>
<AddtlInf>Bank-Code : WELADEDD</AddtlInf>
<AddtlInf>Kontonummer : DE44300500000054627452</AddtlInf>
<AddtlInf>Auftraggeberdaten : XXX</AddtlInf>
<AddtlInf>Anzahl der Zahlungssaetze : 187</AddtlInf>
<AddtlInf>Summe der Betraege (EUR) : 68.672,00</AddtlInf>
<AddtlInf>Ausfuehrungstermin : 01.12.2018</AddtlInf>
<AddtlInf>Hash-Wert : 24 AE 87 34 FE BA 22 12</AddtlInf>

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

<AddtlInf> 34 E4 5A 34 54 33 43 23</AddtlInf>
<AddtlInf> 15 34 55 78 FA F1 33 11</AddtlInf>
<AddtlInf> 93 67 30 03 19 67 BE FA</AddtlInf>
<AddtlInf>G U T S C H R I F T E N</AddtlInf>
<AddtlInf>Datei-ID: 4782647268347</AddtlInf>
<AddtlInf>Datum/Zeit: 28.11.2018/09:30:47</AddtlInf>
<AddtlInf>----------</AddtlInf>
<AddtlInf>Sammlerreferenz : 46573264785</AddtlInf>
<AddtlInf>Bank-Code : WELADEDD</AddtlInf>
<AddtlInf>Kontonummer : DE30300500000035351767</AddtlInf>
<AddtlInf>Auftraggeberdaten : YYY</AddtlInf>
<AddtlInf>Anzahl der Zahlungssaetze : 23</AddtlInf>
<AddtlInf>Summe der Betraege (EUR) : 14.256,00</AddtlInf>
<AddtlInf>Ausfuehrungstermin : 01.12.2018</AddtlInf>
<AddtlInf>Hash-Wert : 29 AE 87 34 FE BA 22 12</AddtlInf>
<AddtlInf> 34 E4 5A 34 54 33 43 23</AddtlInf>
<AddtlInf> 15 34 55 78 FA F1 33 11</AddtlInf>
<AddtlInf> 93 67 30 03 19 67 BE BB</AddtlInf>
</StsRsnInf>
….

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

10.2.3.3.3 Beispiel für DTAZV (deutsches Format, genutzt für internationale Zahlungen)

….
<StsRsnInf>
<AddtlInf>G U T S C H R I F T E N</AddtlInf>
<AddtlInf>Bank-Code : 30040000</AddtlInf>
<AddtlInf>Kundennummer : 0000000001</AddtlInf>
<AddtlInf>Auftraggeberdaten : KARL MUSTERMANN</AddtlInf>
<AddtlInf> MUSTERSTR. 1</AddtlInf>
<AddtlInf> 50825 KOELN</AddtlInf>
<AddtlInf>Erstellungsdatum : 10.05.00</AddtlInf>
<AddtlInf>Auftragswaehrung : USD</AddtlInf>
<AddtlInf>Bank-Code : 30040000</AddtlInf>
<AddtlInf>Kontowaehrung : EUR</AddtlInf>
<AddtlInf>Kontonummer : 1234567890</AddtlInf>
<AddtlInf>Ausfuehrungstermin : 10.11.00
<AddtlInf>Betrag : 20.000,000
<AddtlInf>Anzahl der Datensaetze T : 000000000000001</AddtlInf>
<AddtlInf>Summe der Betraege : 000000000020000</AddtlInf>
</StsRsnInf>
….

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

10.3 Anhang für HAC: External Reason Codes (Ergebnis der Aktion)

Die folgenden Ergebnisse der Aktion sind im Element <Rsn><Cd> zu protokollieren.
Sie sind ein Teil der externen ISO Codeliste “ExternalStatusReason1Code”:

ISO Code	ISO Name	Definition
AM05	Duplication	Der Hashwert der übertragenen Auftragsdaten ist bereits auf dem Bankserver bekannt (Auftragsdaten mit dem gleichen Hashwert wurden bereits übertragen)
AM21	LimitExceeded	Zwischen Kunde und Bank vereinbarter Höchstbetrag überschritten
DS01	ElectronicSignaturesCorrect	Elektronische Unterschrift(en) korrekt
DS02	OrderCancelled	Ein berechtigter Teilnehmer hat den Auftrag storniert
DS03	OrderNotCancelled	Der Versuch des Teilnehmers, den Auftrag zu stornieren, war nicht erfolgreich
DS04	OrderRejected	Der Auftrag wurde bankseitig abgewiesen (aus fachlichen Gründen)
DS05	OrderForwardedForPostprocessing	Der Auftrag war korrekt und konnte zur Weiterverarbeitung weitergegeben werden
DS06	TransferOrder	Der Auftrag wurde an die VEU weitergeleitet
DS07	ProcessingOK	Alle den Auftrag betreffenden Aktionen konnten durch den Bankrechner durchgeführt werden
DS08	DecompressionError	Die Dekomprimierung war nicht erfolgreich
DS09	DecryptionError	Die Entschlüsselung war nicht erfolgreich
DS10	Signer1CertificateRevoked	Das Zertifikat des ersten Unterzeichners wurde widerrufen
DS11	Signer1CertificateNotValid	Das Zertifikat des ersten Unterzeichners ist nicht gültig (widerrufen oder nicht aktiviert)
DS12	IncorrectSigner1Certificate	Für den ersten Unterzeichner ist kein Zertifikat vorhanden
DS13	SignerCertificationAuthority Signer1NotValid	Die CA des Zertifikates des ersten Unterzeichners ist unbekannt/ungültig
DS14	UserDoesNotExist	Der Teilnehmer ist auf dem Bankrechner unbekannt
DS15	IdenticalSignatureFound	Eine identische Unterschrift wurde bereits zur Bank geschickt
DS16	PublicKeyVersionIncorrect	Falsche Public Key Version. Dieser Code wird zurückgegeben, wenn ein Kunde nach dem Umstieg von einer älteren Programmversion (altes EU-Format) auf eine neue Programmversion (neues EU-Format) Unterschriftsdateien an das Kreditinstitut schickt, ohne sich vorher neu initialisiert beziehungsweise eine Public Key-Änderung durchgeführt zu haben.
DS17	DifferentOrderDataInSignatures	Auftragsdaten passen nicht zu der/den Unterschrift(en)
DS18	RepeatOrder	Datei nicht prüfbar, der gesamte Auftrag ist zu wiederholen. Dieser Code wird ausgegeben, wenn eine Betriebsstörung bei der Unterschriftsprüfung auftritt, z.B. zu wenig Speicherplatz

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

ISO Code	ISO Name	Definition
DS19	ElectronicSignatureRightsInsufficient	Die Berechtigungen des Teilnehmers (seine Unterschrift betreffend) sind für eine Ausführung des Auftrags unzureichend
DS20	Signer2CertificateRevoked	Das Zertifikat des zweiten Unterzeichners wurde widerrufen
DS21	Signer2CertificateNotValid	Das Zertifikat des zweiten Unterzeichners ist nicht gültig (widerrufen oder nicht aktiviert)
DS22	IncorrectSigner2Certificate	Für den zweiten Unterzeichner ist kein Zertifikat vorhanden
DS23	SignerCertificationAuthority Signer2NotValid	Die CA des Zertifikates des zweiten Unterzeichners ist unbekannt/ungültig
DS24	WaitingTimeExpired	Wartezeit bei unvollständigem Auftrag abgelaufen
DS25	OrderFileDeleted	Die Auftragsdatei wurde durch die Bank gelöscht (verschiedene Gründe möglich)
DS26	UserSignedMultipleTimes	Derselbe Teilnehmer hat mehrfach unterschrieben
DS27	UserNotYetActivated	Der Teilnehmer ist noch nicht aktiviert (technisch)
DS0A	DataSignRequested	Unterschriftsdaten werden benötigt *In EBICS bedeutet dies, dass die EU(s) noch nicht zum Bankrechner geschickt wurden oder dass die Anzahl der Unterschriften nicht ausreicht*
DS0B	UnknownDataSignFormat	Die Unterschrift ist für das Format nicht verfügbar oder ungültig *In EBICS bedeutet dies, dass die EU(s) nicht korrekt sind*
DS0C	SignerCertificateRevoked	Das Zertifikat des Unterzeichners wurde widerrufen *In EBICS bedeutet dies auch, dass der Teilnehmer gesperrt ist*
DS0D	SignerCertificateNotValid	Das Zertifikat des Unterzeichners ist ungültig (widerrufen oder nicht aktiviert). *In EBICS bedeutet dies, dass der öffentliche Schlüssel noch nicht aktiviert wurde oder das Zertifikat ungültig ist*
DS0E	IncorrectSignerCertificate	Das Zertifikat des Unterzeichners liegt nicht vor. *In EBICS bedeutet dies, dass der öffentliche Schlüssel nicht existiert oder kein Zertifikat vorhanden ist*
DS0F	SignerCertificationAuthority SignerNotValid	CA (Certificate Authority) ist unbekannt
DS0G	NotAllowedPayment	Unterzeichner ist nicht berechtigt, diesen Vorgang zu unterschreiben *In EBICS bedeutet dies, dass der Teilnehmer (für dies) keine Autorisierungsrechte hat*
DS0H	NotAllowedAccount	Der Unterzeichner hat keine Berechtigung für dieses Konto
ID01	CorrespondingOriginalFileStillNotSent	Unterschriftsdatei wurde zur Bank übertragen, die dazugehörige Originaldatei jedoch noch nicht
TA01	TransmissonAborted	Die Übertragung der Datei war nicht erfolgreich – sie musste abgebrochen werden (technische Gründe)
TD01	NoDataAvailable	Keine Daten zur Abholung vorhanden
TD02	FileNonReadable	Die Datei kann nicht gelesen werden (z.B. unbekanntes Format)
TD03	IncorrectFileStructure	Das Dateiformat ist unvollständig oder falsch
TS01	TransmissionSuccessful	Die Übertragung der Datei war erfolgreich
TS04	TransferToSignByHand	Der Auftrag wurde zur Freigabe mittels Begleitzettel weitergereicht

10.4 Anhang für HAC: Art und Ergebnis der Aktion (zulässige Paare)

Falls mehr als ein Ergebniscode passend ist, sollte der präziseste gewählt werden.

<OrgnlPmtInfId> (Art der Aktion)	**Mögliche / Zulässige Werte für <Rsn><Cd> (Ergebnis der Aktion)**	**Beschreibung des Codes (gekürzt, mehr Details siehe Kapitel 10.3)**
FILE_UPLOAD	AM05 TS01 TA01 DS0C DS08 DS09	Upload abgebrochen Upload erfolgreich Upload abgebrochen Teilnehmer/Zertifikat gesperrt Fehler bei Dekomprimierung Fehler bei Entschlüsselung
FILE_DOWNLOAD	TS01 TA01 DS0C DS08 DS09 TD01	Download erfolgreich Download abgebrochen Teilnehmer/Zertifikat gesperrt Fehler bei Dekomprimierung Fehler bei Entschlüsselung Keine Daten für Download vorhanden
ES_UPLOAD	TS01 TA01 DS0C DS08 DS09 ID01	Upload (der EU) erfolgreich Upload (der EU) abgebrochen User gesperrt/Zertifikat widerrufen Fehler bei Dekomprimierung Fehler bei Entschlüsselung Originaldatei wurde noch nicht übertragen
ES_DOWNLOAD		*Wird in EBICS noch nicht verwendet*

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

<OrgnlPmtInfId> (Art der Aktion)	Mögliche / Zulässige Werte für <Rsn><Cd> (Ergebnis der Aktion)	Beschreibung des Codes (gekürzt, mehr Details siehe Kapitel 10.3)
ES_VERIFICATION	AM21	Betragsgrenze überschritten
	TD02	Datei nicht lesbar
	TD03	Dateiformat ungültig
	TS04	Datei ohne EU (kein Signatur-Flag)
	DS01	EU(s) sind korrekt
	DS0A	Anzahl der EU(s) reicht nicht aus
	DS0B	EU(s) sind nicht korrekt
	DS0C	Zertifikat widerrufen/Teilnehmer gesperrt
	DS0D	Zertifikat ungültig/Public Key nicht aktiviert
	DS0E	Zertifikat/Public Key existiert nicht
	DS0F	CA für Zertifikat unbekannt
	DS0G	Unterzeichner hierzu nicht berechtigt
	DS0H	Unterzeichner nicht berechtigt für Konto
	DS10 (DS11;DS12)	Zertifikat widerrufen (ungültig; nicht vorhanden) für ersten Unterzeichner
	DS20 (DS21; DS22)	Zertifikat widerrufen (ungültig; nicht vorhanden) für zweiten Unterzeichner
	DS13/ DS23	CA unbekannt für Zertifikat des ersten/zweiten Unterzeichners
	DS14	Teilnehmer unbekannt auf Server
	DS15	EU bereits schon einmal an Bank übertragen
	DS16	Public Key Version ist nicht korrekt
	DS17	Auftragsdaten und EU passen nicht
	DS18	Auftrag wiederholen (Datei nicht testbar)
	DS19	EU-Berechtigung unzureichend
	DS24	Wartezeit überschritten, Datei gelöscht
	DS25	Datei von Bank gelöscht (versch. Gründe)
	DS26	Gleicher Teilnehmer mehrfach unterschrieben
	DS27	Teilnehmer noch nicht aktiviert
	DS08	Fehler bei Dekomprimierung
	DS09	Fehler bei Entschlüsselung
VEU_FORWARDING	DS06	Auftrag zur VEU weitergeleitet

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

<OrgnlPmtInfId> (Art der Aktion)	Mögliche / Zulässige Werte für <Rsn><Cd> (Ergebnis der Aktion)	Beschreibung des Codes (gekürzt, mehr Details siehe Kapitel 10.3)
VEU_VERIFICATION	AM21 TD02 TD03 DS01 DS0B DS0C DS0D DS0E DS0F DS0G DS0H DS10 (DS11; DS12) DS20 (DS21; DS22) DS13/ DS23 DS14 DS15 DS16 DS17 DS18 DS19 DS24 DS25 DS26 DS27	Betragsgrenze überschritten Datei nicht lesbar Dateiformat ungültig EU(s) sind korrekt EU(s) sind nicht korrekt Zertifikat widerrufen/Teilnehmer gesperrt Zertifikat ungültig/Public Key nicht aktiviert Zertifikat/Public Key existiert nicht CA für Zertifikat unbekannt Unterzeichner hierzu nicht berechtigt Unterzeichner nicht berechtigt für Konto Zertifikat widerrufen (ungültig; nicht vorhanden) für ersten Unterzeichner Zertifikat widerrufen (ungültig; nicht vorhanden) für zweiten Unterzeichner CA unbekannt für Zertifikat des ersten/zweiten Unterzeichners Teilnehmer unbekannt auf Server EU bereits schon einmal an Bank übertragen Public Key version ist nicht korrekt Auftragsdaten und EU passen nicht Auftrag wiederholen (Datei nicht testbar) EU-Berechtigung unzureichend Wartezeit überschritten, Datei gelöscht Datei von Bank gelöscht (versch. Gründe) Gleicher Teilnehmer mehrfach unterschrieben Teilnehmer noch nicht aktiviert
VEU_VERIFICATION_END	DS05	Auftrags war korrekt; wurde zur Weiterverarbeitung weitergeleitet
VEU_CANCEL_ORDER	DS02 DS03	Auftrag storniert Auftrag nicht storniert
ADDITIONAL	*Optional*	Hinweis: eine mögliche Belegung ist nicht im Bereich der EBICS-Spezifikation
ORDER_HAC_FINAL	---	

11 Anhang: Kryptografische Verfahren

In diesem Anhang werden die kryptografischen Verfahren beschrieben, die bei EBICS zum Einsatz kommen.

11.1 Authentifikationssignatur

11.1.1 Verfahren

Authentifikationssignaturen basieren auf dem RSA-Signaturverfahren. Die folgenden Parameter bestimmen das Verfahren zur Authentifikationssignatur: Länge des (geheimen) RSA-Schlüssels, Hashalgorithmus, Paddingverfahren, Kanonisierungsverfahren.

Für die Authentifikationssignatur definiert EBICS das **Verfahren „X002“** mit den folgenden Parametern:

Parameter	**Wert**
Schlüssellänge in KBit	>=2KBit (2048 Bit) und <=16KBit
Hashalgorithmus	SHA-256
Paddingverfahren	PKCS #1
Kanonisierungsverfahren	http://www.w3.org/TR/2001/REC-xml-c14n-20010315

Da die Mindestschlüssellänge bei X002 gegenüber X001 nicht geändert wurde, müssen die Authentifikationsschlüssel beim Umstieg von der Version X001 auf X002 nicht geändert werden.

Die optionalen XML-Signatur-Felder „KeyInfo“ und „Object“ bleiben unbefüllt.
Ab EBICS 2.4 muss das Kundensystem in einem Request den Hashwert des öffentlichen Bankschlüssels X002 verwenden. Die Transaktion wird mit dem Returncode EBICS_INVALID_REQUEST_CONTENT abgebrochen, wenn noch X001 in einem Request verwendet wird.

11.1.2 Format

Authentifikationssignaturen werden in EBICS-Nachrichten gemäß der W3CRecommendation „XML Signature Syntax and Processing“ (http://www.w3.org/TR/xmldsig-core/) dargestellt. Dadurch sind Identifikatoren der Algorithmen zur Bildung der Hashwerte, der Signatur, sowie der Identifikator des Kanonisierungsverfahrens Bestandteil der Authentifikationssignatur. Es ist somit keine Änderung der XML-Schnittstelle notwendig, wenn eine neue Version „X00n“ mit veränderten Parametern definiert wird. Das gilt insbesondere für Versionen, die als Hashfunktion SHA-224, SHA-256, SHA-384 oder SHA-512 einsetzen.

Bei der Einstellung der Authentifikationssignatur im Element `SignatureValue` erfolgt grundsätzlich kein Auffüllen auf die volle Länge des Modulo des RSA-Schlüssels zur Bildung dieser Signatur.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

11.2 Elektronische Unterschriften

11.2.1 Verfahren

Elektronische Unterschriften basieren auf dem RSA-Signaturverfahren. Die Verfahren zur Erzeugung/ Überprüfung Elektronischer Unterschriften sind im Anhang (Kapitel 14) definiert.
EBICS muss die Version „A005“ oder „A006“ der bankfachlichen elektronischen Signatur unterstützen.

11.2.2 Format

Die Schemadatei „ebics_signature.xsd“ enthält UserSignatureData für die Unterschrift der Teilnehmer in EBICS-Nachrichten. Dazu wird ein Instanzdokument zu „ebics_signature.xsd“ erstellt, das für Teilnehmer-EUs `UserSignatureData` als Toplevel-Element enthält. `UserSignatureData` enthält eine Liste von Elementen `OrderSignature` für eine oder mehrere Teilnehmer-EUs.
`UserSignatureData` enthält eine Liste von Elementen `OrderSignatureData` für eine oder mehrere Teilnehmer-EUs (siehe auch Abbildung 4).
Die XML-Schemadatei “ebics_orders_H005.xsd” enthält die Definition der globalen Elemente `BankSignatureData` zur Einbettung der bankseitigen EU (wenn auch geplant, wird diese Struktur heute noch nicht genutzt) – insbesondere enthält `BankSignatureData` noch ein Element `OrderSignature` für die bankseitige EU, das base64-kodiert ist (siehe auch Abbildung 4 EBICS-Autorisierungsschemata)

11.2.3 EBICS-Autorisationsschemata für Unterschriftsklassen

EBICS legt die Autorisierungsschemata für Aufträge fest, die eine oder zwei bankfachliche EUs erfordern. Autorisierungsschemata für Aufträge, die mehr als zwei bankfachliche EUs erfordern, sind nicht im vorliegenden Standard beschrieben, obgleich es nicht verboten ist, mehr als zwei bankfachliche EU’s zu übertragen.

E=Einzelunterschrift, A=Erstunterschrift, B=Zweitunterschrift,
T=Transportunterschrift (nicht bankfachlich).

Autorisierungsschema für Aufträge mit Mindestanzahl EU = 0:

E	A	B	
☑	☑	☑	

Die Mindestanzahl EU = 0 gilt für Aufträge, die über Begleitzettel autorisiert werden **(kein Signatur-Flag))** oder für Aufträge des Schlüsselmanagement, für deren Autorisierung eine Transportunterschrift ausreicht (aber auch E-, A-, B-EU möglich sind). Aufträge die mit EU autorisiert werden (Signatur-Flag ist vorhanden)) müssen ausreichend unterschrieben werden. Da hierfür mindestens eine bankfachliche EU notwendig ist, gelten hier auch die Regeln für Mindestanzahl EU = 1.

Autorisierungsschema für Aufträge mit Mindestanzahl EU = 1:
Autorisierung mit einer einzigen bankfachlichen EU:
Die Autorisierung des Auftrags mit einer einzigen EU kann mit einer Einzelunterschrift

E	A	B	T
☑	☐	☐	☐

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

erfolgen.

Autorisierung mit zwei bankfachlichen EUs:
Die Autorisierung des Auftrags kann auch mit 2 EUs der Klasse E, A oder B erfolgen, wenn mindestens eine der beiden eine Einzel- oder Erstunterschrift ist.

- Mit Ausnahme der Kombination bestehend aus zwei Zweitunterschriften ist die Autorisierung des Auftrags mit einer beliebigen Kombination aus zwei bankfachlichen EUs möglich.

erste EU → / zweite EU ↓	E	A	B	T
E	☑	☑	☑	☐
A	☑	☑	☑	☐
B	☑	☑	☐	☐
T	☐	☐	☐	☐

Allgemein gilt:

- Es ist keine Maximalanzahl von EUs definiert, jedoch müssen die Unterschriften im Falle einer Übertragung von mehr als zwei EUs den Regeln der oben genannten Autorisierungsschemata genügen.
- Einzelunterschriften sind zur Autorisierung grundsätzlich zulässig, jedoch nur bei Aufträgen mit Mindestanzahl EU = 0 oder EU = 1 ausreichend
- Eine Transportunterschrift autorisiert niemals die Ausführung eines Auftrags, sondern ermöglicht lediglich die Auftragseinreichung
- Die Reihenfolge, in der die Unterschriften geleistet werden, ist irrelevant
- Die bankfachlichen EUs eines Auftrags MÜSSEN von unterschiedlichen Teilnehmern (eventuell auch unterschiedlicher Kunden) geleistet werden.

11.3 Verschlüsselung

11.3.1 Verschlüsselung auf TLS-Ebene

11.3.1.1 Verfahren

Kundensystem und Banksystem MÜSSEN sich auf eines der folgenden Verfahren (sog. „Cyphersuites“, siehe RfCs 2246 und 3268) im Rahmen des TLS-Handshake einigen (Details siehe im aktuellen EBICS-Anhang „Transport Layer Security“)

11.3.2 Verschlüsselung auf Anwendungsebene

11.3.2.1 Verfahren

Das Verfahren zur Verschlüsselung der Nutzdaten und EUs eines Auftrags ist die Hybridver-schlüsselung, basierend auf dem symmetrischen Verschlüsselungsverfahren 2-Key-Triple-DES und dem asymmetrischen Verschlüsselungsverfahren RSA.

Die Nutzdaten und EUs einer EBICS-Transaktion werden symmetrisch verschlüsselt. Je EBICS-Transaktion wird ein zufälliger symmetrischer Schlüssel (Transaktionsschlüssel) vom Sender der

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Nutzdaten und/oder EUs generiert, der dann sowohl für die Verschlüsselung der Nutzdaten als auch für die Verschlüsselung der EUs verwendet wird. Der symmetrische Schlüssel wird asymmetrisch verschlüsselt an den Empfänger übertragen.

Ausgehend von Standardverschlüsselungsverfahren definiert EBICS das **Verschlüsselungsverfahren „E002“** mit den folgenden Merkmalen:

- Symmetrischer Verschlüsselungsalgorithmus
  - Generierung des Transaktionsschlüssels
  - AES-128 (Schlüssellänge 128 Bit) im CBC Modus
  - ICV (Initial Chaining Value) = 0
  - Paddingverfahren gemäß ANSI X9.23 / ISO 10126-2.
- RSA-Verschlüsselung des Transaktionsschlüssels, Schlüssellänge >=2KBit (2048 Bit) und <=16KBit
  - Unterschied zu V001: 768
- Paddingverfahren für die RSA-Verschlüsselung: PKCS #1
  - Unterschied zu V001: 0-Padding.

Da die Mindestschlüssellänge bei E002 gegenüber E001 nicht geändert wurde, müssen die Authentifikationsschlüssel beim Umstieg von der Version E001 auf E002 nicht geändert werden.

Das **Verfahren zur asymmetrischen Verschlüsselung** des Transaktionsschlüssels muss für EBICS wie folgt angepasst werden:

- Die Mindestlänge des (geheimen) RSA-Schlüssels beträgt 2048
- Das Paddingverfahren ist PKCS #1-konform.

Diese Anpassungen bedeuten konkret:

- Die Länge von PDEK ist gleich der Länge des verwendeten RSA-Schlüssels (>= 2048)
- PDEK wird aus DEK mittels PKCS #1 Padding erzeugt
- EDEK ist das Ergebnis der RSA-Verschlüsselung von PDEK.

Analog muss für EBICS auch das **Verfahren zur Entschlüsselung** des Transaktionsschlüssels angepasst werden:

- PDEK ist das Ergebnis der RSA-Entschlüsselung von EDEK
- Die 128 niedrigstwertigen Bits von PDEK bilden den geheimen Schlüssel DEK.

Für die **Bildung dieses Hashwertes des öffentlichen RSA-Schlüssels** wird im Kontext von „E002“ das Verfahren SHA-256 verwendet.

11.3.2.2 Formate

Die komprimierten und verschlüsselten EUs und Nutzdatensegmente werden als base64-kodierte Binärdaten in die EBICS-Nachrichten eingebettet.

Innerhalb der EBICS-Nachrichten erfolgt die Übermittlung des asymmetrisch verschlüsselten Transaktionsschlüssels innerhalb eines XML-Elements vom Typ `DataEncryptionInfoType`. Dieser Typ ist in der XML-Schema-Definitionsdatei ebics_types_H005.xsd definiert und seine grafische Darstellung ist in Abbildung 100 enthalten.

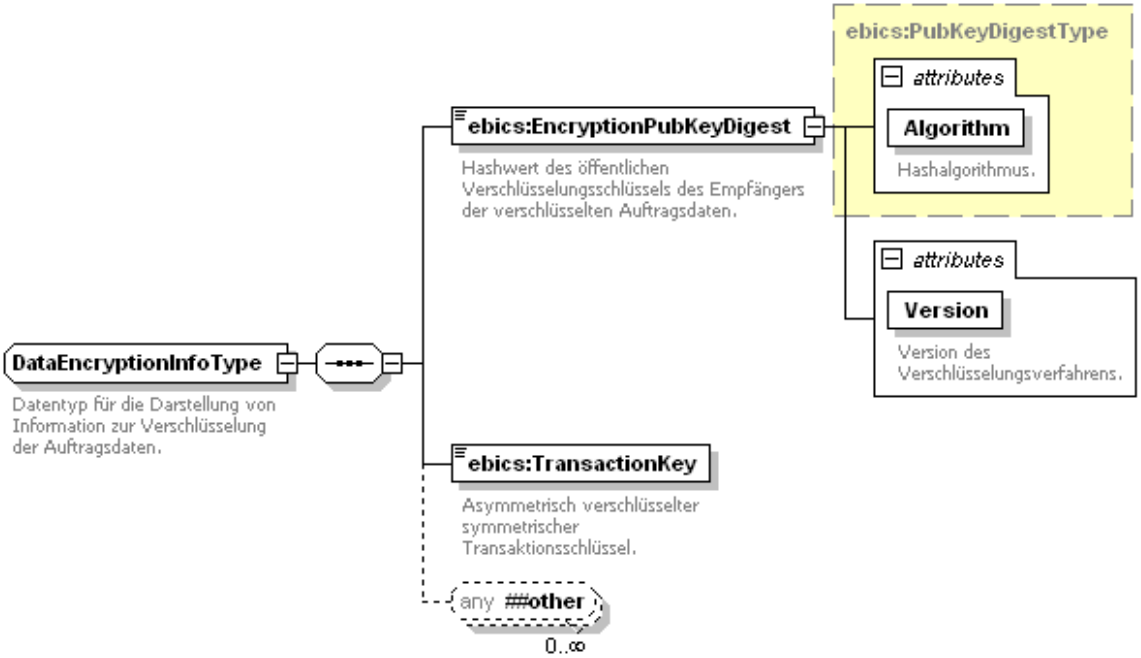


Abbildung 100: Definition des XML-Schema Typs DataEncryptionInfoType

Das Element `ebicsRequest/body/DataTransfer/DataEncryptionInfo` bzw. `ebicsReponse/body/DataTransfer/DataEncryptionInfo` vom Typ `DataEncryptionInfoType` ist Bestandteil des ersten EBICS-Requests einer Upload-Transaktion (vgl. ebics_request_H005.xsd) oder der ersten EBICS-Response einer Download-Transaktion (vgl. ebics_response_H005.xsd).

Im Unterschied zum Vorsatz enthält `DataEncryptionInfoType` keine Teilnehmerangaben. Das ist nicht notwendig, da der Empfänger/ Sender der Auftragsdaten stets der Einreicher des Auftrags ist. Die Teilnehmer-/KundenID des Einreichers ist bereits Bestandteil der Steuerdaten des ersten EBICS-Requests einer jeden EBICS-Transaktion und ist der EBICS-Transaktion fest zugeordnet.

Das Element `EncryptionPubKeyDigest` enthält neben dem Hashwert des öffentlichen RSA-Schlüssels auch die Version des verwendeten Verschlüsselungsverfahrens und den Identifikator des verwendeten Hashalgorithmus.

Es ist keine Änderung von `DataEncryptionInfoType` notwendig, wenn eine neue Version „E00n“ mit veränderten Parametern definiert wird. Das gilt insbesondere für Versionen, die eine oder mehrere der Hashfunktionen SHA-224, SHA-256, SHA-384, SHA-512 zulassen.

Bei der Einstellung des verschlüsselten Transaktionsschlüssels im Element `TransactionKey` erfolgt grundsätzlich kein Auffüllen auf die volle Länge des Modulo des RSA-Schlüssels für die Verschlüsselung.

11.4 Replay-Vermeidung mittels Nonce und Timestamp

11.4.1 Verfahrensbeschreibung

Der erste EBICS-Request, der der Initialisierung einer EBICS-Transaktion dient, enthält die Elemente „**Nonce**“ und „**Timestamp**“, die gemeinsam ein Wiedereinspielen dieses Requests verhindern sollen.

„Nonce“ und „Timestamp“ bilden eine funktionale Einheit zur Replay-Vermeidung:

1. Das Kundensystem generiert einen zufälligen „Nonce“ und setzt „Timestamp“ auf den aktuellen Zeitpunkt des Nachrichtenversands.
2. Das Banksystem vergleicht den empfangenen „Nonce“ mit einer lokal gespeicherten Liste zuvor empfangener „Nonce“-Werte. Außerdem prüft es die Abweichung des „Timestamp“ zur aktuellen Uhrzeit. Ist der gerade empfangene „Nonce“ in der gespeicherten Liste vorhanden oder ist die Abweichung des „Timestamp“ größer als ein vom Kreditinstitut festzulegender Toleranzzeitraum, so wird der Request mit dem technischen Fehlercode EBICS_TX_MESSAGE_REPLAY beantwortet.
3. Verlief die „Nonce“- und „Timestamp“-Prüfung ohne Fehler, so speichert das Banksystem das „Nonce“-„Timestamp“-Paar in der lokalen Liste ab und fährt mit der weiteren Verarbeitung der Nachricht fort.

Das Banksystem kann „Nonce“-„Timestamp“-Paare, deren Zeitstempel außerhalb des Toleranzzeitraums liegen, aus seiner Liste löschen: Nachrichten, die ein solches Paar enthielten, würden bereits wegen der überhöhten Abweichung des „Timestamp“ abgewiesen. Der festgelegte Toleranzzeitraum gilt also gleichermaßen für die Prüfung neuer Paare wie für den Löschvorgang gespeicherter Paare.

Mit den Elementen „Nonce“ und „Timestamp“ garantiert dieses Verfahren, dass der erste EBICS-Request einer Transaktion eindeutig ist. Dadurch wird eine bankseitige Initialisierung neuer EBICS-Transaktionen aufgrund alter, erneut eingespielter Nachrichten verhindert. Gleichzeitig begrenzt „Timestamp“ die zeitliche Notwendigkeit zur bankseitigen „Nonce“-Speicherung.

11.4.2 Aktionen des Kundensystems

11.4.2.1 Erzeugung von „Nonce“ und „Timestamp“

Das Kundensystem MUSS in der Transaktionsphase „Initialisation“ folgende Felder befüllen:

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

- `ebicsRequest/header/static/Nonce` mit einer kryptografisch starken Zufallszahl der Länge 128 Bit
- `ebicsRequest/header/static/Timestamp` mit dem aktuellen Zeitstempel des Versands des EBICS-Requests (Datum und Uhrzeit gemäß ISO 8601).

Ein Beispiel für eine syntaktisch korrekte Befüllung der Werte „Nonce“ und „Timestamp“ gibt folgender XML-Ausschnitt wieder:

```
<?xml version="1.0" encoding="UTF-8"?>
<ebicsRequest
 xmlns="urn:org:ebics:H005"
 xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="urn:org:ebics:H005 ebics_request_H005.xsd "
 Version="H005" Revision="1">
 <header authenticate="true">
   <HostID>EBIXHOST<HostID>
   <Nonce>01A56FF768B3B36C5120E9904A7FB035</Nonce>
   <Timestamp>2005-06-22T17:07:34.123+02:00</Timestamp>
   […]
 </header>
 […]
</ebicsRequest>
```

Weitere Informationen zur korrekten Befüllung der beiden XML-Schema-Elemente sind unter http://www.w3.org/TR/xmlschema-2/#hexBinary (hexBinary) bzw. http://www.w3.org/TR/xmlschema-2/#dateTime (dateTime) zu finden.

11.4.2.2 Verhalten bei Fehler-Rückmeldung EBICS_TX_MESSAGE_REPLAY

Mit dem technischen Fehlercode EBICS_TX_MESSAGE_REPLAY signalisiert das Banksystem, dass die gerade vom Client versandte EBICS-Nachricht einen „Nonce“-Wert enthält, der mit einem banksystemseitig gespeicherten übereinstimmt, oder dass der „Timestamp“ außerhalb des Toleranzzeitraums liegt.

Bei Verwendung kryptografisch starker Zufallszahlen als „Nonce“ und bei vernünftig gewähltem Toleranzzeitraum durch das Kreditinstitut (Richtwert: einige Stunden) kann der Fall einer zufälligen Kollision aufgrund der verschwindend geringen Wahrscheinlichkeit seines Auftretens vernachlässigt werden.

Das Kundensystem muss also bei Empfang der Meldung EBICS_TX_MESSAGE_REPLAY die Möglichkeit eines Replay-Angriffs, einer intolerabel ungenauen Uhreneinstellung auf Kunden- oder Banksystemseite oder einer Fehlfunktion der eigenen Transaktionsverwaltung bei der Vergabe von „Nonce“-Werten in Betracht ziehen.
Möchte der Teilnehmer die betroffene EBICS-Nachricht trotzdem erfolgreich absetzen, so muss er zuvor die Felder `ebicsRequest/header/static/Nonce` und `ebicsRequest/header/static/Timestamp` entsprechend Anhang Kapitel 11.4.2.1 neu erzeugen. Die restlichen Inhalte können unverändert bleiben.

11.4.3 Aktionen des Banksystems

11.4.3.1 Überprüfung von „Nonce“ und „Timestamp“

Wenn das Banksystem eine initiale EBICS-Nachricht eines Teilnehmers erhält, so MUSS es zur Prüfung auf einen Nachrichten-Replay die folgenden Aktionen durchführen. Können diese Prüfungen sämtlich erfolgreich durchlaufen werden, so liegt kein Nachrichten-Replay vor:

1. **Abgleich des empfangenen „Timestamp“ mit dem lokalen Zeitstempel:** Normiert auf UTC muss sich der empfangene „Timestamp“ innerhalb des Toleranzzeitraums befinden, der um den aktuellen Zeitstempel des Banksystems herum aufgespannt ist. Mit diesem Toleranzzeitraum sollen Differenzen der Ganggenauigkeit der beteiligten Uhren zwischen den Systemen und eventuell auch verfrühte/verspätete Sommer-/Winterzeitumstellungen abgefangen werden. Gleichzeitig bestimmt der Toleranzzeitraum auch, wann das Banksystem gespeicherte „Nonce“-„Timestamp“-Paare löschen kann: Ankommende Nachrichten mit einem „Timestamp“ außerhalb des Toleranzzeitraums werden nicht akzeptiert. In der Vergangenheit gespeicherte „Nonce“-„Timestamp“-Paare, deren „Timestamp“ sich mittlerweile außerhalb des Toleranzzeitraums befindet können deshalb gelöscht werden.
   Der Toleranzzeitraum muss vom Banksystem einmalig festgelegt werden. Hierbei vergrößern hohe Werte (= große Toleranzzeiträume) den Speicherbedarf für valide „Nonce“-„Timestamp“-Paare, während niedrige Werte (= kleine Toleranzzeiträume) die Gefahr abgelehnter EBICS-Nachrichten in Folge zu großer Uhrendifferenzen zwischen Kunden- und Banksystem erhöhen.
   Falls sich der empfangene „Timestamp“ nicht im Toleranzzeitraum befindet, besteht die Gefahr eines Nachrichten-Replays. Das Banksystem MUSS daher mit dem technischen Fehlercode EBICS_TX_MESSAGE_REPLAY antworten.
2. **Vergleich des empfangenen „Nonce“ mit den lokal gespeicherten „Nonce“-Werten:** Banksystemseitig sind sämtliche „Nonce“-„Timestamp“-Paare gespeichert, die aus validen EBICS-Requests im Toleranzzeitraum stammen. Falls der empfangene „Nonce“ mit einem gespeicherten „Nonce“ übereinstimmt, MUSS das Banksystem mit dem technischen Fehlercode EBICS_TX_MESSAGE_REPLAY antworten.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

11.5 Initialisierungsbriefe

Initialisierungsbriefe für INI enthalten den öffentlichen Schlüssel für das Teilnehmerzertifikat für die bankfachliche EU, Initialisierungsbriefe für HIA enthalten die öffentlichen Schlüssel der Teilnehmerzertifikate für Authentifikation und Verschlüsselung.

11.5.1 Initialisierungsbrief für INI (Beispiel)

11.5.1.1 Mit Elektronischer Unterschrift der Version A006

User Name	Frank Sample
Date	TT.MM.JJJJ
Time	HH:MM:SS
Host ID	BANKXXXX
Bank	« Name der Bank »
User ID	Xxxxxxxx
Partner ID	Yyyyyyyy
Version	*Name der EU-Version (z.B. A***)*

EU Zertifikat

Zertifikat ausgegeben am: Vor- und Nachname oder Id *(nur vorhanden im Falle von CA-Zertifikaten)*
Zertifikat ausgegeben von : Name der Zertifizierungsinstanz (CA) *(nur vorhanden im Falle von CA-Zertifikaten)*

-----BEGIN CERTIFICATE-----
MIIC6DCCAdCgAwIBAgIIW+dFLrrgUj0wDQYJKoZIhvcNAQELBQAwKjEoMCYGA1UEAwwfRUJJQ1NQQVJUTkVSSURfRUJJQ1NVU0VSS
UQ7Qz1GUjAeFw0xNjExMjMxMTAwMDhaFw0yMTExMjQxMTAwMDhaMCoxKDAmBgNVBAMMH0VCSUNTUEFSVE5FUklEX0VCSUNTV
VNFUklEO0M9RlIwggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQCgDAuAyZ5Qh18LOcu9H+w9fA+FjgIJoK3WkGF5zsyqWDyH
9dlsHo7fp3FQXYaRLGi4VyVrSwgdDOF4gxopOnVO6nYevepqiriBD129YB3rzMxgh/zwuQB60rRyEkr/5mvdddrWpj6RWErRTvQL5CZpeNZ9
G/z96sFa7Rzi2W7K2oHr+piiC5moB2cqP54lnzOIx2Z5V5E9w/Fxq8rIQP6XnXu8iZv9bZbF2jy9iED3umEav+9H0Gn67GFxy2i9OkKOGvcmLy9
wwiDPF756t2xSrpNhVEFCek8pJPnDkQji93XqtTSfZXezKuT2L59MhuICH4IMVOOD2xaOvsmHYml5AgMBAAGjEjAQMA4GA1UdDwEB/wQ
EAwIGQDANBgkqhkiG9w0BAQsFAAOCAQEAWONno+PpSXFBciiqS76x0NkYiYTSk3rOUjK/Q/sC+FQX60TgBEbybLXvbGB2fkkUeoQopCu
qWkVamJ01tnD+sUPRducSMMh1YERLyTRk/15YjtoeeiEGJNKldGbaR9W6KaTMdY6SOIloyAm/t6HDDhpgL83rN8d5C1uiIpkrPbmqGJ5iOlkJ
BWzRBxOXAxIGa9OZ4r/RoIF7wAwSbfr2cB7rfySrJOUvdZYafwlKZVw8PtSft2JCOnrT5iz0+wGBCIboLiaBOSIO4o6Y+qAC15hbkfdEC5JiK0+
+vxsDHqHdggtTLU9DP36+KwEPyf8HjW2tCt8F4eucxT8GrfqZcw==
-----END CERTIFICATE-----

Hashwert des EU-Zertifikates (SHA-256):

72 46 B6 32 85 DF 35 B8
1B 08 66 51 6D E0 FC 1D
DF 35 C7 DD F9 90 EA 6E
A2 C8 A0 ED A9 AB 32 49

Ich bestätige hiermit den obigen öffentlichen Schlüssel für meine Elektronische Unterschrift.

Ort / Datum	Firma / Name	Unterschrift

11.5.2 Initialisierungsbrief für HIA (Beispiel)

User Name	Frank Sample
Date	TT.MM.JJJJ
Time	HH:MM:SS
Host ID	BANKXXXX
Bank	« Name der Bank »
User ID	Xxxxxxxx
Partner ID	Yyyyyyyy
Version	*Name der Authentifikationssignatur (z.B.X***)* *und Name des Verschlüsselungsverfahrens (z.B.E****)

Zertifikat für Authentifikation - Typ X***

-----BEGIN CERTIFICATE-----
MIIC6DCCAdCgAwIBAgIIMQ36MhxHh34wDQYJKoZIhvcNAQELBQAwKjEoMCYGA1UEAwwfRUJJQ1NQQVJUTkVSSURfRUJJQ1NVU0VS
SUQ7Qz1GUjAeFw0xNjExMjMxMTAwMDdaFw0yMTExMjQxMTAwMDdaMCoxKDAmBgNVBAMMH0VCSUNTUEFSVE5FUklEX0VCSUNTV
VNFUklEO0M9RlIwggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQDLoPa1O8XJ8L066vGQ9yzXm1NRyvjGxO4c/2GTNxnzAN9e
gbawaouw/OUyMZ0Pof6zRfcSm4NhxqnxkE18FIpBUzxZDEiDy0CITSHDmknI7xJibk+zCdnHYE3QS5Kg7CeGXnZm30Gwl4UnOvfhKONgPK
8/DRXdZDdzrSkaOt+Xqhi1i8qUerGoAEt7HNrs2gWfnirEBksj3Mj0OrTdIwdgWAuLEuk7CnA4gweqsRjha/EaXrQbUB4KJHOS2NsJczh4HKoao
HdEiyFq8Asm4mFhqQmarpv59zzsnnsep+ho0z59+7ETBaW5KgQZsRE3dbZVHhPPONsPkQs6OLKacn5pAgMBAAGjEjAQMA4GA1UdDwE
B/wQEAwIHgDANBgkqhkiG9w0BAQsFAAOCAQEAGw2nxKLtmzpEnPmA89tnSO1lFE49y/aGPYpabWjveR+P1VMglGNp7RWK+NVoZBBJu
SBRqnmYEbieQK2Mo/hShFVpMiEyRB5mdkoRu58PHl4pvpVVUtpVLHjjw8SVKXm8nw6l+8laYwRAuUn63pOl7de3Hy0DsrBYDwcxMpr8RSC
5l4ZZmeKlGwW1GqPzCUu74M+8eqZhlOD2TFruhlITsGO3zeQKeUZ/uy8Y9PMfrjPrtwrrGgx105agdyKHSuY3FkslsVrTVNOHRQWcMP84zd
NL1F2PuZofnJ1zc+unctpq3flHaZ663fUKDMKleKMOoXfQ13Vugg4cHXS/DiaE9A==
-----END CERTIFICATE-----

Hashwert Zertifikat für Authentifikation (SHA-256):

8E 98 E6 46 FC E4 E9 5E
3E 50 5B A2 DF AA 8D 89
81 38 AE 17 B1 51 D3 12
87 96 F4 C1 FF BA E7 82

Zertifikat für Verschlüsselung - Typ E*

-----BEGIN CERTIFICATE-----
MIIC6DCCAdCgAwIBAgIIbZ163qos4DQwDQYJKoZIhvcNAQELBQAwKjEoMCYGA1UEAwwfRUJJQ1NQQVJUTkVSSURfRUJJQ1NVU0VS
SUQ7Qz1GUjAeFw0xNjExMjMxMTAwMDhaFw0yMTExMjQxMTAwMDhaMCoxKDAmBgNVBAMMH0VCSUNTUEFSVE5FUklEX0VCSUNTV
VNFUklEO0M9RlIwggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQC3K5Lgva6kZizCJVgFTIMjwjLvPl6wfWWP4ei/eqABeslZ6Zv9
z/EauplDZzK0ulyIuCwtyO7V36EiCLZ0VS7V20izpbllIwVyYVi950/Q9Pznvz0p9KvwquheXLFHTwdUATuAEKHT8wc2347j5vRfYCxjxk1Xgk8sg
bnyXBwJxy/XkaaALxEfY/60jUz7ip1jilB4AH03I+qn5lsI1d7EZFCTNwGIHXRgivLiNJ7sF5Q+MTqwZ2kYgReBzY6rDD7SMaOlcfopqDDkbyayoh
KnJxaSXUCpLTKjHfi2ZPhCxwyZRTKG16y1jscbb4Asocmcsrg/SeHY/G1YERe6Xfa/AgMBAAGjEjAQMA4GA1UdDwEB/wQEAwIFIDANBgkqh
kiG9w0BAQsFAAOCAQEAfINilMSoo1O4ms3qQTTEH6KJLrC9UYRJMzTO0YpTXROOB4n3NHG/q1ElZU41UB8VcCrpWKWBroqx98oRNrFy
OD1wGjB+ine5bxT71ncALEk7ZneUSE3anZKaQV6mZbaJWRq/HSNTQ3G6Ml1LZ8/ZFy5Bt+VnYIXG/tASv5U/jW0+67ceNs/j94zzrH9auvL7h
6PP6260znqGKKgxuX6+XMT21ff7jyG3h+BwGWrtCwU1qmbZGW3wRYTR6x9kCoiV+Wl5gqWSpOoai7Oi7nBGklrLbeOL3UFSvrfjEhpS9az4
5vn0ldFs+C8eYSgs+ZsBWNcuu8UaLt9S57UmlaR8Gg==
-----END CERTIFICATE-----

Hashwert Zertifikat für Verschlüsselung (SHA-256):

E3 FA 11 A3 A4 40 CF 29
6D 25 1B 09 F4 1A 38 F7
33 E5 3A 96 FF DF 6C 5F
30 DF B2 9D 72 40 3E D7

Ich bestätige hiermit die obigen öffentlichen Schlüssel für Authentifikation und Verschlüsselung

Ort / Datum	Firma / Name	Unterschrift

11.6 Erzeugen der TransaktionsIDs

TransaktionsIDs sind kryptografisch starke Zufallszahlen der Länge 128 Bit. Dadurch wird erreicht, dass für zwei beliebige Banksysteme die Wahrscheinlichkeit, zu einem Zeitpunkt dieselbe TransaktionsID zu verwenden, hinreichend gering ist.

TransaktionsIDs werden von kryptografischen Pseudozufallszahlengeneratoren (PRNG) erzeugt, welche mit einer echten Zufallszahl (Seed) initialisiert wurden. Die Entropie des Seed sollte mindestens 100 Bit betragen.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

12 Übersicht ausgewählter EBICS-Details

12.1 Optionale EBICS-Features

Bei EBICS sind nicht alle Funktionen als verpflichtend definiert. Für einige administrative Auftragsarten sowie Features innerhalb des Transaktionsablaufs steht einem Kreditinstitut, das den EBICS-Standard implementiert, die Unterstützung frei.

12.1.1 Optionale administrative Auftragsarten

Folgende EBICS-Auftragsarten KÖNNEN aus Sicht eines Kreditinstituts unterstützt werden (d.h. sie sind optional):

- HAA (abrufbare Auftragsarten abholen)
- HKD (Kunden- und Teilnehmerdaten des Kunden abholen)
- HTD (Kunden- und Teilnehmerdaten des Teilnehmers abholen)

12.1.2 Optionale Funktionalitäten im Transaktionsablauf

Folgende EBICS-Funktionalitäten KANN ein Kreditinstitut oder ein Kundenprodukt unterstützen (d.h. sie sind für beide Seiten optional):

Vorabprüfung (siehe Kapitel 3.6 und 5.3)

Recovery (siehe Kapitel 3.4 und 5.4).

12.2 EBICS-Bankparameter

Mit der administrativen EBICS-Auftragsart HPD (siehe auch Kapitel 9.2) kann sich der Teilnehmer über die spezifischen Zugangs- (`AccessParams`) und Protokollparameter (`ProtocolParams`) des Kreditinstituts informieren.

Zugangsparameter (`AccessParams`):

Parametername	#	Bedeutung	Beispiel
`URL`	1..∞	URL oder IP-Adresse für den elektronischen Zugang zum Kreditinstitut. Es besteht die Möglichkeit, mehrere URLs anzugeben. Jede URL mit erreichtem valid_from-Datum (oder wenn dieses Feld leer ist) ist gültig. Ist eine URL nicht erreichbar, kann der Kunde eine andere gültige Adresse nutzen.	„http://www.die-bank.de“
`URL@valid_from`	0..1	Gültigkeitsbeginn der URL/IP. Falls nicht angegeben, ist die Angabe ab sofort gültig	„2005-01-30T15:30:45.123Z“

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Institute	1	Bezeichnung des Kreditinstituts	„Die Bank“
HostID	0..1	ID des EBICS Banksystems	„bank01“

Protokollparameter (ProtocolParams):

Parametername	#	Bedeutung	abh. admin. Auftrags-arten
Version	1	zulässige Versionen (jeweils als Auflistung) für EBICS-Protokoll (Protocol), Verschlüsselung (Encryption), Signatur (Signature) und Authentifikation (Authentication)	alle
Recovery	0..1	Unterstützung für das Wiederaufsetzen von Transaktionen	alle
PreValidation	0..1	Unterstützung der Vorabprüfung. Falls dieser Parameter gesetzt ist, sichert das Kreditinstitut damit zu, dass es zumindest einen Teil der vom Teilnehmer im Rahmen der Vorabprüfung übermittelten Daten prüft. Das Kreditinstitut ist aber nicht verpflichtet, die Daten in vollem Umfang zu prüfen	Uploads
ClientDataDownload	0..1	Unterstützung der administrativen Auftragsarten HKD (Download von Kundendaten, Kapitel 9.3) und HTD (Download von Teilnehmerdaten, Kapitel 9.4)	HKD, HTD
DownloadableOrder» Data	0..1	Unterstützung der administrativen Auftragsart HAA (abrufbare BTF abholen, Kapitel 9.1)	HAA

12.3 Sicherheitsmedien bankfachlicher Schlüssel

Für die Angabe des Sicherheitsmediums (geheimer) bankfachlicher Schlüssel definiert EBICS die folgenden Wertekategorien:

Sicherheitsmedium	Belegung
Keine Angabe	0000
Diskette	01dd
Chipkarte	02dd
Sonstiges Wechselmedium	03dd
Nicht wechselbares Medium	04dd

In der obigen Tabelle steht „dd“ für eine beliebige Zahlenkombination, die institutsindividuell festgelegt wird.

12.4 Muster für TeilnehmerIDs, KundenIDs, AuftragsIDs, HostIDs

Die nachfolgende Tabelle legt die in EBICS zulässigen Muster unterschiedlicher IDs fest. Für jede ID werden zudem all die XML-Typen aufgelistet, die in EBICS verwendet werden, um entsprechende IDs aufzunehmen.

ID	TeilnehmerID / ID Technischer Teilnehmer	KundenID	AuftragsIDs	Host ID

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Pattern	[a-zA-Z0-9,=]{1,35}	[a-zA-Z0-9,=]{1,35}	[A-Z]{1}[A-Z0-9]{3}	---
XML-Typ Definiert in ebics_types_H005.xsd	Beide vom Typ `UserIDType`	`PartnerIDType`	`OrderIDType`	`HostIDType`

13 Anhang: Auftragsartenkennungen

Die administrativen Auftragsarten in den folgenden Tabellen werden in den folgenden Kapiteln ausführlich erläutert:

Schlüsselmanagement (Kapitel 4)
Verteilte Elektronische Unterschrift (Kapitel 8.3))
HAC (Kapitel 10)
und Sonstige administrative Auftragsarten (Kapitel 9)

Kennung	Über-tragungs-richtung	Text	Optional / Verpflichtend durch EBICS-Bankrechnerlösungen zu unterstützen
BTD	D	Abholen einer Datei, die durch eine BTF-Struktur näher gekennzeichnet ist	Verpflichtend
BTU	U	Senden einer Datei, die durch eine BTF-Struktur näher gekennzeichnet ist	Verpflichtend
HAA	D	Abrufbare Auftragsarten abholen	Optional
HAC	D	Abholen Kundenprotokoll (XML-Format)	Verpflichtend
HCA	U	Änderung der Teilnehmerschlüssel für Authentifikation und Verschlüsselung senden	Verpflichtend
HCS	U	Änderung der Teilnehmerschlüssel für EU, Authentifikation und Verschlüsselung senden	Verpflichtend
HEV	D	Unterstützte EBICS-Versionen abrufen	Verpflichtend
HIA	U	Übermittlung der Teilnehmerschlüssel für Authentifikation und Verschlüsselung im Rahmen der Teilnehmerinitialisierung	Verpflichtend
HKD	D	Kunden- und Teilnehmerdaten des Kunden abholen	Optional
HPB	D	Transfer der öffentlichen Bankschlüssel (Abholen)	Verpflichtend
HPD	D	Bankparameter abholen	Verpflichtend
HTD	D	Kunden- und Teilnehmerdaten des Teilnehmers abholen	Optional
HVD	D	VEU-Status abrufen	Verpflichtend
HVE	U	VEU-Unterschrift hinzufügen	Verpflichtend
HVS	U	VEU-Storno	Verpflichtend
HVT	D	VEU-Transaktionsdetails abrufen	Verpflichtend
HVU	D	VEU-Übersicht abholen	Verpflichtend
HVZ	D	VEU-Übersicht mit Zusatzinformationen abholen	Verpflichtend
H3K	U	Übermittlung aller drei Teilnehmerschlüssel (für EU, Authentifikation und Verschlüsselung im Rahmen der Teilnehmerinitialisierung im Falle der Verwendung von CA-Zertifikaten)	Optional

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Die konkrete Verwendung der administrativen Auftragsarten, die für die VEU notwendig sind, ist mit Bank und Kunden zu vereinbaren.

Weitere administrative Auftragsarten des Schlüsselmanagements:

Kennung	Über-tragungs-richtung	Text	Format
INI	U	Senden Passwort-Initialisierung	Public Key des Kunden für die EU (siehe Anhang Kapitel 14)
PTK	D	Abholen Kundenprotokoll	Format siehe Implementation Guide (Kapitel 4.3.1, verpflichtend für deutsche Kreditinstitute)
PUB	U	Senden Public Key zur Unterschriftenverifizierung	Public Key des Kunden für die EU (siehe Anhang Kapitel 14)
SPR	U	Senden Sperre der Zugangsberechtigung	Übertragung einer EU-Datei mit der Signatur für eine Dummy-Datei, die nur ein Leerzeichen enthält (verpflichtend)

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

14 Anhang: Signaturverfahren für die Elektronische Unterschrift

Die zum Einsatz kommenden Sicherheitsverfahren müssen die Elektronische Unterschrift für die zu übertragenden Daten leisten. Hierbei ist folgendes Anforderungsprofil zu erfüllen:

- Die Unterschrift darf nur vom Unterzeichner geleistet werden können, so dass der Unterzeichner die Unterschrift nicht leugnen kann bzw. dass beweisbar ist, dass der Ursprung eines eventuellen Missbrauchs nur im Verantwortungsbereich des Unterzeichners liegen kann.

- Jeder mögliche Empfänger muss die Echtheit der Unterschrift prüfen können, wobei zusätzlich gewährleistet sein muss, dass diese Prüfung auch zu einem späteren Zeitpunkt (z. B. durch juristische Instanzen) möglich ist.

- Die Unterschrift muss in einem direkten Zusammenhang zu dem unterschriebenen Dateiinhalt stehen, so dass sie gleichzeitig den entsprechenden Dateiinhalt authentifiziert und so jeder mögliche Empfänger (insbesondere juristische Instanzen auch noch zu einem späteren Zeitpunkt) anhand der Unterschrift auch den Dateiinhalt verifizieren kann (Prüfung der Dateiintegrität).

- Die Unterschriftslösung muss auf jeden beliebigen Kontext anwendbar sein.

- Das Unterschriftsverfahren muss unter Performancegesichtspunkten auch auf weniger leistungsfähigen PC‘s mit vertretbarer Rechenzeitintensität einsetzbar sein.

- Der Verwaltungsaufwand für die notwendige Aufbewahrung der zur Unterschriften-erzeugung und insbesondere -prüfung benötigten Daten (Kennungen) muss möglichst gering sein (einfaches Key Management).

- Die konkrete technische Lösung muss für die gängigen Betriebssysteme, die beim Unterzeichner bzw. Empfänger zum Einsatz kommen können, kompatibel einsetzbar sein.

- The betriebssystemabhängigem Zeichen CR, LF und Ctrl-Z werden in die Hashwertbildung der A005/A006-EU nicht einbezogen.

Dieses Anforderungsprofil kann nur durch den Einsatz asymmetrischer kryptografischer Verfahren erfüllt werden.

Die Verwendung der Elektronischen Unterschrift wird für alle Datenübertragungen, die nicht der reinen Informationsbeschaffung dienen, dringend empfohlen, soweit in den besonderen Vereinbarungen für einzelne Verfahren nichts Abweichendes bestimmt ist.

Für jedes konkret zum Einsatz kommende Sicherheitsverfahren muss eine detaillierte Beschreibung der darin verwendeten mathematischen Verfahren sowie der verwendeten Datenstrukturen kostenlos offengelegt werden. Diese Beschreibung muss geeignet sein, ein funktional kompatibles Produkt durch beliebige Hersteller erstellen zu lassen. Außerdem muss ein positives Gutachten eines vom Kreditgewerbe bestimmten Gutachters zu dem jeweiligen Gesamtverfahren und speziell zu den darin zum Einsatz kommenden mathematischen Prozeduren vorgelegt werden.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

14.1 Die Elektronische Unterschrift der Version A005/A006

Unter Beachtung der unter Anhang Kapitel 14 formulierten Anforderungen sind die nachfolgend beschriebenen Verfahren für die Elektronische Unterschrift bankseitig verpflichtend zu unterstützen. Für die Signaturverfahren A005 und A006 wird ein Intervall von 2048 Bit (Minimum) und 4096 Bit (Maximum) für die Schlüssellänge festgelegt.

14.1.1 Vorbemerkung und Einleitung

Das Kapitel 14.1 beschreibt in den folgenden Unterkapiteln zwei neue Signaturverfahren, die beide auf den Vorgaben aus [PKCS1] und dem Einsatz von SHA-256 als Hash-Algorithmus basieren. Die beiden Verfahren unterscheiden sich jedoch bezüglich des zum Einsatz kommenden Padding-Verfahrens aus [PKCS1].
Seit der Fertigstellung von [A005] wurde die Namensgebung für die beiden Signaturverfahren geändert. Wurden die beiden Signaturverfahren in [A005] noch als A005_V1.5 und A005_PSS bezeichnet, werden sie zukünftig als A005 und A006 bezeichnet. Die folgende Tabelle zeigt die Zuordnung zwischen neuen und alten Namen und zu den Signaturverfahren aus [PKCS1]:

Zukünftiger Name	Name in [A005]	[PKCS1]
A005	A005_V1.5	EMSA-PKCS1-v1_5 mit SHA-256
A006	A005_PSS (mit vorher erzeugtem SHA-256 Hash-Wert als Eingabewert)	EMSA-PSS mit SHA-256 (mit vorher erzeugtem SHA-256 Hash-Wert als Eingabewert)

Die folgende Beschreibung der beiden Signaturverfahren basiert auf den entsprechenden Abschnitten aus der Spezifikation von SECCOS 6 [SECCOS6]. Beide Signaturverfahren werden durch eine ZKA Signaturkarte unterstützt, die auf SECCOS 6 basiert und die zugehörige Signatur Anwendung der DK (ehemals ZKA, daher unter dem Namen ZKA-Siganturkarte bekannt) [ZKASigAnw] (oder eine höhere Version) enthält.
Da bei kreditwirtschaftlichen Anwendungen zukünftig sowohl der AUT-Schlüssel als auch der DS-Schlüssel einer auf SECCOS basierenden ZKA-Signaturkarte mit Signatur Anwendung gemäß [ZKASigAnw] für das Erstellen einer Signatur genutzt werden sollen, muss die folgende Besonderheit berücksichtigt werden:

- Bei Nutzung des AUT-Schlüssels wird das Kommando INTERNAL AUTHENTICATE verwendet. Soll dieser Schlüssel mit dem PSS-Padding aus [PKCS1] verwendet werden, erzeugt die SECCOS-Chipkarte bei Aufruf des Kommandos INTERNAL AUTHENTICATE immer intern den benötigten Hash-Wert über die Eingabedaten. Da die aufrufende Anwendung aber vor Aufruf ebenfalls einen Hash-Wert über die eigentliche Nachricht M erzeugt, kommt es hierdurch zu einer doppelten Berechnung des Hash-Wertes, d. h. es wird der Wert Hash(Hash(M)) berechnet.

- Aus diesem Grund wird A006 so definiert, dass in das eigentliche Signaturverfahren bereits ein vorher berechneter Hash-Wert über die Nachricht M als Eingabewert eingeht, und nicht die Nachricht M selber.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Die von der ZKA-Chipkarte bereitgestellten asymmetrischen kryptografischen Algorithmen basieren auf dem RSA-Algorithmus mit ungeraden öffentlichen Exponenten ([RSA]).
In Kapitel 14.1.2 des vorliegenden Dokuments werden das Konstruktionsprinzip und die **Schlüsselkomponenten** der öffentlichen und privaten RSA-Schlüssel gemäß Anhang F von [EMV CA] und [PKCS1] für ungerade öffentliche Exponenten erläutert.
Ein ***Signatur-Algorithmus*** besteht aus einem Algorithmus zur Signaturerzeugung und einem hierzu inversen Algorithmus zur Klartextrückgewinnung. Der als Standard von der ZKA-Chipkarte unterstützte Signatur-Algorithmus wird in Kapitel 14.1.3 beschrieben.
Der beschriebene Signatur-Algorithmus auf Basis des RSA-Algorithmus wird durch die ZKA-Chipkarte nur im Rahmen von Signaturverfahren eingesetzt. Ein ***Signaturverfahren*** legt fest, in welcher Weise eine Nachricht M zu einer Bytefolge aufzubereiten ist, die dann in die Signaturerzeugung eines Signatur-Algorithmus eingeht. Die durch ein Signaturverfahren erzeugte Bytefolge wird als ***Digital Signature Input*** (***DSI***) bezeichnet.
Die Signatur Anwendung der DK [ZKASigAnw] unterstützt mehrere verschiedene Signaturverfahren. In Kapitel 14.1.4 des vorliegenden Dokuments werden die beiden neuen Signaturverfahren A005 und A006 beschrieben, die beide auf den Padding-Verfahren aus [PKCS1] und dem Hash-Algorithmus SHA-256 aufbauen.

14.1.2 RSA

Ein RSA-Schlüsselpaar besteht aus

- einem ***öffentlichen Schlüssel P_K*** und
- einem ***privaten Schlüssel S_K***.

Öffentlicher und privater Schlüssel bestehen aus ***Schlüsselkomponenten***. RSA-Schlüssel werden auch als ***asymmetrische Schlüssel*** bezeichnet.
Zur Erzeugung eines RSA-Schlüsselpaares mit einem ungeraden ***öffentlichen Exponenten e*** werden zwei verschiedene ***Primzahlen p und q (Primfaktoren)*** verwendet, für die e teilerfremd zu (p-1) und (q-1) ist.
Der zugehörige ***private Exponent d*** ist dann bestimmt durch

$$e*d \equiv 1 \bmod kgV(p-1, q-1).$$

Die Primzahlen p und q sowie der private Exponent d müssen geheim gehalten werden.
Das Produkt der Primzahlen n = p*q wird als ***Modulus*** bezeichnet.
Der ***öffentliche Schlüssel P_K*** des RSA-Schlüsselpaares besteht aus den Komponenten

- ***Modulus n*** und
- ***öffentlicher Exponent e***.

Der ***private Schlüssel S_K*** des RSA-Schlüsselpaares kann auf zwei Arten durch Komponenten dargestellt werden (vgl. [PKCS1]):

1. Darstellung von S_K durch die Komponenten
   - ***Modulus n*** und

- ***privater Exponent d***, oder

2. Darstellung von S_K durch die Komponenten

- ***Primfaktor p,***
- ***Primfaktor q,***
- d_p = ***d mod (p-1),***
- d_q = ***d mod (q-1) und***
- ***qInv*** = q^{-1} ***mod p.***

Bei der ersten Darstellung muss nur die Komponente d geheim gehalten werden. Die Komponenten der 2. Darstellung werden als ***Chinese Remainder Theorem-Parameter*** (***CRT-Parameter***) bezeichnet. Die CRT-Parameter müssen sämtlich geheim gehalten werden.
Die ZKA-Chipkarte unterstützt den RSA-Algorithmus mit beliebigen ungeraden öffentlichen Exponenten. In der Regel wird aber einer der ungeraden öffentlichen Exponenten 3 oder $F_4 = 2^{16}+1$ verwendet.

In dem vorliegenden Dokument wird die folgende Notation verwendet:
k bezeichnet die Bit-Länge des Modulus n eines RSA-Schlüsselpaares.
k ist durch die Gleichung $2^{k-1} <= n < 2^k$ eindeutig definiert.
n lässt sich darstellen als Folge von Bits

$n = b_k\ b_{k-1} \ldots b_1$, wobei $b_k <> 0$ ist.

Die durch n repräsentierte ganze Zahl wird dadurch bestimmt, dass das erste, linke Bit b_k das höchstwertige Bit und das letzte, rechte Bit b_1 das niedrigstwertige Bit in der Binärdarstellung von n ist. Zu k existieren eindeutige Zahlen N >= 1 und 8 >= r >=1 mit k = 8*(N-1) + r. Dann lässt sich n auch als Folge von Bits schreiben als

$n = b_r\ b_{r-1} \ldots b_1\ b_{8*(N-1)} \ldots b_{8*(N-2)+1} \ldots b_8 \ldots b_1$.

Wenn r = 8 ist, lässt sich n direkt als Folge von N Bytes schreiben:

$n = B_N\ B_{N-1} \ldots B_1$, wobei $B_N <>$ '00' ist.

Falls r < 8 ist, stellt man 8-r binäre 0 der Bitfolge $b_r\ b_{r-1} \ldots b_1\ b_{8*(N-1)} \ldots b_{8*(N-2)+1} \ldots b_8 \ldots b_1$ voran:

$n = 0 \ldots 0\ b_r\ b_{r-1} \ldots b_1\ b_{8*(N-1)} \ldots b_{8*(N-2)+1} \ldots b_8 \ldots b_1$.

und erhält wiederum eine Bytefolge

$n = B_N\ B_{N-1} \ldots B_1$, wobei $B_N <>$ '00' ist.

Der ganzzahlige Wert von n wird durch führende 0 in der Binärdarstellung nicht geändert, so dass die Darstellung von n als Folge von N Bytes den durch eine Folge von k Bits dargestellten Zahlwert unverändert lässt.
N ist die Byte-Länge von n.
N ist durch die Gleichung $2^{8*(N-1)} <= n < 2^{8*N}$ eindeutig definiert.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

14.1.3 Signaturalgorithmus

14.1.3.1 Standardfunktion für die Signaturerzeugung

Sei S_K ein privater RSA-Schlüssel bestehend aus dem Modulus n und dem privaten Exponenten d oder bestehend aus den CRT-Parametern. Der zugehörige öffentliche RSA-Schlüssel P_K bestehe aus dem Modulus n und dem öffentlichen Exponenten e.
Dann können mit S_K binär kodierte Bytefolgen x signiert werden, aus denen sich aus der Binärdarstellung von x ein ganzzahliger Wert zwischen 0 und n-1 ergibt. x lässt sich somit darstellen als Bytefolge mit einer Länge von N Bytes und als Bitfolge mit einer Länge von k Bits. Das k-te Bit in der darstellenden Byte- oder Bitfolge kann, muss aber nicht, den Wert 1 haben. Sofern vorhanden haben die Bits b_{8*N} ... b_{k+1} in der darstellenden Bytefolge den Wert 0.
Für die Signaturerzeugung mit dem aus n und d bestehenden privaten Schlüssel wird die folgende Notation verwendet:

$$sign(S_K)[x] = x^d \bmod n$$

Falls der private Schlüssel S_K aus den CRT-Parametern besteht, berechnet sich $sign(S_K)[x] = x^d \bmod n$ wie folgt:

$$sign(S_K)[x] = s_2 + h*q$$

wobei s_2 und h wie folgt berechnet werden:

$$s_1 = x^{dp} \bmod p,$$

$$s_2 = x^{dq} \bmod q,$$

$$h = qInv * (s_1 - s_2) \bmod p.$$

Hierbei werden die Potenzierungen $x^d \bmod n$, $x^{dp} \bmod p$, $x^{dq} \bmod q$ mit der ganzen Zahl ausgeführt, deren Wert sich aus der Binärdarstellung von x ergibt.
Das Ergebnis der Signaturerzeugung ist wiederum eine Bytefolge s, die sich als Binärdarstellung des ganzzahligen Wertes der Potenz $x^d \bmod n$ bzw. von $s_2 + h*q$ ergibt. Dieser ganzzahlige Wert liegt wieder zwischen 0 und n-1. s lässt sich somit darstellen als Bytefolge mit einer Länge von N Bytes und als Bitfolge mit einer Länge von k Bits. Das k-te Bit in der darstellenden Byte- oder Bitfolge kann, muss aber nicht, den Wert 1 haben. Sofern vorhanden haben die Bits b_{8*N} ... b_{k+1} in der darstellenden Bytefolge den Wert 0.

14.1.3.2 Standardfunktion für die Klartextrückgewinnung

Sei P_K ein öffentlicher RSA-Schlüssel bestehend aus dem Modulus n und dem öffentlichen Exponenten e.
Dann kann mit P_K aus einer binär kodierten Bytefolge s Klartext zurück gewonnen werden, wenn der sich aus der Binärdarstellung von s ergebende ganzzahlige Wert zwischen 0 und n-1 liegt. s lässt sich somit darstellen als Bytefolge mit einer Länge von N Bytes und als Bitfolge mit einer Länge von k Bits. Das k-te Bit in der darstellenden Byte- oder Bitfolge kann, muss aber nicht, den Wert 1 haben. Sofern vorhanden haben die Bits b_{8*N} ... b_{k+1} in der darstellenden Bytefolge den Wert 0.
Für die Klartextrückgewinnung wird die folgende Notation verwendet:

$$recover(P_K)[s] = s^e \bmod n$$

Hierbei wird die Potenzierung $s^e \bmod n$ mit der ganzen Zahl ausgeführt, deren Wert sich aus der Binärdarstellung von s ergibt.
Das Ergebnis der Klartextrückgewinnung ist eine ganze Zahl, deren Wert zwischen 0 und n-1 liegt. Es lässt sich somit darstellen als Bytefolge mit einer Länge von N Bytes und als Bitfolge mit einer Länge von k Bits. Das k-te Bit in der darstellenden Byte- oder Bitfolge kann, muss aber nicht, den Wert 1 haben. Sofern vorhanden haben die Bits b_{8*N} ... b_{k+1} in der darstellenden Bytefolge den Wert 0.
Für ein RSA-Schlüsselpaar P_K und S_K gilt

$$recover(P_K)[sign(S_K)[x]] = x$$

14.1.4 Signaturverfahren A005 und A006 der DK

Die Signaturverfahren A005 und A006 basieren beide auf dem Industriestandard PKCS#1 [PKCS1] und dem Einsatz von SHA-256 als Hash-Algorithmus. Beides sind Signaturverfahren ohne Nachrichtenrückgewinnung.
Ein Hash-Algorithmus bildet Bitfolgen beliebiger Länge (***Eingabe-Bitfolgen***) auf Bytefolgen einer festen, durch den Hash-Algorithmus festgelegten Länge ab. Das Ergebnis der Anwendung eines Hash-Algorithmus auf eine Bitfolge wird als ***Hash-Wert*** bezeichnet.
Der Hash-Algorithmus SHA-256 wird in [FIPS H2] spezifiziert. SHA-256 bildet Eingabe-Bitfolgen beliebiger Länge auf eine Bytefolge von 32 Bytes Länge ab. Das Padding der Eingabe-Bitfolge auf ein Vielfaches von 64 Bytes ist Bestandteil des Hash-Algorithmus. Das Padding erfolgt auch dann, falls die Länge der Eingabe-Bitfolge bereits ein Vielfaches von 64 Bytes ist.
SHA-256 verarbeitet die Eingabe-Bitfolge in Blöcken von 64 Bytes Länge.
Der Hash-Wert einer Eingabe-Bitfolge x unter dem Hash-Algorithmus SHA-256 wird wie folgt bezeichnet:

$$SHA\text{-}256(x)$$

Für das Generieren des Eingabewertes für die eigentliche Signaturerzeugung **(Digital Signature Input DSI)** unterscheidet PKCS#1 [PKCS1] die beiden Methoden EMSA-PKCS1-v1_5 und EMSA-PSS. Entsprechend werden die zwei Signaturverfahren der DK unterschieden, genannt A005 und A006.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

14.1.4.1 Signaturverfahren A005

Für das Berechnen und das Verifizieren einer digitalen Signatur mit dem in [PKCS1] beschriebenen Signaturverfahren mit Nutzung der Methode EMSA-PKCS1-v1_5 müssen die folgenden Punkte angegeben werden:

- der zu verwendende Hash-Algorithmus HASH,
- die Länge H des generierten Hash-Wertes (in Bytes),
- der zum Einsatz kommende Signatur-Algorithmus und
- die maximale Länge N des erzeugten DSI als Eingabe für den Signatur-Algorithmus.

Das Signaturverfahren A005 ist identisch mit EMSA-PKCS1-v1_5, wobei der Hash-Algorithmus SHA-256 verwendet wird. Die Länge H des Hash-Wertes beträgt 32 Bytes.
Als Signatur-Algorithmus wird bei ZKA Chipkarten RSA verwendet. Der Wert N entspricht daher der Bytelänge des Modulus n des verwendeten RSA-Schlüsselpaares.
In den folgenden Abschnitten wird das Berechnen und das Verifizieren einer digitalen Signatur mit dem Signaturverfahren A005 beschrieben. Dabei werden die in Kapitel 14.1.2 definierten Abkürzungen verwendet.

14.1.4.1.1 Berechnen der digitalen Signatur

Gemäß [PKCS1] müssen (bei Nutzung der Methode EMSA-PKCS1-v1_5) die folgenden Schritte für das Berechnen einer digitalen Signatur über eine Nachricht M mit der Bitlänge m ausgeführt werden.

1. Der Hash-Wert HASH(M) mit einer Länge von H Bytes muss berechnet werden. Für das Signaturverfahren A005 SHA-256(M) mit einer Länge von 32 Bytes.

2. Der Eingabewert DSI für den Signatur-Algorithmus muss erzeugt werden.

   Der DSI ist eine Bytefolge von N-1 Bytes, die wie folgt aufgebaut wird:

Bezeichnung	**Länge (Bytes)**	**Wert**
Block-Typ	1	'01'
Padding-Feld	N-3-D	'FF..FF'
Separator	1	'00'
Digest-Info	D	BER-TLV kodiertes Datenobjekt mit der OID und den Parametern des Hash-Algorithmus HASH und dem Hash-Wert HASH(M)

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Bei einer Nutzung von SHA-256 ist das Digest-Info wie folgt aufgebaut:

Tag	**Length** (in byte)	**Value**	**Description**
'30'	'31'		Tag und Länge SEQUENCE
'30'	'0D'		Tag und Länge SEQUENCE
'06'	'09'	'60 86 48 01 65 03 04 02 01'	OID von SHA-256 (2 16 840 1 101 3 4 2 1)
'05'	'00'	-	TLV Kodierung von ZERO (keine Parameter)
'04'	'20'	'XX..XX'	Hash-Wert

Das Digest-Info hat bei Nutzung von SHA-256 eine Länge D von 51 Bytes. Das Padding-Feld hat eine Länge von N-54 Bytes. Da N mindestens den Wert 128 hat (entspricht einer minimalen Schlüssellänge von 1024 Bit), müssen mindestens 74 Bytes 'FF' als Padding eingefügt werden.

3. Die digitale Signatur wird mit der Standardfunktion für die Signaturerzeugung über die Eingabebytefolge DSI wie in Abschnitt 14.1.3.1 beschrieben berechnet.

   Da der DSI eine Bytefolge von N-1 Bytes ist, ist die durch den DSI (betrachtet als Binärdarstellung) gegebene ganze Zahl immer kleiner als der Wert n des Modulus.

   Die berechnete digitale Signatur kann als Bytefolge der Länge N dargestellt werden. Bei der Darstellung des Modulus n als Bytefolge hat das Bit b_k den Wert 1 und die Bits b_{8*N} b_{8*N-1} ... b_{k+1} haben, falls vorhanden, den Wert 0. In der Darstellung der digitalen Signatur als Bytefolge müssen daher die Bits b_{8*N} b_{8*N-1} ... b_{k+1} ebenfalls den Wert 0 haben.

14.1.4.1.2 Verifizieren der digitalen Signatur

Gemäß [PKCS1] müssen (bei Nutzung der Methode EMSA-PKCS1-v1_5) die folgenden Schritte für das Verifizieren einer digitalen Signatur ausgeführt werden. Die zu verifizierende Signatur und die Nachricht M' müssen dabei als Bytefolgen vorliegen.

1. Die digitale Signatur muss eine Bytefolge der Länge N sein. Dabei müssen die Bits b_{8*N} b_{8*N-1} ... b_{k+1}, falls vorhanden, den Wert 0 haben. Ist dies nicht der Fall, muss die Signatur abgelehnt werden.

   Die durch die Signatur (betrachtet als Binärdarstellung) gegebene ganze Zahl muss kleiner sein als der Modulus n. Ist dies nicht der Fall, wird die digitale Signatur abgelehnt.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

2. Die Standardfunktion für die Klartextrückgewinnung muss wie in Abschnitt 14.1.3.2 beschrieben auf die Signatur angewendet werden. Das Ergebnis muss eine Bytefolge der Länge N-1 sein. Ist dies nicht der Fall, muss die Signatur abgelehnt werden.

3. Der Vergleichswert DSI' mit einer Länge von N-1 Bytes wird aus der Nachricht M' wie in den Schritten 1 und 2 von Abschnitt 14.1.4.1.1 beschrieben erzeugt.

4. DSI' muss mit dem in Schritt 2 aus der Signatur zurück gewonnenen Klartext verglichen werden. Stimmen beide Werte überein, wurde die Signatur erfolgreich verifiziert. Andernfalls muss die Signatur abgelehnt werden.

14.1.4.1.3 Notation

Für die Berechnung einer Signatur zu einer Nachricht M mit dem Signaturverfahren A005 und dem privaten RSA-Schlüssel S_K wird die folgende Notation verwendet:

$s = sign_{A005}(S_K)[M]$.

Für die Prüfung einer Signatur s zu einer Nachricht M mit dem Signaturverfahren A005 und dem öffentlichen RSA-Schlüssel P_K wird die folgende Notation verwendet:

$verify_{A005}(P_K)[s,M]$.

14.1.4.2 Signaturverfahren A006

Für das Berechnen und das Verifizieren einer digitalen Signatur mit dem in [PKCS1] beschriebenen Signaturverfahren mit Nutzung der Methode EMSA-PSS müssen die folgenden Punkte angegeben werden:

- der zu verwendende Hash-Algorithmus HASH,
- die Länge H des generierten Hash-Wertes (in Bytes),
- die Länge S des zu nutzenden Zufallswertes salt (in Bytes),
- die zum Einsatz kommenden Maskierfunktion (mask generation function),
- der zum Einsatz kommende Signatur-Algorithmus,
- die maximale Länge k (in Bits) des erzeugten DSI als Eingabe für den Signatur-Algorithmus sowie
- die maximale Länge N (in Bytes) des erzeugten DSI als Eingabe für den Signatur-Algorithmus.

Das Signaturverfahren A006 basiert auf EMSA-PSS, wobei der Hash-Algorithmus SHA-256 verwendet wird. Die Länge H des Hash-Wertes beträgt 32 Byte.
Die Länge S des Zufallswertes salt wird durch den zum Einsatz kommenden Hash-Algorithmus definiert, d.h. die Länge S des Zufallswertes salt ist identisch mit der Länge H des Hash-Wertes.
Im Rahmen des Signaturverfahrens A006 wird nur die in [PKCS1] definierte Maskierfunktion MGF1 verwendet.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Notation: k ist die Länge des Modulus n (in Bits) des verwendeten RSA-Schlüssels. Die Länge des erzeugten DSI (in Bits) ist k-1 und wird mit emBits bezeichnet. Die Länge des Modulus n (in Bytes) wird mit N bezeichnet. Die Länge des DSI (in Bytes) wird mit emLen bezeichnet.

14.1.4.2.1 Maskierfunktion MGF1

Das in den Abschnitten 8.1 und 9.1 von [PKCS1] beschriebene Signaturverfahren nutzt die Maskierfunktion MGF1, die in Abschnitt B.2 von [PKCS1] beschrieben wird.
Die Maskierfunktion MGF1 basiert auf einem Hash-Algorithmus HASH, der Hash-Werte der Länge H (in Bytes) erzeugt. MGF1 erzeugt dann wie folgt aus einem Startwert **mgfSeed** eine Bytefolge der gegebenen Länge **maskLen**:

1. Sei T eine leere Bytefolge.

2. Für einen Zähler von 0 bis $\lceil maskLen / H \rceil - 1$ führe das folgende durch:

   a. Konvertiere den Zähler in eine Bytefolge C der Länge 4 Bytes.

   b. Berechne den Hashwert HASH(mgfSeed | C) und konkateniere diesen mit der aktuellen Bytefolge T, d.h.:

   $$T = T \mid HASH(mgfSeed \mid C)$$

3. Das Ergebnis MGF1(mgfSeed, maskLen) sind die höchstwertigen ("leftmost") maskLen Bytes der Bytefolge T.

Hinweis: $\lceil maskLen / H \rceil$ ist definiert als die kleinste ganze Zahl, die größer als oder gleich (masklen / H) ist.

14.1.4.2.2 Berechnen der digitalen Signatur mit EMSA-PSS

Gemäß den Abschnitten 8.1.1 und 9.1.1 von [PKCS1] müssen (bei Nutzung der Methode EMSA-PSS) die folgenden Schritte für das Berechnen einer digitalen Signatur über eine Nachricht M mit Bitlänge m ausgeführt werden:

1. Der Hash-Wert HASH(M) mit der Länge H (in Bytes) muss berechnet werden. Wird EMSA-PSS als Basis für das Signaturverfahren A006 genutzt, wird SHA-256(M) mit einer Länge von 32 Bytes berechnet.

2. Der Eingabewert DSI für den Signatur-Algorithmus muss wie folgt erzeugt werden:

   Erzeuge eine Zufallszahl der Länge S Bytes, die als Zufallswert salt genutzt wird.

   Erzeuge die Nachricht M' wie folgt:

   M' = '00 00 00 00 00 00 00 00' | HASH(M) | salt

   Berechne über M' den Hash-Wert HASH(M') der Länge H.

   Bilde eine Padding-Bytefolge PS der Länge emLen – H – S – 2 Bytes bestehend aus '00' Bytes.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Setze DB = PS | '01' | salt; DB ist eine Bytefolge der Länge emLen – H – 1.

Sei dbMask = MGF(HASH(M'), emLen – H – 1) das Ergebnis der Maskierfunktion. Hinweis: Wird EMSA-PSS als Basis für A006 genutzt, wird an dieser Stelle MGF1 aus Abschnitt 14.1.4.2.1 verwendet.

Setze maskedDB = DB ⊕ dbMask.

Setze die 8*emLen – emBits höchstwertigen ("leftmost") Bits des höchstwertigen ("leftmost") Byte von maskedDB auf 0.

Setze DSI = maskedDB | HASH(M') | 'BC'.

3. Die digitale Signatur wird mit der Standardfunktion für die Signaturerzeugung über die Eingabebytefolge DSI wie in Abschnitt 14.1.3.1 beschrieben berechnet.

   Die Bytefolge DSI hat eine Länge von emLen Bytes. Die durch den DSI (betrachtet als Binärdarstellung) gegebene ganze Zahl ist immer kleiner als der Wert n des Modulus, da die Bitlänge emBits des DSI kleiner ist als die Bitlänge des Modulus.

   Die berechnete digitale Signatur kann als Bytefolge der Länge N dargestellt werden. Bei der Darstellung des Modulus n als Bytefolge hat das Bit b_k den Wert 1 und die Bits b_{8*N} b_{8*N-1} ... b_{k+1} haben, falls vorhanden, den Wert 0. In der Darstellung der digitalen Signatur als Bytefolge müssen daher die Bits b_{8*N} b_{8*N-1} ... b_{k+1} ebenfalls den Wert 0 haben.

14.1.4.2.3 Verifizieren der digitalen Signatur

Gemäß den Abschnitten 8.1.2 und 9.1.2 von [PKCS1] müssen (bei Nutzung der Methode EMSA-PSS) die folgenden Schritte für das Verifizieren einer digitalen Signatur ausgeführt werden. Die zu verifizierende Signatur und die Nachricht M müssen dabei als Bytefolgen vorliegen.

1. Die digitale Signatur muss eine Bytefolge der Länge N sein. Dabei müssen die Bits b_{8*N} b_{8*N-1} ... b_{k+1}, falls vorhanden, den Wert 0 haben. Ist dies nicht der Fall, muss die Signatur abgelehnt werden.

   Die durch die Signatur (betrachtet als Binärdarstellung) gegebene ganze Zahl muss kleiner sein als der Modulus n. Ist dies nicht der Fall, wird die digitale Signatur abgelehnt.

2. Die Standardfunktion für die Klartextrückgewinnung muss wie in Abschnitt 14.1.3.2 beschrieben auf die Signatur angewendet werden. Das Ergebnis muss eine Bytefolge der Länge emLen sein. Ist dies nicht der Fall, muss die Signatur abgelehnt werden.

3. Der zurück gewonnene Klartext muss wie folgt überprüft werden:

   Berechne den Hash-Wert HASH(M) mit der Länge H Bytes.

   Das niedrigstwertige Byte des Klartexts muss den Wert 'BC' haben. Ist dies nicht der Fall, muss die Signatur abgelehnt werden.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Seien maskedDB die emLen – H – 1 höchstwertigen ("leftmost") Bytes des zurück gewonnenen Klartexts und HM' die nächsten H Bytes des Klartexts.

Falls die 8*emLen – emBits höchstwertigen ("leftmost") Bits des höchstwertigen ("leftmost") Byte von maskedDB nicht den Wert 0 haben, muss die Signatur abgelehnt werden.

Setze dbMask = MGF (HM', emLen – H – 1) mit der Maskierfunktion MGF1.

Sei DB = maskedDB ⊕ dbMask.

Setze die 8*emLen – emBits höchstwertigen ("leftmost ") Bits des höchstwertigen Byte von DB auf 0.

Falls die emLen – H – S – 2 höchstwertigen ("leftmost") Bytes von DB nicht alle den Wert '00' haben oder das Byte an der Position emLen – H – S – 1 nicht den Wert '01' hat, muss die Signatur abgelehnt werden.

Sei salt die S niedrigstwertigen ("rightmost") Bytes von DB.

Setze M' = '00 00 00 00 00 00 00 00' | HASH(M) | salt

und berechne den Hash-Wert HASH(M') der Länge H.

Falls HM' = HASH(M') gilt, wurde die Signatur erfolgreich verifiziert. Andernfalls muss die Signatur abgelehnt werden.

14.1.4.2.4 Notation für EMSA-PSS

Für die Berechnung einer Signatur zu einer Nachricht M mit der Methode EMSA-PSS gemäß [PKCS1] und dem privaten RSA-Schlüssel S_K wird die folgende Notation verwendet:

$s = sign_{EMSA\text{-}PSS}(S_K)[M]$.

Für die Prüfung einer Signatur s zu einer Nachricht M mit der Methode EMSA-PSS gemäß [PKCS1] und dem öffentlichen RSA-Schlüssel P_K wird die folgende Notation verwendet:

$verify_{EMSA\text{-}PSS}(P_K)[s,M]$.

14.1.4.2.5 Berechnen der digitalen Signatur mit A006

Wie bereits in den Vorbemerkungen erwähnt, soll bei kreditwirtschaftlichen Anwendungen auch der (eigentlich für Authentifikationszwecke konzipierte) AUT-Schlüssel für die Berechnung von Signaturen eingesetzt werden. Dabei berechnet eine SECCOS-Chipkarte bei Nutzung des Kommandos INTERNAL AUTHENTICATE (AUT-Schlüssel) im Rahmen des Signaturverfahrens EMSA-PSS immer intern den Hash-Wert über den Eingabewert. Bei diesen Anwendungen sind die Nachrichten, über die die Signaturen berechnet werden, im allgemein sehr lang. Sie können daher nicht direkt als Eingabewert mit dem Kommando INTERNAL AUTHENTICATE an die SECCOS-Chipkarte übergeben werden. Unter anderem aus diesem Grunde berechnet die Anwendung vor Aufruf des SECCOS-Kommandos ebenfalls einen Hash-Wert über die Nachricht und übergibt diesen dann als Eingabewert mit dem Kommando.

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Folglich kommt es bei Nutzung des AUT-Schlüssels immer zu einer doppelten Berechnung des Hash-Werts. Diesen Umstand trägt die Definition des Signaturverfahrens A006 Rechnung.
Um über eine Nachricht M eine Signatur s mit dem privaten Schlüssel S_K und dem Signaturverfahren A006 zu berechnen, führe die beiden folgenden Schritte durch:

- Berechne den Hashwert HM = SHA-256(M).
- Berechne dann die Signatur $s = sign_{EMSA\text{-}PSS}(S_K)[HM]$.

14.1.4.2.6 Verifizieren der digitalen Signatur mit A006

Um eine Signatur s zu einer Nachricht M mit dem öffentlichen Schlüssel P_K und dem Signaturverfahren A006 zu überprüfen, führe die beiden folgenden Schritte durch:

- Berechne den Hashwert HM = SHA-256(M).
- Überprüfe dann s mittels $verify_{EMSA\text{-}PSS}(P_K)[s,HM]$.

14.1.4.2.7 Notation für A006

Für die Berechnung einer Signatur zu einer Nachricht M mit dem Signaturverfahren A006 und dem privaten RSA-Schlüssel S_K wird die folgende Notation verwendet:

$s = sign_{A006}(S_K)[M]$.

Für die Prüfung einer Signatur s zu einer Nachricht M mit dem Signaturverfahren A006 und dem öffentlichen RSA-Schlüssel P_K wird die folgende Notation verwendet:

$verify_{A006}(P_K)[s,M]$.

14.1.5 Referenzen

[EMV CA] Europay International, MasterCard International and Visa International, Integrated Circuit Card Specifications for Payment Systems, Annexes, Version 3.1.1, 31.05.1998

[FIPS H2] FIPS 180-2, Secure Hash Signature Standard, Federal Information Processing Standards Publication 180-2, U. S. Department of Commerce / N.I.S.T., National Technical Information Service, August 2002

[PKCS1] PKCS #1: RSA Encryption Standard, Version 2.1, 14.06.2002

[RSA] R. L. Rivest, A. Shamir, and L. Adleman, A method for obtaining digital signatures and public key cryptosystems, Communications of the ACM, vol. 21, n. 2, 1978, 120-126

[SECCOS6] Interface Specifications for the SECCOS ICC, Secure Chip Card Operating System (SECCOS), Version 6.1, 19.05.2006 (with revisions as on October 16th, 2006)

[ZKASigAnw] Interface Specifications for the SECCOS ICC, Digital Signature Application for SECCOS 6, Version 1.1, 25.05.2007

14.1.6 XML-Struktur der Unterschriftsversionen A005/A006

Die bankfachliche Elektronische Unterschrift (EU) in strukturierter Form hat folgenden Aufbau:

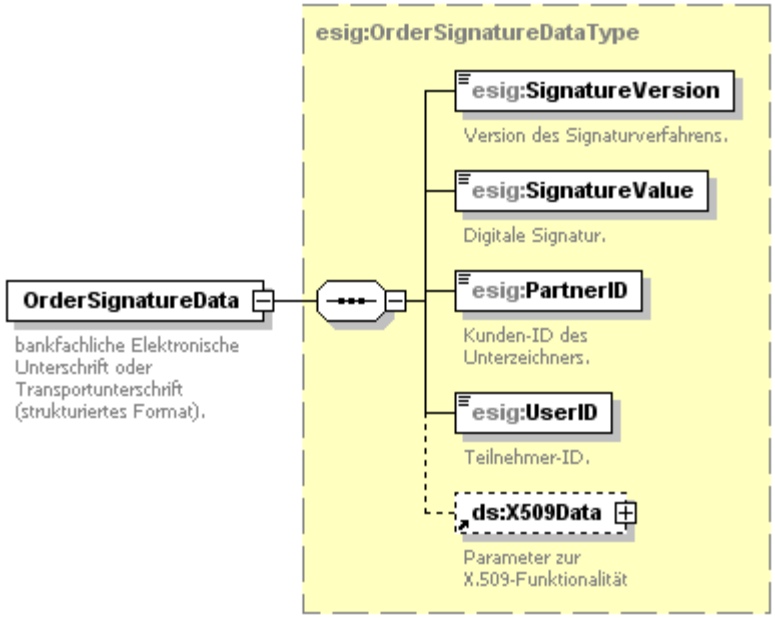


Abbildung 101: OrderSignatureData – strukturierte Elektronische Unterschrift

`OderSignatureData` darf immer nur als Bestandteil eines XML Dokuments mit dem Rootelement `UserSignatureData` übertragen werden. Weitere Erläuterungen und Abbildung siehe Kapitel 3.5.3.

Mit der Intention, die strukturierte Form der EU auch außerhalb von EBICS nutzen zu können, wurden alle notwendigen Datenstrukturen in einer eigenständigen XSD-Datei (ebics_signature_S002.xsd) definiert. Sie ist unter http://www.ebics.de (siehe „Spezifikation“) abrufbar.
Für den Transport der öffentlichen Signaturschlüssel wird das Format `SignaturePubKeyInfoType` verwendet (siehe Kapitel 4.2).

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

15 Anhang: Standards und Verweise

Im EBICS-Feinkonzept wird Bezug auf einige Verfahren, Algorithmen und Formatfestlegungen genommen.

Die zugehörigen Standard-Dokumentkennungen sowie Links zu den referenzierten Dokumenten werden im Folgenden aufgeführt.

Standard	Eigenschaft	Standard-kennung	Verweis
EBICS	multibankfähige Schnittstelle zur internetbasierten Kommunikation	H005	http://www.ebics.de, Kategorie „Spezifikation“ (XML-Schema)
ZIP	universeller Komprimierungs-algorithmus	RFC 1950, RFC 1951	http://www.ietf.org/rfc/rfc1950.txt http://www.ietf.org/rfc/rfc1951.txt
base64	Kodierungsformat zum textuellen Bytecode-Transport	RFC 1421, RFC 2045	http://www.ietf.org/rfc/rfc1421.txt http://www.ietf.org/rfc/rfc2045.txt
UTF-8	Kodierungsformat für Unicode-Zeichen	RFC 3629 (ISO 10646)	http://www.ietf.org/rfc/rfc3629.txt
HTTP 1.1	Internet-Anwendungs-protokoll	RFC 2616	http://www.ietf.org/rfc/rfc2616.txt
TLS	Transportschicht-Verschlüsselung	RFC 2246, RFC 3268 (+AES), RFC 2818 (HTTP über TLS)	http://www.ietf.org/rfc/rfc2246.txt http://www.ietf.org/rfc/rfc3268.txt http://www.ietf.org/rfc/rfc2818.txt
TCP	Internet-Übertragungs-protokoll	RFC 793	http://www.ietf.org/rfc/rfc793.txt
IP(v4)	Internet-Netzwerkprotokoll	RFC 791	http://www.ietf.org/rfc/rfc791.txt
XML	hierarchische Dokumenten-sprache	(W3C-Rec.)	http://www.w3.org/TR/REC-xml/
XML-Signature	Verfahren zur digitalen Signatur von XML-	RFC 3275	http://www.ietf.org/rfc/rfc3275.txt http://www.w3.org/TR/xmldsig-core/

	Dokumenten		
X.509v3	Format und Profil für PKI-Zertifikatsdaten	RFC 5280	http://www.ietf.org/rfc/rfc52805.txt
Länder-codes	Format für Länderkürzel	RFC 1766, ISO 639	http://www.ietf.org/rfc/rfc1766.txt
Zeitstempel	Format für Datums- und Zeitstempel	ISO 8601 (2004)	http://www.iso.org/iso/en/CatalogueDetail»Page.CatalogueDetail?CSNUMBER=40874
SHA-1	Hashalgorithmus	RFC 3174, FIPS 180-2 (SHA allg.)	http://www.ietf.org/rfc/rfc3174.txt http://csrc.nist.gov/publications/fips/fips»180-2/fips180-2withchangenotice.pdf
AES	symmetrischer Verschlüsse-lungsalgorithmus	FIPS 197	http://csrc.nist.gov/publications/fips/fips197/»fips-197.pdf

16 Anhang: Glossar

Administrative Auftragsart	Dreistellige alphanumerische Kennzeichnung für EBICS-interne administrative Aufträge.
AES	„Advanced Encryption Standard“: ein symmetrischer Verschlüsselungsalgorithmus, der DES ablösen soll. Im EBICS-Kontext findet AES Anwendung bei TLS sowie bei der Verschlüsselung bankfachlicher Nutzdaten (gemäß RFC 3268).
Auftrag	Bankfachlicher oder systembedingter Geschäftsvorfall, dessen Typ über die sogenannte Auftragsart identifiziert wird.
Auftragsdaten	siehe „bankfachliche Nutzdaten“ und „BTF-Parameterwerte“
Auftragsnummer	Eindeutige, durch den Bankrechner vergebene Auftragsnummer, die dem Kunden(system) mittels Upload-Response mitgeteilt wird. Sie dient insbesondere zur Synchronisation von Auftragsdaten und elektronischen Unterschriften. Durch die Anwendung ist sicherzustellen, dass pro Kunden-ID und pro BTF-Parameterwertkombination eindeutige Auftragsnummern vergeben werden. Aufbau der 4-stelligen Auftragsnummer: Erste Stelle: Alphazeichen (A-Z) frei wählbar Restliche 3 Stellen: Alphanumerisch (A-Z oder 0-9) hochzählen
Auftrags-parameter	Zusätzliche Parameter zum Auftrag, die der Client im ersten Transaktionsschritt an den Server übermittelt. Siehe Kapitel 3.11.
Authentifikations-schlüssel (öffentlich/privat)	RSA-Schlüsselpaar, dessen privater Schlüssel zur Bildung der Authentifikationssignatur und dessen öffentlicher Schlüssel zu deren Prüfung eingesetzt wird.
Authentifikations-signatur	Digitale Signatur zur Sicherung der Authentizität der Steuerdaten einer EBICS-Nachricht. Als Signaturformat kommt XML-Signatur zur Anwendung.
bankfachliche Elektronische Unterschrift	EU eines Teilnehmers der Unterschriftsklasse „E“, „A“ oder „B“, mit der die Durchführung eines Auftrags autorisiert wird.
bankfachliche Nutzdaten	Daten, die für die Durchführung eines Auftrags benötigt werden. Das Format dieser Daten ist abhängig von den BTF-Parameterwerten und/oder der administrativen Auftragsart. Die Mehrheit der Datenformate, die in EBICS verwendet werden, ist bereits definiert. Die Datenformate der für EBICS neu definierten administrativen Auftragsarten (wie etwa Auftragsarten für die Verteilte Elektronische Unterschrift) sind in EBICS mittels XML-Schema definiert. Die Nutzdaten eines Auftrags werden transparent (in komprimierter verschlüsselter Form) in EBICS-Nachrichten eingebettet.
Bankfachlicher Schlüssel (öffentlich/privat)	RSA-Schlüsselpaar, dessen privater Schlüssel zur Bildung der bankfachlichen Elektronischen Unterschrift und dessen öffentlicher Schlüssel zu deren Verifikation verwendet wird.
Bankfachliches Zielsystem	Komponente in der Verantwortung des Kreditinstituts, die für die Kunden-/ Teilnehmerverwaltung sowie die Durchführung

	bankfachlicher Aufträge zuständig ist. Im Rahmen der EBICS-Spezifikation gilt das bankfachliche Zielsystem als „sichere Blackbox“.
Banksystem	Komponenten in der Verantwortung des Kreditinstituts, die an der Durchführung einer EBICS-Transaktion beteiligt sind. Dies umfasst sowohl das bankfachliche Zielsystem als auch den/die HTTP-Server, die die EBICS-Nachricht empfangen und an das bankfachliche Zielsystem weiterreichen.
Base64	Kodierungsalgorithmus und —format gemäß RFCs 1421 und 2045. Das Resultat eines base64-Kodierungsdurchlaufs ist komplett in ASCII darstellbar.
BTF-Parameterwerte	Satz von Informationen, die die Art eines Auftrags kennzeichnen, in erster Linie durch die Informationen aus der XML-Struktur „Service“
CA	Abkürzung für Certificate Authority (Zertifizierungsinstanz)
Client	Kommunikationseinheit, die EBICS-Requests verschickt und EBICS-Responses empfängt. Siehe auch „Kundensystem“.
Download-Transaktion	EBICS-Transaktion zur Übermittlung eines Abholauftrags. Die Transaktionsphasen einer Download-Transaktion sind: Transaktionsinitialisierung, Datentransfer, Quittierung der Abholdaten.
EBICS-Nachricht	EBICS-Request eines Teilnehmers oder EBICS-Response des Kreditinstituts. EBICS-Nachrichten setzen sich im Wesentlichen aus Steuerdaten, der Authentifikationssignatur und aus bankfachlichen Daten zusammen.
EBICS-Request	Request eines Teilnehmers im XML-Format, das in EBICS definiert wurde.
EBICS-Response	Response des Kreditinstituts im XML-Format, das in EBICS definiert wurde.
EBICS-Transaktion	Sequenzielle Abfolge von EBICS-Transaktionsphasen, die notwendig sind, um einen Auftrag an das bankfachliche Zielsystem zu übermitteln**.** EBICS-Transaktionen können Upload- oder Download-Transaktionen sein.
EBICS-Transaktions-phase	Abfolge zusammengehöriger EBICS-Transaktionsschritte. Bei EBICS werden folgende Transaktionsphasen unterschieden: Transaktionsinitialisierung („initialisation“), Datentransfer („transfer“) und Quittierung („receipt“).
EBICS-Transaktions-schritt	Paar aus EBICS-Request und zugehöriger EBICS-Response. Ein EBICS-Request wird stets vom Kundensystem initiiert.
EBICS-Transaktions-verwaltung	Komponente des Banksystems, deren Verantwortlichkeit die Verwaltung von EBICS-Transaktionen ist.
EDS	Electronic Distributed Signature ,siehe „Verteilte Elektronische Unterschrift“.
Elektronische Unterschrift (EU)	Willentliche Signatur der Auftragsdaten eines Sendeauftrags durch einen Teilnehmer, mit welcher der entsprechende Auftrag eingereicht oder autorisiert werden kann, oder auch Signatur des Kreditinstituts

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

	über Abholdaten. In EBICS werden EUs gemäß Anhang (Kapitel 14) verwendet, die mindestens nach dem Verfahren A005 gebildet sind
EU	siehe „Elektronische Unterschrift“
EU-Signatur-schlüssel	siehe „bankfachlicher Schlüssel (öffentlich/privat)“.
Host-ID	EBICS Host-ID zur Identifikation des EBICS Bankrechners in jeder Request-Nachricht des Kundensystems. Das Kreditinstitut teilt dem Kunden die EBICS Host-ID zusammen mit der URL des Bankzugangs mit.
Kunde	Organisatorische Einheit (Unternehmen oder Privatperson), die mit dem Kreditinstitut seinen Vertrag abschließt. Der Vertrag legt fest, welche Geschäftsvorfälle der Kunde mit dem Kreditinstitut abwickelt, welche Konten betroffen sind, welche Teilnehmer des Kunden mit dem Banksystem arbeiten und welche Berechtigungen die Teilnehmer besitzen.
Kundensystem	Komponente, welche von Teilnehmern verwendet wird, um Aufträge an ein Kreditinstitut zu senden und Information zu Aufträgen oder Teilnehmerkonten vom Kreditinstitut zu erhalten.
OrderData	siehe „Auftragsart“.
Partner	siehe „Kunde“.
RFC	Request for Comments
Schlüssel-verwaltung	Komponente des Banksystems, die für Zuordnung öffentlicher Schlüssel zu Teilnehmern zuständig ist und den Zugriff auf die von ihr verwalteten Schlüssel kontrolliert.
Segmentierung	Aufteilung des Datenvolumens der Auftragsdaten nach Komprimierung, Verschlüsselung und base64-Kodierung in Segmente von max. 1 MB Größe. Siehe auch Kapitel 7.
Server	Kommunikationseinheit, die EBICS-Requests empfängt und EBICS-Responses verschickt. Siehe auch „Banksystem“.
Steuerdaten	Daten einer EBICS-Nachricht, die für die Steuerung des Ablaufs einer EBICS-Transaktion benötigt werden. Es handelt sich dabei sowohl um Daten für die Authentisierung des Teilnehmers gegenüber dem bankfachlichen Zielsystem, als auch um Daten zur Identifikation des nächsten auszuführenden Transaktionsschritts oder um technische Returncodes, als auch um Auftragsparameter, Daten zur Vorabprüfung eines Auftrags oder bankfachliche Returncodes.
Teilnehmer	Menschlicher Nutzer („nichttechnischer Teilnehmer“) oder technisches System („technischer Teilnehmer“), der/das einem Kunden zugeordnet ist. Wird über die Kombination aus TeilnehmerID und KundenID identifiziert. Der technische Teilnehmer dient dem Datenaustausch zwischen einem Kunden und einem Kreditinstitut und ist nicht gleichzusetzen mit einer technischen ID für Service-Dienstleister.
Teilnehmer-initialisierung	Verfahren, nach welchem die öffentlichen Teilnehmerschlüssel dem Kreditinstitut übermittelt werden und anschließend vom Kreditinstitut

	freigegeben werden. Nach dem erfolgreichen Durchlauf der Teilnehmerinitialisierung haben die Teilnehmer im Banksystem den Zustand „Bereit“.
TLS	„Transport Layer Security“: Protokoll gemäß RFCs 2246 und 3268 zur kryptografischen Absicherung von Nachrichten, die TCP/IP als Übertragungsprotokoll benutzen. Im EBICS-Kontext findet TLS als Transportverschlüsselung für HTTP-Nachrichten (HTTPS) Anwendung.
Transaktion	siehe „EBICS-Transaktion“.
Transaktions-phase	siehe „EBICS-Transaktionsphase“.
Transaktions-schlüssel	Symmetrischer Schlüssel, der innerhalb einer EBICS-Transaktion zur Verschlüsselung bankfachlicher Daten verwendet wird.
Transaktions-schritt	siehe „EBICS-Transaktionsschritt“.
Transaktions-verwaltung	siehe „EBICS-Transaktionsverwaltung“.
Transport-unterschrift	EU eines Teilnehmers der Unterschriftsklasse „T“, mit der ein Auftrag eingereicht (und nicht seine Durchführung autorisiert) wird.
Unterschrifts-klasse	Bezieht sich auf EUs von Teilnehmern. EBICS definiert die folgenden Unterschriftsklassen: Einzelunterschrift (Typ „E“), Erstunterschrift (Typ „A“), Zweitunterschrift (Typ „B“), Transportunterschrift (Typ „T“). Details siehe Kapitel 3.5.1.
Upload-Transaktion	EBICS-Transaktion zur Übermittlung eines Sendeauftrags. Die Transaktionsphasen einer Upload-Transaktion sind: Transaktionsinitialisierung, Datentransfer.
UTF-8	„Unicode Transformation Format“, ein Zeichenkodierungsstandard gemäß RFC 3629.
Verschlüsselungs-schlüssel (öffentlich/privat)	RSA-Schlüsselpaar, dessen öffentlicher Schlüssel vom Kommunikationspartner zur Verschlüsselung des symmetrischen Transaktionsschlüssels und dessen privater Schlüssel vom Inhaber zur Entschlüsselung desselben Transaktionsschlüssels eingesetzt wird.
Verteilte bankfachliche Unterschrift	siehe „Verteilte Elektronische Unterschrift“.
Verteilte Elektronische Unterschrift	Verfahren, mit dem bankfachliche Elektronische Unterschriften zu einem bestimmten Auftrag orts- und zeitunabhängig voneinander geleistet werden können. Siehe hierzu Kapitel 8. Die deutsche Abkürzung lautet „VEU“, die internationale Abkürzung lautet; „EDS“ (Electronic Distributed Signature)“.
Vertrauensanker	Im Kontext der Zertifikatsprüfung bezeichnet ein Vertrauensanker (Point of Trust) ein Zertifikat, das als vertrauenswürdig gilt. Normalerweise handelt es sich hier um das Zertifikat einer CA

	(Certification Authority).
VEU	siehe „Verteilte Elektronische Unterschrift“.
Zertifikat	Wenn der Begriff „Zertifikat“ in den EBICS-Spezifikationsdokumenten verwendet wird, ist das Public Key Format X.509 gemeint. Schlüssel, die durch eine Zertifizierungsinstanz (CA) herausgegeben werden, werden CA-Zertifikate genannt. In den EBICS-Dokumenten werden die **Hashwerte der Schlüssel** als Zusammensetzung der **Hashwerte des jeweiligen Zertifikates** verstanden.
ZIP	Verlustfreier Komprimierungsalgorithmus gemäß RFCs 1950 und 1951.

17 Abbildungsverzeichnis

Abbildung 1: Symbolik für XML-Schema 14

Abbildung 2 Verschachtelung von Aktivitäten 15

Abbildung 3: Wurzelstruktur des EBICS-Protokolls 22

Abbildung 4: XML-Strukturen UserSignatureData für die EUs eines Auftrags (in strukturierter Form) 29

Abbildung 5: X509DataType 33

Abbildung 6: OrderParams 34

Abbildung 7: Mögliche Ausprägungen für die Auftragsparameter (OrderParams) 35

Abbildung 8: Beispielhafter Ablauf einer EBICS-Transaktion für einen Sendeauftrag 37

Abbildung 9: Beispielhafter Ablauf einer EBICS-Transaktion für einen Abholauftrag 38

Abbildung 10: Definition des XML-Schema-Typs AuthenticationPubKeyInfoType 43

Abbildung 11: Definition des XML-Schema-Typs SignaturePubKeyInfoType 43

Abbildung 12: Definition des XML-Schema-Typs EncryptionPubKeyInfoType 43

Abbildung 13: Notwendige Schritte im Vorfeld der eigentlichen Durchführung von Geschäftsvorfällen über EBICS (wenn mit INI/HIA initialisiert wird) 45

Abbildung 14: Beispielhafter Ablauf: Teilnehmerinitialisierung mit anschließender Abholung und Überprüfung der Bankschlüssel (wenn mit INI/HIA initialisiert wird) 46

Abbildung 15: Bankseitige Verarbeitung eines INI-Requests 50

Abbildung 16: Bankseitige Verarbeitung eines HIA-Requests 53

Abbildung 17: Zustandsübergangsdiagramm für Teilnehmer 56

Abbildung 18: Definition des XML-Schema-Elements SignaturePubKeyOrderData der Auftragsdaten für INI (identisch mit PUB, siehe eigenes Kapitel) 57

Abbildung 19: Definition des XML-Schema-Elements HIARequestOrderData der Auftragsdaten für HIA 57

Abbildung 20: EBICS-Request für die administrative Auftragsart INI 59

Abbildung 21: EBICS-Response für die Auftragsart INI 60

Abbildung 22: EBICS-Request für die Auftragsart HIA 61

Abbildung 23: EBICS-Response für die Auftragsart HIA 61

Abbildung 24: Definition des XML-Schema-Elements H3KRequestOrderData für H3KOrder Data 63

Abbildung 25: Bankseitige Verarbeitung eines HPB-Requests 66

Abbildung 26: Definition des XML-Schema-Elements HPBResponseOrderData der Auftragsdaten für HPB 68

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Abbildung 27: EBICS-Request für die administrative Auftragsart HPB 70

Abbildung 28: EBICS-Response für die administrative Auftragsart HPB 71

Abbildung 29: Änderung des bankfachlichen Teilnehmerschlüssels mittels PUB 75

Abbildung 30: Änderung des Authentifikationsschlüssels sowie des Verschlüsselungsschlüssels mittels HCA 76

Abbildung 31: Änderung des bankfachlichen Teilnehmerschlüssels sowie Authentifikationsschlüssel und Verschlüsselungsschlüssel mittels HCS 77

Abbildung 32: Definition des XML-Schema-Elements SignaturePubKeyOrderData der Auftragsdaten für PUB (identisch mit INI, siehe eigenes Kapitel) 78

Abbildung 33: Definition des XML-Schema-Elements HCARequestOrderData der Auftragsdaten für HCA 78

Abbildung 34: Definition des XML-Schema-Elements HCSRequestOrderData der Auftragsdaten für HCS 79

Abbildung 35: Fehlerfreier Ablauf einer Upload-Transaktion 89

Abbildung 36: EBICS-Request für die Transaktionsinitialisierung für einen BTF-Upload-Auftrag 93

Abbildung 37: XML-Dokument, das die EUs der Unterzeichner des Sendeauftragsenthält 93

Abbildung 38: EBICS-Response der Transaktionsinitialisierung für einen BTF-Upload 94

Abbildung 39: EBICS-Request für die Übertragung des letzten Nutzdatensegments für einen BTF-Upload 95

Abbildung 40: EBICS-Response der Übertragung des letzten Nutzdatensegments für einen BTF-Upload 96

Abbildung 41: BTF-Struktur für Senden (verwendet eingeschränkte Service-Struktur) 98

Abbildung 42: Detaillierte Beschreibung des Ablaufschritts „Überprüfung der Authentizität des EBICS-Requests“ 107

Abbildung 43: Detaillierte Beschreibung des Ablaufschritts „Teilnehmerbezogene Prüfungen des Auftrags“ 108

Abbildung 44: Detaillierte Beschreibung des Ablaufschritts „Erzeugung einer EBICS-Transaktion“ 109

Abbildung 45: Verarbeitung des EBICS-Requests aus der Transaktionsinitialisierung 110

Abbildung 46: Detaillierte Beschreibung des Ablaufschritts „Überprüfung einer EBICS-Transaktion“ 114

Abbildung 47: Verarbeitung eines EBICS-Requests zur Übermittlung eines Nutzdatensegments (1.Teil) 114

Abbildung 48: Verarbeitung eines EBICS-Requests zur Übermittlung eines Nutzdatensegments (2.Teil) 115

Abbildung 49: Abbruch des Recovery einer Upload-Transaktion aufgrund der Überschreitung der maximal erlaubten Zahl von Recovery-Versuchen 117

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Abbildung 50: Recovery einer Upload-Transaktion mit expliziter Synchronisation zwischen Kunden- und Banksystem 118

Abbildung 51: EBICS-Response mit technischem Fehler EBICS_TX_RECOVERY_SYNC 119

Abbildung 52: Fehlerfreier Ablauf einer Download-Transaktion 119

Abbildung 53: EBICS-Request für die Transaktionsinitialisierung für das Abholen eines Kontoauszugs im Format MT940 122

Abbildung 54: EBICS-Response der Transaktionsinitialisierung für das Abholen eines Kontoauszugs im Format MT940 124

Abbildung 55: EBICS-Request für die Übertragung des nächsten Nutzdatensegments für den Kontoauszug im Format MT940 125

Abbildung 56: EBICS-Response der Übertragung des letzten Nutzdatensegments für den Kontoauszug im Format MT940 127

Abbildung 57: EBICS-Request für die Quittierung der Download-Daten 128

Abbildung 58: EBICS-Response der Quittierung der Download-Daten 129

Abbildung 59: BTF-Struktur für Abholung (verwendet eingeschränkte Service-Struktur) 130

Abbildung 60: Verarbeitung des EBICS-Requests der Initialisierungsphase einer Download-Transaktion 135

Abbildung 61: Detaillierte Beschreibung des Ablaufschritts „Überprüfung der Download-Transaktion“ 136

Abbildung 62: Verarbeitung eines EBICS-Requests zur Anforderung eines Nutzdatensegments 137

Abbildung 63: Verarbeitung eines EBICS-Requests zur Quittierung im Rahmen einer Download-Transaktion 139

Abbildung 64: Abbruch des Recovery einer Download-Transaktion aufgrund der Überschreitung der maximal erlaubten Zahl von Recovery-Versuchen 140

Abbildung 65: Recovery einer Download-Transaktion mit expliziter Synchronisation zwischen Kunden- und Banksystem 141

Abbildung 66: EBICS-Response mit dem technischen Fehler EBICS_TX_RECOVERY_SYNC 142

Abbildung 67: Ablaufdiagramm zur VEU 149

Abbildung 68: HVUOrderParams 152

Abbildung 69: HVUResponseOrderData 154

Abbildung 70: HVUSigningInfoType (zu SigningInfo) 155

Abbildung 71: SignerInfoType (zu SignerInfo) 155

Abbildung 72: HVUOriginatorInfoType (zu OriginatorInfo) 156

Abbildung: 73 HVZOrderParams 161

Abbildung 74: HVZResponseOrderData 164

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)

Abbildung 75 HVZPaymentOrderDetailsStructure 165
Abbildung 76: HVDOrderParams 177
Abbildung 77: HVDResponseOrderData 179
Abbildung 78: HVTOrderParams 184
Abbildung 79: HVTResponseOrderData 189
Abbildung 80: HVTOrderInfoType (zu OrderInfo) 190
Abbildung 81: HVTAccountInfoType (zu AccountInfo) 191
Abbildung 82: HVEOrderParams 205
Abbildung 83: HVSOrderParams 209
Abbildung 84: Nicht-eingeschränkte BTF-Service-Struktur nur für HVU und HVZ-Requests 212
Abbildung 85: „Standard“-BTF-Servicestruktur für alle anderen Fälle 213
Abbildung 86: HAAResponseOrderData 214
Abbildung 87: HPDResponseOrderData 217
Abbildung 88: HPDAccessParamsType (zu AccessParams) 217
Abbildung 89: HPDProtocolParamsType (zu ProtocolParams) 218
Abbildung 90: HPDVersionType (zu Version) 219
Abbildung 91: HKDResponseOrderData 225
Abbildung 92: PartnerInfoType (zu PartnerInfo) 226
Abbildung 93: AddressInfoType (zu AddressInfo) 227
Abbildung 94: BankInfoType (zu BankInfo) 227
Abbildung 95: AuthOrderInfoType (zu OrderInfo) 228
Abbildung 96: UserInfoType (zu UserInfo) 228
Abbildung 97: UserPermissionType (zu Permission) 229
Abbildung 98: HTDResponseOrderData 244
Abbildung 99: HEVRequest / HEVResponse 248
Abbildung 100: Definition des XML-Schema Typs DataEncryptionInfoType 270
Abbildung 101: OrderSignatureData – strukturierte Elektronische Unterschrift 296

© EBICS SCRL (für das fachliche Konzept) / © Die Deutsche Kreditwirtschaft (für die deutsche Übersetzung)